

Experimentos para WD

Pre-procesamiento

```
In [1]: import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from pathlib import Path

from tensorflow.keras.models import load_model, Model
from tensorflow.keras import layers
from matplotlib.lines import Line2D
from sklearn.decomposition import PCA, KernelPCA
from sklearn.manifold import TSNE
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, DBSCAN, SpectralClustering
from sklearn.metrics import silhouette_score

from ipywidgets import interact, widgets

import seaborn as sns
from sklearn.metrics.cluster import contingency_matrix
```

```
In [2]: project_dir = Path().resolve().parent

model_path = os.path.join(project_dir, r"models\model_4C_3FC.h5")
data_path = os.path.join(project_dir, r"data\raw\sdss_dat_files")
labels_path = os.path.join(project_dir, r"data\raw\label_data")

print(project_dir)
print(model_path)
print(data_path)
print(labels_path)
```

```
C:\Users\javip\OneDrive - Universidad Técnica Federico Santa María\6to año\elo308\ML_NPF
C:\Users\javip\OneDrive - Universidad Técnica Federico Santa María\6to año\elo308\ML_NPF\models\model_4C_3FC.h5
C:\Users\javip\OneDrive - Universidad Técnica Federico Santa María\6to año\elo308\ML_NPF\data\raw\sdss_dat_files
C:\Users\javip\OneDrive - Universidad Técnica Federico Santa María\6to año\elo308\ML_NPF\data\raw\label_data
```

Funciones

Función process_spectrum

```
In [3]: def process_spectrum(file_path):
    try:
        sed = np.loadtxt(file_path, unpack=True)
        flux = sed[1, 48:-400] # 4200 puntos

        # Ajuste a 3750 puntos, como espera la red
        if len(flux) > 3750:
            diff = len(flux) - 3750
            start = diff // 2
            flux = flux[start:start + 3750]
        elif len(flux) < 3750:
            flux = np.pad(flux, (0, 3750 - len(flux)), mode='constant')

        norm_val = flux[2100]
        if norm_val == 0 or np.isnan(norm_val):
            norm_val = np.mean(flux)

        if norm_val == 0 or np.isnan(norm_val):
            return None

        flux = flux / norm_val

        if np.isnan(flux).any():
            return None

        return flux.astype(np.float32)

    except Exception:
        return None
```

```
In [4]: fields = ['File Name', 'MJD', 'Target ID', 'DB ID', 'Classification', 'Data Quality']
raw_labels_df = pd.DataFrame()

for filename in os.listdir(labels_path):
    if filename.endswith('.csv'):
        file_path = os.path.join(labels_path, filename)
        df = pd.read_csv(file_path, usecols=fields)
        raw_labels_df = pd.concat([raw_labels_df, df], ignore_index=True)

print(raw_labels_df.shape)
print(raw_labels_df['Classification'].value_counts().head(20))
```

```
(35253, 6)
Classification
WDA          13748
STAR         8749
WD+MS        2555
DUNNO        2086
UNCLASS      1441
WDC          1353
WDB          1186
WDELM        1069
sdX          744
WD           743
WDZ          383
CV           356
EXGAL        301
WDH          272
WDO          184
WDQ          81
WELM         1
DA           1
Name: count, dtype: int64
```

```
In [5]: raw_labels_df
```

Out[5]:

	Classification	DB ID	Data Quality	File Name	MJD	Target ID
0	WDA	21899	NaN	spec-15249-59265-04601916346-21899.png	59265	4601916346
1	sdX	23360	BLEND	spec-15301-59338-04602344336-23360.png	59338	4602344336
2	WDA	18900	SNR	spec-15113-59217-04538755416-18900.png	59217	4538755416
3	WDA	18575	NaN	spec-15086-59267-04474143209-18575.png	59267	4474143209
4	WD	20528	SNR	spec-15198-59269-04351438385-20528.png	59269	4351438385
...	...	...	...	...	...	...
35248	WDA	15895	SNR	spec-15011-59222-04399876473-15895.png	59222	4399876473
35249	WDA	20605	NaN	spec-15200-59324-04589320996-20605.png	59324	4589320996
35250	WDA	19388	NaN	spec-15193-59243-04594923329-19388.png	59243	4594923329
35251	DUNNO	19161	SNR	spec-15121-59212-04342114705-19161.png	59212	4342114705
35252	WDB	24506	FC	spec-15336-59294-04524328661-24506.png	59294	4524328661

35253 rows × 6 columns

```
In [6]: file_dict = {}

for folder in os.listdir(data_path):
    folder_full_path = os.path.join(data_path, folder)
    if os.path.isdir(folder_full_path):
        for filename in os.listdir(folder_full_path):
            if filename.endswith('.dat'):
                key = filename.split('-')[-2] + '_' + filename.split('_')[-1]
                file_dict[key] = os.path.join(folder, filename)

print(f"Archivos .dat indexados: {len(file_dict)}")
```

Archivos .dat indexados: 33698

```
In [7]: full_model = load_model(model_path)

dummy_input = np.zeros((1, 3750, 1), dtype=np.float32)
_ = full_model.predict(dummy_input, verbose=0)

full_model.summary()
```



```
c:\Users\javip\OneDrive - Universidad Técnica Federico Santa María\6to año\elo308\ML_NPF\.venv\lib\site-packages\keras\src\layers\convolutional\base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
c:\Users\javip\OneDrive - Universidad Técnica Federico Santa María\6to año\elo308\ML_NPF\.venv\lib\site-packages\keras\src\optimizers\base_optimizer.py:86: UserWarning: Argument `decay` is no longer supported and will be ignored.
```

```
warnings.warn(
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_metrics` will be empty until you train or evaluate the model.
WARNING:absl>Error in loading the saved optimizer state. As a result, your model is starting with a freshly initialized optimizer.
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 1874, 128)	640
max_pooling1d (MaxPooling1D)	(None, 937, 128)	0
conv1d_1 (Conv1D)	(None, 467, 64)	32,832
max_pooling1d_1 (MaxPooling1D)	(None, 233, 64)	0
conv1d_2 (Conv1D)	(None, 115, 32)	8,224
max_pooling1d_2 (MaxPooling1D)	(None, 57, 32)	0
conv1d_3 (Conv1D)	(None, 27, 16)	2,064
max_pooling1d_3 (MaxPooling1D)	(None, 13, 16)	0
flatten (Flatten)	(None, 208)	0
dense (Dense)	(None, 128)	26,752
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8,256
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 32)	2,080
dense_3 (Dense)	(None, 12)	396

Total params: 81,246 (317.37 KB)

Trainable params: 81,244 (317.36 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 2 (12.00 B)

Construir extractores de capas

```
In [8]: def build_extractor_until(layer_name, full_model):
        input_layer = layers.Input(shape=(3750, 1))
        x = input_layer

        for layer in full_model.layers:
            x = layer(x)
            if layer.name == layer_name:
                break

        extractor = Model(inputs=input_layer, outputs=x)
        return extractor

        flatten_extractor = build_extractor_until("flatten", full_model)
        dense_extractor = build_extractor_until("dense", full_model)
        dense1_extractor = build_extractor_until("dense_1", full_model)
        dense2_extractor = build_extractor_until("dense_2", full_model)

        print("extrat listos")
```

extrat listos

```
In [9]: def build_class_dataframe(class_name):
        star_df = raw_labels_df[raw_labels_df['Classification'] == class_name].copy()

        spectrum_list = []
        for _, row in star_df.iterrows():
            filekey = str(row['MJD']) + '_' + str(row['Target ID']) + '.dat'
            if filekey in file_dict:
                spectrum_list.append({
                    'filename': file_dict[filekey],
                    'classID': row['Classification']
                })

        spectrum_df = pd.DataFrame(spectrum_list)
        return spectrum_df
```

```
In [10]: # fc que carga espectros de una clase, los preprocesa y los deja listo para entrar
def load_class_spectra(class_name, n_samples=1100, random_state=42):
    spectrum_df = build_class_dataframe(class_name)
    print(f"{class_name}: disponibles con match a .dat = {len(spectrum_df)}")

    if len(spectrum_df) < n_samples:
        raise ValueError(f"No hay suficientes espectros para {class_name}. Disponib

    selected_df = spectrum_df.sample(n=n_samples, random_state=random_state).reset_

    X_list = []
    valid_rows = []

    for _, row in selected_df.iterrows():
        f_path = os.path.join(data_path, row['filename'])
        spec = process_spectrum(f_path)
```

```

        if spec is not None:
            X_list.append(spec)
            valid_rows.append(row)

X_final = np.array(X_list, dtype=np.float32)
X_input = X_final.reshape((X_final.shape[0], X_final.shape[1], 1))
valid_df = pd.DataFrame(valid_rows).reset_index(drop=True)

print(f"{class_name}: espectros válidos finales = {X_input.shape[0]}")
return X_input, valid_df

```

```

In [11]: # fc que carga los espectros y los conserva en formato 2D para poder graficarlos de
# retorna X_final, que son los espectros originales preprocesados, listos para graf
def load_class_spectra_DRACULA(class_name, n_samples=1100, random_state=42):
    spectrum_df = build_class_dataframe(class_name)
    print(f"{class_name}: disponibles con match a .dat = {len(spectrum_df)}")

    if len(spectrum_df) < n_samples:
        raise ValueError(f"No hay suficientes espectros para {class_name}. Disponib

    selected_df = spectrum_df.sample(n=n_samples, random_state=random_state).reset_

    X_list = []
    valid_rows = []

    for _, row in selected_df.iterrows():
        f_path = os.path.join(data_path, row['filename'])
        spec = process_spectrum(f_path)
        if spec is not None:
            X_list.append(spec)
            valid_rows.append(row)

    X_final = np.array(X_list, dtype=np.float32)
    X_input = X_final.reshape((X_final.shape[0], X_final.shape[1], 1))
    valid_df = pd.DataFrame(valid_rows).reset_index(drop=True)

    print(f"{class_name}: espectros válidos finales = {X_input.shape[0]}")
    return X_final, X_input, valid_df

```

```

In [12]: # fc que carga espectros, conserva el flujo (eje y) normalizado y conserva el eje r
def load_class_spectra_DRACULA_with_wavelength(class_name, n_samples=1100, random_s
    spectrum_df = build_class_dataframe(class_name)
    print(f"{class_name}: disponibles con match a .dat = {len(spectrum_df)}")

    if len(spectrum_df) < n_samples:
        raise ValueError(f"No hay suficientes espectros para {class_name}. Disponib

    selected_df = spectrum_df.sample(n=n_samples, random_state=random_state).reset_

    wave_list = []
    flux_list = []
    valid_rows = []

    for _, row in selected_df.iterrows():
        f_path = os.path.join(data_path, row['filename'])

```

```

    wave, flux = process_spectrum_with_wavelength(f_path)

    if wave is not None and flux is not None:
        wave_list.append(wave)
        flux_list.append(flux)
        valid_rows.append(row)

W_final = np.array(wave_list, dtype=np.float32)
X_final = np.array(flux_list, dtype=np.float32)

X_input = X_final.reshape((X_final.shape[0], X_final.shape[1], 1))
valid_df = pd.DataFrame(valid_rows).reset_index(drop=True)

print(f"{class_name}: espectros válidos finales = {X_input.shape[0]}")
print(f"{class_name}: shape wavelength = {W_final.shape}")
print(f"{class_name}: shape flux = {X_final.shape}")

return W_final, X_final, X_input, valid_df

'''
Uso de esta fc -> mostrar un espectro / muestra
W_wda, X_wda_spec, X_wda_input, df_wda_DRAC = load_class_spectra_DRACULA_with_wavel
    "WDA",
    n_samples=1100,
    random_state=42
)

sample_idx = 0

plt.figure(figsize=(10, 5))
plt.plot(W_wda[sample_idx], X_wda_spec[sample_idx])
plt.xlabel("Wavelength [Å]")
plt.ylabel("Normalized flux")
plt.title(f"WDA - sample {sample_idx}")
plt.tight_layout()
plt.show()
'''

'''
Uso de esta fc -> mostrar el promedio + lo anterior
mean_wave = np.nanmean(W_wda, axis=0)
mean_flux = np.nanmean(X_wda_spec, axis=0)

plt.figure(figsize=(10, 5))
plt.plot(mean_wave, mean_flux, label="Total mean spectrum")
plt.xlabel("Wavelength [Å]")
plt.ylabel("Normalized flux")
plt.title("WDA - Total mean spectrum")
plt.legend()
plt.tight_layout()
plt.show()
'''

```

```
Out[12]: '\nUsó de esta fc -> mostrar el promedio + lo anterior\nmean_wave = np.nanmean(W_w
da, axis=0)\nmean_flux = np.nanmean(X_wda_spec, axis=0)\n\nplt.figure(figsize=(10,
5))\nplt.plot(mean_wave, mean_flux, label="Total mean spectrum")\nplt.xlabel("Wave
length [Å])\nplt.ylabel("Normalized flux")\nplt.title("WDA - Total mean spectru
m")\nplt.legend()\nplt.tight_layout()\nplt.show()\n'
```

```
In [13]: def extract_layer_features(X_input): # fc que extrae representaciones internas de L
flatten_features = flatten_extractor.predict(X_input, batch_size=64, verbose=1)
dense_features = dense_extractor.predict(X_input, batch_size=64, verbose=1)
dense1_features = dense1_extractor.predict(X_input, batch_size=64, verbose=1)
dense2_features = dense2_extractor.predict(X_input, batch_size=64, verbose=1)

print("flatten:", flatten_features.shape)
print("dense:", dense_features.shape)
print("dense_1:", dense1_features.shape)
print("dense_2:", dense2_features.shape)

return flatten_features, dense_features, dense1_features, dense2_features
```

```
In [14]: def reduce_all_to_32(flatten_features, dense_features, dense1_features, dense2_feat
flatten_32 = PCA(n_components=32, random_state=42).fit_transform(flatten_features)
dense_32 = PCA(n_components=32, random_state=42).fit_transform(dense_features)
dense1_32 = PCA(n_components=32, random_state=42).fit_transform(dense1_features)
dense2_32 = dense2_features.copy() # ya está en 32

print("flatten_32:", flatten_32.shape)
print("dense_32:", dense_32.shape)
print("dense1_32:", dense1_32.shape)
print("dense2_32:", dense2_32.shape)

return {
    "flatten": flatten_32,
    "dense": dense_32,
    "dense_1": dense1_32,
    "dense_2": dense2_32
}
```

Funciones para graficar

```
In [15]: # fc que grafica los features_nd en 2D
def plot_internal_structure_2d(
    features_nd,
    class_name,
    layer_name,
    method="pca",
    xlim=None,
    ylim=None,
    kpca_kernel="rbf",
    kpca_gamma=None,
    kpca_degree=3,
    kpca_coef0=1,
    scale_before_pca=False,
    scale_before_tsne=False,
    scale_before_kpca=True
```

```

):
    method = method.lower()

    if method == "pca":
        X_plotPCA = features_nd.copy()

        if scale_before_pca:
            X_plotPCA = StandardScaler().fit_transform(X_plotPCA)

        emb_2d = PCA(n_components=2, random_state=42).fit_transform(X_plotPCA)

    elif method == "tsne":
        X_plotTSNE = features_nd.copy()

        if scale_before_tsne:
            X_plotTSNE = StandardScaler().fit_transform(X_plotTSNE)

        emb_2d = TSNE(
            n_components=2,
            random_state=42,
            perplexity=30,
            init='pca'
        ).fit_transform(X_plotTSNE)

    elif method == "kpca":
        X_kpca = features_nd.copy()

        if scale_before_kpca:
            X_kpca = StandardScaler().fit_transform(X_kpca)

        emb_2d = KernelPCA(
            n_components=2,
            kernel=kpca_kernel,
            gamma=kpca_gamma,
            degree=kpca_degree,
            coef0=kpca_coef0
        ).fit_transform(X_kpca)

    else:
        raise ValueError("method debe ser 'pca', 'tsne' o 'kpca'")

    plt.figure(figsize=(6, 5))
    plt.scatter(emb_2d[:, 0], emb_2d[:, 1], s=10, alpha=0.65)

    if method == "kpca":
        extra = " + scaled" if scale_before_kpca else ""
        n_samples = features_nd.shape[0]
        plt.title(f"{class_name} - {layer_name} - estructura interna (KPCA-{kpca_ke

    elif method == "pca":
        extra = " + scaled" if scale_before_pca else ""
        n_samples = features_nd.shape[0]
        plt.title(f"{class_name} - {layer_name} - estructura interna (PCA{extra}, n

    elif method == "tsne":
        extra = " + scaled" if scale_before_tsne else ""

```

```

n_samples = features_nd.shape[0]
plt.title(f"{class_name} - {layer_name} - estructura interna (TSNE{extra},

plt.xlabel("Comp. 1")
plt.ylabel("Comp. 2")

if xlim is not None:
    plt.xlim(xlim)
if ylim is not None:
    plt.ylim(ylim)

plt.tight_layout()
plt.show()

```

```

In [16]: # funcion para graficar el espectro promedio total
def plot_total_mean_spectrum(
    spectra_2d,
    title="Total mean spectrum",
    wavelength=None,
    figsize=(10, 5),
    save_path=None
):
    """
    Grafica el espectro promedio total de todas las muestras.

    Parameters
    -----
    spectra_2d : np.ndarray
        Matriz shape (n_samples, n_wavelengths)
    title : str
        Título del gráfico
    wavelength : np.ndarray or None
        Eje x. Si None, usa índice espectral
    figsize : tuple
    save_path : str or None
        Si se entrega, guarda la figura
    """

    spectra_2d = np.asarray(spectra_2d)

    if spectra_2d.ndim != 2:
        raise ValueError(f"spectra_2d debe ser 2D. Shape actual: {spectra_2d.shape}")

    mean_spec = spectra_2d.mean(axis=0)

    if wavelength is None:
        wavelength = np.arange(spectra_2d.shape[1])

    plt.figure(figsize=figsize)
    plt.plot(wavelength, mean_spec, label="Total mean spectrum")
    plt.xlabel("Spectral index")
    plt.ylabel("Normalized flux")
    plt.title(title)
    plt.legend()
    plt.tight_layout()

```

```

if save_path is not None:
    plt.savefig(save_path, dpi=300, bbox_inches="tight")

plt.show()

```

```

In [17]: # función para graficar una muestra individual
def plot_single_spectrum(
    spectra_2d,
    sample_idx,
    valid_df=None,
    title=None,
    wavelength=None,
    figsize=(10, 5),
    save_path=None
):
    """
    Grafica un espectro individual a partir de su índice.

    Parameters
    -----
    spectra_2d : np.ndarray
        Matriz shape (n_samples, n_wavelengths)
    sample_idx : int
        Índice de la muestra a graficar
    valid_df : pd.DataFrame or None
        Metadata opcional para mostrar info adicional en el título
    title : str or None
        Si None, construye uno automático
    wavelength : np.ndarray or None
        Eje x. Si None, usa índice espectral
    figsize : tuple
    save_path : str or None
        Si se entrega, guarda la figura
    """

    spectra_2d = np.asarray(spectra_2d)

    if spectra_2d.ndim != 2:
        raise ValueError(f"spectra_2d debe ser 2D. Shape actual: {spectra_2d.shape}")

    if sample_idx < 0 or sample_idx >= spectra_2d.shape[0]:
        raise IndexError(f"sample_idx fuera de rango. Debe estar entre 0 y {spectra_2d.shape[0]}")

    spectrum = spectra_2d[sample_idx]

    if wavelength is None:
        wavelength = np.arange(spectra_2d.shape[1])

    if title is None:
        title = f"Single spectrum - sample {sample_idx}"

        if valid_df is not None and sample_idx < len(valid_df):
            row = valid_df.iloc[sample_idx]

            extra_parts = []
            if "filename" in row.index:

```



```

        extra_parts.append(f"file: {row['filename']}")
    if "classID" in row.index:
        extra_parts.append(f"class: {row['classID']}")

    if len(extra_parts) > 0:
        title += "\n" + " | ".join(extra_parts)

plt.figure(figsize=figsize)
plt.plot(wavelength, spectrum, label=f"Sample {sample_idx}")
plt.xlabel("Spectral index")
plt.ylabel("Normalized flux")
plt.title(title)
plt.legend()
plt.tight_layout()

if save_path is not None:
    plt.savefig(save_path, dpi=300, bbox_inches="tight")

plt.show()

```

```

In [18]: def plot_multiple_spectra(
    spectra_2d,
    sample_indices,
    valid_df=None,
    title="Selected spectra",
    wavelength=None,
    figsize=(10, 5),
    alpha=0.9,
    save_path=None
):
    """
    Grafica varias muestras individuales en la misma figura.

    Parameters
    -----
    spectra_2d : np.ndarray
        Matriz shape (n_samples, n_wavelengths)
    sample_indices : list or array-like
        Índices de las muestras a graficar
    valid_df : pd.DataFrame or None
    title : str
    wavelength : np.ndarray or None
    figsize : tuple
    alpha : float
    save_path : str or None
    """

    spectra_2d = np.asarray(spectra_2d)

    if spectra_2d.ndim != 2:
        raise ValueError(f"spectra_2d debe ser 2D. Shape actual: {spectra_2d.shape}")

    if wavelength is None:
        wavelength = np.arange(spectra_2d.shape[1])

    plt.figure(figsize=figsize)

```

```

for idx in sample_indices:
    if idx < 0 or idx >= spectra_2d.shape[0]:
        raise IndexError(f"sample_idx {idx} fuera de rango.")

    label = f"sample {idx}"

    if valid_df is not None and idx < len(valid_df):
        row = valid_df.iloc[idx]
        if "filename" in row.index:
            label += f" | {row['filename']}"

    plt.plot(wavelength, spectra_2d[idx], alpha=alpha, label=label)

plt.xlabel("Spectral index")
plt.ylabel("Normalized flux")
plt.title(title)
plt.legend(fontsize=8)
plt.tight_layout()

if save_path is not None:
    plt.savefig(save_path, dpi=300, bbox_inches="tight")

plt.show()

```

Espectro representativo de clase — diagnóstico de normalización

```

In [19]: def plot_class_mean_spectrum(
    spectra_2d, wavelength, class_name,
    use_median=True,
    show_individuals=False,
    n_individuals=10,
    percentile_band=(10, 90),
    flux_clip=(-0.5, 5.0),
    figsize=(13, 5)
):
    """
    Muestra el espectro promedio (o mediana) de una clase completa.

    Parametros
    -----
    spectra_2d      : array (n x 3750) con los espectros normalizados
    wavelength      : array de longitudes de onda
    class_name      : str, solo para el titulo
    use_median      : True -> usa mediana (mas robusta a outliers)
                    False -> usa media
    show_individuals : si True, grafica n_individuals espectros individuales
    n_individuals    : cuantos espectros individuales mostrar
    percentile_band  : tupla (p_low, p_high) para la banda de dispersion
    flux_clip        : (min, max) para detectar espectros anomalos.
                    None = no filtrar
    """
    spectra_2d = np.asarray(spectra_2d, dtype=np.float32)
    wavelength = np.asarray(wavelength)

```

```

n_total      = spectra_2d.shape[0]

# — Diagnostico de espectros anomalos —————
flux_min = spectra_2d.min(axis=1)  # minimo de cada espectro
flux_max = spectra_2d.max(axis=1)  # maximo de cada espectro

print(f'\n=== Diagnostico {class_name} (n={n_total}) ===')
print(f'  Flux minimo por espectro:  media={flux_min.mean():.3f}  '
      f'p5={np.percentile(flux_min,5):.3f}  '
      f'p95={np.percentile(flux_min,95):.3f}')
print(f'  Flux maximo por espectro:  media={flux_max.mean():.3f}  '
      f'p5={np.percentile(flux_max,5):.3f}  '
      f'p95={np.percentile(flux_max,95):.3f}')

if flux_clip is not None:
    clip_lo, clip_hi = flux_clip
    bad_mask = (flux_min < clip_lo) | (flux_max > clip_hi)
    n_bad     = bad_mask.sum()
    pct_bad   = 100 * n_bad / n_total
    print(f'  Espectros fuera de [{clip_lo}, {clip_hi}]: '
          f'{n_bad} ({pct_bad:.1f}%)')
    spectra_clean = spectra_2d[~bad_mask]
    label_clean   = f'n={spectra_clean.shape[0]} (sin anomalos)'
else:
    spectra_clean = spectra_2d
    label_clean   = f'n={n_total}'

# — Calcular curvas representativas —————
if use_median:
    central = np.median(spectra_clean, axis=0)
    central_label = f'Mediana {class_name} ({label_clean})'
else:
    central = np.mean(spectra_clean, axis=0)
    central_label = f'Media {class_name} ({label_clean})'

p_lo = np.percentile(spectra_clean, percentile_band[0], axis=0)
p_hi = np.percentile(spectra_clean, percentile_band[1], axis=0)

# tambien con todos (sin filtrar) para comparar
if flux_clip is not None:
    central_all = (np.median(spectra_2d, axis=0) if use_median
                  else np.mean(spectra_2d, axis=0))
else:
    central_all = None

# — Grafico —————
fig, ax = plt.subplots(figsize=figsize)

# Espectros individuales (opcional)
if show_individuals:
    idx_sample = np.random.choice(spectra_clean.shape[0],
                                  size=min(n_individuals, spectra_clean.shape[0]
                                             replace=False))

    for i in idx_sample:
        ax.plot(wavelength, spectra_clean[i], color='steelblue',
                linewidth=0.4, alpha=0.3)

```

```

# Banda de percentiles
ax.fill_between(wavelength, p_lo, p_hi,
                alpha=0.15, color='steelblue',
                label=f'P{percentile_band[0]}-P{percentile_band[1]}')

# Curva representativa SIN filtrar (si se filtro)
if central_all is not None:
    ax.plot(wavelength, central_all, linewidth=1.2, color='gray',
            linestyle='--', alpha=0.6, label='Todos (con anomalos)')

# Curva representativa Limpia
ax.plot(wavelength, central, linewidth=2.2, color='steelblue',
        label=central_label)

# Linea en flux=1 (punto de normalizacion)
ax.axhline(1.0, linestyle=':', linewidth=1.0, color='black',
           alpha=0.5, label='flux = 1 (punto normaliz. ~6158 A)')

add_balmer_lines(ax)

ax.set_xlabel('Wavelength [A]')
ax.set_ylabel('Normalized flux (norm. en ~6158 A)')
ax.set_title(f'Espectro representativo de clase {class_name}')
ax.legend(fontsize=9)
plt.tight_layout()
plt.show()

return {
    'central'      : central,
    'p_lo'         : p_lo,
    'p_hi'         : p_hi,
    'spectra_clean' : spectra_clean,
    'n_bad'        : int(bad_mask.sum()) if flux_clip is not None else 0
}

```

Función que agrega líneas de Balmer (mientras no tengo el paquete)

```

In [20]: def add_balmer_lines(ax, y_text=None, alpha=0.6, fontsize=9):
        """
        Agrega líneas verticales de Balmer al eje.
        """
        balmer_lines = {
            "Hε": 3970.1,
            "Hδ": 4101.7,
            "Hγ": 4340.5,
            "Hβ": 4861.3,
            "Hα": 6562.8
        }

        ymin, ymax = ax.get_ylim()

        if y_text is None:
            y_text = ymax - 0.03 * (ymax - ymin)

```

```

for name, lam in balmer_lines.items():
    ax.axvline(lam, linestyle="--", alpha=alpha, linewidth=1.0)
    ax.text(
        lam,
        y_text,
        name,
        rotation=90,
        va="top",
        ha="right",
        fontsize=fontsize
    )

```

Función que genera archivos de salida

```

In [21]: # =====
# FUNCIÓN PRINCIPAL PARA GENERAR OUTPUTS ESPECTRALES -> ESPECTROS POR CLUSTER
# =====
def generate_cluster_spectra_outputs(
    spectra_2d,          # shape (n_samples, n_wavelengths)
    cluster_labels,      # shape (n_samples,)
    valid_df,            # metadata alineada con spectra_2d
    class_name,
    layer_name,
    method_name,
    output_dir="dracula_outputs",
    ignore_noise=False,
    wavelength=None
):
    """
    Genera salidas para análisis espectral por cluster.

    Guarda:
    - sample_cluster_map.csv
    - cluster_summary.csv
    - spectra.dat general
    - clusters.dat general
    - wavelength.dat
    - spectra individuales por cluster
    - mean por cluster
    - std por cluster

    Parameters
    -----
    spectra_2d : np.ndarray
        Matriz de espectros originales, shape (n_samples, n_wavelengths)
    cluster_labels : np.ndarray
        Etiquetas de clustering, shape (n_samples,)
    valid_df : pd.DataFrame
        Metadata alineada con spectra_2d
    class_name : str
    layer_name : str
    method_name : str
    output_dir : str
    ignore_noise : bool
    """

```

```

        Si True, elimina cluster -1 (útil para DBSCAN)
wavelength : np.ndarray or None
        Vector de longitudes de onda. Si None, usa 0..n_wavelengths-1

Returns
-----
dict con rutas y dataframes útiles
"""

spectra_2d = np.asarray(spectra_2d)
cluster_labels = np.asarray(cluster_labels)

# -----
# Validaciones
# -----
if spectra_2d.ndim != 2:
    raise ValueError(f"spectra_2d debe ser 2D. Shape actual: {spectra_2d.shape}")

if len(cluster_labels) != spectra_2d.shape[0]:
    raise ValueError(
        f"cluster_labels y spectra_2d no coinciden: "
        f"{len(cluster_labels)} vs {spectra_2d.shape[0]}"
    )

if len(valid_df) != spectra_2d.shape[0]:
    raise ValueError(
        f"valid_df y spectra_2d no coinciden: "
        f"{len(valid_df)} vs {spectra_2d.shape[0]}"
    )

if wavelength is None:
    wavelength = np.arange(spectra_2d.shape[1], dtype=np.float32)

# -----
# Crear carpeta base
# -----
base_name = f"{class_name}_{layer_name}_{method_name}"
base_dir = os.path.join(output_dir, class_name, layer_name, method_name)
os.makedirs(base_dir, exist_ok=True)

# -----
# Crear dataframe muestra -> cluster
# -----
cluster_df = valid_df.copy().reset_index(drop=True)
cluster_df["sample_idx"] = np.arange(len(cluster_df))
cluster_df["cluster"] = cluster_labels

# -----
# Manejo de ruido DBSCAN (-1)
# -----
if ignore_noise:
    mask = cluster_df["cluster"] != -1
    cluster_df = cluster_df.loc[mask].reset_index(drop=True)
    spectra_used = spectra_2d[mask.to_numpy()]
    labels_used = cluster_df["cluster"].to_numpy()
else:

```

```

spectra_used = spectra_2d
labels_used = cluster_labels

unique_clusters = np.unique(labels_used)

# -----
# Guardar mapa muestra -> cluster
# -----
cluster_map_csv = os.path.join(base_dir, f"{base_name}_sample_cluster_map.csv")
cluster_df.to_csv(cluster_map_csv, index=False)

# -----
# Guardar archivos generales
# -----
spectra_dat = os.path.join(base_dir, f"{base_name}_spectra.dat")
labels_dat = os.path.join(base_dir, f"{base_name}_clusters.dat")
wavelength_dat = os.path.join(base_dir, f"{base_name}_wavelength.dat")

np.savetxt(spectra_dat, spectra_used, fmt="%.8e")
np.savetxt(labels_dat, labels_used.astype(int), fmt="%d")
np.savetxt(wavelength_dat, wavelength, fmt="%.8e")

# -----
# Guardar resumen y archivos por cluster
# -----
summary_rows = []

for c in unique_clusters:
    mask_c = labels_used == c
    cluster_spectra = spectra_used[mask_c]

    mean_spec = cluster_spectra.mean(axis=0)
    std_spec = cluster_spectra.std(axis=0)

    n_cluster = int(mask_c.sum())

    summary_rows.append({
        "cluster": int(c),
        "n_samples": n_cluster
    })

# metadata del cluster
cluster_meta = cluster_df.loc[cluster_df["cluster"] == c].copy()
cluster_meta_csv = os.path.join(base_dir, f"{base_name}_cluster_{c}_metadat")
cluster_meta.to_csv(cluster_meta_csv, index=False)

# todos los espectros del cluster
cluster_specs_path = os.path.join(base_dir, f"{base_name}_cluster_{c}_spect")
np.savetxt(cluster_specs_path, cluster_spectra, fmt="%.8e")

# media del cluster
mean_path = os.path.join(base_dir, f"{base_name}_cluster_{c}_mean.dat")
np.savetxt(mean_path, mean_spec, fmt="%.8e")

# std del cluster
std_path = os.path.join(base_dir, f"{base_name}_cluster_{c}_std.dat")

```

```

np.savetxt(std_path, std_spec, fmt="%.8e")

summary_df = pd.DataFrame(summary_rows).sort_values("cluster").reset_index(drop=True)
summary_csv = os.path.join(base_dir, f"{base_name}_cluster_summary.csv")
summary_df.to_csv(summary_csv, index=False)

return {
    "base_dir": base_dir,
    "cluster_map_csv": cluster_map_csv,
    "spectra_dat": spectra_dat,
    "labels_dat": labels_dat,
    "wavelength_dat": wavelength_dat,
    "summary_csv": summary_csv,
    "summary_df": summary_df,
    "cluster_df": cluster_df
}

```

```

In [22]: # fc que automatiza la generación de archivos de salida para todos los experimentos
def generate_all_outputs_for_class(
    spectra_2d,
    valid_df,
    class_name,
    labels_dict_for_class,
    output_dir="dracula_outputs",
    ignore_noise_for_dbscan=False,
    wavelength=None
):
    """
    Recorre todas las capas y métodos de una clase y genera outputs espectrales.

    Parameters
    -----
    spectra_2d : np.ndarray
        Espectros originales shape (n_samples, n_wavelengths)
    valid_df : pd.DataFrame
    class_name : str
    labels_dict_for_class : dict
        Ejemplo:
        cluster_labels_all["WDA"]
    output_dir : str
    ignore_noise_for_dbscan : bool
    wavelength : np.ndarray or None

    Returns
    -----
    dict
    """

    all_outputs = {}

    for layer_name, methods_dict in labels_dict_for_class.items():
        all_outputs[layer_name] = {}

        for method_name, labels in methods_dict.items():
            ignore_noise = (method_name == "dbscan" and ignore_noise_for_dbscan)

```



```

        result = generate_cluster_spectra_outputs(
            spectra_2d=spectra_2d,
            cluster_labels=labels,
            valid_df=valid_df,
            class_name=class_name,
            layer_name=layer_name,
            method_name=method_name,
            output_dir=output_dir,
            ignore_noise=ignore_noise,
            wavelength=wavelength
        )

        all_outputs[layer_name][method_name] = result

    return all_outputs

```

Contenedores -> diccionarios anidados que contienen array de enteros donde cada posición es una muestra y el valor es el cluster al que fue asignado.

Tiene más o menos esta forma: cluster_labels_all -> "WDA" -> "flatten" -> "spectral" -> np.array([0,1,2,0,2,1,...]) shape (n_muestras,)

```

In [23]: cluster_labels_all = {
        "WD": {}
    }

```

```

In [24]: #guarda labels dentro del diccionario general cluster_labels_all.
def save_cluster_labels(container, class_name, layer_name, method_name, labels):
    """
    estructura final:
    cluster_labels_all["WDA"]["dense_1"]["spectral"] = labels
    """

    if class_name not in container:
        container[class_name] = {}

    if layer_name not in container[class_name]:
        container[class_name][layer_name] = {}

    container[class_name][layer_name][method_name] = np.asarray(labels).copy()

```

Funciones que consideran las longitudes de onda

```

In [25]: def process_spectrum_with_wavelength(file_path):
        try:
            sed = np.loadtxt(file_path, unpack=True)

            wave = sed[0, 48:-400]
            flux = sed[1, 48:-400]

            # Ajuste a 3750 puntos, como espera la red

```

```

if len(flux) > 3750:
    diff = len(flux) - 3750
    start = diff // 2

    flux = flux[start:start + 3750]
    wave = wave[start:start + 3750]

elif len(flux) < 3750:
    pad_len = 3750 - len(flux)

    flux = np.pad(flux, (0, pad_len), mode='constant')

    # Para wavelength, mejor evitar inventar demasiado.
    # Extendemos linealmente usando el paso medio.
    if len(wave) > 1:
        dw = np.nanmedian(np.diff(wave))
        extra_wave = wave[-1] + dw * np.arange(1, pad_len + 1)
        wave = np.concatenate([wave, extra_wave])
    else:
        wave = np.pad(wave, (0, pad_len), mode='constant')

norm_val = flux[2100]

if norm_val == 0 or np.isnan(norm_val):
    norm_val = np.nanmean(flux)

if norm_val == 0 or np.isnan(norm_val):
    return None, None

flux = flux / norm_val

if np.isnan(flux).any() or np.isnan(wave).any():
    return None, None

return wave.astype(np.float32), flux.astype(np.float32)

except Exception:
    return None, None

```

```

In [26]: def load_class_spectra_DRACULA_with_wavelength(class_name, n_samples=1100, random_s
spectrum_df = build_class_dataframe(class_name)
print(f"{class_name}: disponibles con match a .dat = {len(spectrum_df)}")

if len(spectrum_df) < n_samples:
    raise ValueError(f"No hay suficientes espectros para {class_name}. Disponib

selected_df = spectrum_df.sample(n=n_samples, random_state=random_state).reset_

wave_list = []
flux_list = []
valid_rows = []

for _, row in selected_df.iterrows():
    f_path = os.path.join(data_path, row['filename'])

    wave, flux = process_spectrum_with_wavelength(f_path)

```

```

        if wave is not None and flux is not None:
            wave_list.append(wave)
            flux_list.append(flux)
            valid_rows.append(row)

W_final = np.array(wave_list, dtype=np.float32)
X_final = np.array(flux_list, dtype=np.float32)

X_input = X_final.reshape((X_final.shape[0], X_final.shape[1], 1))
valid_df = pd.DataFrame(valid_rows).reset_index(drop=True)

print(f"{class_name}: espectros válidos finales = {X_input.shape[0]}")
print(f"{class_name}: shape wavelength = {W_final.shape}")
print(f"{class_name}: shape flux = {X_final.shape}")

return W_final, X_final, X_input, valid_df

```

Función para evaluación de K con Silhouette Score

```

In [27]: def evaluate_kmeans_k(features_32, class_name, layer_name, k_values=range(2, 9)):
        Xs = StandardScaler().fit_transform(features_32)

        #inertias = []
        silhouettes = []

        for k in k_values:
            km = KMeans(n_clusters=k, random_state=42, n_init=10)
            labels = km.fit_predict(Xs)

            #inertias.append(km.inertia_)
            silhouettes.append(silhouette_score(Xs, labels))

        plt.figure(figsize=(12,4))

        ...
        plt.subplot(1,2,1)
        plt.plot(list(k_values), inertias, marker='o')
        plt.title(f"{class_name} - {layer_name} - Elbow")
        plt.xlabel("k")
        plt.ylabel("Inercia")
        ...

        #plt.subplot(1,2,2)
        n_samples = features_32.shape[0]
        plt.plot(list(k_values), silhouettes, marker='o')
        plt.title(f"{class_name} - {layer_name} - Silhouette(n={n_samples})")
        #plt.legend([f"n={n_samples}"])
        plt.xlabel("k")
        plt.ylabel("Silhouette")

        plt.tight_layout()
        plt.show()

```

Función para hacer matrices (de contingencia) y analizar cambios de k para Spectral Clustering

```
In [28]: def plot_migration_matrix(labels_a, labels_b, ka, kb, title=None, figsize=(6, 5)):
cm = contingency_matrix(labels_a, labels_b)

rows_labels = [f'C{i}' for i in range(cm.shape[0])]
cols_labels = [f'C{j}' for j in range(cm.shape[1])]

fig, ax = plt.subplots(figsize=figsize)
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=cols_labels,
    yticklabels=rows_labels,
    ax=ax
)
ax.set_xlabel(f'Clústeres k={kb}', fontsize=11)
ax.set_ylabel(f'Clústeres k={ka}', fontsize=11)
_title = title if title else f'Migración k={ka} → k={kb} (Flatten, WDA)'
ax.set_title(_title, fontsize=12, fontweight='bold')
plt.tight_layout()
plt.show()
```

Visualización de un espectro con la longitud de onda correcta

```
In [29]: W_wd, X_wd_spec, X_wd_input, df_wd_DRAC = load_class_spectra_DRACULA_with_wavelength
"WD",
n_samples=700,
random_state=42
)

sample_idx = 3

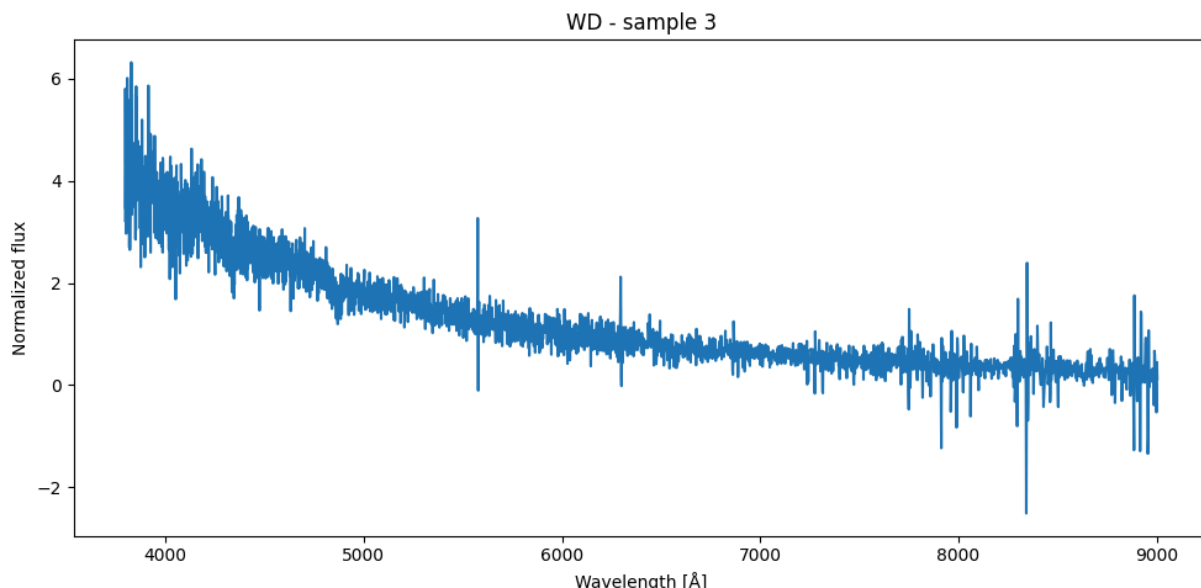
plt.figure(figsize=(10, 5))
plt.plot(W_wd[sample_idx], X_wd_spec[sample_idx])
plt.xlabel("Wavelength [Å]")
plt.ylabel("Normalized flux")
plt.title(f"WD - sample {sample_idx}")
plt.tight_layout()
plt.show()
```

WD: disponibles con match a .dat = 736

WD: espectros válidos finales = 629

WD: shape wavelength = (629, 3750)

WD: shape flux = (629, 3750)



Funciones que hacen cluster y plot según método, y generan espectros a partir de los clusters encontrados en cada método. Se implementa interact

```
In [30]: def _add_cluster_legend(ax, labels, cmap=plt.cm.viridis):
    unique_labels = np.unique(labels)
    label_min, label_max = unique_labels.min(), unique_labels.max()
    handles = []
    for lab in unique_labels:
        count = int(np.sum(labels == lab))
        if lab == -1:
            color, text = "black", f"ruido (-1): n={count}"
        else:
            norm = 0.5 if label_max == label_min else (lab - label_min) / (label_max - label_min)
            color, text = cmap(norm), f"cluster {lab}: n={count}"
        handles.append(Line2D([0], [0], marker='o', color='w',
                               markerfacecolor=color, markersize=8, label=text))
    ax.legend(handles=handles, title="Clusters", loc="best", fontsize=9)
```

```
In [31]: def _print_metrics_box(class_name, layer_name, method, param_str, metrics, dbcv=None):
    k = metrics['n_clusters']
    noise = metrics['n_noise']
    sil = f"{metrics['silhouette']:.4f}"
    db = f"{metrics['davies_bouldin']:.4f}"
    ch = f"{metrics['calinski_harabasz']:.2f}"
    sep = chr(9472) * 64
    print(f'\n \u250c{sep}\u2510')
    print(f' \u2502 {class_name} | {layer_name} | {method} | {param_str} ')
    print(f' \u2502 clusters={k} noise={noise} sil={sil} DB={db} CH={ch} ')
    if dbcv is not None:
        dbcv_str = f'{dbcv:.4f}' if dbcv == dbcv else 'N/A'
        print(f' \u2502 DBCV = {dbcv_str} (rango [-1,1], específico DBSCAN) ')
    print(f' \u2514{sep}\u2518\n')
```

```

def cluster_plot_KMeans(features_32, class_name, layer_name,
                        kmeans_k=3, random_state=42,
                        vis_method="pca", xlim=None, ylim=None,
                        container=None):

    Xs = StandardScaler().fit_transform(features_32)

    if vis_method.lower() == "pca":
        emb_2d = PCA(n_components=2, random_state=42).fit_transform(Xs)
    elif vis_method.lower() == "tsne":
        emb_2d = TSNE(n_components=2, random_state=42,
                      perplexity=30, init='pca').fit_transform(Xs)
    else:
        raise ValueError("vis_method debe ser 'pca' o 'tsne'")

    km_labels = KMeans(n_clusters=kmeans_k, random_state=random_state,
                      n_init=10).fit_predict(Xs)

    if container is not None:
        save_cluster_labels(container, class_name, layer_name, "kmeans", km_labels)

    fig, ax = plt.subplots(figsize=(9, 6))
    ax.scatter(emb_2d[:, 0], emb_2d[:, 1], c=km_labels, s=30, alpha=0.6, cmap='viridis')
    ax.set_title(f"{class_name} \u2014 {layer_name} \u2014 KMeans "
                f"(k={kmeans_k}, n={features_32.shape[0]})", fontsize=15)
    _add_cluster_legend(ax, km_labels)
    ax.set_xlabel("Comp. 1"); ax.set_ylabel("Comp. 2")
    if xlim: ax.set_xlim(xlim)
    if ylim: ax.set_ylim(ylim)
    plt.tight_layout(); plt.show()

    metrics = compute_cluster_metrics(Xs, km_labels)
    _print_metrics_box(class_name, layer_name, "kmeans", f"k={kmeans_k}", metrics)
    return {"kmeans": km_labels, "metrics": metrics}

def cluster_plot_Spect(features_32, class_name, layer_name,
                        spectral_k=3, random_state=42,
                        vis_method="pca", xlim=None, ylim=None,
                        container=None):

    Xs = StandardScaler().fit_transform(features_32)

    if vis_method.lower() == "pca":
        emb_2d = PCA(n_components=2, random_state=42).fit_transform(Xs)
    elif vis_method.lower() == "tsne":
        emb_2d = TSNE(n_components=2, random_state=42,
                      perplexity=30, init='pca').fit_transform(Xs)
    else:
        raise ValueError("vis_method debe ser 'pca' o 'tsne'")

    sc_labels = SpectralClustering(n_clusters=spectral_k, random_state=random_state,
                                  affinity='nearest_neighbors').fit_predict(Xs)

    if container is not None:
        save_cluster_labels(container, class_name, layer_name, "spectral", sc_labels)

```

```

fig, ax = plt.subplots(figsize=(9, 6))
ax.scatter(emb_2d[:, 0], emb_2d[:, 1], c=sc_labels, s=30, alpha=0.6, cmap='viridis')
ax.set_title(f"{class_name} \u2014 {layer_name} \u2014 Spectral "
             f"(k={spectral_k}, n={features_32.shape[0]})", fontsize=15)
_add_cluster_legend(ax, sc_labels)
ax.set_xlabel("Comp. 1"); ax.set_ylabel("Comp. 2")
if xlim: ax.set_xlim(xlim)
if ylim: ax.set_ylim(ylim)
plt.tight_layout(); plt.show()

metrics = compute_cluster_metrics(Xs, sc_labels)
_print_metrics_box(class_name, layer_name, "spectral", f"k={spectral_k}", metrics)
return {"spectral": sc_labels, "metrics": metrics}

def cluster_plot_DBSCAN(features_32, class_name, layer_name,
                        dbscan_eps=1.5, dbscan_min_samples=10,
                        vis_method="pca", xlim=None, ylim=None,
                        container=None):

    Xs = StandardScaler().fit_transform(features_32)

    if vis_method.lower() == "pca":
        emb_2d = PCA(n_components=2, random_state=42).fit_transform(Xs)
    elif vis_method.lower() == "tsne":
        emb_2d = TSNE(n_components=2, random_state=42,
                      perplexity=30, init='pca').fit_transform(Xs)
    else:
        raise ValueError("vis_method debe ser 'pca' o 'tsne'")

    db_labels = DBSCAN(eps=dbscan_eps,
                      min_samples=dbscan_min_samples).fit_predict(Xs)

    if container is not None:
        save_cluster_labels(container, class_name, layer_name, "dbscan", db_labels)

    fig, ax = plt.subplots(figsize=(9, 6))
    ax.scatter(emb_2d[:, 0], emb_2d[:, 1], c=db_labels, s=30, alpha=0.3, cmap='viridis')
    ax.set_title(f"{class_name} \u2014 {layer_name} \u2014 DBSCAN "
                 f"(eps={dbscan_eps}, n={features_32.shape[0]})", fontsize=15)
    _add_cluster_legend(ax, db_labels)
    ax.set_xlabel("Comp. 1"); ax.set_ylabel("Comp. 2")
    if xlim: ax.set_xlim(xlim)
    if ylim: ax.set_ylim(ylim)
    plt.tight_layout(); plt.show()

    metrics = compute_cluster_metrics(Xs, db_labels)
    n_pts = int(np.sum(db_labels != -1))
    dbcv = dbcv_score(Xs, db_labels) if n_pts <= 5000 else None
    if n_pts > 5000:
        print(f" [DBCV omitido: {n_pts} puntos no-ruido > 5000] ")
    _print_metrics_box(class_name, layer_name, "dbscan", f"eps={dbscan_eps}", metrics)
    return {"dbscan": db_labels, "metrics": metrics, "dbcv": dbcv}

```

```

In [32]: def _plot_cluster_spectra_base(
    spectra_2d, labels, wavelength,
    class_name, layer_name, method_name,
    linewidth_mean=2.4,
    figsize=(11, 5),
    mark_balmer=True,
    robust_ylim=True,
    ignore_noise=False,
    diff_percentiles=(1, 99),
    show_std=False,
    smooth_sigma=3          # suavizado Gaussian para el plot de diferencia (pix
):
    spectra_2d = np.asarray(spectra_2d)
    labels      = np.asarray(labels)
    wavelength  = np.asarray(wavelength)

    if ignore_noise:
        mask      = labels != -1
        spectra_2d = spectra_2d[mask]
        labels     = labels[mask]

    unique_clusters = np.sort(np.unique(labels))
    total_mean      = np.mean(spectra_2d, axis=0)
    group_means     = {int(c): np.mean(spectra_2d[labels == c], axis=0)
                       for c in unique_clusters}

    n_clusters = len(unique_clusters)
    cmap       = plt.cm.get_cmap("tab10", n_clusters)

    # promedio de cada cluster por separado
    for i, c in enumerate(unique_clusters):
        group_spectra = spectra_2d[labels == c]
        group_mean     = group_means[int(c)]
        n              = int(group_spectra.shape[0])
        #color          = cmap(i)

        if c == -1:
            color = "gray"
            label_mean = f"Ruido (-1) mean (n={n})"
            title_tag = f"Ruido (-1) (n={n})"
        else:
            color = cmap(i)
            label_mean = f"Cluster {c} mean (n={n})"
            title_tag = f"Cluster {c} (n={n})"

    fig, ax = plt.subplots(figsize=figsize)

    if show_std:
        std = np.std(group_spectra, axis=0)
        ax.fill_between(wavelength,
                        group_mean - std,
                        group_mean + std,
                        alpha=0.18, color=color, label="±1σ")

    ax.plot(wavelength, group_mean, linewidth=linewidth_mean,

```



```

        color=color, label=f"Cluster {c} mean (n={n})")

    ax.set_xlabel("Wavelength [Å]")
    ax.set_ylabel("Normalized flux")
    ax.set_title(f"{class_name} - {layer_name} - {method_name}\n{n{title_tag}}")

    if robust_ylim:
        p_low, p_high = np.percentile(group_mean, [1, 99])
        margin = 0.10 * (p_high - p_low)
        ax.set_ylim(p_low - margin, p_high + margin)

    if mark_balmer:
        add_balmer_lines(ax)

    ax.legend()
    plt.tight_layout()
    plt.show()

# promedio total —
fig, ax = plt.subplots(figsize=figsize)
ax.plot(wavelength, total_mean, linewidth=linewidth_mean,
        color="black", label=f"Total mean (n={spectra_2d.shape[0]})")

ax.set_xlabel("Wavelength [Å]")
ax.set_ylabel("Normalized flux")
ax.set_title(f"{class_name} - {layer_name} - {method_name}\n"
            f"Total mean spectrum")

if robust_ylim:
    p_low, p_high = np.percentile(total_mean, [1, 99])
    margin = 0.10 * (p_high - p_low)
    ax.set_ylim(p_low - margin, p_high + margin)

if mark_balmer:
    add_balmer_lines(ax)

ax.legend()
plt.tight_layout()
plt.show()

# diferencia: promedio cluster - promedio total
from scipy.ndimage import gaussian_filter1d
for i, c in enumerate(unique_clusters):
    diff = group_means[int(c)] - total_mean
    diff_smooth = gaussian_filter1d(diff, sigma=smooth_sigma)
# ruido dbscan
    if c == -1:
        color = "gray"
        diff_label = "Ruido (-1) - total mean"
        diff_title = "Diff: Ruido (-1) mean - total mean"
    else:
        color = cmap(i)
        diff_label = f"Cluster {c} - total mean"
        diff_title = f"Diff: Cluster {c} mean - total mean"

fig, ax = plt.subplots(figsize=figsize)

```

```

ax.plot(wavelength, diff, linewidth=0.8, color=color,
        alpha=0.25, label="raw")
ax.plot(wavelength, diff_smooth, linewidth=linewidth_mean, color=color,
        alpha=1.0, label=f"{diff_label} ( $\sigma$ ={{smooth_sigma}}px)")
ax.axhline(0.0, linestyle="--", linewidth=1.2, color="gray", alpha=0.7)

ax.set_xlabel("Wavelength [Å]")
ax.set_ylabel("Δ Normalized flux")
ax.set_title(f"{{class_name}} - {{layer_name}} - {{method_name}}\n{{diff_title}}")

if robust_ylim:
    p_low, p_high = np.percentile(diff_smooth, diff_percentiles)
    margin = 0.15 * (p_high - p_low + 1e-8)
    ax.set_ylim(p_low - margin, p_high + margin)

if mark_balmer:
    add_balmer_lines(ax)

ax.legend()
plt.tight_layout()
plt.show()

# todos los promedios juntos
fig, ax = plt.subplots(figsize=figsize)
all_curves = []

for i, c in enumerate(unique_clusters):
    group_spectra = spectra_2d[labels == c]
    mean_spec = group_means[int(c)]
    n_in_cluster = int(np.sum(labels == c))
    #color = cmap(i)
    if c == -1:
        color = "gray"
        leg = f"Ruido (-1) (n={{n_in_cluster}})"
    else:
        color = cmap(i)
        leg = f"Cluster {c} mean (n={{n_in_cluster}})"

    if show_std:
        std = np.std(spectra_2d[labels==c], axis=0)
        ax.fill_between(wavelength,
                        mean_spec - std,
                        mean_spec + std,
                        alpha=0.18, color=color)

    ax.plot(wavelength, mean_spec, linewidth=2.2,
            color=color, label=leg)
    all_curves.append(mean_spec)

ax.plot(wavelength, total_mean, linewidth=2.4, linestyle="--",
        color="black", label=f"Total mean (n={{spectra_2d.shape[0]}})")

ax.set_xlabel("Wavelength [Å]")
ax.set_ylabel("Normalized flux")
ax.set_title(f"{{class_name}} - {{layer_name}} - {{method_name}}\n"
            f"All cluster means")

```

```

if robust_ylim:
    stacked = np.vstack([*all_curves, total_mean])
    p_low, p_high = np.percentile(stacked, [1, 99])
    margin = 0.10 * (p_high - p_low + 1e-8)
    ax.set_ylim(p_low - margin, p_high + margin)

if mark_balmer:
    add_balmer_lines(ax)

ax.legend()
plt.tight_layout()
plt.show()

return {
    "total_mean"      : total_mean,
    "group_means"     : group_means,
    "unique_clusters": unique_clusters
}

```

```

In [33]: def plot_spectra_kmeans(spectra_2d, labels, wavelength, class_name, layer_name,
                                linewidth_mean=2.4, figsize=(11,5),
                                mark_balmer=True, robust_ylim=True, show_std=False,
                                smooth_sigma=3):
    return _plot_cluster_spectra_base(
        spectra_2d, labels, wavelength,
        class_name, layer_name, method_name="kmeans",
        linewidth_mean=linewidth_mean, figsize=figsize,
        mark_balmer=mark_balmer, robust_ylim=robust_ylim,
        ignore_noise=False, show_std=show_std, smooth_sigma=smooth_sigma
    )

def plot_spectra_spectral(spectra_2d, labels, wavelength, class_name, layer_name,
                          linewidth_mean=2.4, figsize=(11,5),
                          mark_balmer=False, robust_ylim=True, show_std=False,
                          smooth_sigma=3):
    return _plot_cluster_spectra_base(
        spectra_2d, labels, wavelength,
        class_name, layer_name, method_name="spectral",
        linewidth_mean=linewidth_mean, figsize=figsize,
        mark_balmer=mark_balmer, robust_ylim=robust_ylim,
        ignore_noise=False, show_std=show_std, smooth_sigma=smooth_sigma
    )

def plot_spectra_dbscan(spectra_2d, labels, wavelength, class_name, layer_name,
                        ignore_noise=False, linewidth_mean=2.4, figsize=(11,5),
                        mark_balmer=False, robust_ylim=True, show_std=False,
                        smooth_sigma=3):
    return _plot_cluster_spectra_base(
        spectra_2d, labels, wavelength,
        class_name, layer_name, method_name="dbscan",
        linewidth_mean=linewidth_mean, figsize=figsize,
        mark_balmer=mark_balmer, robust_ylim=robust_ylim,
        ignore_noise=ignore_noise, show_std=show_std, smooth_sigma=smooth_sigma
    )

```

```

In [34]: # Widget interactivo para explorar los espectros promedio por cluster
def interactive_plot_cluster_spectra(
    spectra_2d,
    wavelength,
    class_name,
    container,
    linewidth_mean=2.4,
    figsize=(13, 5),
    mark_balmer=True
):
    """
    Controles:
    - Capa      : flatten / dense / dense_1 / dense_2
    - Método    : kmeans / spectral / dbscan
    - Mostrar  $\pm 1\sigma$ : activa/desactiva la banda de desviación estándar
    """
    spectra_2d = np.asarray(spectra_2d)
    wavelength = np.asarray(wavelength)

    # Leer capas y métodos disponibles desde el container
    available_layers = list(container[class_name].keys())
    available_methods = list(next(iter(container[class_name].values())).keys())

    layer_dropdown = widgets.Dropdown(
        options=available_layers,
        value=available_layers[0],
        description="Capa:"
    )
    method_dropdown = widgets.Dropdown(
        options=available_methods,
        value=available_methods[0],
        description="Método:"
    )
    std_toggle = widgets.ToggleButtons(
        options=["No", "Sí"],
        value="No",
        description="Mostrar  $\pm 1\sigma$ :",
        button_style=""
    )

    def _plot(layer_name, method_name, mostrar_std):
        if method_name not in container[class_name].get(layer_name, {}):
            print(f"Sin labels para {class_name} - {layer_name} - {method_name}")
            return

        labels = np.asarray(container[class_name][layer_name][method_name])
        unique_clusters = np.sort(np.unique(labels))
        total_mean = np.mean(spectra_2d, axis=0)
        n_clusters = len(unique_clusters)
        cmap = plt.cm.get_cmap("tab10", n_clusters)
        show_std = mostrar_std == "Sí"

        fig, ax = plt.subplots(figsize=figsize)

        for i, c in enumerate(unique_clusters):

```

```

mask          = labels == c
group_spectra = spectra_2d[mask]
group_mean    = np.mean(group_spectra, axis=0)
n_in_cluster  = int(mask.sum())
color         = cmap(i)

if show_std:
    std = np.std(group_spectra, axis=0)
    ax.fill_between(wavelength,
                    group_mean - std,
                    group_mean + std,
                    alpha=0.18, color=color)

ax.plot(wavelength, group_mean, linewidth=linewidth_mean,
        color=color, label=f"Cluster {c} (n={n_in_cluster})")

ax.plot(wavelength, total_mean, linewidth=2.0, linestyle="--",
        color="black", label=f"Total mean (n={spectra_2d.shape[0]})")

ax.set_xlabel("Wavelength [Å]")
ax.set_ylabel("Normalized flux")
ax.set_title(f"{class_name} - {layer_name} - {method_name}"
            + (" [±1σ]" if show_std else ""))

if mark_balmer:
    add_balmer_lines(ax)

ax.legend()
plt.tight_layout()
plt.show()

interact(
    _plot,
    layer_name=layer_dropdown,
    method_name=method_dropdown,
    mostrar_std=std_toggle
)

```

Función para el cálculo de métricas Davies Bouldin Score, Calinski-Harabasz Score y Silhouette Score para un clustering dado

In [35]: `from sklearn.metrics import davies_bouldin_score, calinski_harabasz_score, silhouette`

```

def compute_cluster_metrics(Xs, labels):
    """
    Calcula DB, CH y Silhouette sobre el espacio escalado.
    Excluye ruido (-1) automáticamente.
    Retorna dict con métricas, o None en cada métrica si hay menos de 2 clusters.
    """
    mask          = labels != -1
    Xs_clean      = Xs[mask]
    labels_clean  = labels[mask]

```

```

n_clusters = len(np.unique(labels_clean))
n_noise    = int(np.sum(~mask))

if n_clusters < 2:
    return {
        "n_clusters"      : n_clusters,
        "n_noise"         : n_noise,
        "silhouette"      : None,
        "davies_bouldin"   : None,
        "calinski_harabasz" : None
    }

return {
    "n_clusters"      : n_clusters,
    "n_noise"         : n_noise,
    "silhouette"      : round(silhouette_score(Xs_clean, labels_clean), 6),
    "davies_bouldin"   : round(davies_bouldin_score(Xs_clean, labels_clean), 6),
    "calinski_harabasz" : round(calinski_harabasz_score(Xs_clean, labels_clean), 6)
}

```

```

In [36]: # censity based clustering validation (DBCV)
from scipy.spatial.distance import cdist
from scipy.sparse.csgraph import minimum_spanning_tree
from scipy.sparse import csr_matrix

def dbcv_score(X, labels, noise_id=-1):
    """
    rango [-1, 1]. Mas alto = mejor.
    los puntos de ruido (noise_id=-1) se excluyen automaticamente.
    complejidad O(n^2). Recomendado para n <= 5000.
    """
    X      = np.asarray(X, dtype=np.float64)
    labels = np.asarray(labels)

    mask   = labels != noise_id
    X      = X[mask]
    labels = labels[mask]

    unique = np.unique(labels)
    if len(unique) < 2:
        return np.nan

    n = X.shape[0]

    # matriz de distancias completa
    D = cdist(X, X)
    np.fill_diagonal(D, np.inf)

    # distancia core: media armonica de distancias dentro del cluster
    # d_core(x_i) = (|C|-1) / sum_{j in C, j!=i} (1/d(x_i,x_j))
    core_dists = np.full(n, np.inf)
    for label in unique:
        idx = np.where(labels == label)[0]
        if len(idx) <= 1:

```

```

        continue
    D_sub = D[np.ix_(idx, idx)]
    with np.errstate(divide='ignore'):
        inv_d = np.where(np.isinf(D_sub), 0.0, 1.0 / D_sub)
    harmonic_core = (len(idx) - 1) / inv_d.sum(axis=1)
    for k, i in enumerate(idx):
        core_dists[i] = harmonic_core[k]

# Mutual reachability distance: mrd(i,j) = max(core_i, core_j, d(i,j))
D_fin = np.where(np.isinf(D), 0.0, D)
mrd = np.maximum(
    np.maximum(core_dists[:, None], core_dists[None, :]),
    D_fin
)

# Arbol de expansion minima sobre mrd
mst = minimum_spanning_tree(csr_matrix(mrd)).toarray()
mst_sym = np.maximum(mst, mst.T)

# DSC: max arista interna del MST por cluster (dispersion interna)
dsc = {}
for label in unique:
    idx = np.where(labels == label)[0]
    internal = mst_sym[np.ix_(idx, idx)]
    dsc[label] = float(internal.max()) if internal.size > 0 else 0.0

# DSPC: min MRD entre clusters distintos (separacion entre clusters)
dspc = {}
for i, li in enumerate(unique):
    for j, lj in enumerate(unique):
        if i >= j:
            continue
        ix_i = np.where(labels == li)[0]
        ix_j = np.where(labels == lj)[0]
        val = float(mrd[np.ix_(ix_i, ix_j)].min())
        dspc[(li, lj)] = dspc[(lj, li)] = val

# Validez por cluster: (separacion - dispersion) / max(separacion, dispersion)
validity = {}
for label in unique:
    min_sep = min(dspc[(label, o)] for o in unique if o != label)
    denom = max(min_sep, dsc[label])
    validity[label] = 0.0 if denom == 0 else (min_sep - dsc[label]) / denom

# Score global: promedio ponderado por tamaño de cluster
score = sum(
    (np.sum(labels == label) / n) * validity[label]
    for label in unique
)
return round(float(score), 6)

```

```

In [37]: # tabla de métricas
def build_metrics_table(
    representations_32,
    class_name,
    spectral_k_values = [2, 3, 4, 5],

```

```

dbscan_eps_values = [0.5, 1.0, 1.5, 2.0, 2.5, 3.0],
dbscan_min_samples = 10,
random_state      = 42
):
    """
    Itera sobre todas las combinaciones capa × método × parámetro
    y devuelve una tabla con las métricas de clustering.

    Solo considera Spectral Clustering y DBSCAN.

    Parameters
    -----
    representations_32 : dict {layer_name: np.ndarray shape (n, 32)}
    class_name         : str
    spectral_k_values  : list de k a probar para Spectral
    dbscan_eps_values  : list de eps a probar para DBSCAN
    dbscan_min_samples : int

    Returns
    -----
    pd.DataFrame con columnas:
        class | layer | method | param | param_value |
        n_clusters | n_noise | silhouette | davies_bouldin | calinski_harabasz
    """
    rows = []

    for layer_name, feats in representations_32.items():
        Xs = StandardScaler().fit_transform(feats)

        # — Spectral Clustering: barra sobre k —
        for k in spectral_k_values:
            labels = SpectralClustering(
                n_clusters = k,
                affinity    = "nearest_neighbors",
                assign_labels = "kmeans",
                random_state = random_state
            ).fit_predict(Xs)

            metrics = compute_cluster_metrics(Xs, labels)

            rows.append({
                "class"          : class_name,
                "layer"          : layer_name,
                "method"         : "spectral",
                "param"          : "k",
                "param_value"    : k,
                "**metrics"
            })

        # — DBSCAN: barra sobre eps —
        for eps in dbscan_eps_values:
            labels = DBSCAN(
                eps = eps,
                min_samples = dbscan_min_samples
            ).fit_predict(Xs)

```



```

metrics = compute_cluster_metrics(Xs, labels)

rows.append({
    "class"          : class_name,
    "layer"          : layer_name,
    "method"         : "dbscan",
    "param"          : "eps",
    "param_value"    : eps,
    **metrics
})

df = pd.DataFrame(rows, columns=[
    "class", "layer", "method", "param", "param_value",
    "n_clusters", "n_noise",
    "silhouette", "davies_bouldin", "calinski_harabasz"
])

return df

```

```

In [38]: # identificar las mejores configuraciones
def get_best_configs(metrics_df, top_n=5, min_clusters=2):
    """
    Rankea las configuraciones usando las tres métricas combinadas.

    Criterios:
        silhouette      → mayor es mejor
        davies_bouldin  → menor es mejor
        calinski_harabasz → mayor es mejor

    Se filtran filas con métricas None o n_clusters < min_clusters.

    Parameters
    -----
    metrics_df : output de build_metrics_table
    top_n      : cuántas configuraciones mostrar
    min_clusters: mínimo de clusters para considerar válida la fila

    Returns
    -----
    pd.DataFrame con las top_n configuraciones y su score_compuesto
    """
    df = metrics_df.dropna(subset=["silhouette", "davies_bouldin",
                                   "calinski_harabasz"]).copy()
    df = df[df["n_clusters"] >= min_clusters].reset_index(drop=True)

    # Ranking por cada métrica (1 = mejor)
    df["rank_sil"] = df["silhouette"].rank(ascending=False)
    df["rank_db"]  = df["davies_bouldin"].rank(ascending=True) # menor es mejor
    df["rank_ch"]  = df["calinski_harabasz"].rank(ascending=False)

    # Score compuesto: promedio de los tres rankings (menor = mejor)
    df["score_compuesto"] = (df["rank_sil"] + df["rank_db"] + df["rank_ch"]) / 3

    top = (df.sort_values("score_compuesto")
           .head(top_n)
           .reset_index(drop=True))

```

```

cols_show = [
    "class", "layer", "method", "param_value", "n_clusters", "n_noise",
    "silhouette", "davies_bouldin", "calinski_harabasz", "score_compuesto"
]

return top[cols_show]

```

```

In [39]: # visualización tabla
def plot_metrics_heatmap(metrics_df, metric="silhouette", class_name=None):
    """
    Heatmap de una métrica para todas las combinaciones capa × configuración.

    Parameters
    -----
    metric : "silhouette" | "davies_bouldin" | "calinski_harabasz"
    """
    if class_name:
        df = metrics_df[metrics_df["class"] == class_name].copy()
    else:
        df = metrics_df.copy()

    df["config"] = df["method"] + " " + df["param"] + "=" + df["param_value"].astype(str)

    pivot = df.pivot_table(index="layer", columns="config",
                            values=metric, aggfunc="first")

    # Para DB, invertir la escala visualmente (menor = mejor = más oscuro)
    ascending = metric != "davies_bouldin"

    fig, ax = plt.subplots(figsize=(max(10, len(pivot.columns)), 4))
    sns.heatmap(
        pivot,
        annot=True, fmt=".4f",
        cmap="RdYlGn" if ascending else "RdYlGn_r",
        ax=ax,
        linewidths=0.5,
        cbar_kws={"label": metric}
    )
    title_note = "(↑ mejor)" if ascending else "(↓ mejor)"
    ax.set_title(f"{class_name or ''} - {metric} {title_note}", fontsize=13)
    ax.set_xlabel("")
    ax.set_ylabel("Capa")
    plt.xticks(rotation=30, ha="right")
    plt.tight_layout()
    plt.show()

```

Caracterización espectral — profundidad de líneas Balmer por cluster

```

In [40]: BALMER_LINES = {
    'Hε': 3970,
    'Hδ': 4101,
    'Hγ': 4340,

```

```

    'H $\beta$ ': 4861,
    'H $\alpha$ ': 6563,
}

def _measure_line(flux, wave, center,
                  line_hw=20, cont_offset=50, cont_hw=25):
    wave = np.asarray(wave)
    flux = np.asarray(flux)

    line_mask = np.abs(wave - center) <= line_hw
    cont_mask = (
        ((wave >= center - cont_offset - cont_hw) & (wave <= center - cont_offset))
        ((wave >= center + cont_offset) & (wave <= center + cont_offset +
    )

    if line_mask.sum() == 0 or cont_mask.sum() == 0:
        return np.nan, np.nan

    F_cont = np.mean(flux[cont_mask])
    if F_cont == 0:
        return np.nan, np.nan

    F_line = flux[line_mask]
    W_line = wave[line_mask]
    depth = F_cont - np.min(F_line)
    d_lam = np.abs(np.gradient(W_line)).mean()
    ew = np.sum(np.clip(1.0 - F_line / F_cont, 0, None)) * d_lam
    return round(float(depth), 6), round(float(ew), 4)

def compute_balmer_depths(group_means, wavelength,
                           class_name, layer_name, method_name,
                           label_counts=None):
    """
    Calcula profundidad y pseudo-EW de lineas Balmer para cada cluster.

    group_means : dict {cluster_id: array espectro medio} <- viene de _plot_clust
    label_counts : dict {cluster_id: n_muestras} (opcional)

    Retorna pd.DataFrame filas=clusters, columnas=profundidad/EW por linea
    """
    wavelength = np.asarray(wavelength)
    rows = []

    for cid in sorted(group_means.keys()):
        mean_spec = np.asarray(group_means[cid])
        row = {
            'class' : class_name,
            'layer' : layer_name,
            'method' : method_name,
            'cluster': cid,
        }
        if label_counts is not None:
            row['n'] = label_counts.get(cid, None)

```

```

    for line_name, center in BALMER_LINES.items():
        if center < wavelength.min() or center > wavelength.max():
            row[f'{line_name}_depth'] = np.nan
            row[f'{line_name}_ew'] = np.nan
            continue
        depth, ew = _measure_line(mean_spec, wavelength, center)
        row[f'{line_name}_depth'] = depth
        row[f'{line_name}_ew'] = ew

    rows.append(row)

    return pd.DataFrame(rows)

def plot_balmer_comparison(balmer_df, metric='depth', figsize=(10, 4)):
    """
    Barras agrupadas: profundidad (o EW) de cada linea Balmer por cluster.
    metric : 'depth' o 'ew'
    """
    cols = [f'{ln}_{metric}' for ln in BALMER_LINES if f'{ln}_{metric}' in balmer_df]
    n_clusters = len(balmer_df)
    x = np.arange(len(cols))
    width = 0.8 / n_clusters
    cmap = plt.cm.get_cmap('tab10', n_clusters)

    fig, ax = plt.subplots(figsize=figsize)
    for i, (_, row) in enumerate(balmer_df.iterrows()):
        vals = [row[c] for c in cols]
        label = f"Cluster {int(row['cluster'])}"
        if 'n' in row and row['n'] is not None:
            label += f" (n={int(row['n'])})"
        ax.bar(x + i * width, vals, width=width, color=cmap(i), alpha=0.85, label=label)

    ax.set_xticks(x + width * (n_clusters - 1) / 2)
    ax.set_xticklabels([c.replace(f'_{metric}', '') for c in cols], fontsize=11)
    ax.set_ylabel('Profundidad (flux units)' if metric == 'depth' else 'Pseudo-EW [')
    row0 = balmer_df.iloc[0]
    ax.set_title(f"Balmer {metric} por cluster - {row0['class']} | {row0['layer']}")
    ax.legend()
    plt.tight_layout(); plt.show()

```

Estabilidad de clusters — ARI entre múltiples corridas

```

In [41]: from sklearn.metrics import adjusted_rand_score
        from itertools import combinations as _combinations

        def _pairwise_ari(all_labels):
            pairs = list(_combinations(range(len(all_labels)), 2))
            return [adjusted_rand_score(all_labels[i], all_labels[j]) for i, j in pairs]

        def test_kmeans_stability(features_32, k, class_name, layer_name,
                                   n_runs=30, random_state_base=0):
            """

```

```

Corre KMeans n_runs veces con semillas distintas.
ARI ~ 1.0 => muy estable.  ARI < 0.6 => inestable.
"""
Xs = StandardScaler().fit_transform(features_32)
all_labels = [
    KMeans(n_clusters=k, n_init=10, random_state=seed).fit_predict(Xs)
    for seed in range(random_state_base, random_state_base + n_runs)
]
aris = _pairwise_ari(all_labels)
mu, sd = np.mean(aris), np.std(aris)
print(f"[{class_name} | {layer_name} | KMeans k={k}] "
      f"ARI = {mu:.3f} +/- {sd:.3f} "
      f"(min={min(aris):.3f}, max={max(aris):.3f}) [{n_runs} corridas]")
return {'mean_ari': mu, 'std_ari': sd, 'all_ari': aris}

def test_spectral_stability(features_32, k, class_name, layer_name,
                           n_runs=20, random_state_base=0):
    """
    Corre SpectralClustering n_runs veces con semillas distintas.
    """
    Xs = StandardScaler().fit_transform(features_32)
    all_labels = [
        SpectralClustering(
            n_clusters=k, affinity='nearest_neighbors',
            assign_labels='kmeans', random_state=seed
        ).fit_predict(Xs)
        for seed in range(random_state_base, random_state_base + n_runs)
    ]
    aris = _pairwise_ari(all_labels)
    mu, sd = np.mean(aris), np.std(aris)
    print(f"[{class_name} | {layer_name} | Spectral k={k}] "
          f"ARI = {mu:.3f} +/- {sd:.3f} "
          f"(min={min(aris):.3f}, max={max(aris):.3f}) [{n_runs} corridas]")
    return {'mean_ari': mu, 'std_ari': sd, 'all_ari': aris}

def test_subsample_stability(features_32, cluster_func, class_name, layer_name,
                             n_runs=30, subsample_frac=0.80, random_state=42):
    """
    Estabilidad por submuestreo sin reposicion (recomendado para DBSCAN).
    cluster_func(Xs) -> labels (Xs ya escalado).
    """
    rng = np.random.default_rng(random_state)
    Xs = StandardScaler().fit_transform(features_32)
    n = Xs.shape[0]
    n_sub = int(n * subsample_frac)

    runs = []
    for _ in range(n_runs):
        idx = np.sort(rng.choice(n, size=n_sub, replace=False))
        labels = cluster_func(Xs[idx])
        runs.append((idx, labels))

    aris = []
    for (idx_i, lbl_i), (idx_j, lbl_j) in _combinations(runs[:20], 2):

```

```

        common = np.intersect1d(idx_i, idx_j)
        if len(common) < 20:
            continue
        pos_i = np.searchsorted(idx_i, common)
        pos_j = np.searchsorted(idx_j, common)
        aris.append(adjusted_rand_score(lbl_i[pos_i], lbl_j[pos_j]))

    if not aris:
        print(f"[{class_name} | {layer_name}] Sin pares comparables.")
        return {}

    mu, sd = np.mean(aris), np.std(aris)
    print(f"[{class_name} | {layer_name} | subsample {subsample_frac*100:.0f}%] "
          f"ARI = {mu:.3f} +/- {sd:.3f} "
          f"(min={min(aris):.3f}, max={max(aris):.3f}) [{n_runs} corridas]")
    return {'mean_ari': mu, 'std_ari': sd, 'all_aris': aris}

def plot_ari_distribution(stability_result, title='Distribucion ARI', figsize=(7, 3))
    aris = stability_result.get('all_aris', [])
    if not aris:
        return
    fig, ax = plt.subplots(figsize=figsize)
    ax.hist(aris, bins=20, color='steelblue', edgecolor='white', alpha=0.85)
    ax.axvline(np.mean(aris), color='red', linestyle='--',
               label=f'Media = {np.mean(aris):.3f}')
    ax.set_xlabel('ARI'); ax.set_ylabel('Frecuencia')
    ax.set_title(title); ax.legend()
    plt.tight_layout(); plt.show()

```

Funciones interactive

```

In [42]: # interactive de Spectral Clustering (tSNE + PCA)

from IPython.display import display

# Almacen de experimentos hechos via el widget:
# clave = (layer_name, k) -> {labels, emb_tsne, emb_pca, metrics}
spectral_store = {}

def _run_spectral_cached(features_32, k, random_state=42):
    """Corre SpectralClustering + embeddings tSNE y PCA una sola vez y los cachea."""
    Xs = StandardScaler().fit_transform(features_32)
    emb_tsne = TSNE(n_components=2, random_state=42,
                    perplexity=30, init='pca').fit_transform(Xs)
    emb_pca = PCA(n_components=2, random_state=42).fit_transform(Xs)
    labels = SpectralClustering(n_clusters=k, random_state=random_state,
                               affinity='nearest_neighbors').fit_predict(Xs)
    metrics = compute_cluster_metrics(Xs, labels)
    return {"labels": labels, "emb_tsne": emb_tsne,
            "emb_pca": emb_pca, "metrics": metrics}

```

```

def _plot_spectral_views(data, class_name, layer_name, k):
    """Dibuja los scatter tSNE y PCA a partir de datos cacheados."""
    labels = data["labels"]
    views = [
        ("tsne", data["emb_tsne"], None, None),
        ("pca", data["emb_pca"], (-10, 10), (-10, 10))
    ]
    for vis_name, emb, xlim, ylim in views:
        fig, ax = plt.subplots(figsize=(9, 6))
        ax.scatter(emb[:, 0], emb[:, 1], c=labels, s=30, alpha=0.6, cmap='viridis')
        ax.set_title(f"{class_name} - {layer_name} - Spectral "
                    f"(k={k}, vis={vis_name}, n={len(labels)})", fontsize=14)
        _add_cluster_legend(ax, labels)
        ax.set_xlabel("Comp. 1"); ax.set_ylabel("Comp. 2")
        if xlim: ax.set_xlim(xlim)
        if ylim: ax.set_ylim(ylim)
        plt.tight_layout(); plt.show()
    _print_metrics_box(class_name, layer_name, "spectral", f"k={k}", data["metrics"])

def spectral_explorer(features_dict, class_name, suffix,
                      store=None,
                      layers_avail=("flatten", "dense", "dense_1", "dense_2"),
                      k_values=range(2, 13)):

    if store is None:
        store = spectral_store

    def _show(capa, k, recalcular):
        k = int(k)
        key = (capa, k)

        hechos = sorted(f"{l}/k={kk}" for (l, kk) in store)
        print("Experimentos guardados:",
              ", ".join(hechos) if hechos else "ninguno")
        print("-" * 60)

        if key not in store or recalcular:
            print(f"Calculando {class_name} | {capa} | k={k} ...")
            store[key] = _run_spectral_cached(features_dict[capa], k)
            save_cluster_labels(cluster_labels_all, class_name,
                              capa, "spectral", store[key]["labels"])
            varname = f"labels_{capa}_k{k}_{suffix}"
            globals()[varname] = store[key]["labels"]
            print(f"Guardado en store[('{capa}', {k})], en "
                  f"cluster_labels_all[('{class_name}')][('{capa}')]['spectral'] "
                  f"y en la variable global `{varname}`")
        else:
            print(f"Mostrando experimento cacheado: {class_name} | {capa} | k={k}")
            print("-" * 60)

        _plot_spectral_views(store[key], class_name, capa, k)

    interact(
        _show,
        capa=widgets.Dropdown(options=list(layers_avail), description="Capa:"),
        k=widgets.Dropdown(options=list(k_values), value=3, description="k:"),
        recalcular=widgets.Checkbox(value=False, description="Forzar recalculo"),

```

```

)

def get_spectral_labels(layer, k, store=None):
    """Devuelve las etiquetas de un experimento guardado por el explorador."""
    if store is None:
        store = spectral_store
    key = (layer, int(k))
    if key not in store:
        raise KeyError(f"No existe el experimento ({layer}, k={k}). "
                       f"Correlo primero en el explorador.")
    return store[key]["labels"]

```

In [43]: # interactive de DBSCAN (tSNE + PCA)

```

dbscan_store = {} # cache: (layer, eps, min_samples) -> {labels, emb_tsne, emb_pca}

def _run_dbscan_cached(features_32, eps, min_samples):
    """Corre DBSCAN + embeddings tSNE y PCA y los cachea."""
    Xs = StandardScaler().fit_transform(features_32)
    emb_tsne = TSNE(n_components=2, random_state=42,
                    perplexity=30, init='pca').fit_transform(Xs)
    emb_pca = PCA(n_components=2, random_state=42).fit_transform(Xs)
    labels = DBSCAN(eps=eps, min_samples=min_samples).fit_predict(Xs)
    metrics = compute_cluster_metrics(Xs, labels)
    n_pts = int(np.sum(labels != -1))
    dbcv = dbcv_score(Xs, labels) if 0 < n_pts <= 5000 else None
    if n_pts > 5000:
        print(f' [DBCV omitido: {n_pts} puntos no-ruido > 5000]')
    return {"labels": labels, "emb_tsne": emb_tsne,
            "emb_pca": emb_pca, "metrics": metrics, "dbcv": dbcv}

def _plot_dbscan_views(data, class_name, layer_name, eps, min_samples):
    """Dibuja scatter tSNE y PCA a partir de datos cacheados."""
    labels = data["labels"]
    views = [("tsne", data["emb_tsne"]), ("pca", data["emb_pca"])]
    for vis_name, emb in views:
        fig, ax = plt.subplots(figsize=(9, 6))
        ax.scatter(emb[:, 0], emb[:, 1], c=labels,
                  s=30, alpha=0.4, cmap='viridis')
        ax.set_title(
            f"{class_name} \u2014 {layer_name} \u2014 DBSCAN "
            f"(eps={eps}, ms={min_samples}, vis={vis_name}, n={len(labels)}",
            fontsize=14)
        _add_cluster_legend(ax, labels)
        ax.set_xlabel("Comp. 1"); ax.set_ylabel("Comp. 2")
        plt.tight_layout(); plt.show()
    _print_metrics_box(class_name, layer_name, "dbscan",
                       f"eps={eps} ms={min_samples}",
                       data["metrics"], dbcv=data["dbcv"])

def dbscan_explorer(features_dict, class_name, suffix,

```



```

        store=None,
        layers_avail=("flatten", "dense", "dense_1", "dense_2"),
        eps_values=None,
        ms_values=None):

    if store is None:
        store = dbscan_store
    if eps_values is None:
        eps_values = [0.3, 0.5, 0.7, 1.0, 1.2, 1.5, 2.0, 2.5, 3.0, 4.0]
    if ms_values is None:
        ms_values = [3, 5, 8, 10, 12, 15, 20, 25, 30]

    def _show(capa, eps, min_samples, recalcular):
        key = (capa, eps, min_samples)
        hechos = sorted(f"{l}/eps={e}/ms={m}" for (l, e, m) in store)
        print("Experimentos guardados:",
              ", ".join(hechos) if hechos else "ninguno")
        print("-" * 60)

        if key not in store or recalcular:
            print(f"Calculando {class_name} | {capa} | eps={eps} ms={min_samples} .")
            store[key] = _run_dbscan_cached(features_dict[capa], eps, min_samples)
            save_cluster_labels(cluster_labels_all, class_name,
                               capa, "dbscan", store[key]["labels"])
            eps_str = str(eps).replace(".", "p")
            varname = f"labels_dbs_{capa}_e{eps_str}_ms{min_samples}_{suffix}"
            globals()[varname] = store[key]["labels"]
            print(f"Guardado en store[{key}], en "
                  f"cluster_labels_all['{class_name}']['{capa}']['dbscan'] "
                  f"y en la variable global `{varname}`")
        else:
            print(f"Mostrando cacheado: {class_name} | {capa} | eps={eps} ms={min_s")
            print("-" * 60)
            _plot_dbscan_views(store[key], class_name, capa, eps, min_samples)

    interact(
        _show,
        capa=widgets Dropdown(options=list(layers_avail), description="Capa:"),
        eps=widgets Dropdown(options=eps_values, value=1.5, description="eps:"),
        min_samples=widgets Dropdown(options=ms_values, value=10, description="min_")
        recalcular=widgets.Checkbox(value=False, description="Forzar recalculo"),
    )

def get_dbscan_labels(layer, eps, min_samples, store=None):
    """Devuelve etiquetas de un experimento guardado por el explorador DBSCAN."""
    if store is None:
        store = dbscan_store
    key = (layer, float(eps), int(min_samples))
    if key not in store:
        raise KeyError(
            f"No existe el experimento ({layer}, eps={eps}, ms={min_samples}). "
            f"Correlo primero en el explorador DBSCAN.")
    return store[key]["labels"]

```

```

In [44]: # espectros por cluster (Spectral + DBSCAN)

def _plot_cluster_spectra_interactive(
    spectra_2d, labels, wavelength, total_mean,
    class_name, layer_name, method_name, param_str,
    show_media=True, show_balmer=False, show_std=False,
    figsize=(11, 5), linewidth_mean=2.4
):
    """Dibuja graficos de espectro por cluster + grafico combinado."""
    labels = np.asarray(labels)
    unique_cls = np.sort(np.unique(labels))
    n_clusters = len(unique_cls)
    cmap = plt.cm.get_cmap("tab10", n_clusters)

    # --- graficos individuales por cluster ---
    for i, c in enumerate(unique_cls):
        mask = labels == c
        g_spec = spectra_2d[mask]
        g_mean = np.mean(g_spec, axis=0)
        n = int(mask.sum())
        color = "gray" if c == -1 else cmap(i)
        lbl = "Ruido (-1)" if c == -1 else f"Cluster {c}"

        fig, ax = plt.subplots(figsize=figsize)
        if show_std:
            std = np.std(g_spec, axis=0)
            ax.fill_between(wavelength, g_mean - std, g_mean + std,
                           alpha=0.18, color=color, label="\u00b11\u03c3")
        ax.plot(wavelength, g_mean, linewidth=linewidth_mean,
                color=color, label=f"{lbl} mean (n={n})")
        ax.set_xlabel("Wavelength [\u00c5]")
        ax.set_ylabel("Normalized flux")
        ax.set_title(
            f"{class_name} \u2014 {layer_name} \u2014 "
            f"{method_name} ({param_str})\n{lbl} (n={n})")
        if show_balmer:
            add_balmer_lines(ax)
        ax.legend()
        plt.tight_layout()
        plt.show()

    # --- grafico combinado ---
    fig, ax = plt.subplots(figsize=figsize)
    all_curves = []
    for i, c in enumerate(unique_cls):
        mask = labels == c
        g_spec = spectra_2d[mask]
        g_mean = np.mean(g_spec, axis=0)
        n = int(mask.sum())
        color = "gray" if c == -1 else cmap(i)
        leg = "Ruido (-1)" if c == -1 else f"Cluster {c} (n={n})"
        if show_std:
            std = np.std(g_spec, axis=0)
            ax.fill_between(wavelength, g_mean - std, g_mean + std,
                           alpha=0.12, color=color)

```

```

ax.plot(wavelength, g_mean, linewidth=2.2, color=color, label=leg)
all_curves.append(g_mean)

if show_media and len(all_curves) > 0:
    ax.plot(wavelength, total_mean, linewidth=2.4, linestyle="--",
            color="black", label=f"Media clase (n={len(labels)})")
    all_curves.append(total_mean)

ax.set_xlabel("Wavelength [\u00c5]")
ax.set_ylabel("Normalized flux")
ax.set_title(
    f"{class_name} \u2014 {layer_name} \u2014 "
    f"{method_name} ({param_str})\nAll cluster means")

if all_curves:
    stacked = np.vstack(all_curves)
    p_lo, p_hi = np.percentile(stacked, [1, 99])
    margin = 0.10 * (p_hi - p_lo + 1e-8)
    ax.set_ylim(p_lo - margin, p_hi + margin)

if show_balmer:
    add_balmer_lines(ax)
ax.legend()
plt.tight_layout()
plt.show()

def spectra_viewer(spectra_2d, wavelength, class_name, suffix,
                  spectral_store_ref=None, dbscan_store_ref=None):
    """
    Widget interactivo para visualizar espectros de clusters ya calculados.

    Metodos soportados: spectral (usa spectral_store) y dbscan (usa dbscan_store).
    Cuando se selecciona 'spectral' aparece el dropdown de k;
    cuando se selecciona 'dbscan' aparecen los dropdowns de eps y min_samples.

    Controles de visualizacion:
    - Media clase : muestra/oculta el espectro promedio en el grafico combinado
    - Balmer      : superpone lineas de Balmer en todos los graficos
    - +/-1sigma   : banda de desviacion estandar en los graficos individuales
    """
    if spectral_store_ref is None:
        spectral_store_ref = spectral_store
    if dbscan_store_ref is None:
        dbscan_store_ref = dbscan_store

    spectra_2d = np.asarray(spectra_2d)
    wavelength = np.asarray(wavelength)
    total_mean = np.mean(spectra_2d, axis=0)

    W = widgets.Layout

    metodo_dd = widgets.Dropdown(
        options=["spectral", "dbscan"], description="Metodo:",
        layout=W(width="190px"))
    capa_dd = widgets.Dropdown(

```

```

options=["flatten", "dense", "dense_1", "dense_2"],
description="Capa:", layout=W(width="200px"))
k_dd = widgets.Dropdown(
    options=list(range(2, 13)), value=3,
    description="k:", layout=W(width="140px"))
eps_dd = widgets.Dropdown(
    options=[0.3, 0.5, 0.7, 1.0, 1.2, 1.5, 2.0, 2.5, 3.0, 4.0],
    value=1.5, description="eps:", layout=W(width="150px"))
ms_dd = widgets.Dropdown(
    options=[3, 5, 8, 10, 12, 15, 20, 25, 30],
    value=10, description="min_s:", layout=W(width="160px"))

media_btn = widgets.ToggleButton(
    value=True, description="Media clase",
    button_style="info", layout=W(width="150px"))
balmer_btn = widgets.ToggleButton(
    value=False, description="Balmer",
    button_style="", layout=W(width="110px"))
std_btn = widgets.ToggleButton(
    value=False, description="\u00b1\u03c3 individual",
    button_style="", layout=W(width="150px"))

out = widgets.Output()

def _update_visibility(change=None):
    is_sp = metodo_dd.value == "spectral"
    k_dd.layout.display = "" if is_sp else "none"
    eps_dd.layout.display = "none" if is_sp else ""
    ms_dd.layout.display = "none" if is_sp else ""

metodo_dd.observe(_update_visibility, names="value")
_update_visibility()

def _render(change=None):
    with out:
        out.clear_output(wait=True)
        metodo = metodo_dd.value
        capa = capa_dd.value

        if metodo == "spectral":
            k = int(k_dd.value)
            key = (capa, k)
            if key not in spectral_store_ref:
                print(f"No hay experimento spectral ({capa}, k={k}).\n"
                    "Correlo primero en el explorador Spectral Clustering.")
                return
            labels = spectral_store_ref[key]["labels"]
            param_str = f"k={k}"
        else:
            eps = float(eps_dd.value)
            ms = int(ms_dd.value)
            key = (capa, eps, ms)
            if key not in dbscan_store_ref:
                print(f"No hay experimento DBSCAN ({capa}, eps={eps}, ms={ms}).\n"
                    "Correlo primero en el explorador DBSCAN.")
                return

```

```

labels      = dbscan_store_ref[key]["labels"]
param_str   = f"eps={eps} ms={ms}"

_plot_cluster_spectra_interactive(
    spectra_2d, labels, wavelength, total_mean,
    class_name, capa, metodo, param_str,
    show_media = media_btn.value,
    show_balmer = balmer_btn.value,
    show_std    = std_btn.value,
)

for w in [metodo_dd, capa_dd, k_dd, eps_dd, ms_dd,
          media_btn, balmer_btn, std_btn]:
    w.observe(_render, names="value")

row1 = widgets.HBox([metodo_dd, capa_dd, k_dd, eps_dd, ms_dd])
row2 = widgets.HBox([media_btn, balmer_btn, std_btn])
display(widgets.VBox([row1, row2, out]))
_render()

```

Experimentos

Usar n = 700 muestras

```

In [45]: # Datos para DRACULA
X_wd_spec2, X_wd_input2, df_wd_DRAC2 = load_class_spectra_DRACULA("WD", n_samples=7)
flatten_wd_DRAC, dense_wd_DRAC, dense1_wd_DRAC, dense2_wd_DRAC = extract_layer_feat
repr_wd_32 = reduce_all_to_32(flatten_wd_DRAC, dense_wd_DRAC, dense1_wd_DRAC, dense

```

```

WD: disponibles con match a .dat = 736
WD: espectros válidos finales = 629
10/10 ————— 1s 43ms/step
10/10 ————— 1s 46ms/step
10/10 ————— 1s 48ms/step
10/10 ————— 1s 45ms/step
flatten: (629, 208)
dense: (629, 128)
dense_1: (629, 64)
dense_2: (629, 32)
flatten_32: (629, 32)
dense_32: (629, 32)
dense1_32: (629, 32)
dense2_32: (629, 32)

```

```

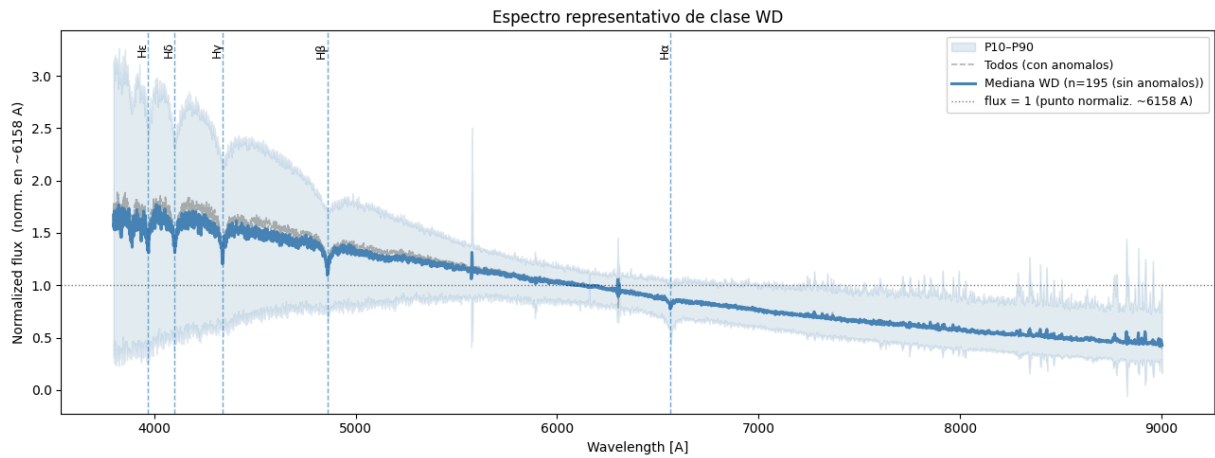
In [46]: plot_class_mean_spectrum(X_wd_spec, W_wd[0], class_name="WD")

```

```

=== Diagnostico WD (n=629) ===
Flux minimo por espectro: media=-11056143400960.000 p5=-13.569 p95=0.204
Flux maximo por espectro: media=1536618004480.000 p5=1.773 p95=20.829
Espectros fuera de [-0.5, 5.0]: 434 (69.0%)

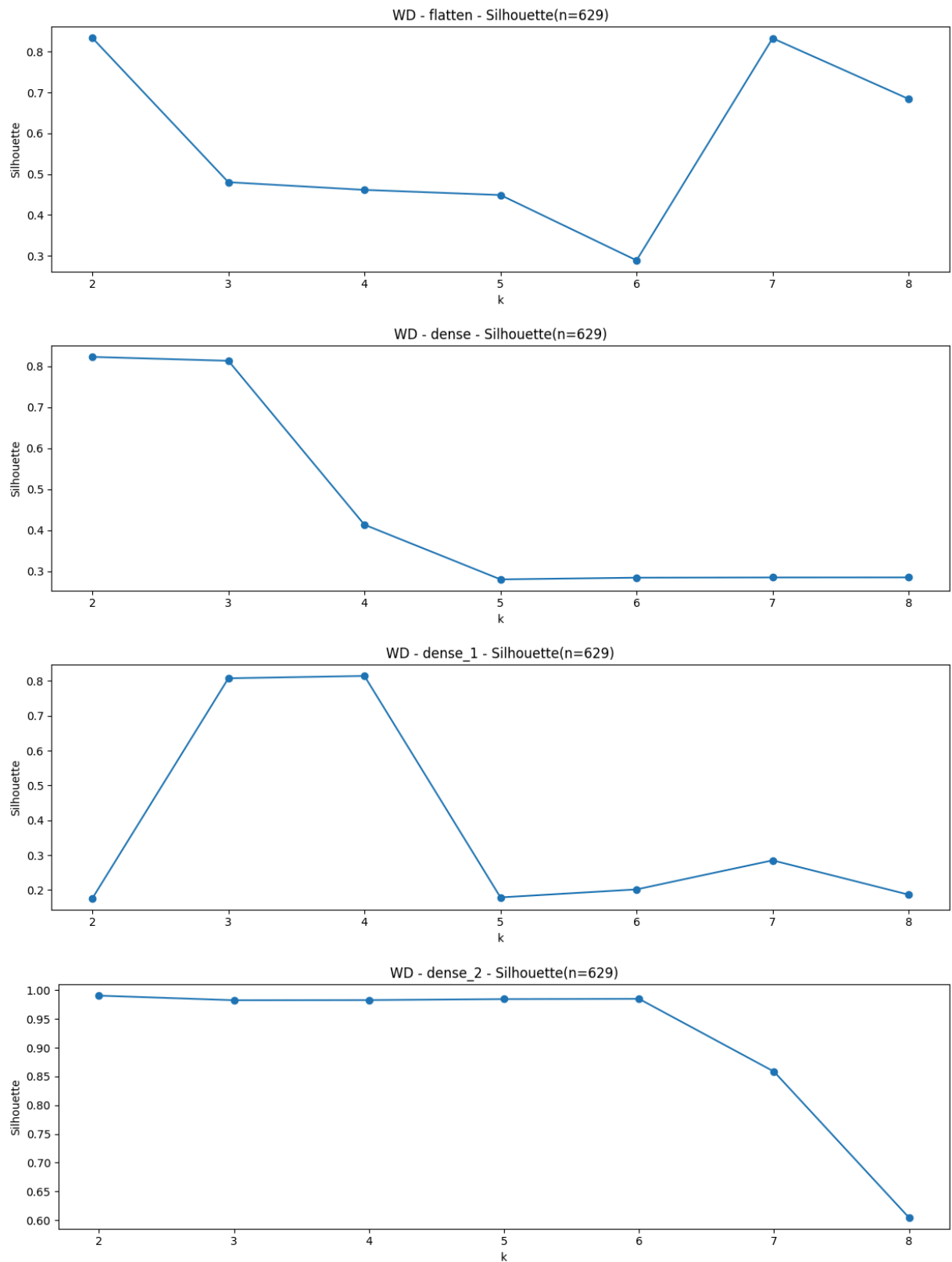
```



```
Out[46]: {'central': array([1.6699731 , 1.5867871 , 1.5563612 , ..., 0.47176507, 0.4505432
,
        0.42810085], dtype=float32),
        'p_lo': array([0.35359904, 0.30790576, 0.44183597, ..., 0.26503329, 0.16455931,
        0.16358801]),
        'p_hi': array([3.11234345, 3.13163075, 3.09151602, ..., 0.97381082, 1.01288364,
        0.91308331]),
        'spectra_clean': array([[4.17038   , 3.4223802 , 4.080973   , ..., 0.14180057, 0.2
30642   ,
        0.21507654],
        [1.7501309 , 1.7100465 , 1.5806053 , ..., 0.46198675, 0.45528954,
        0.4611065   ],
        [1.2320663 , 1.1932751 , 1.6273049 , ..., 0.5038261 , 0.4704357 ,
        0.519842   ],
        ...,
        [1.1189705 , 2.0492811 , 1.6283233 , ..., 0.35338846, 0.44005775,
        0.42222202],
        [2.568726   , 2.7166257 , 2.7557666 , ..., 0.3852875 , 0.41284925,
        0.2426779   ],
        [1.7967786 , 1.2851976 , 0.4589212 , ..., 0.81418544, 0.9586785 ,
        0.4274985   ]], dtype=float32),
        'n_bad': 434}
```

Evaluación con Silhouette (n=629)

```
In [47]: evaluate_kmeans_k(repr_wd_32["flatten"], "WD", "flatten")
evaluate_kmeans_k(repr_wd_32["dense"], "WD", "dense")
evaluate_kmeans_k(repr_wd_32["dense_1"], "WD", "dense_1")
evaluate_kmeans_k(repr_wd_32["dense_2"], "WD", "dense_2")
```



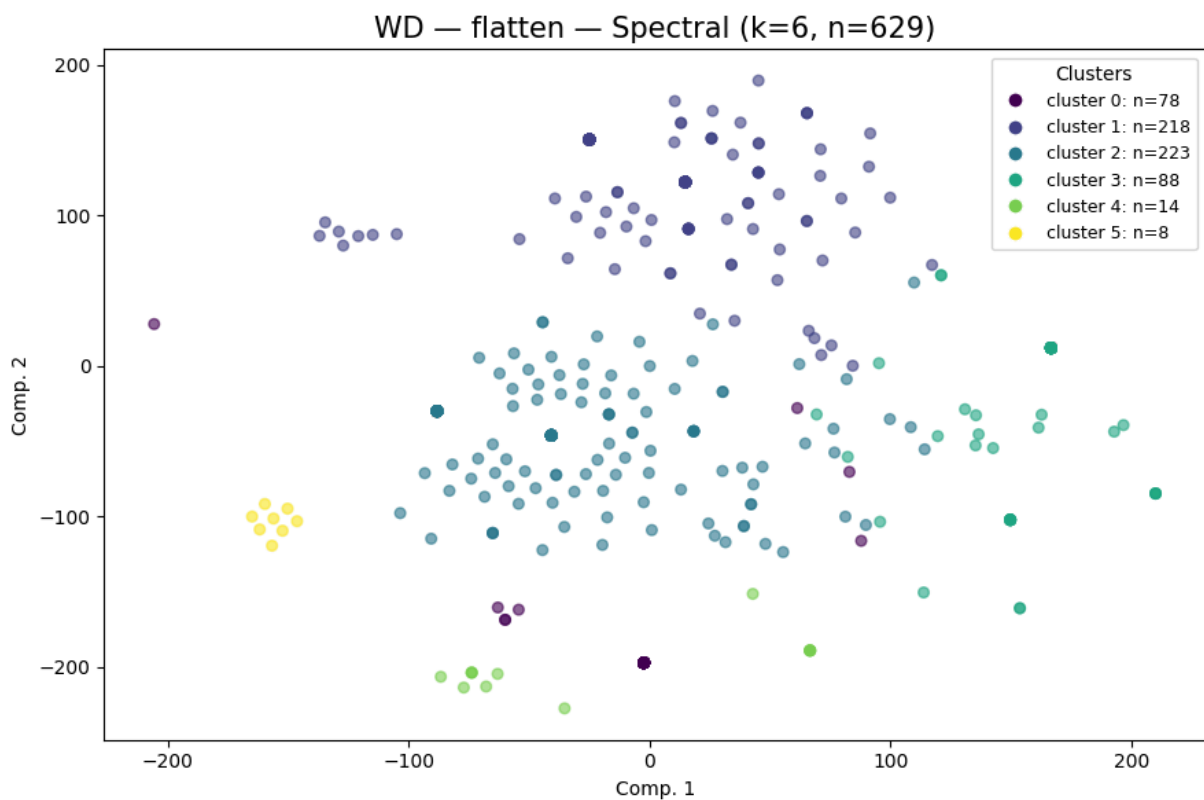
Gráficos t-SNE y PCA -> para Spectral Clustering y DBSCAN

Resultados capa flatten, con Spectral Clustering + tSNE y PCA para n=700

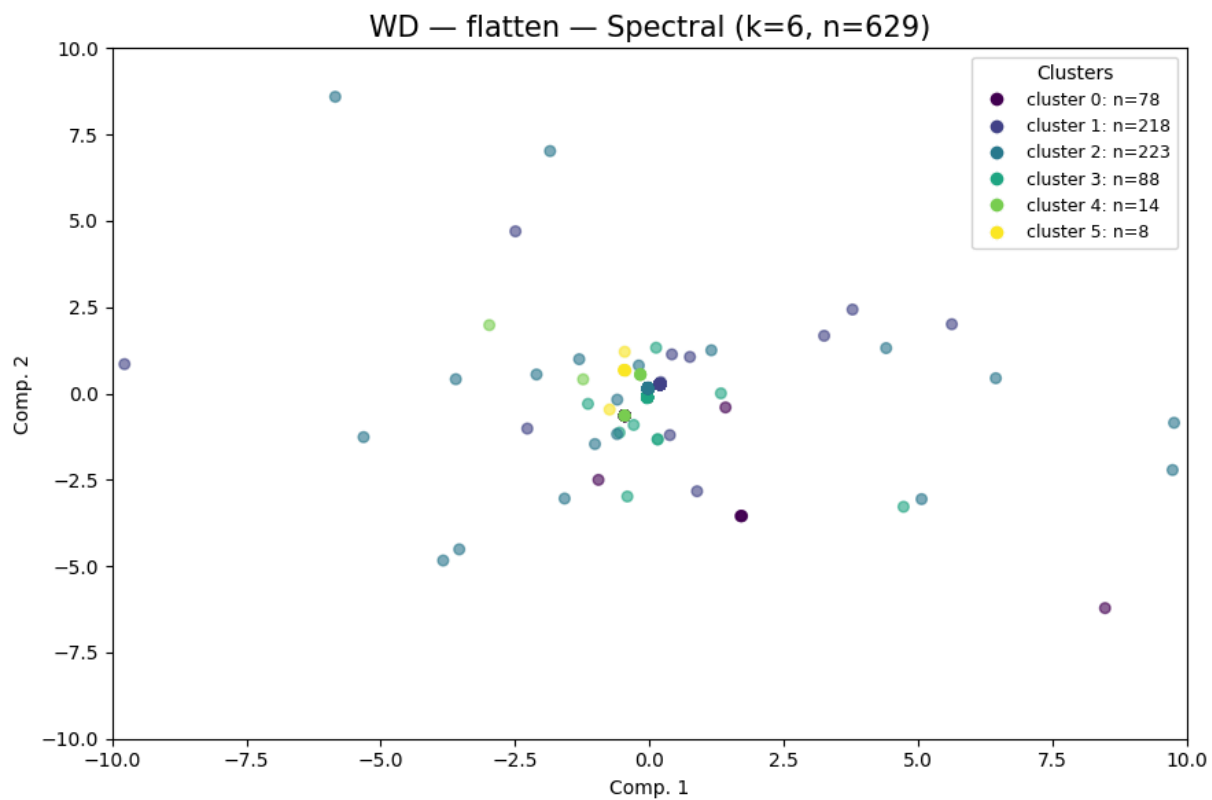
Spectral Clustering k = 6

```
In [48]: res_k6 = cluster_plot_Spect(
    repr_wd_32["flatten"],
    "WD",
    "flatten",
    spectral_k=6,
    vis_method="tsne",
    container=cluster_labels_all
)

labels_flatten_k6_wd = res_k6["spectral"]
cluster_plot_Spect(
    repr_wd_32["flatten"],
    "WD",
    "flatten",
    spectral_k=6,
    vis_method="pca",
    xlim=(-10,10), ylim=(-10,10)
)
```



```
WD | flatten | spectral | k=6
clusters=6 noise=0 sil=0.5040 DB=2.0300 CH=16.70
```

```
WD | flatten | spectral | k=6  
clusters=6 noise=0 sil=0.5040 DB=2.0300 CH=16.70
```

```

Out[48]: {'spectral': array([2, 2, 2, 2, 0, 2, 1, 1, 3, 3, 2, 4, 2, 2, 3, 2, 3, 3, 0, 3, 1,
2,
1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 1, 1, 1, 1,
1, 1, 2, 2, 2, 2, 2, 2, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1,
2, 2, 2, 2, 2, 2, 1, 0, 0, 0, 0, 0, 0, 0, 2, 1, 1, 1, 1, 1, 2, 2,
2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2,
2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 1, 1, 1, 1, 1, 1, 2, 4, 2, 3, 2, 2,
0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 0, 0,
0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3,
3, 3, 3, 3, 4, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3,
3, 3, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 2, 2, 2, 1, 1, 1,
1, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 1, 1, 1, 1, 1,
1, 3, 2, 2, 2, 2, 2, 1, 1, 1, 1, 2, 2, 2, 1, 1, 1, 1, 1, 1, 2, 2,
2, 2, 2, 2, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2,
2, 2, 1, 1, 1, 1, 2, 2, 2, 1, 1, 1, 4, 4, 1, 2, 2, 2, 2, 2, 2, 3,
3, 3, 3, 3, 3, 3, 1, 1, 1, 4, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1, 1,
1, 2, 2, 2, 1, 1, 1, 1, 1, 4, 2, 2, 2, 2, 2, 2, 0, 0, 0, 0, 0, 0,
0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 2, 2, 2, 1,
1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 1, 1, 1,
1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 2, 1, 2, 1, 1, 1, 1, 1,
2, 2, 2, 2, 2, 2, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2,
2, 2, 2, 2, 1, 1, 1, 1, 2, 2, 2, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2,
2, 3, 3, 3, 3, 1, 3, 3, 3, 2, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1,
2, 1, 1, 2, 2, 2, 1, 1, 1, 4, 1, 1, 2, 2, 2, 2, 2, 2, 0, 0, 0, 0,
0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 2, 2,
2, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 1,
1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 2, 2, 2, 1, 1, 1, 1,
1, 1, 2, 2, 2, 2, 2, 2, 4, 4, 4, 4, 4, 4, 2, 2, 0, 2, 3, 2, 2, 0,
5, 5, 1, 5, 5, 5, 0, 0, 0, 0, 5, 5, 5]),
'metrics': {'n_clusters': 6,
'n_noise': 0,
'silhouette': 0.50396,
'davies_bouldin': 2.029956,
'calinski_harabasz': 16.7015}}

```

Spectral Clustering k = 7

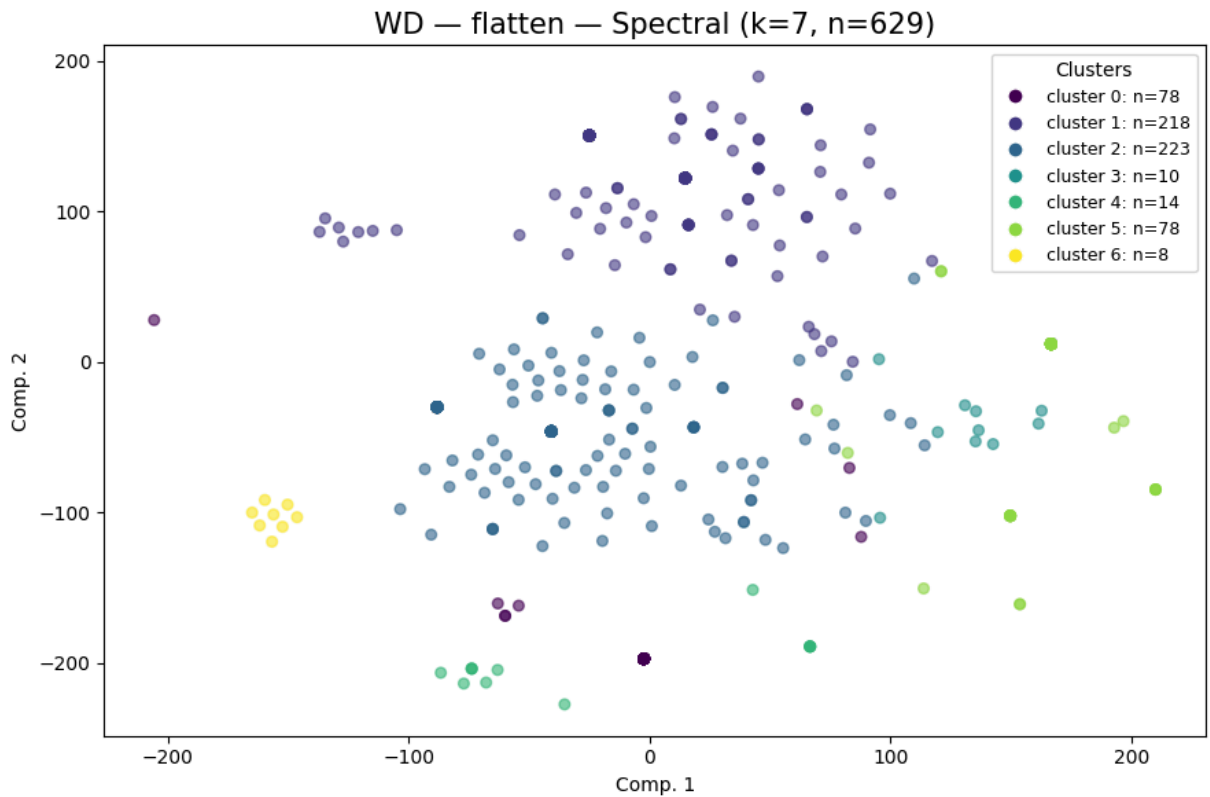
```

In [49]: res_k7 = cluster_plot_Spect(
    repr_wd_32["flatten"],
    "WD",
    "flatten",
    spectral_k=7,
    vis_method="tsne",
    container=cluster_labels_all
)

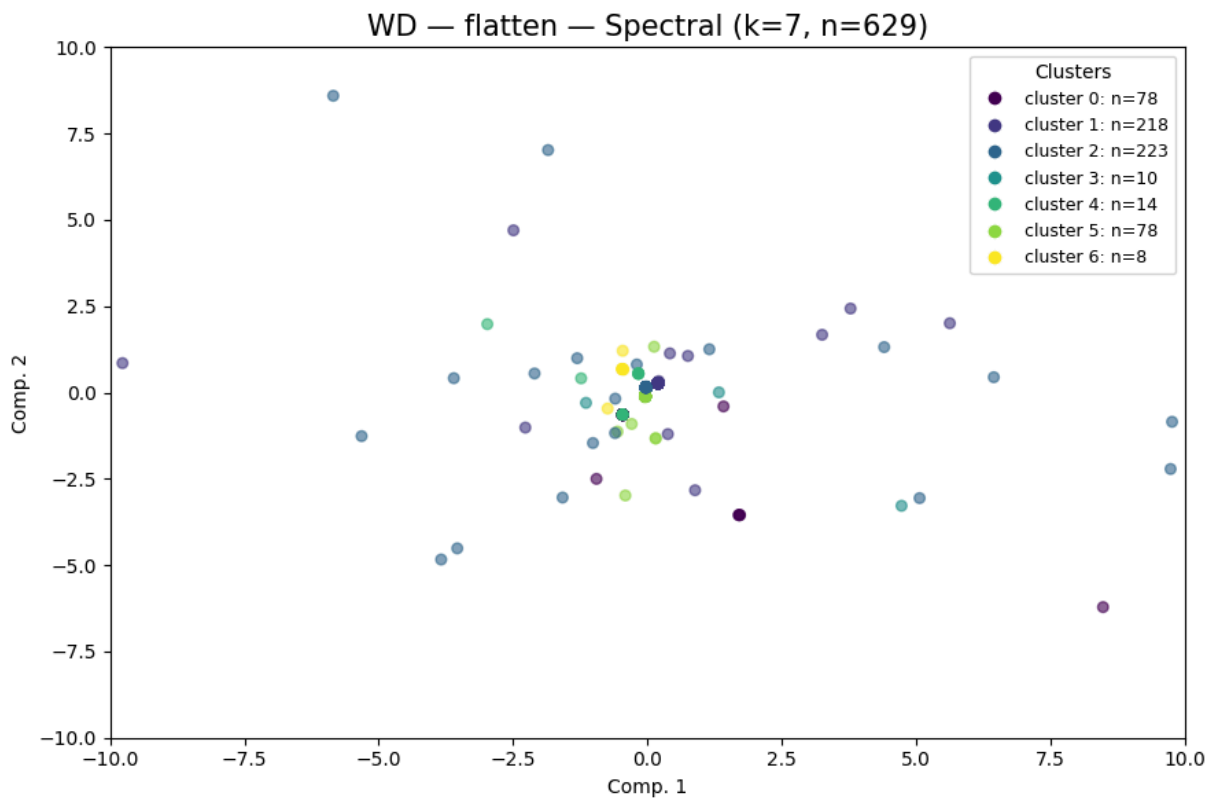
labels_flatten_k7_wd = res_k7["spectral"]
cluster_plot_Spect(
    repr_wd_32["flatten"],
    "WD",
    "flatten",
    spectral_k=7,
    vis_method="pca",

```

```
xlim=(-10,10), ylim=(-10,10)
)
```



```
WD | flatten | spectral | k=7
clusters=7 noise=0 sil=0.4941 DB=2.1556 CH=14.27
```



WD flatten spectral k=7
clusters=7 noise=0 sil=0.4941 DB=2.1556 CH=14.27

```
Out[49]: {'spectral': array([2, 2, 2, 2, 0, 2, 1, 1, 5, 5, 2, 4, 2, 2, 3, 2, 3, 5, 0, 5, 1,
2,
1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 5, 3, 3, 3, 3, 1, 1, 1, 1,
1, 1, 2, 2, 2, 2, 2, 2, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1,
2, 2, 2, 2, 2, 2, 1, 0, 0, 0, 0, 0, 0, 0, 2, 1, 1, 1, 1, 1, 2, 2,
2, 2, 2, 2, 5, 5, 5, 5, 5, 5, 5, 5, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2,
2, 2, 5, 5, 5, 5, 5, 5, 5, 5, 1, 1, 1, 1, 1, 1, 2, 4, 2, 5, 2, 2,
0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 0, 0,
0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 5, 5, 5, 5,
5, 5, 5, 5, 4, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 5, 5, 5, 5, 5, 5,
5, 5, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 2, 2, 2, 1, 1, 1,
1, 1, 1, 2, 2, 2, 2, 2, 2, 5, 5, 5, 5, 5, 5, 5, 5, 1, 1, 1, 1, 1,
1, 5, 2, 2, 2, 2, 2, 1, 1, 1, 1, 2, 2, 2, 1, 1, 1, 1, 1, 2, 2,
2, 2, 2, 2, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2,
2, 2, 1, 1, 1, 1, 2, 2, 2, 1, 1, 1, 4, 4, 1, 2, 2, 2, 2, 2, 2, 5,
5, 5, 5, 5, 5, 5, 1, 1, 1, 4, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1, 1,
1, 2, 2, 2, 1, 1, 1, 1, 1, 4, 2, 2, 2, 2, 2, 2, 0, 0, 0, 0, 0, 0,
0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 2, 2, 2, 1,
1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 5, 5, 5, 5, 5, 5, 5, 5, 1, 1, 1,
1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 2, 1, 2, 1, 1, 1, 1, 1,
2, 2, 2, 2, 2, 2, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2,
2, 2, 2, 2, 1, 1, 1, 1, 2, 2, 2, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2,
2, 5, 5, 5, 5, 1, 5, 5, 5, 2, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1,
2, 1, 1, 2, 2, 2, 1, 1, 1, 4, 1, 1, 2, 2, 2, 2, 2, 2, 0, 0, 0, 0,
0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 2, 2,
2, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 5, 5, 5, 5, 5, 5, 5, 5, 1,
1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 2, 2, 2, 1, 1, 1, 1,
1, 1, 2, 2, 2, 2, 2, 2, 4, 4, 4, 4, 4, 4, 2, 2, 0, 2, 3, 2, 2, 0,
6, 6, 1, 6, 6, 6, 0, 0, 0, 0, 6, 6, 6]),
'metrics': {'n_clusters': 7,
'n_noise': 0,
'silhouette': 0.494103,
'davies_bouldin': 2.155638,
'calinski_harabasz': 14.2694}}
```

Spectral Clustering k = 8

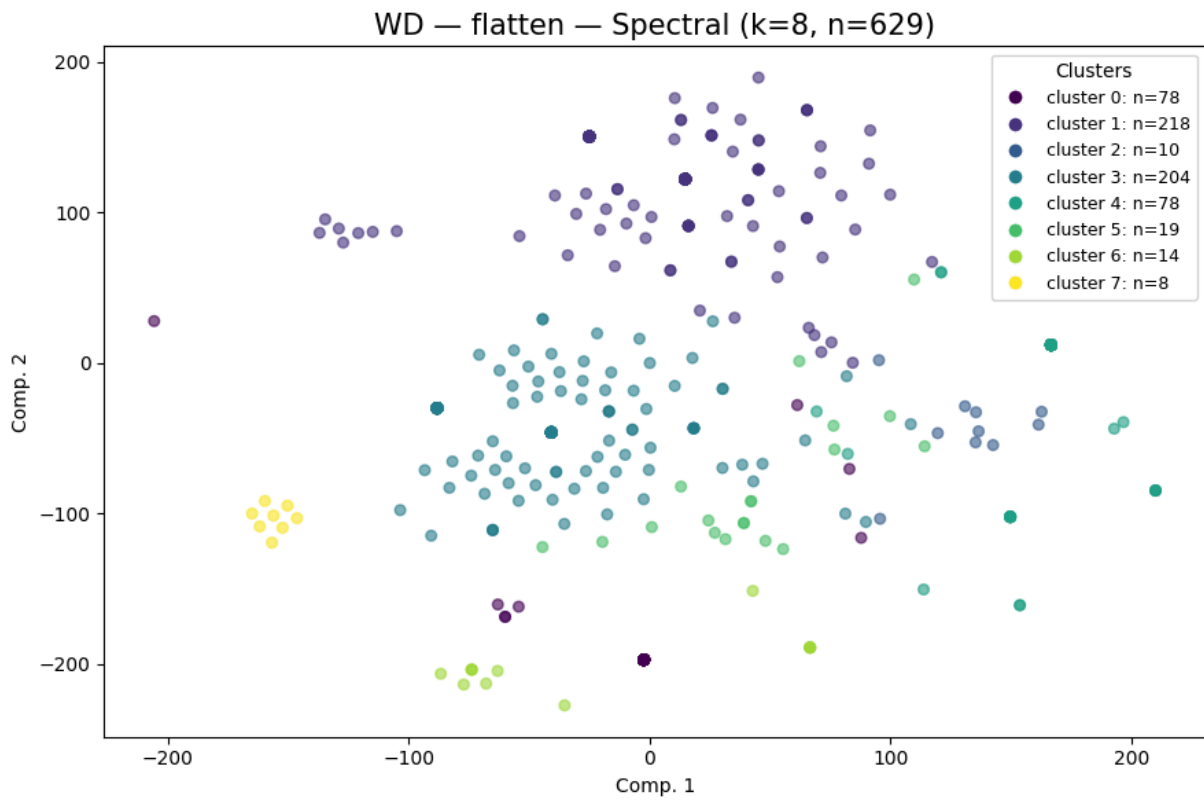
```
In [50]: res_k8 = cluster_plot_Spect(
    repr_wd_32["flatten"],
    "WD",
    "flatten",
    spectral_k=8,
    vis_method="tsne",
    container=cluster_labels_all
)

labels_flatten_k8_wd = res_k8["spectral"]
cluster_plot_Spect(
    repr_wd_32["flatten"],
```

```

"WD",
"flatten",
spectral_k=8,
vis_method="pca",
xlim=(-10,10), ylim=(-10,10)
)

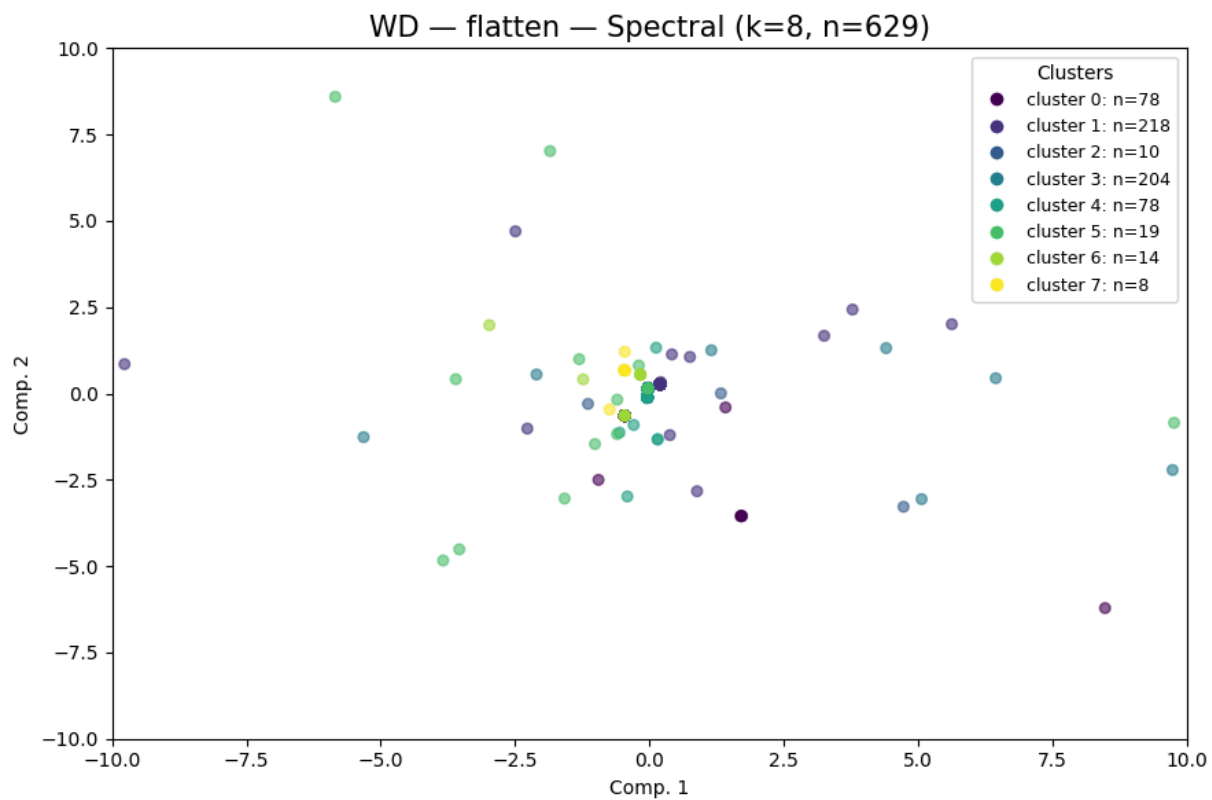
```



```

WD | flatten | spectral | k=8
clusters=8 noise=0 sil=0.5500 DB=3.3894 CH=13.55

```



```
WD | flatten | spectral | k=8  
clusters=8 noise=0 sil=0.5500 DB=3.3894 CH=13.55
```

```

Out[50]: {'spectral': array([5, 3, 5, 5, 0, 5, 1, 1, 4, 4, 3, 6, 5, 5, 2, 3, 2, 4, 0, 4, 1,
5,
1, 1, 1, 1, 3, 5, 5, 5, 3, 3, 2, 2, 2, 4, 2, 2, 2, 2, 1, 1, 1, 1,
1, 1, 3, 3, 3, 3, 3, 3, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1,
3, 3, 3, 3, 3, 3, 1, 0, 0, 0, 0, 0, 0, 0, 5, 1, 1, 1, 1, 1, 3, 3,
3, 3, 3, 3, 4, 4, 4, 4, 4, 4, 4, 4, 1, 1, 1, 1, 1, 1, 3, 3, 3, 3,
3, 3, 4, 4, 4, 4, 4, 4, 4, 4, 1, 1, 1, 1, 1, 1, 3, 6, 3, 4, 3, 3,
0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 3, 3, 3, 3, 3, 3, 0, 0,
0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 3, 3, 3, 3, 3, 3, 4, 4, 4, 4,
4, 4, 4, 4, 6, 1, 1, 1, 1, 1, 3, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4, 4,
4, 4, 1, 1, 1, 1, 1, 1, 3, 3, 3, 3, 3, 3, 0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 3, 3, 3, 3, 3, 3, 1, 1, 1, 1, 3, 3, 3, 1, 1, 1,
1, 1, 1, 3, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4, 4, 4, 1, 1, 1, 1, 1, 1,
1, 4, 3, 3, 3, 3, 3, 1, 1, 1, 1, 3, 3, 3, 1, 1, 1, 1, 1, 1, 3, 3,
3, 3, 3, 3, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 3, 3, 3, 3,
3, 3, 1, 1, 1, 1, 3, 3, 3, 1, 1, 1, 6, 6, 1, 3, 3, 3, 3, 3, 3, 4,
4, 4, 4, 4, 4, 4, 1, 1, 1, 6, 1, 1, 3, 3, 3, 3, 3, 3, 1, 1, 1,
1, 3, 3, 3, 1, 1, 1, 1, 1, 6, 3, 3, 3, 3, 3, 3, 0, 0, 0, 0, 0, 0,
0, 0, 1, 1, 1, 1, 1, 1, 3, 3, 3, 3, 3, 3, 1, 1, 1, 1, 3, 3, 3, 1,
1, 1, 1, 1, 1, 3, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4, 4, 4, 1, 1, 1,
1, 1, 1, 3, 3, 3, 3, 3, 3, 1, 1, 1, 1, 3, 1, 3, 1, 1, 1, 1, 1, 1,
3, 3, 3, 3, 3, 3, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 3, 3,
3, 3, 3, 3, 1, 1, 1, 1, 3, 3, 3, 1, 1, 1, 1, 1, 1, 3, 3, 3, 3, 3,
3, 4, 4, 4, 4, 1, 4, 4, 4, 5, 1, 1, 1, 1, 1, 3, 3, 3, 3, 3, 3, 1,
5, 1, 1, 3, 3, 3, 1, 1, 1, 6, 1, 1, 3, 3, 3, 3, 3, 3, 0, 0, 0, 0,
0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 3, 3, 3, 3, 3, 3, 1, 1, 1, 1, 3, 3,
3, 1, 1, 1, 1, 1, 1, 3, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4, 4, 4, 4, 1,
1, 1, 1, 1, 1, 3, 3, 3, 3, 3, 1, 1, 1, 1, 1, 3, 3, 3, 1, 1, 1, 1,
1, 1, 3, 3, 5, 5, 5, 5, 6, 6, 6, 6, 6, 6, 5, 5, 0, 3, 2, 3, 3, 0,
7, 7, 1, 7, 7, 7, 0, 0, 0, 0, 7, 7, 7])),
'metrics': {'n_clusters': 8,
'n_noise': 0,
'silhouette': 0.549979,
'davies_bouldin': 3.389443,
'calinski_harabasz': 13.5509}}

```

Análisis de migración de clústeres

Se compara cómo los datos migran entre diferentes valores de k.

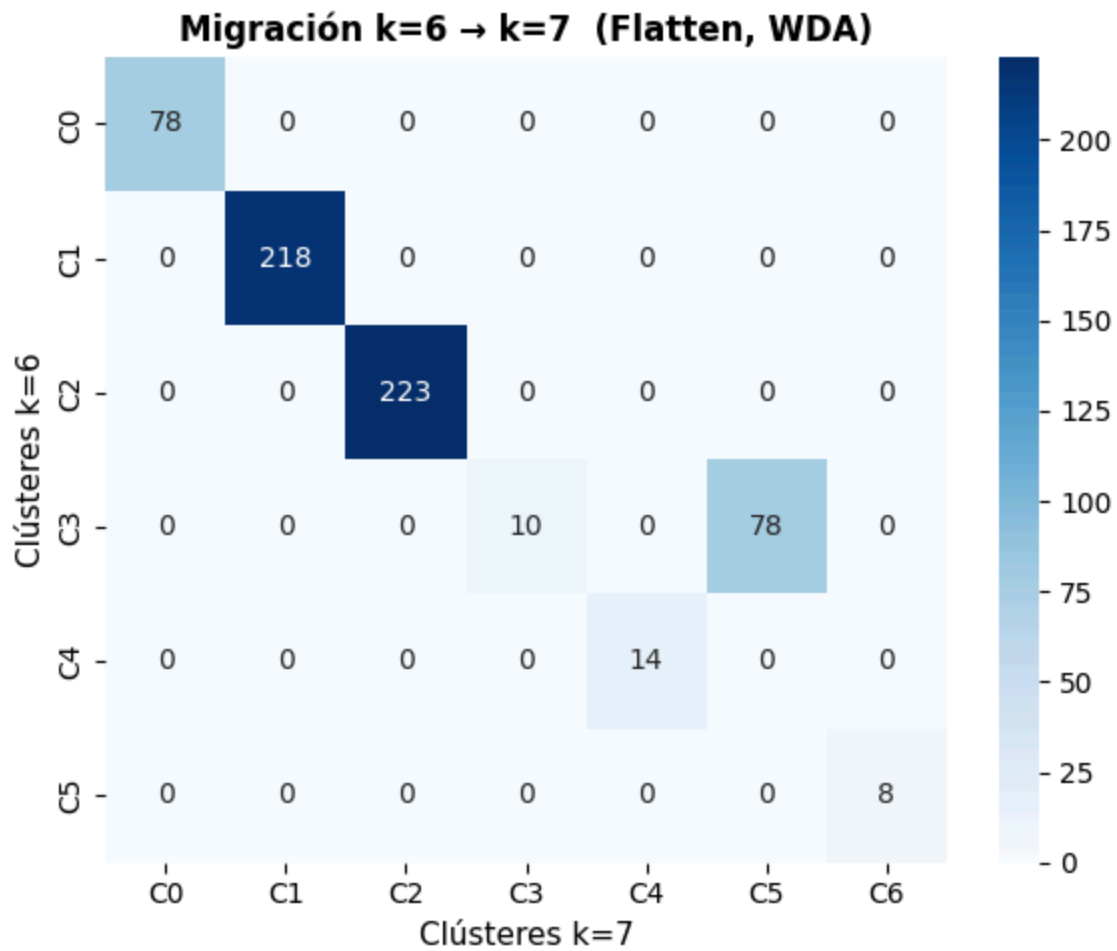
```

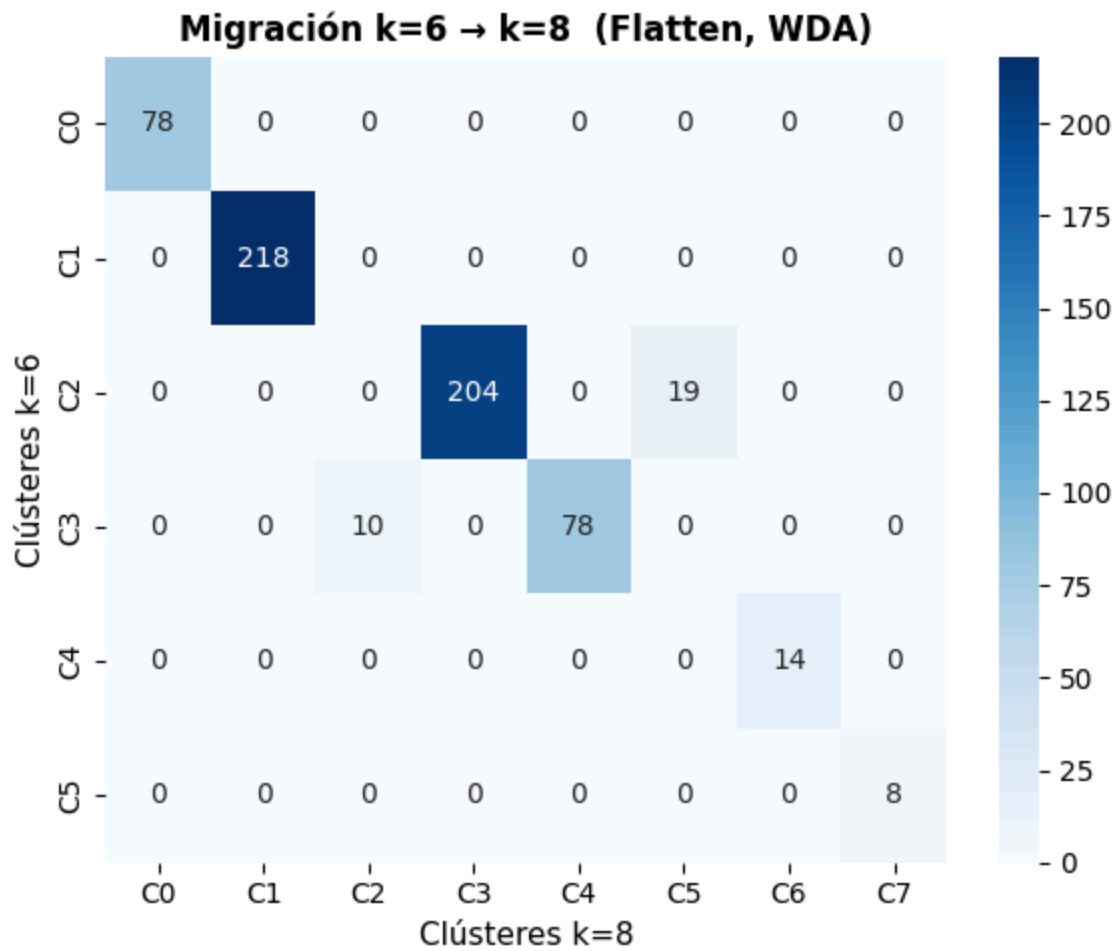
In [51]: print(f'Muestras en variables - k=3: {labels_flatten_k6_wd.shape}')
print(f'Muestras en variables - k=4: {labels_flatten_k7_wd.shape}')
print(f'Muestras en variables - k=5: {labels_flatten_k8_wd.shape}')

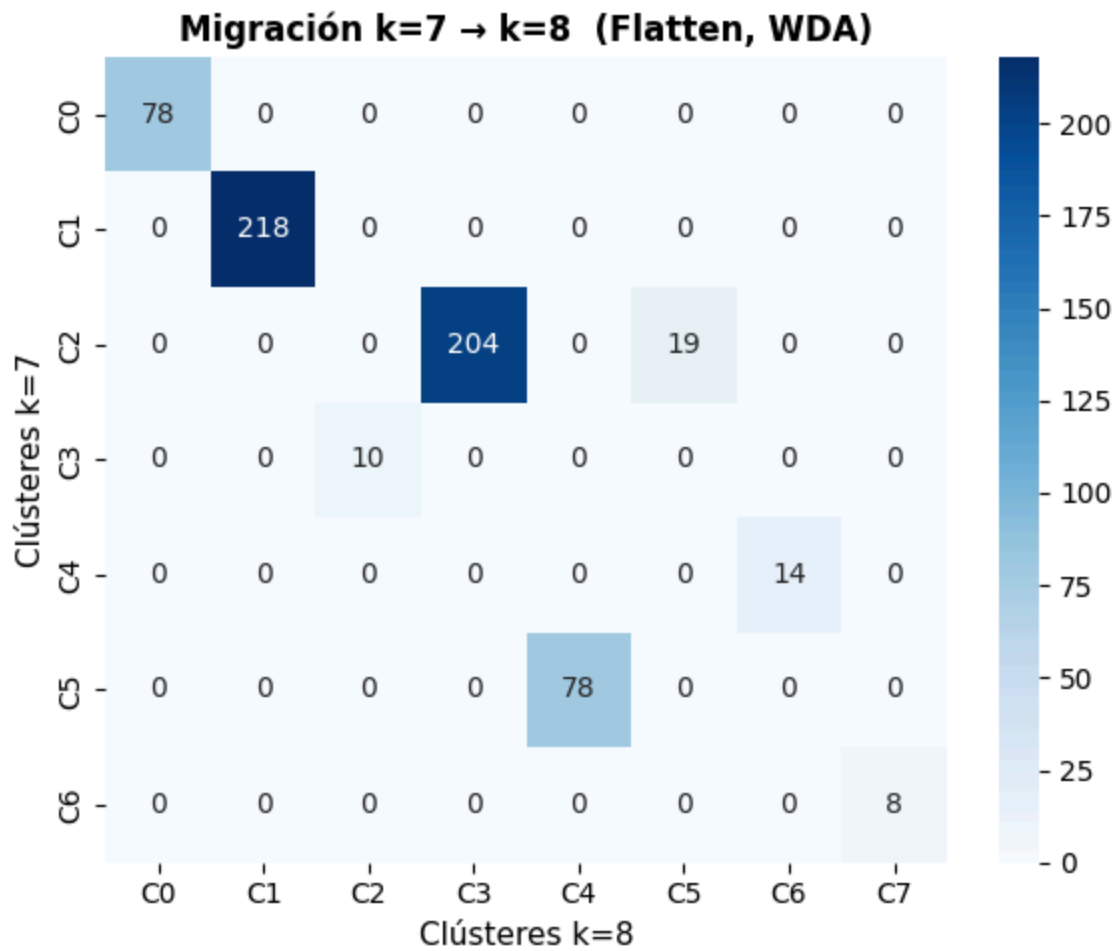
plot_migration_matrix(labels_flatten_k6_wd, labels_flatten_k7_wd, ka=6, kb=7)
plot_migration_matrix(labels_flatten_k6_wd, labels_flatten_k8_wd, ka=6, kb=8)
plot_migration_matrix(labels_flatten_k7_wd, labels_flatten_k8_wd, ka=7, kb=8)

Muestras en variables - k=3: (629,)
Muestras en variables - k=4: (629,)
Muestras en variables - k=5: (629,)

```





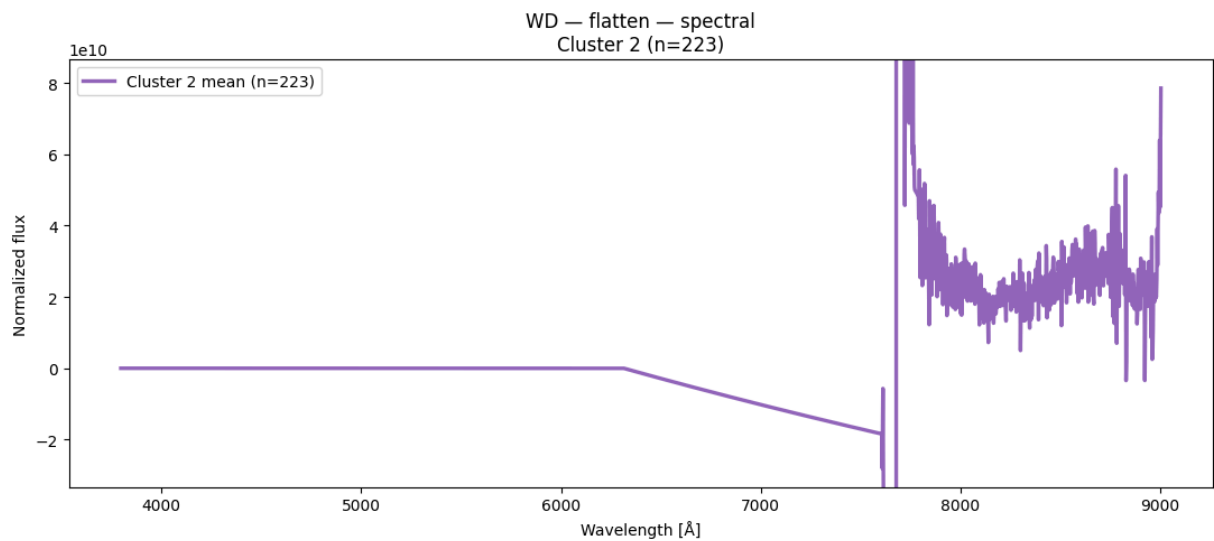
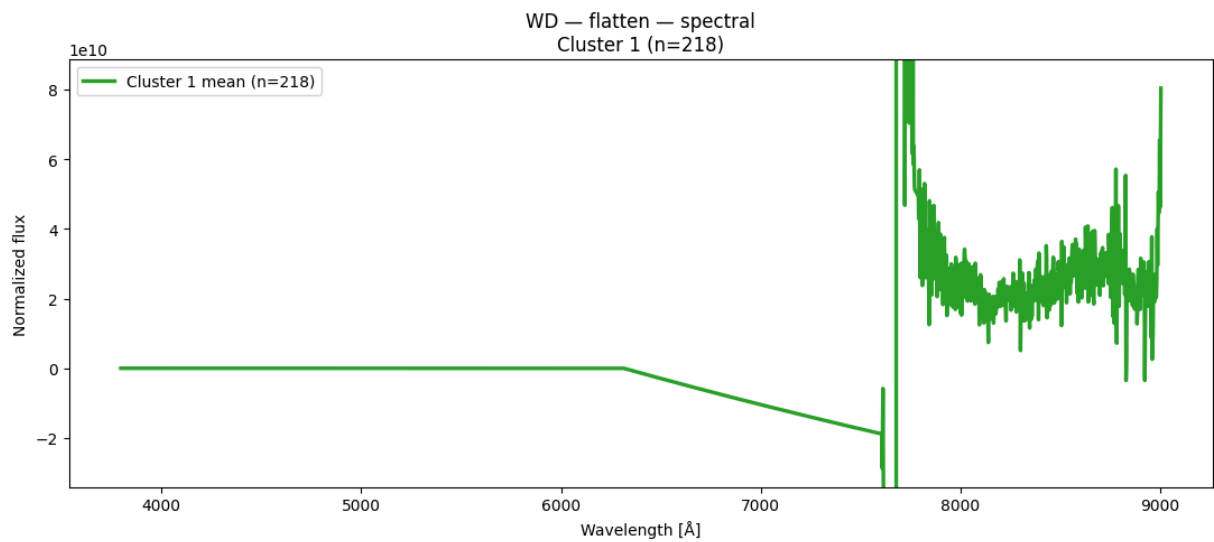
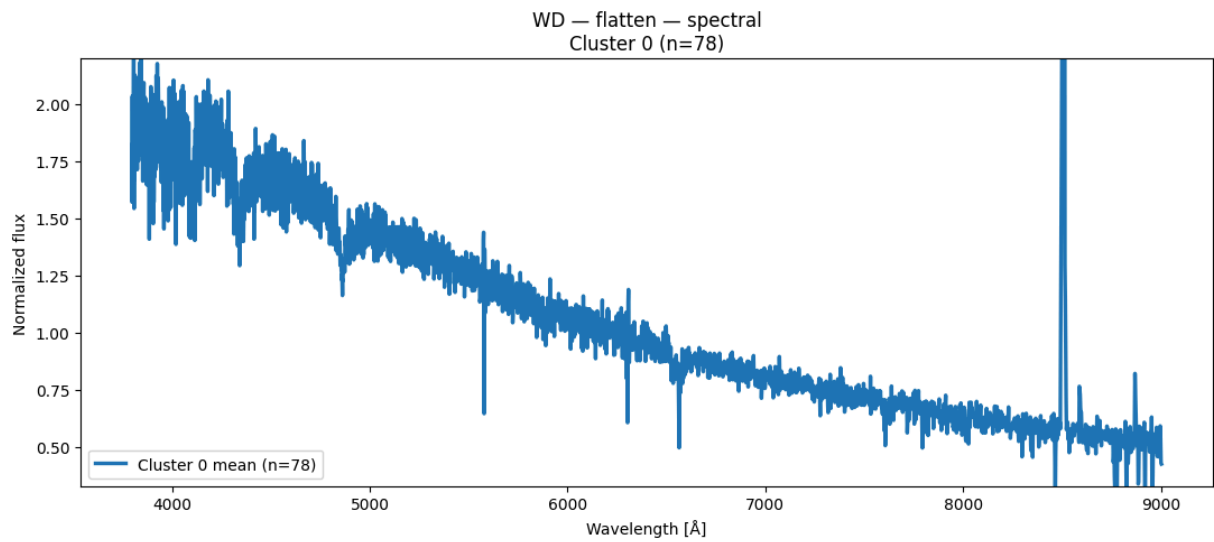


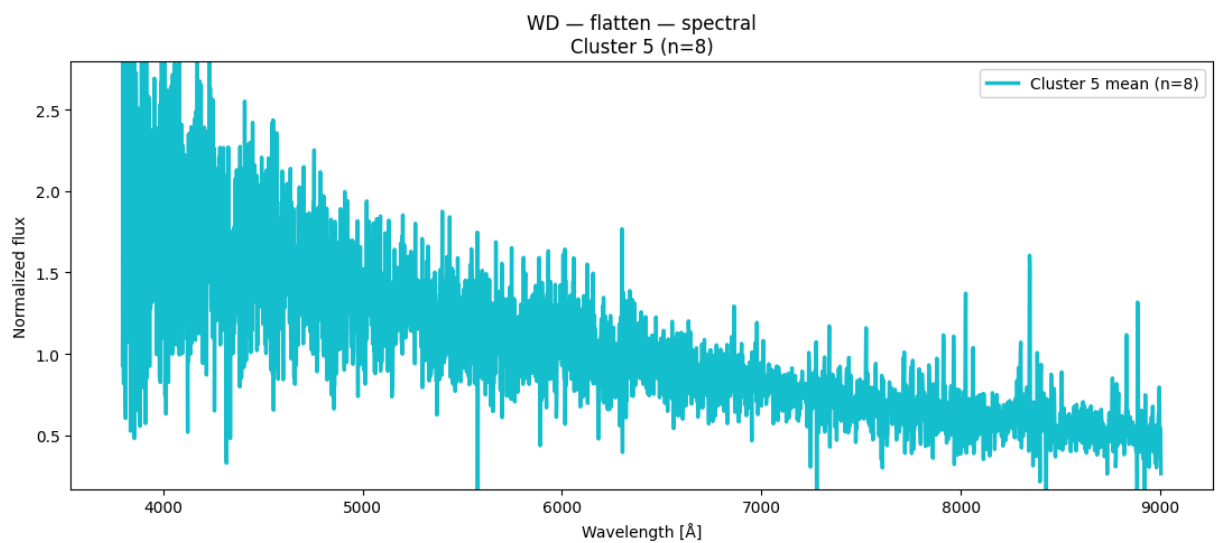
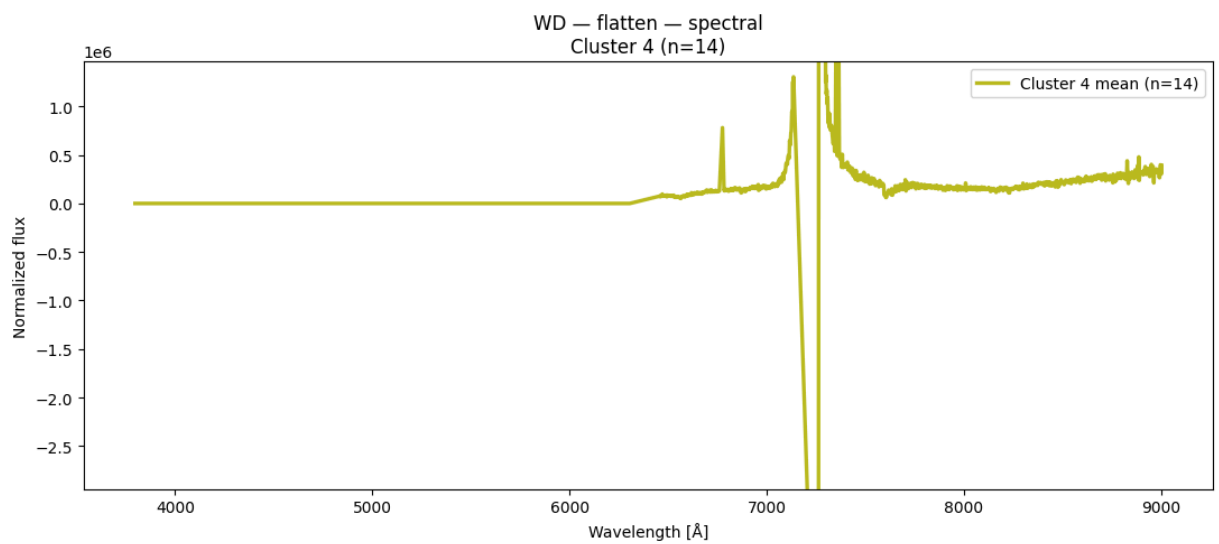
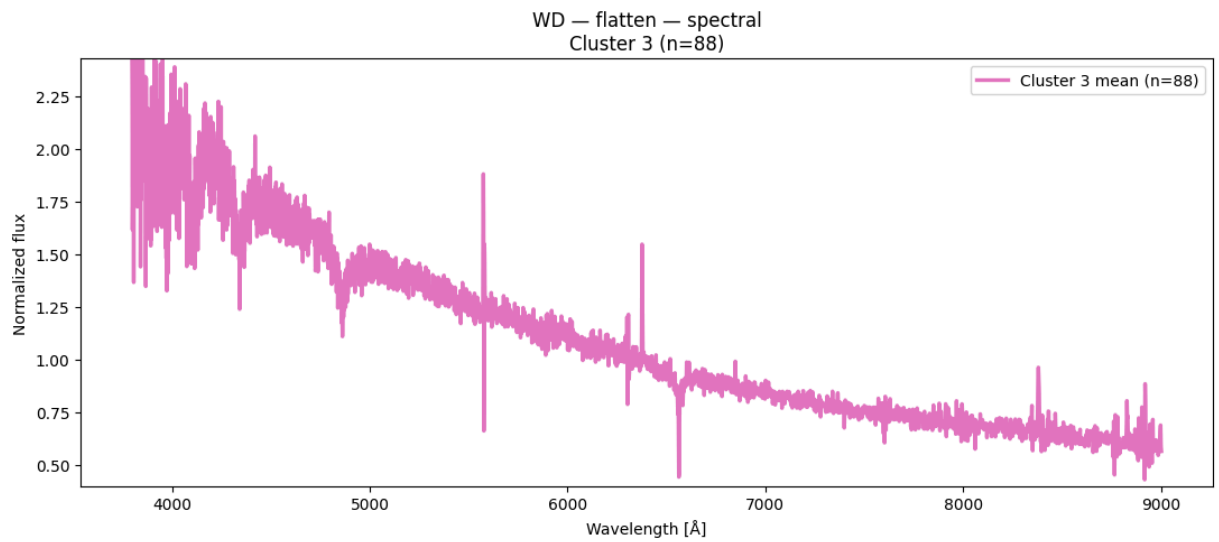
Espectros obtenidos de Spectral Clustering para k=6 y n=629

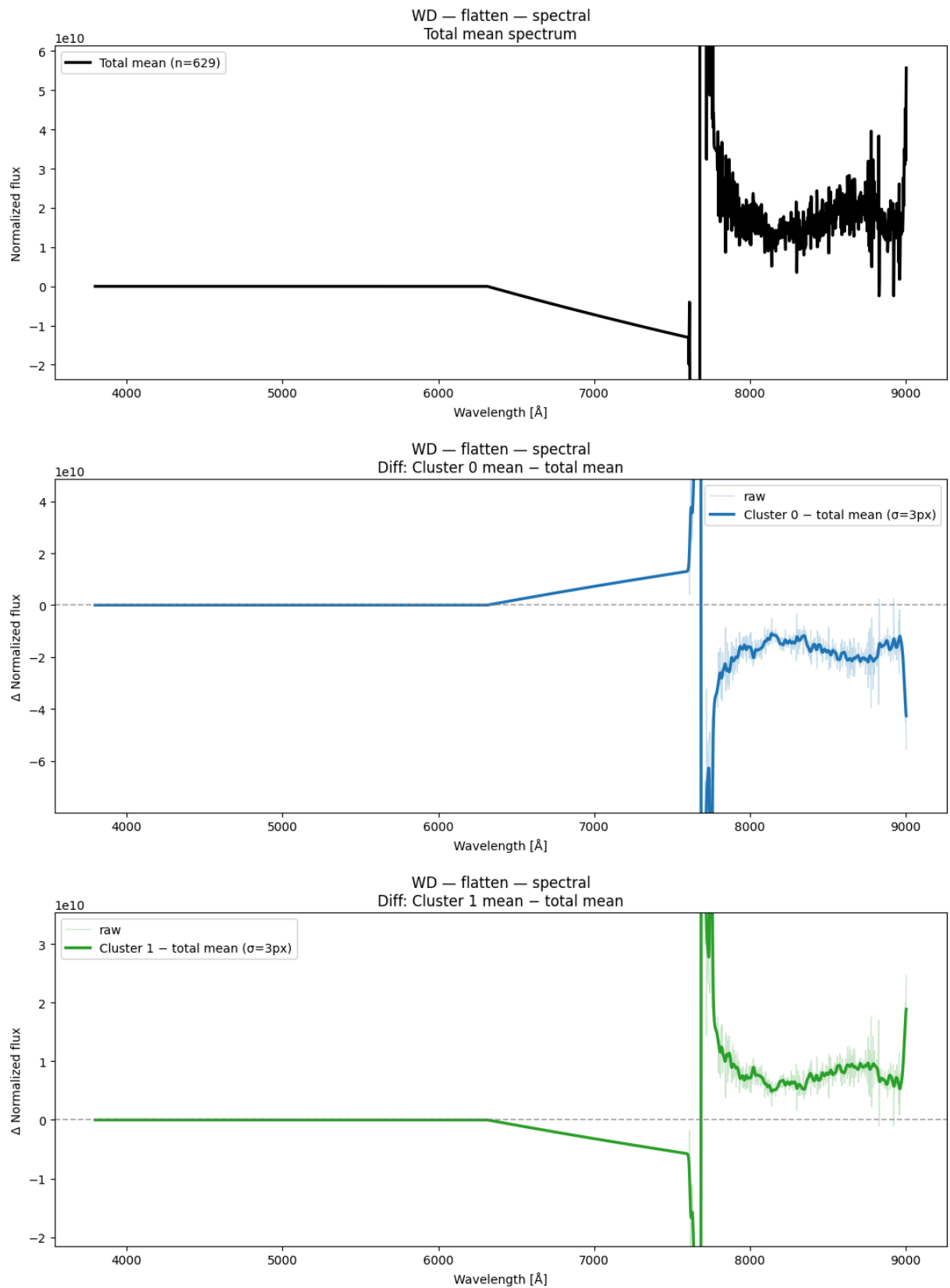
```
In [52]: result = plot_spectra_spectral(X_wd_spec, labels_flatten_k6_wd, W_wd[0],
                                         class_name="WD", layer_name="flatten")
```

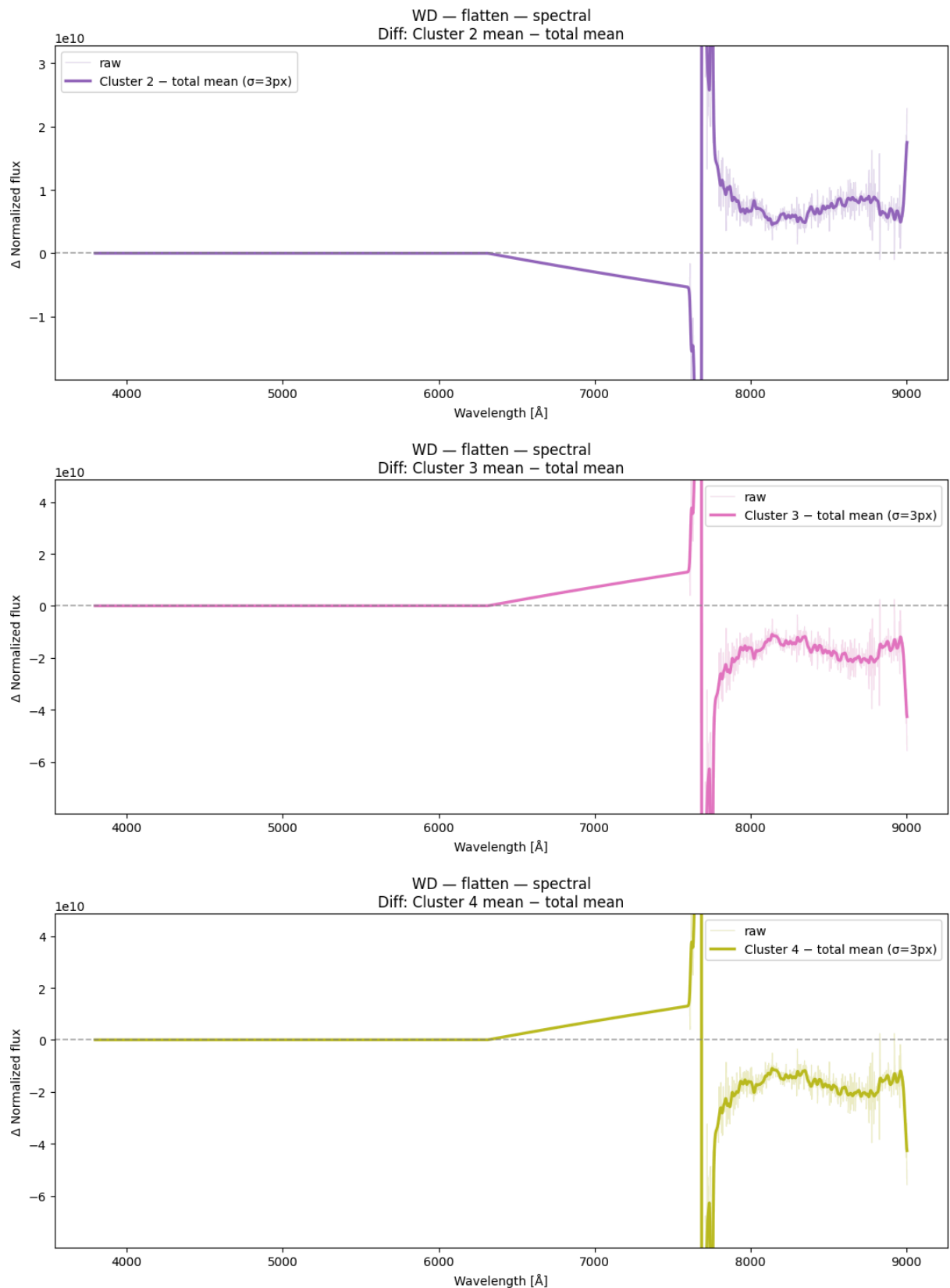
C:\Users\javip\AppData\Local\Temp\ipykernel_52368\531566132.py:28: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get\_cmap()`` or ``pyplot.get\_cmap()`` instead.

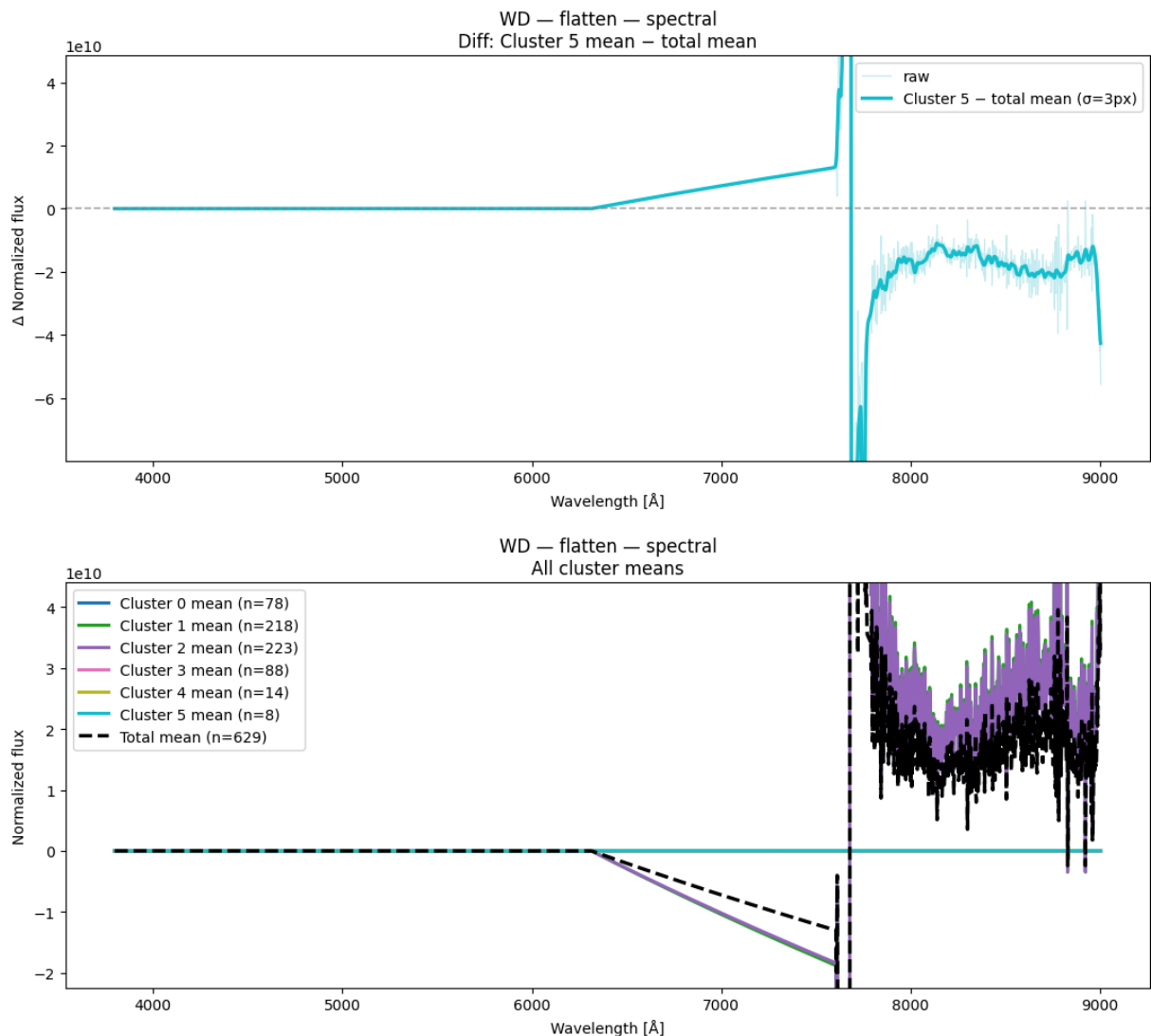
```
cmap = plt.cm.get_cmap("tab10", n_clusters)
```









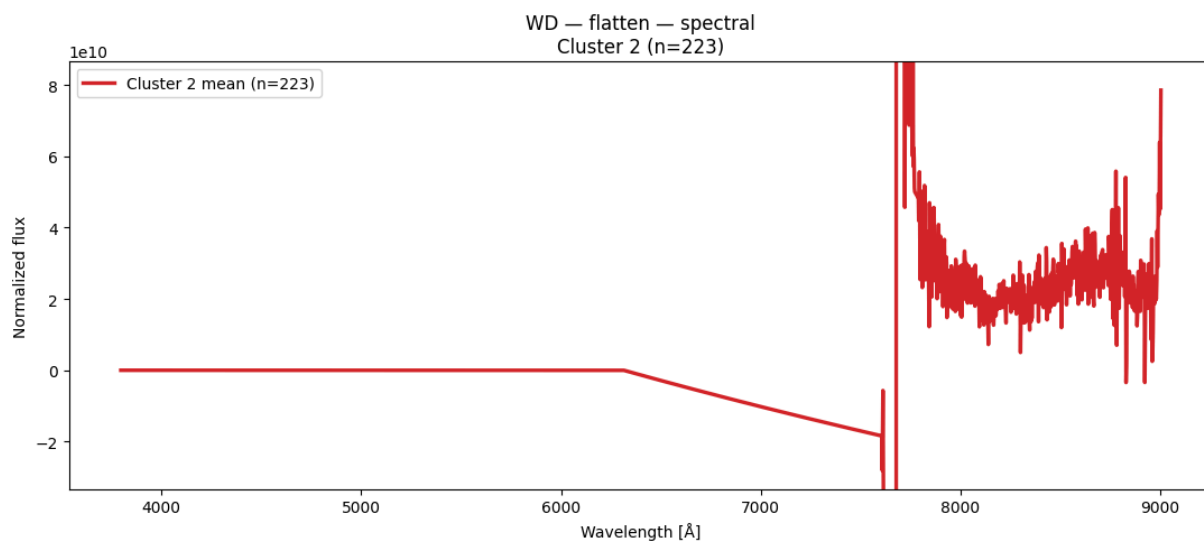
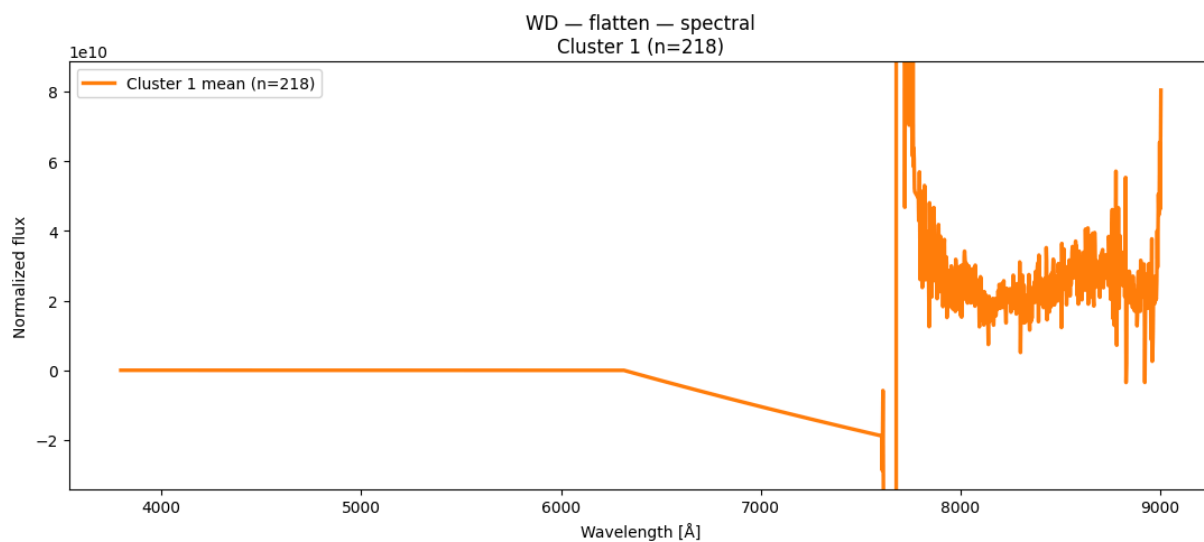
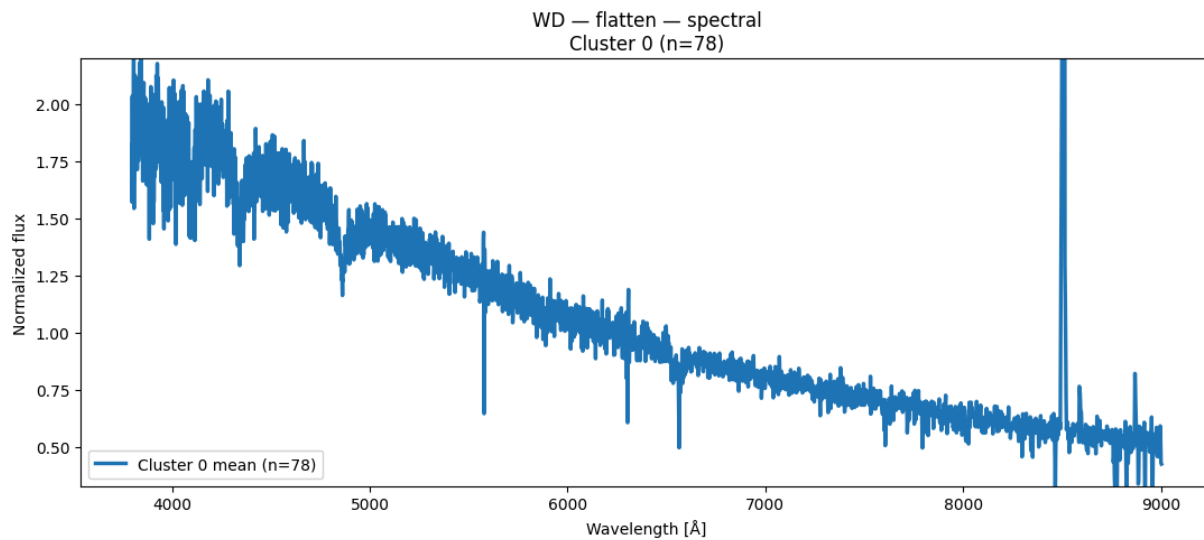


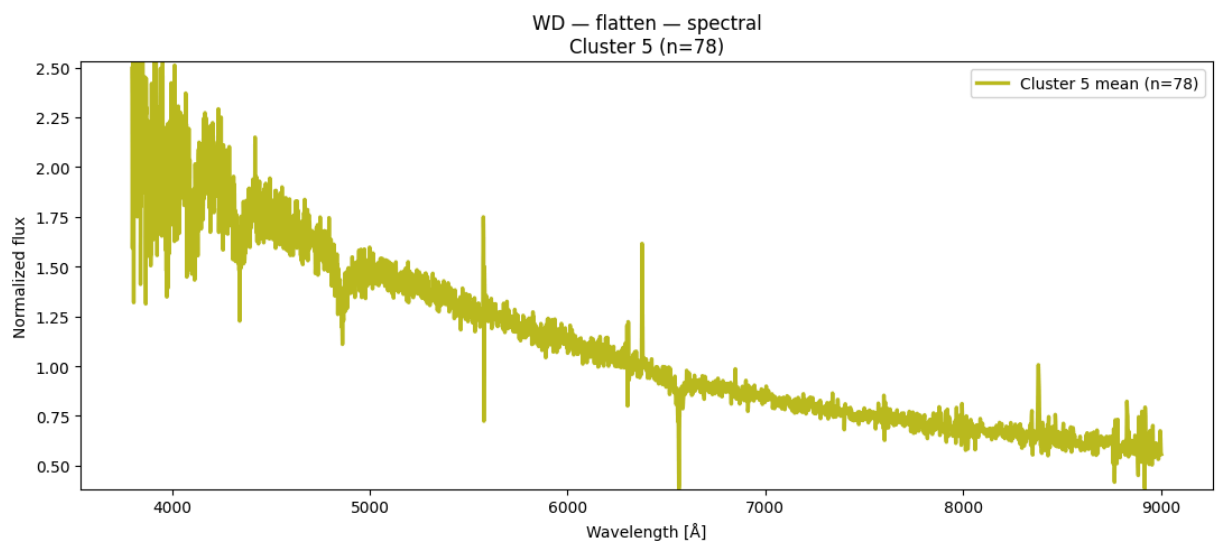
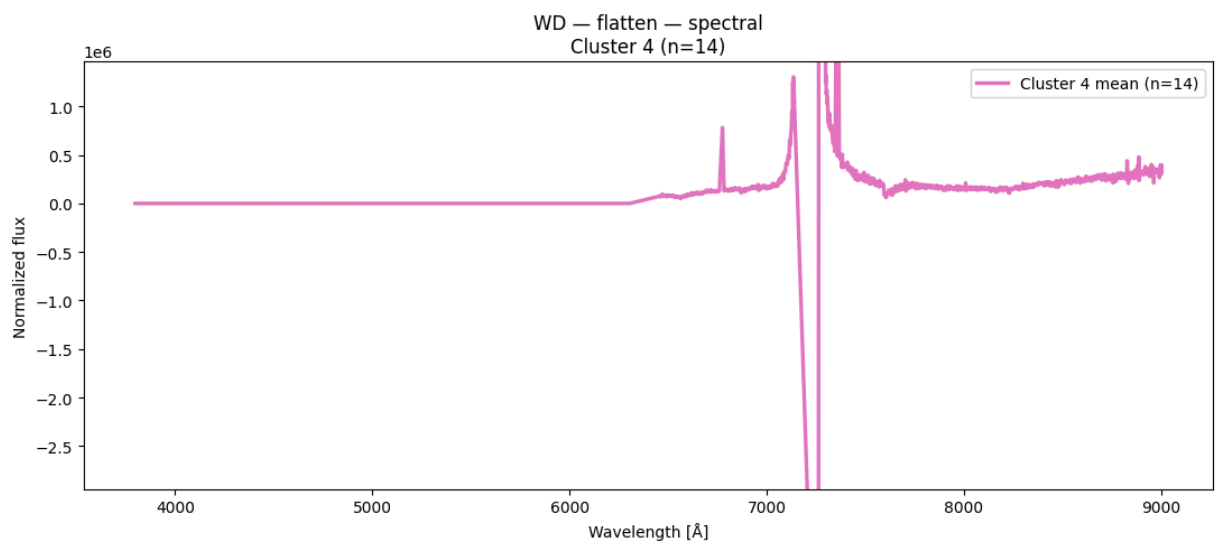
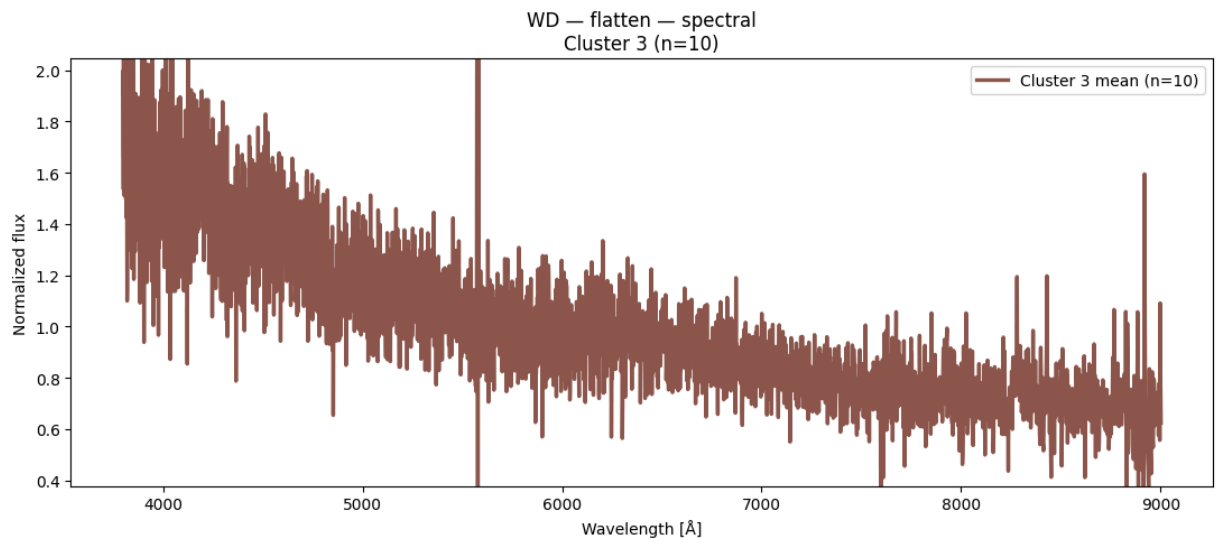
Espectros obtenidos de Spectral Clustering para $k=7$ y $n=629$

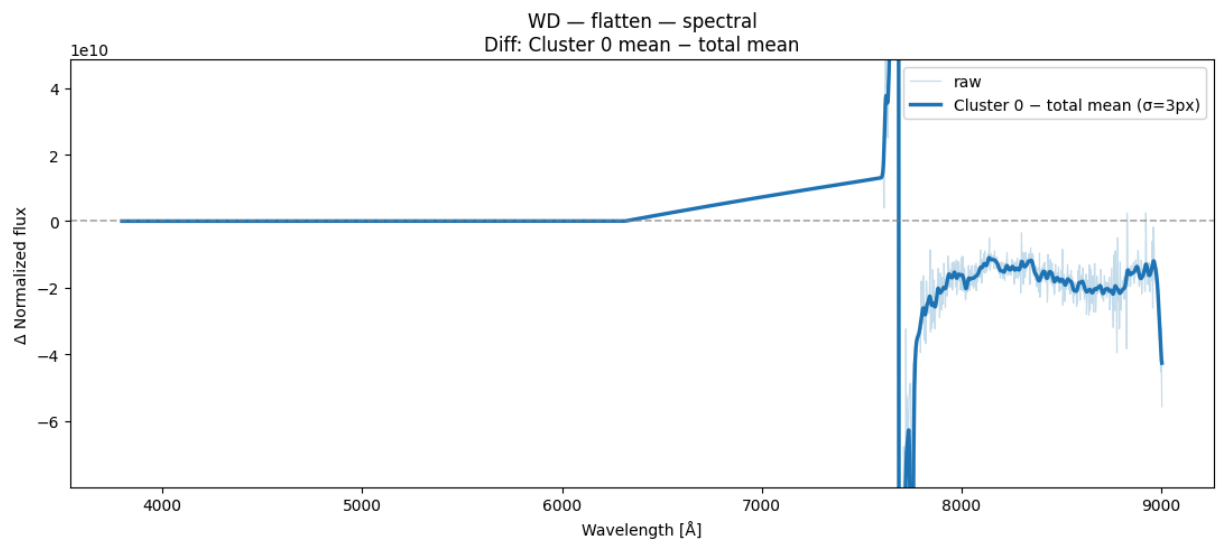
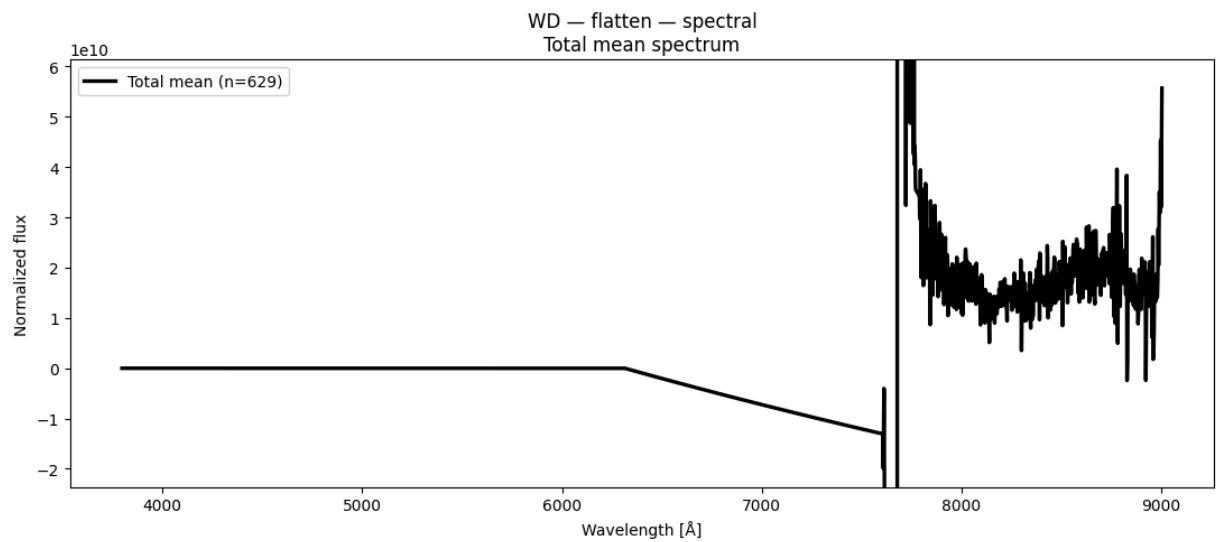
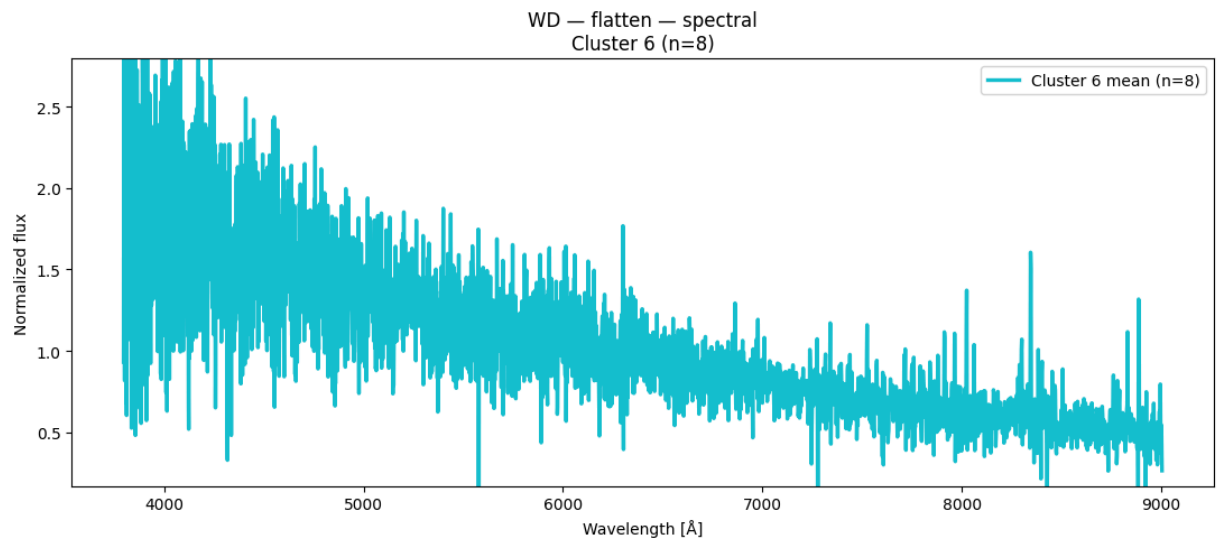
```
In [53]: plot_spectra_spectral(
    X_wd_spec,
    labels_flatten_k7_wd,
    W_wd[0],
    class_name="WD",
    layer_name="flatten",
)
```

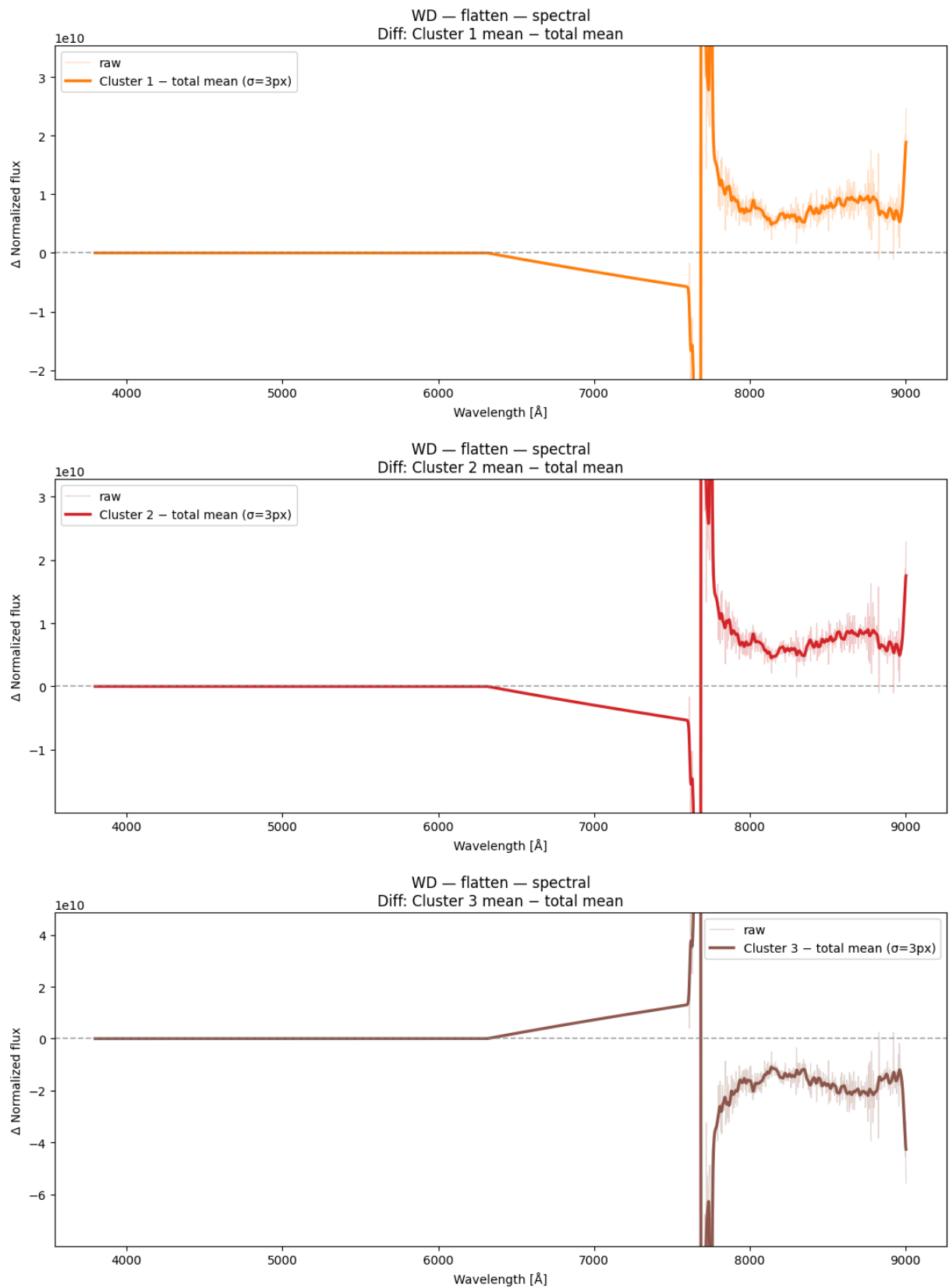
C:\Users\javip\AppData\Local\Temp\ipykernel_52368\531566132.py:28: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get\_cmap()`` or ``pyplot.get\_cmap()`` instead.

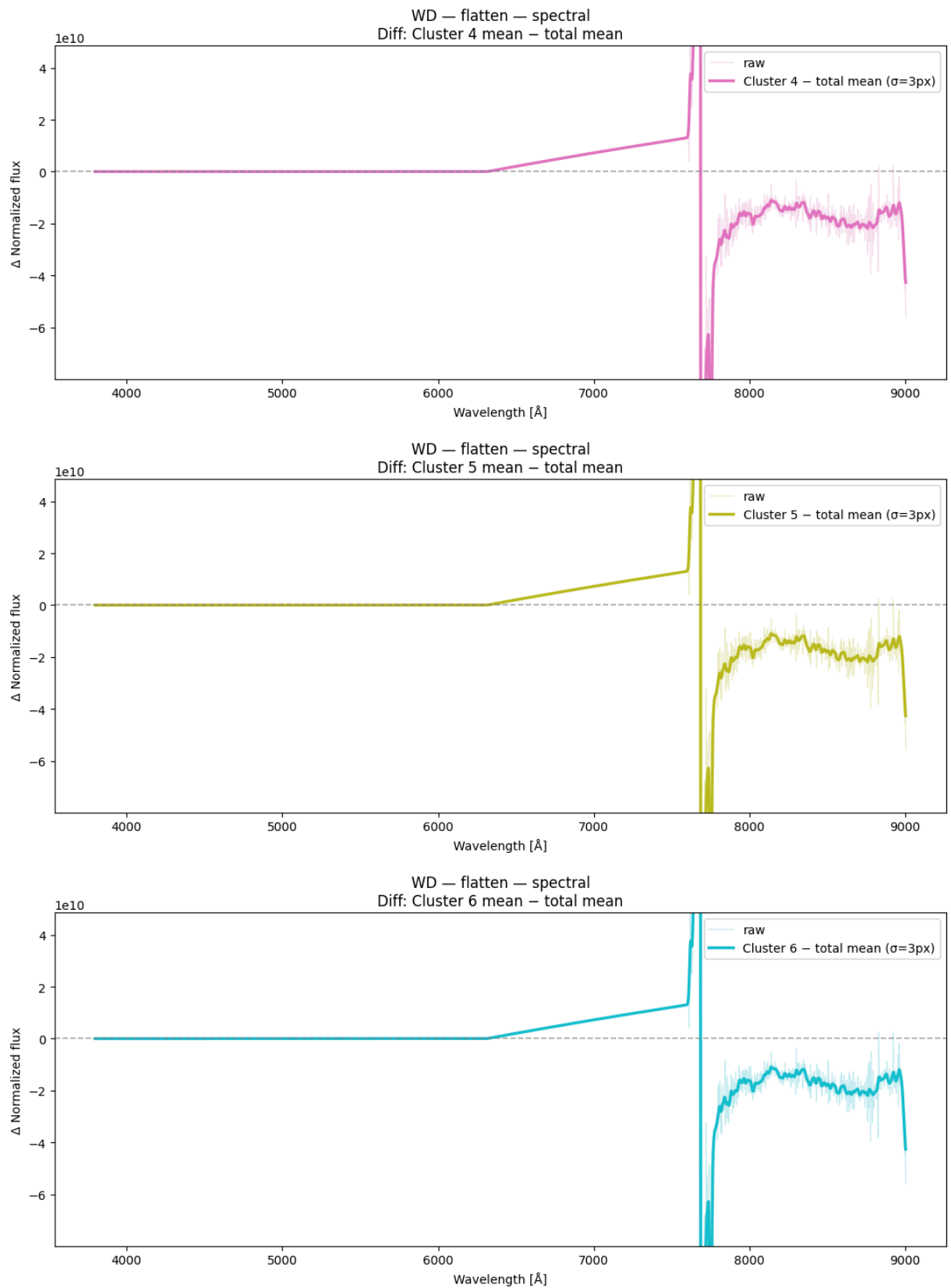
```
cmap = plt.cm.get_cmap("tab10", n_clusters)
```

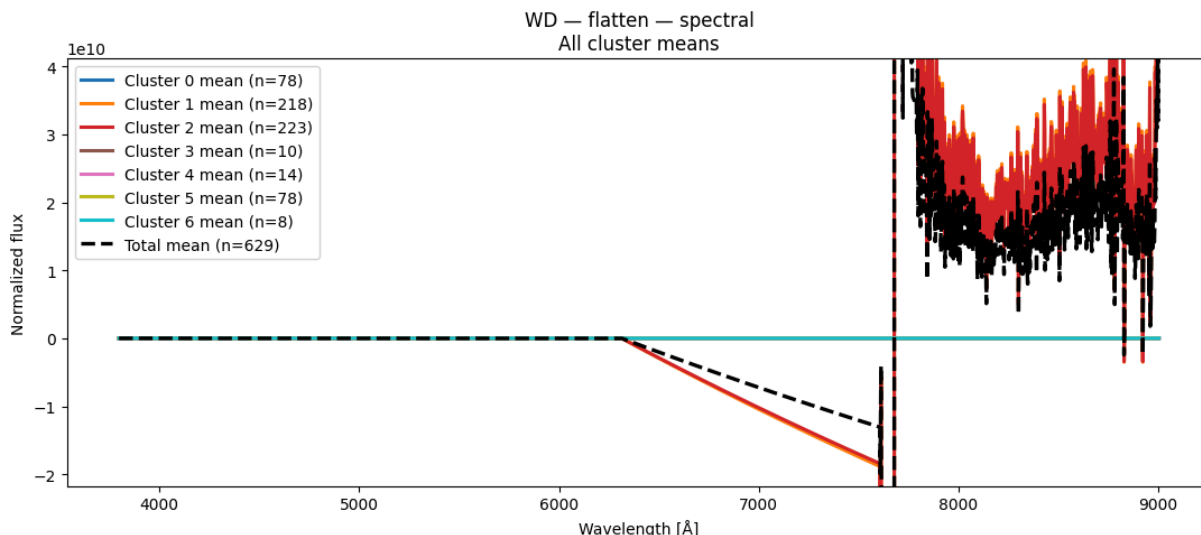












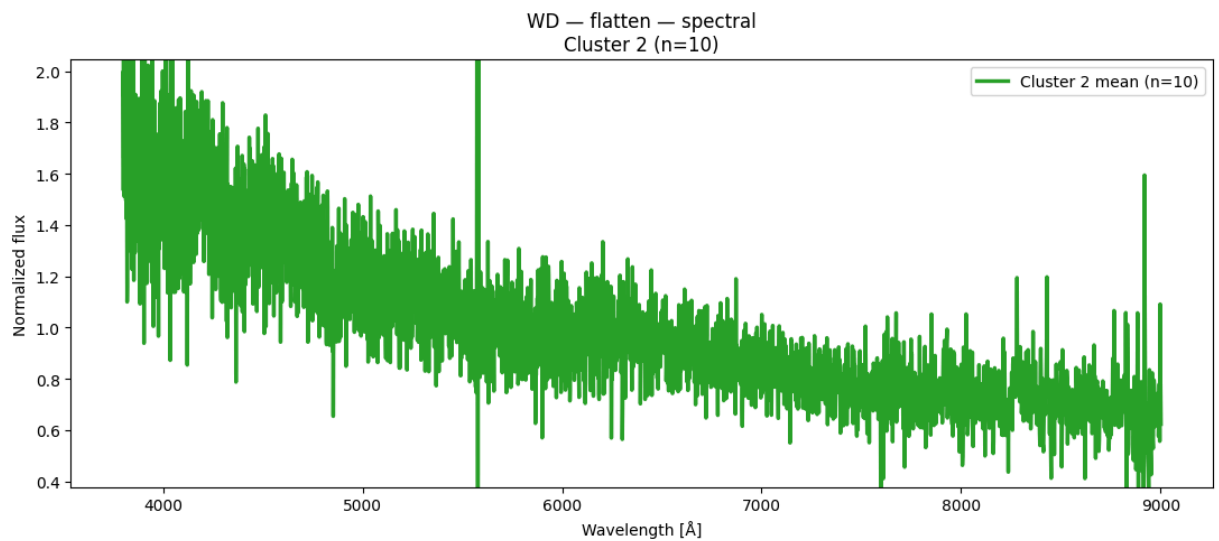
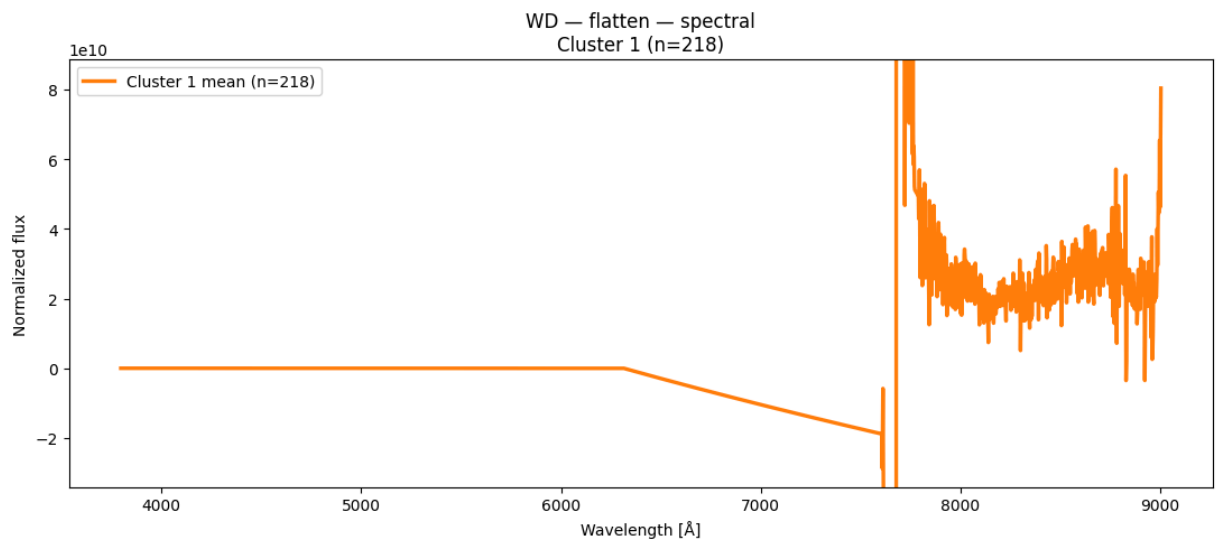
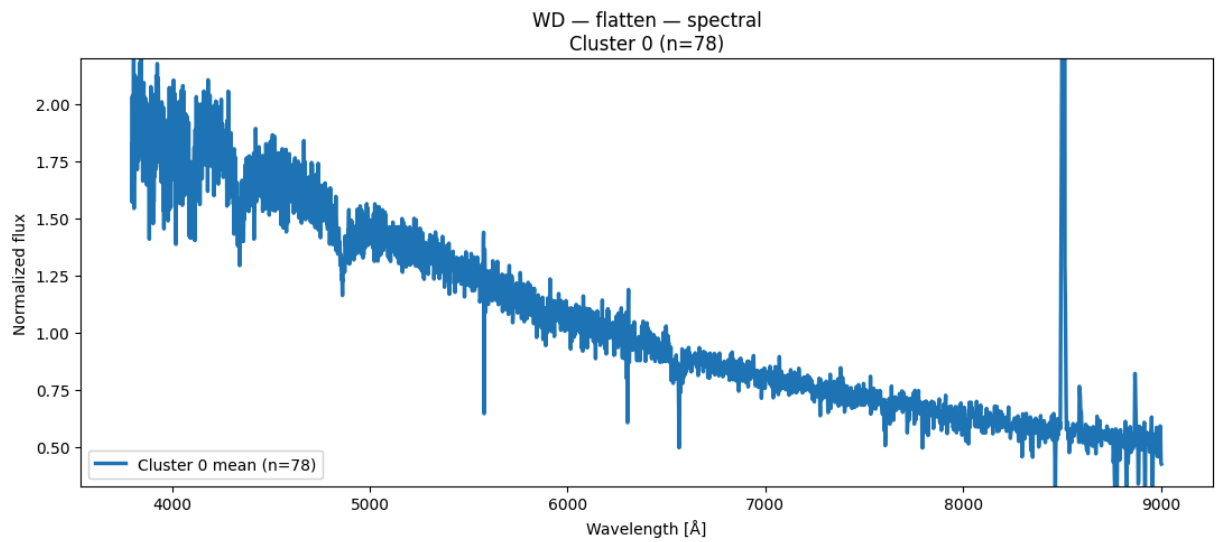
```
Out[53]: {'total_mean': array([1.8663167e+00, 1.4631674e+00, 2.3618588e+00, ..., 3.2000662e
+10,
      4.3846500e+10, 5.5692263e+10], dtype=float32),
  'group_means': {0: array([1.8198066 , 1.5705259 , 2.0340953 , ..., 0.53136235, 0.
42942575,
      0.42539307], dtype=float32),
    1: array([1.9775230e+00, 2.2498171e+00, 2.1407335e+00, ..., 4.6166102e+10,
      6.3255630e+10, 8.0345039e+10], dtype=float32),
    2: array([2.4182165e+00, 2.2892640e+00, 2.5385358e+00, ..., 4.5130957e+10,
      6.1837300e+10, 7.8543536e+10], dtype=float32),
    3: array([1.940815 , 1.9095039, 1.997124 , ..., 1.0910794, 0.780286 ,
      0.6237645], dtype=float32),
    4: array([-1.1421267e+01, -3.0009708e+01, 6.1820192e+00, ...,
      3.9323972e+05, 3.0205722e+05, 3.9240778e+05], dtype=float32),
    5: array([2.4954133 , 2.3517096 , 2.0646846 , ..., 0.6370731 , 0.56343603,
      0.5575099 ], dtype=float32),
    6: array([0.9316737 , 1.809056 , 3.3264894 , ..., 0.54613864, 0.48039228,
      0.26362562], dtype=float32)},
  'unique_clusters': array([0, 1, 2, 3, 4, 5, 6])}
```

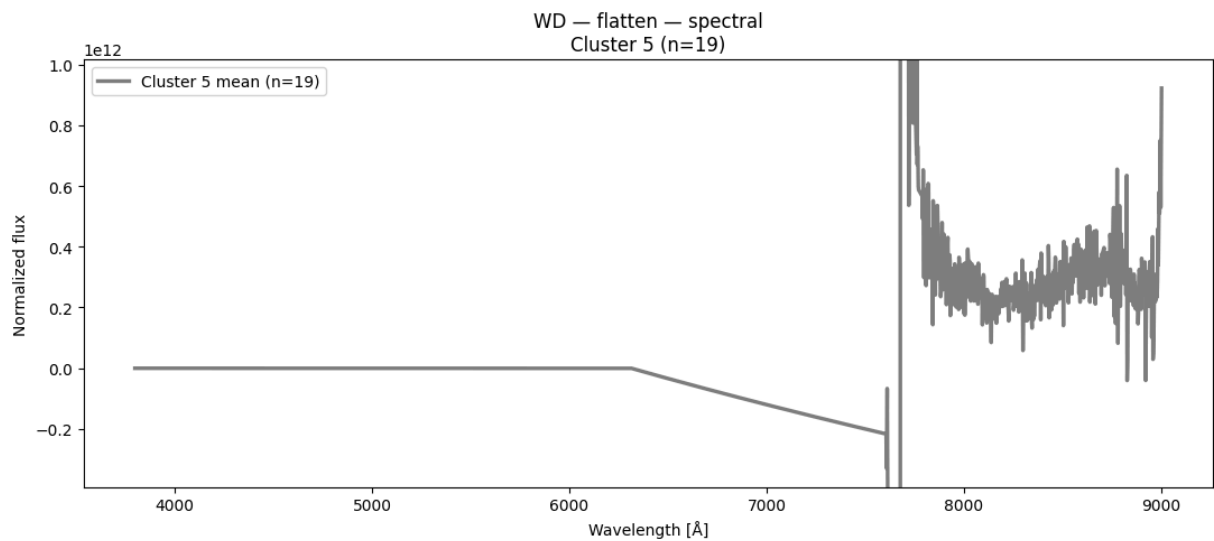
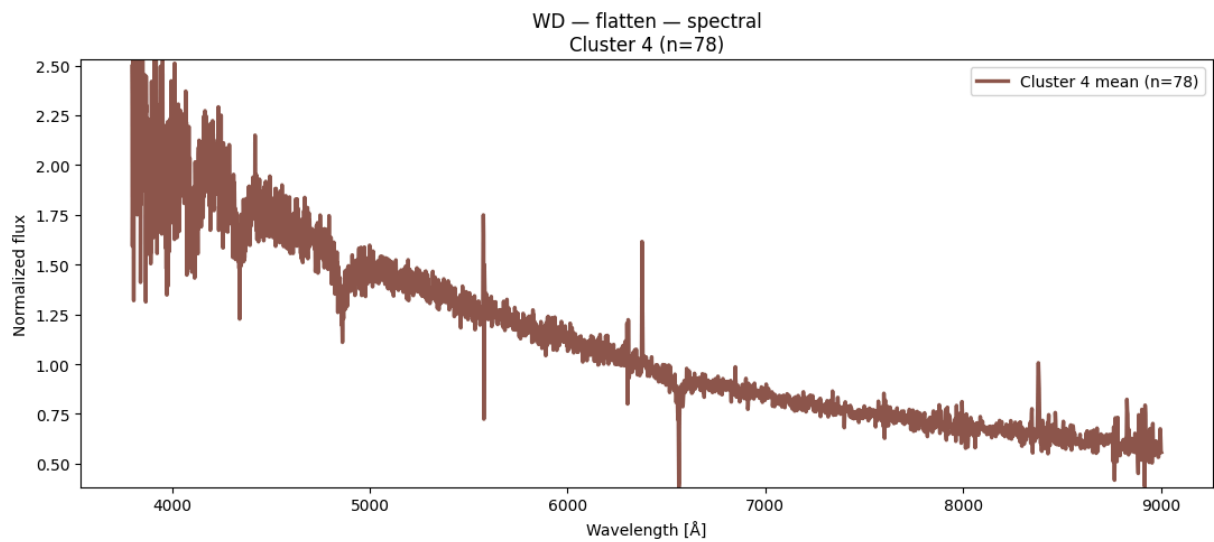
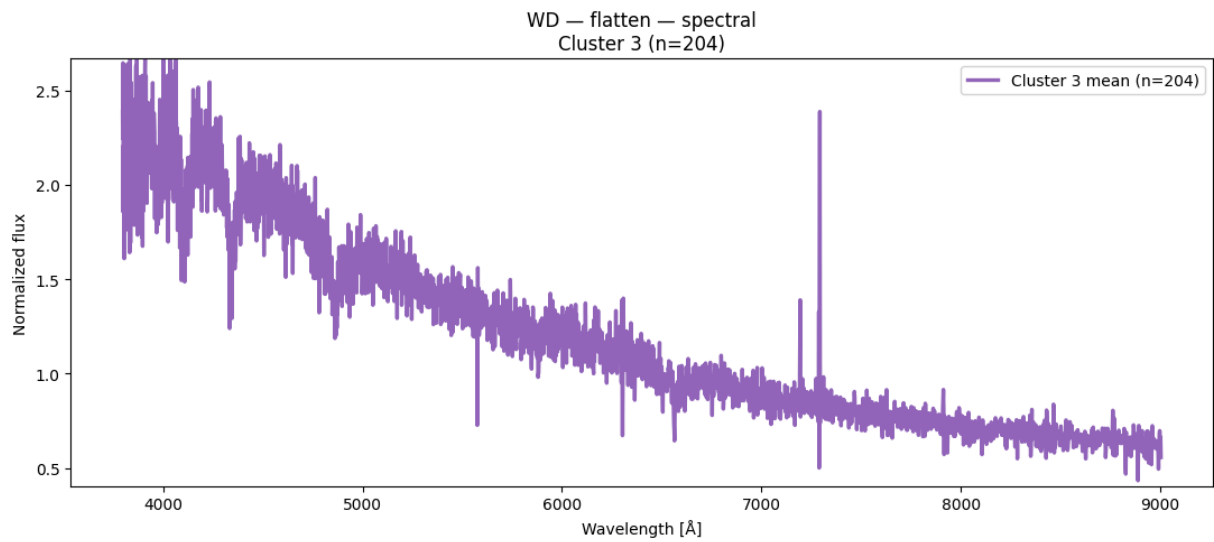
Espectros obtenidos de Spectral Clustering para k=8 y n=629

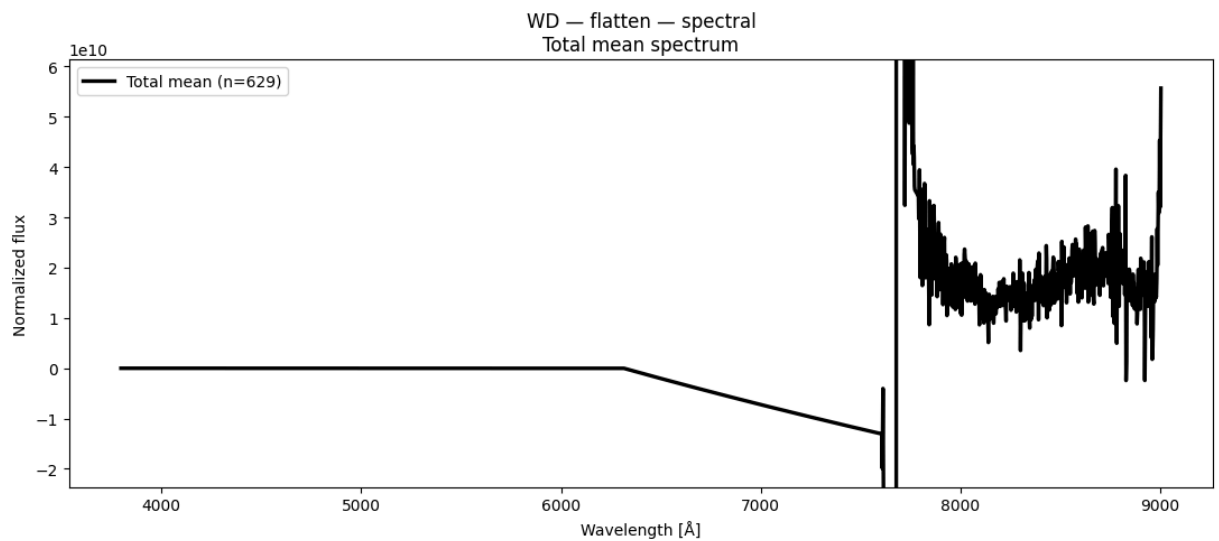
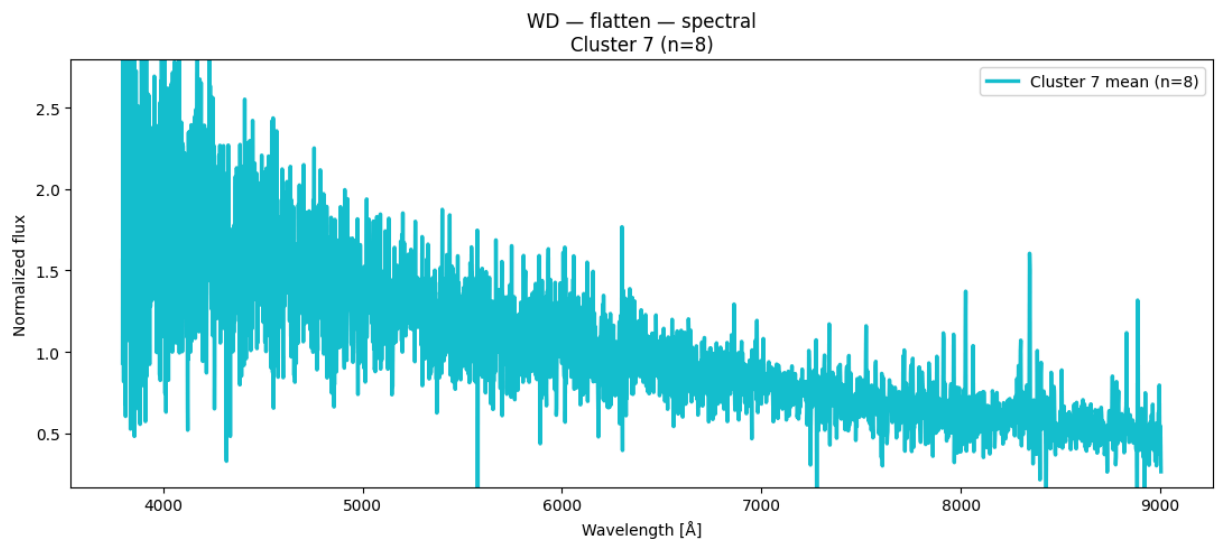
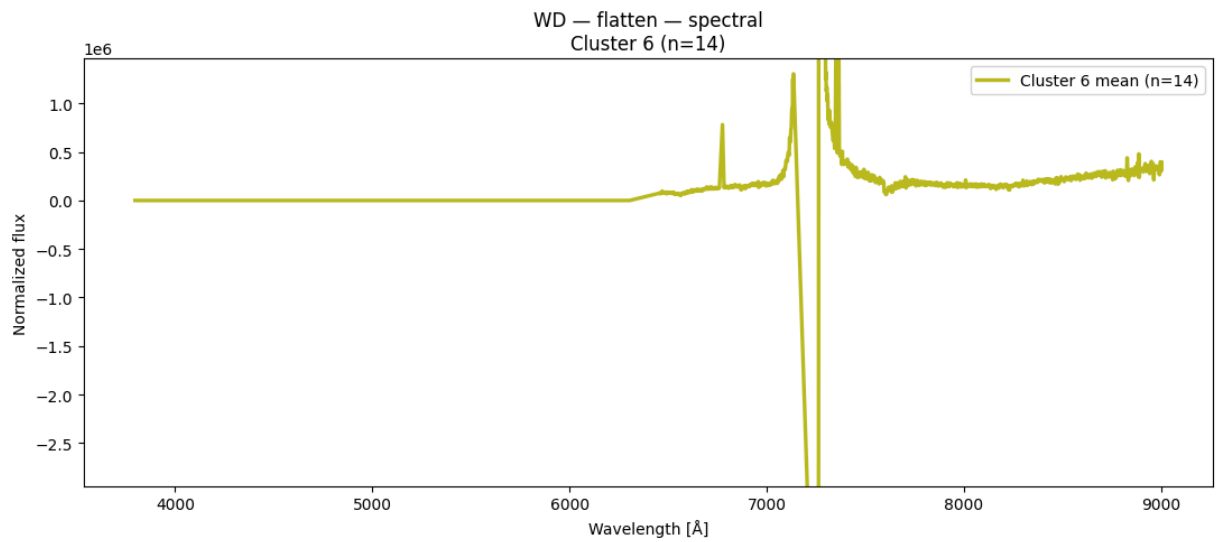
```
In [54]: plot_spectra_spectral(
    X_wd_spec,
    labels_flatten_k8_wd,
    W_wd[0],
    class_name="WD",
    layer_name="flatten"
)
```

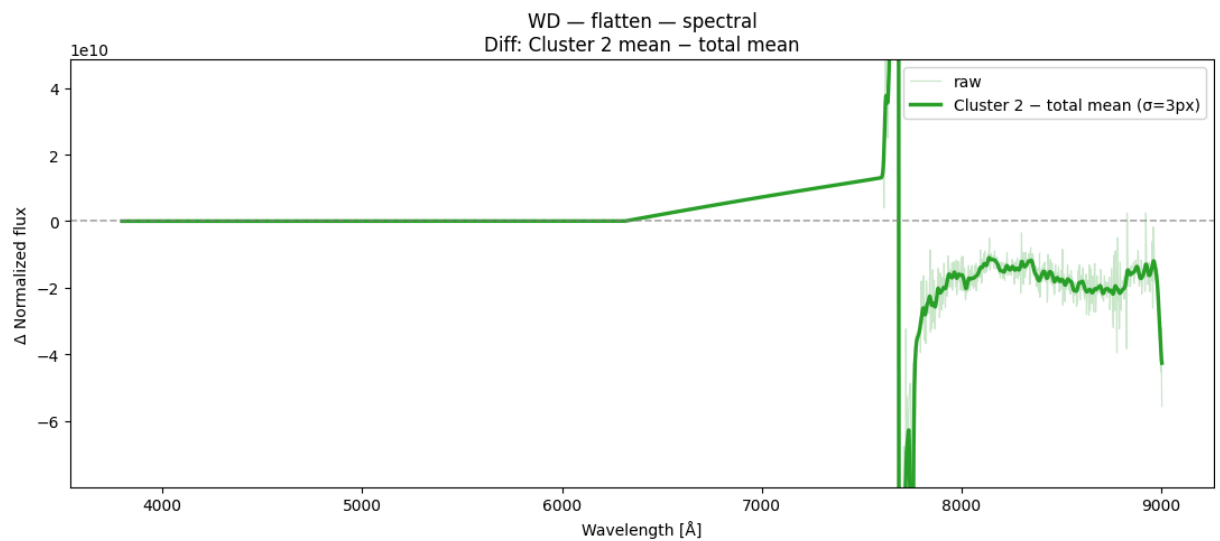
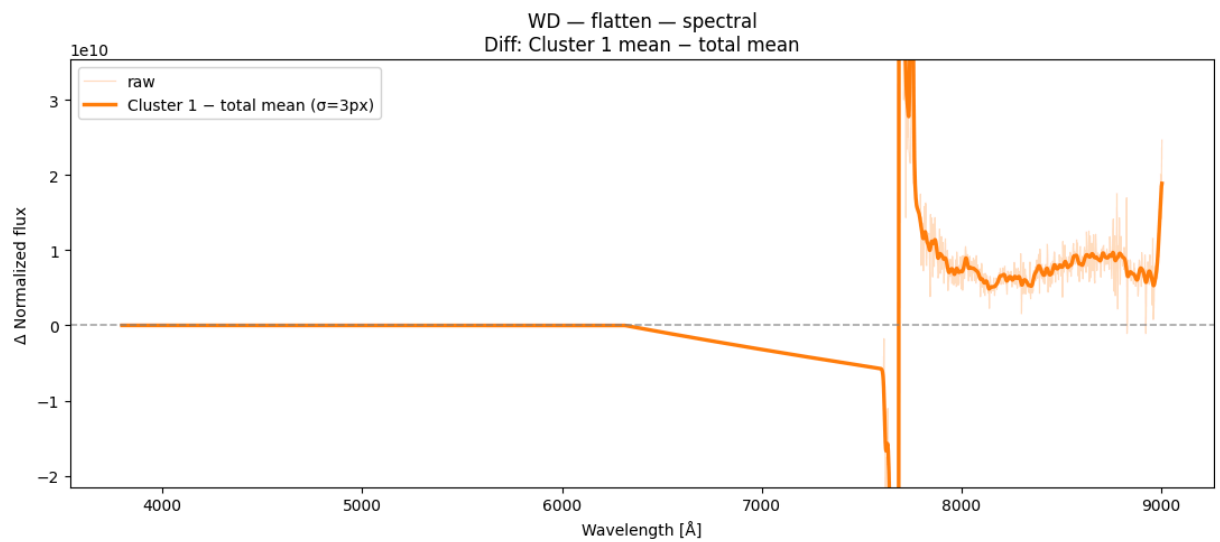
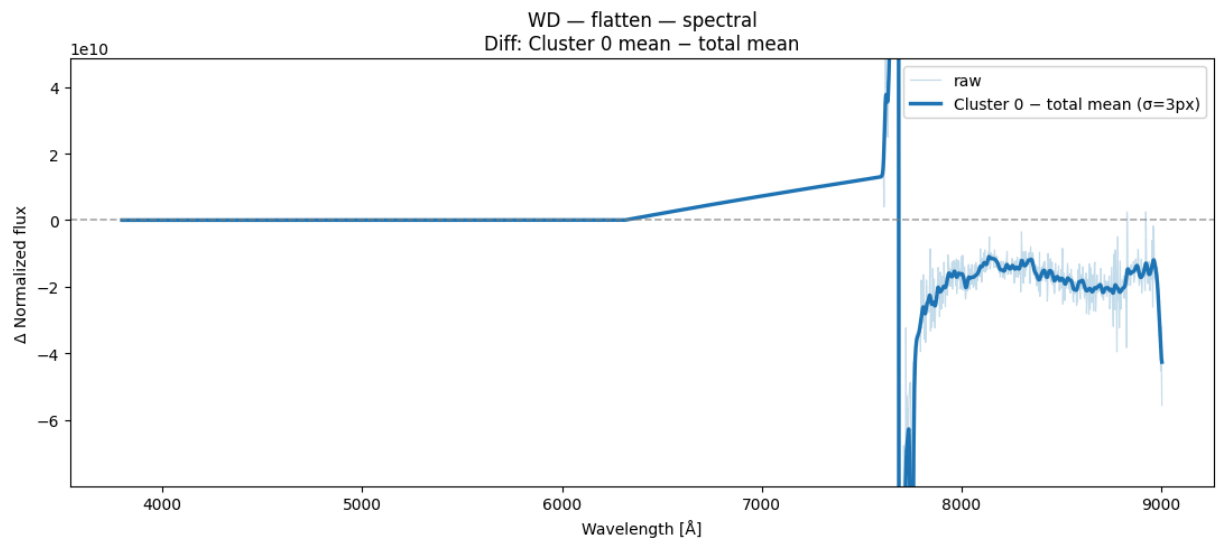
C:\Users\javip\AppData\Local\Temp\ipykernel_52368\531566132.py:28: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get\_cmap()`` or ``pyplot.get\_cmap()`` instead.

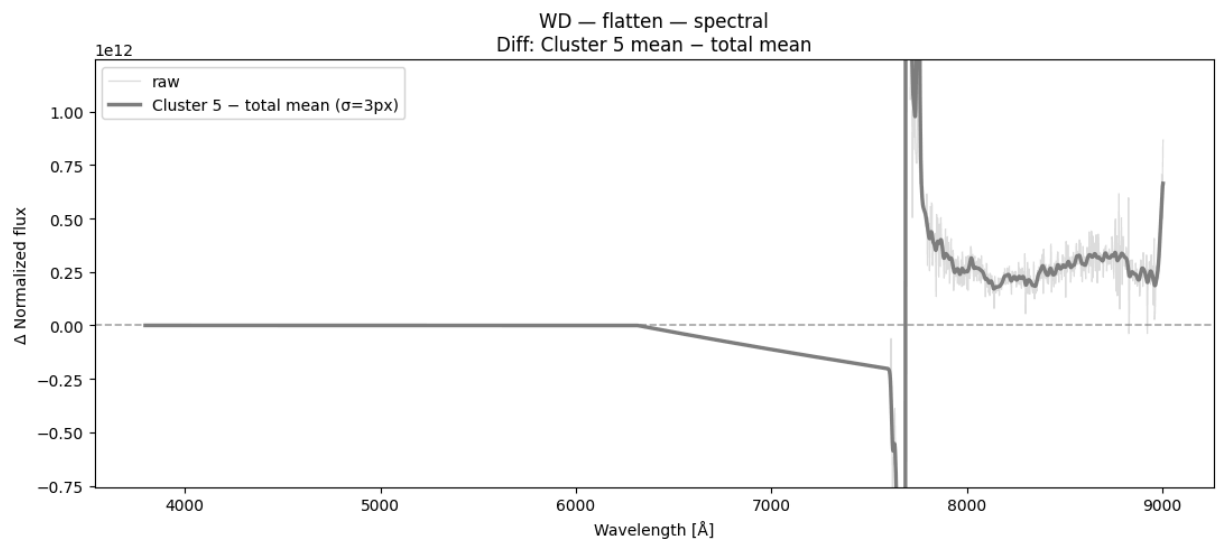
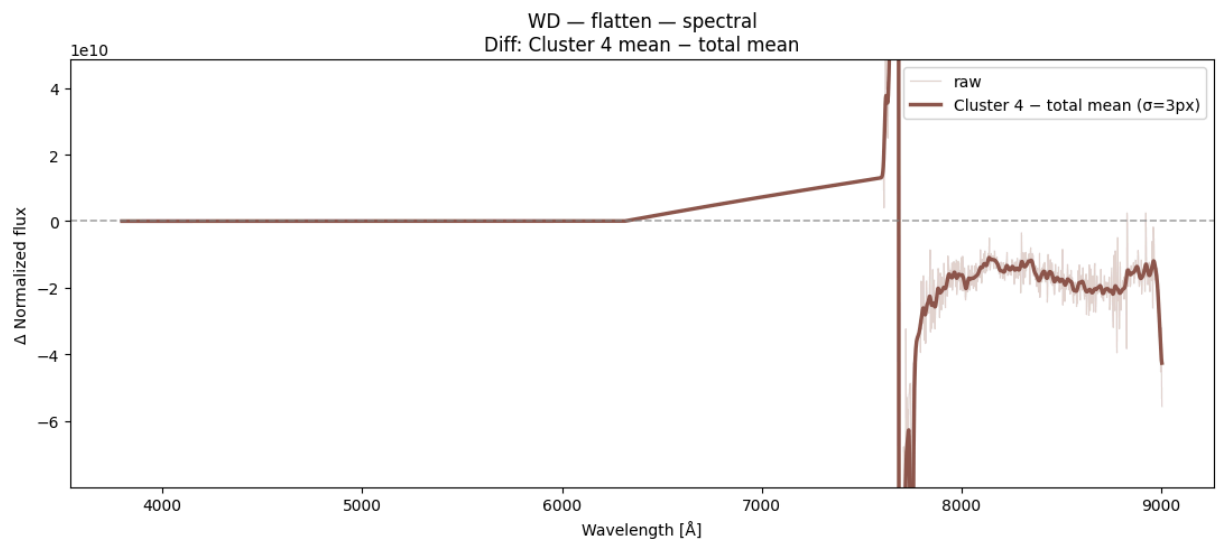
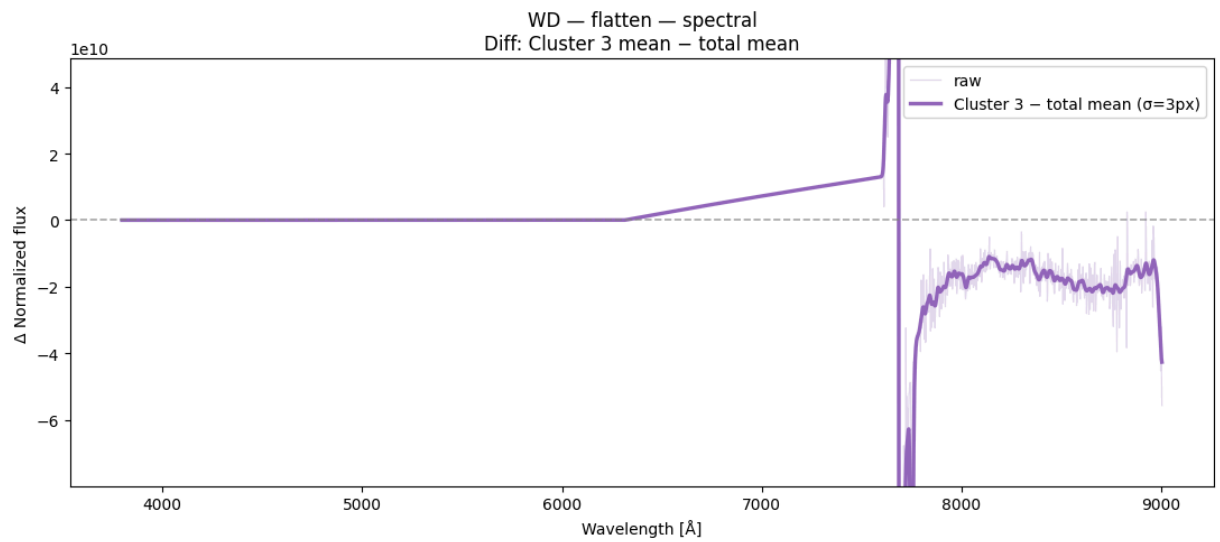
```
cmap = plt.cm.get_cmap("tab10", n_clusters)
```

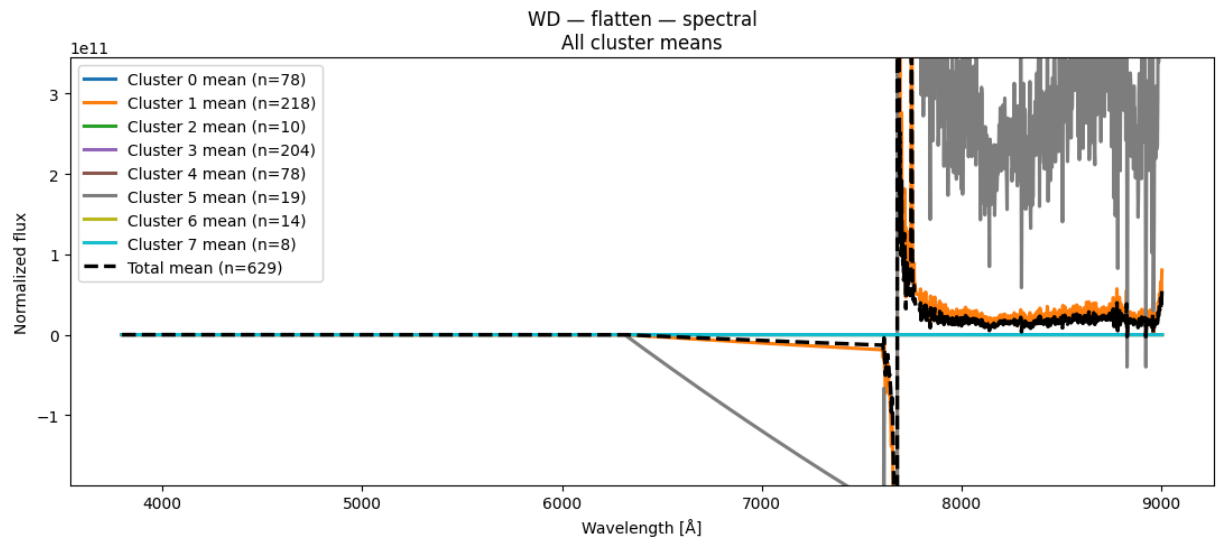
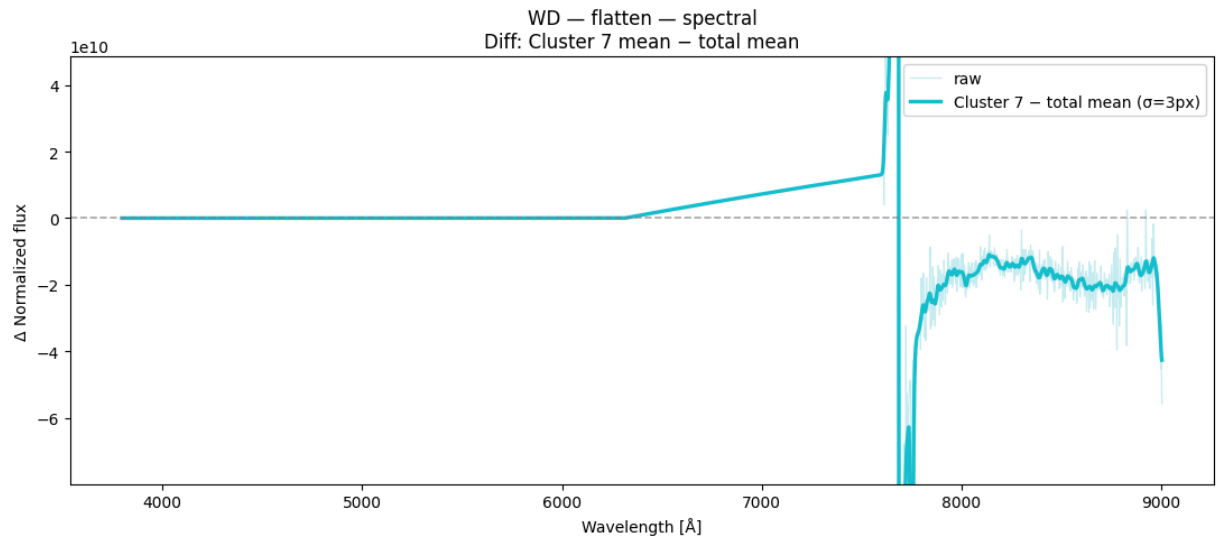
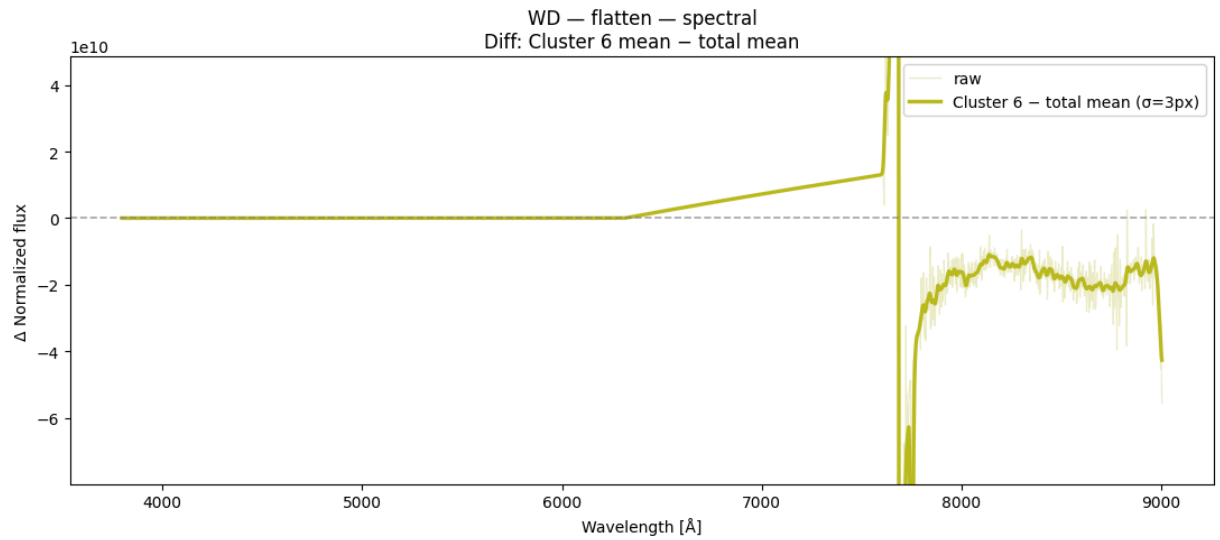












```

Out[54]: {'total_mean': array([1.8663167e+00, 1.4631674e+00, 2.3618588e+00, ..., 3.2000662e
+10,
        4.3846500e+10, 5.5692263e+10], dtype=float32),
        'group_means': {0: array([1.8198066 , 1.5705259 , 2.0340953 , ..., 0.53136235, 0.
42942575,
        0.42539307], dtype=float32),
        1: array([1.9775230e+00, 2.2498171e+00, 2.1407335e+00, ..., 4.6166102e+10,
        6.3255630e+10, 8.0345039e+10], dtype=float32),
        2: array([1.940815 , 1.9095039, 1.997124 , ..., 1.0910794, 0.780286 ,
        0.6237645], dtype=float32),
        3: array([2.3851926 , 2.2446141 , 2.6454017 , ..., 0.65866596, 0.66407424,
        0.5570027 ], dtype=float32),
        4: array([2.4954133 , 2.3517096 , 2.0646846 , ..., 0.6370731 , 0.56343603,
        0.5575099 ], dtype=float32),
        5: array([2.7727911e+00, 2.7686646e+00, 1.3911316e+00, ..., 5.2969488e+11,
        7.2577463e+11, 9.2185303e+11], dtype=float32),
        6: array([-1.1421267e+01, -3.0009708e+01, 6.1820192e+00, ...,
        3.9323972e+05, 3.0205722e+05, 3.9240778e+05], dtype=float32),
        7: array([0.9316737 , 1.809056 , 3.3264894 , ..., 0.54613864, 0.48039228,
        0.26362562], dtype=float32)},
        'unique_clusters': array([0, 1, 2, 3, 4, 5, 6, 7])}

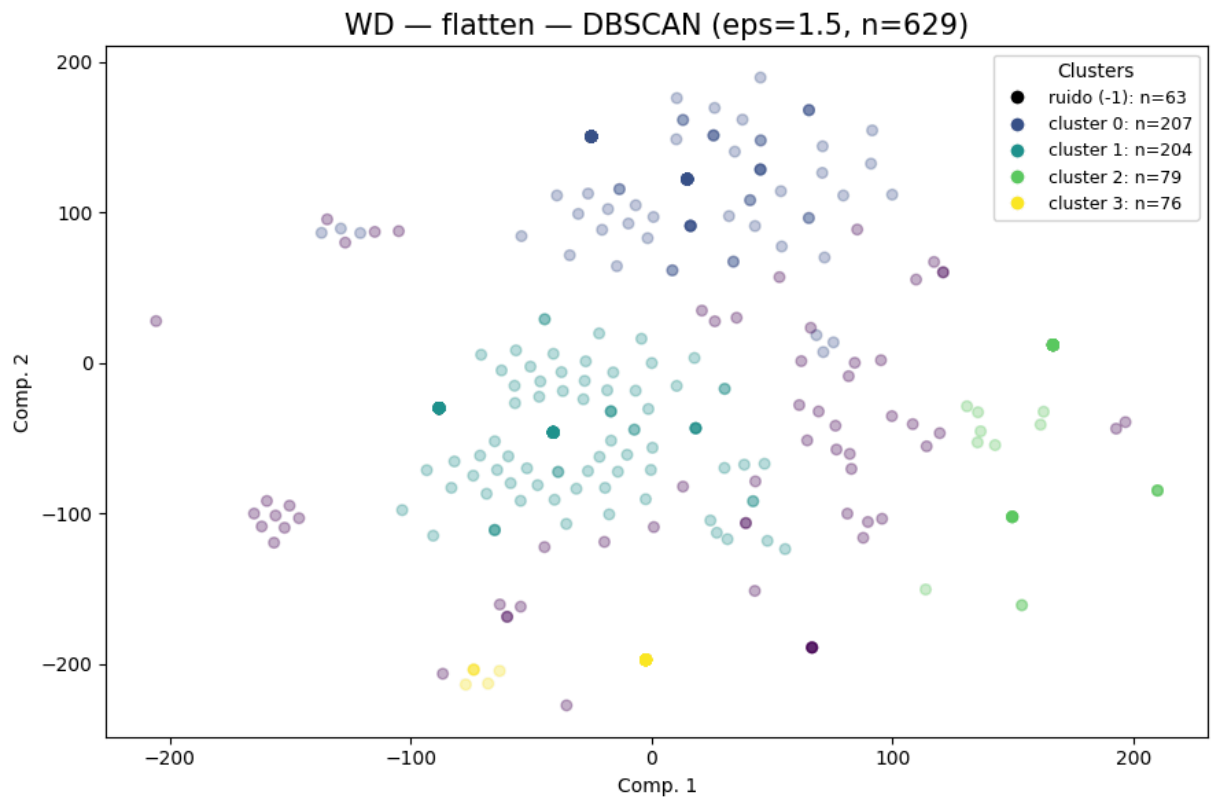
```

DBSCAN

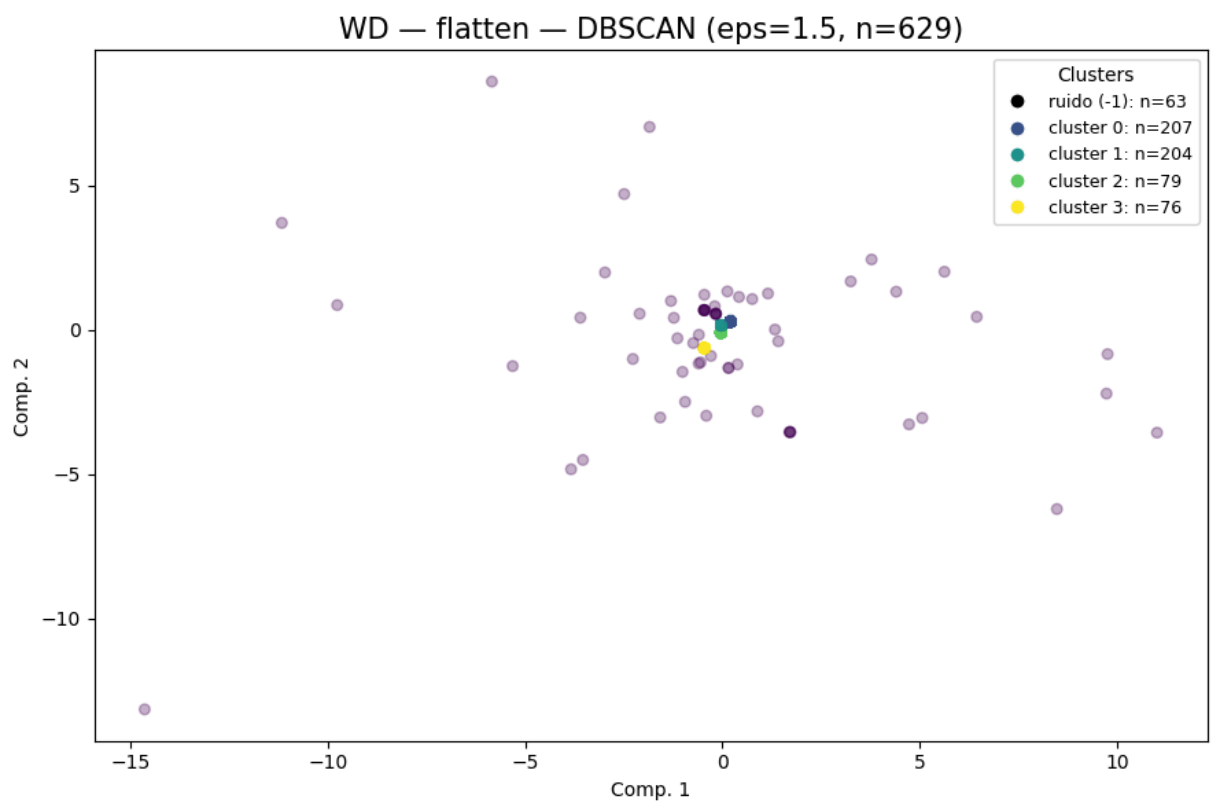
```

In [55]: res_dbs_flatten = cluster_plot_DBSCAN(
        repr_wd_32["flatten"],
        "WD",
        "flatten",
        dbscan_eps=1.5,
        dbscan_min_samples=10,
        vis_method="tsne",
        xlim=None,
        ylim=None,
        container=cluster_labels_all
    )
labels_dbs_flatten_wd = res_dbs_flatten["dbscan"]
cluster_plot_DBSCAN(
    repr_wd_32["flatten"],
    "WD",
    "flatten",
    dbscan_eps=1.5,
    dbscan_min_samples=10,
    vis_method="pca",
    xlim=None,
    ylim=None
)

```



```
WD | flatten | dbscan | eps=1.5
clusters=4 noise=63 sil=0.9969 DB=0.0050 CH=240358.93
DBCV = 0.7633 (rango [-1,1], específico DBSCAN)
```



```
WD | flatten | dbscan | eps=1.5
clusters=4 noise=63 sil=0.9969 DB=0.0050 CH=240358.93
DBCV = 0.7633 (rango [-1,1], específico DBSCAN)
```

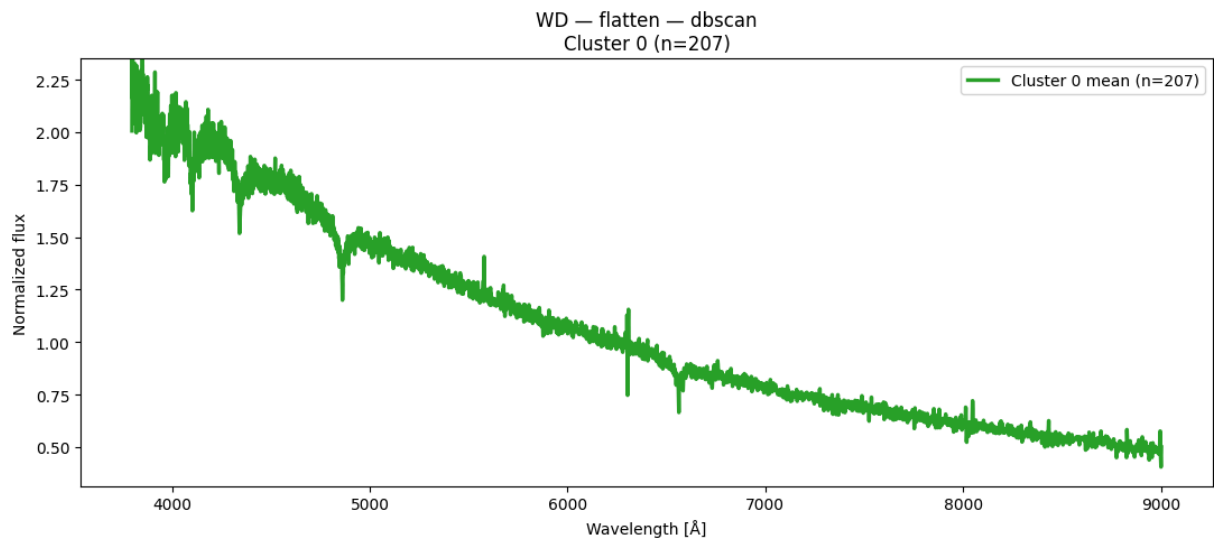
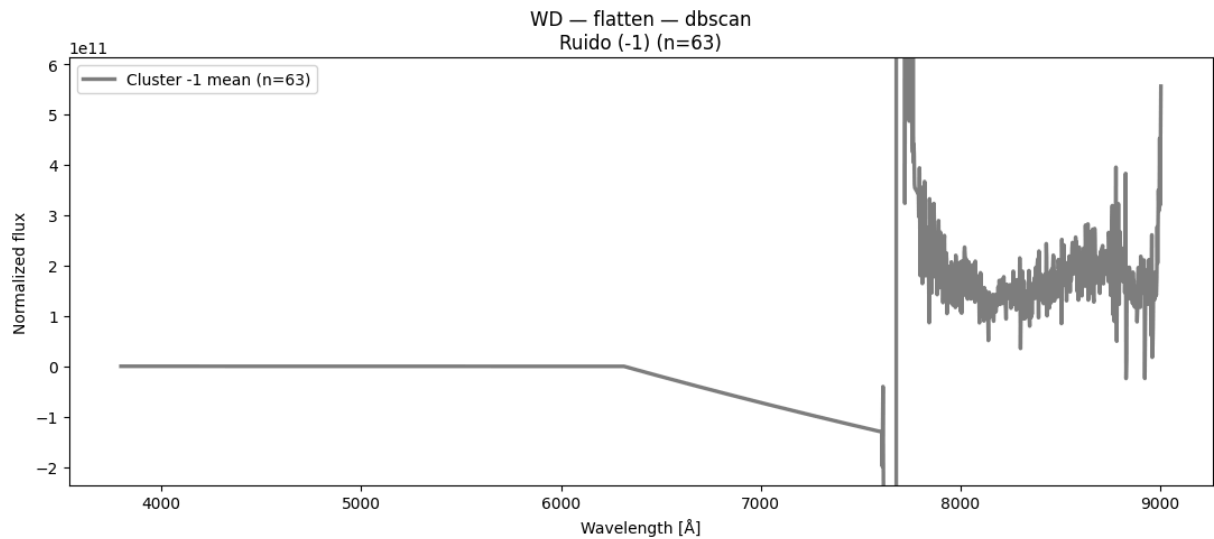
```
Out[55]: {'dbscan': array([-1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1,
-1,
-1, -1, -1, -1, -1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2,
2, 2, 2, 2, 2, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1,
1, -1, 3, 3, 3, 3, 3, 3, 3, 3, 0, 0, 0, 0, 0, 0, 1, 1,
1, 1, 1, 1, -1, 3, 3, 3, 3, 3, 3, 3, -1, 0, 0, 0, 0,
0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 0, 0,
0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2,
2, 0, 0, 0, 0, 0, 0, 0, 1, -1, 1, -1, 1, 1, 3, 3, 3, 3,
3, 3, 3, 3, 0, 0, 0, 0, -1, 0, 1, 1, 1, 1, 1, 1, 3,
3, 3, 3, 3, 3, 3, 3, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1,
1, 1, 2, 2, 2, 2, 2, 2, 2, 2, -1, 0, 0, 0, 0, 0, 1,
1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 0, 0, 0, 0,
0, 0, 1, 1, 1, 1, 1, 1, 3, 3, 3, 3, 3, 3, 3, 3, 0,
0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1,
1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 2, 2, 2, 2,
2, 2, 2, 2, 0, 0, 0, 0, 0, 0, 0, -1, 1, 1, 1, 1, 1, 0,
0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1,
1, 3, 3, 3, 3, 3, 3, 3, 3, 3, 0, 0, 0, 0, 0, 0, 1, 1,
1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, -1, -1, 0,
1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 0, 0, 0,
-1, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 0,
0, 0, 0, 0, -1, 1, 1, 1, 1, 1, 1, 3, 3, 3, 3, 3, 3,
3, 3, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0,
0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 2,
2, 2, 2, 2, 2, 2, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1,
1, -1, 0, 0, 0, 0, 1, -1, 1, 0, 0, 0, 0, 0, 0, 1, 1,
1, 1, 1, 1, 3, 3, 3, 3, 3, 3, 3, 3, 0, 0, 0, 0, 0,
0, 1, 1, 1, 1, 1, 1, 0, 0, -1, 0, 1, 1, 1, 0, 0, 0,
0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, -1, 2, 2, 2,
-1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, -1, 0, 0, 1,
1, 1, 0, 0, 0, -1, 0, 0, 1, 1, 1, 1, 1, 1, 3, 3, 3,
3, 3, 3, 3, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1,
0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1,
1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 0, 0, 0, 0, 0, 0, 1,
1, 1, 1, 1, -1, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0,
0, 1, 1, 1, 1, 1, 1, 3, 3, 3, 3, 3, 3, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1],
dtype=int64),
'metrics': {'n_clusters': 4,
'n_noise': 63,
'silhouette': 0.996886,
'davies_bouldin': 0.005012,
'calinski_harabasz': 240358.9325},
'dbcv': 0.763256}
```

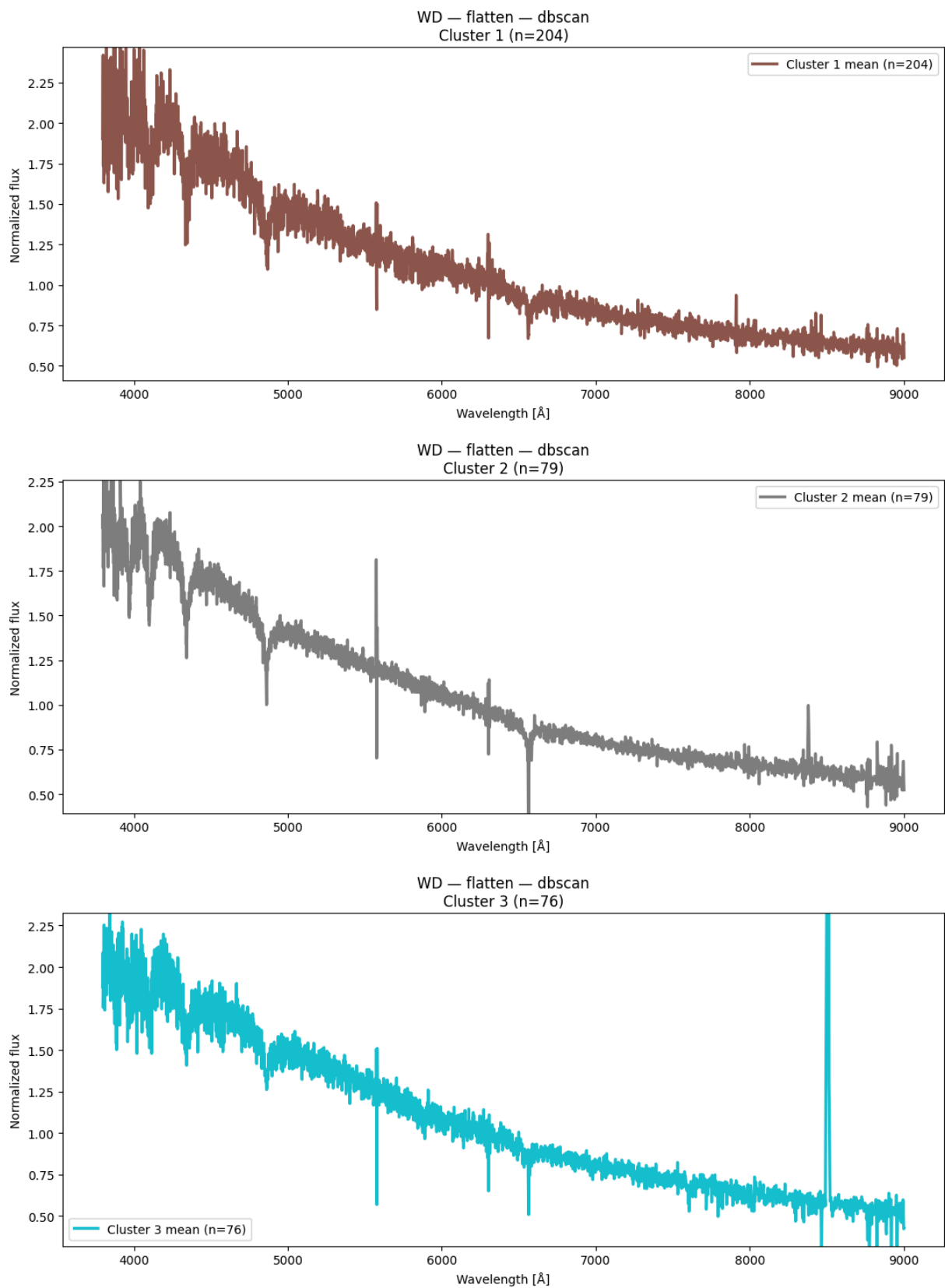
Espectros obtenidos de DBSCAN n=629

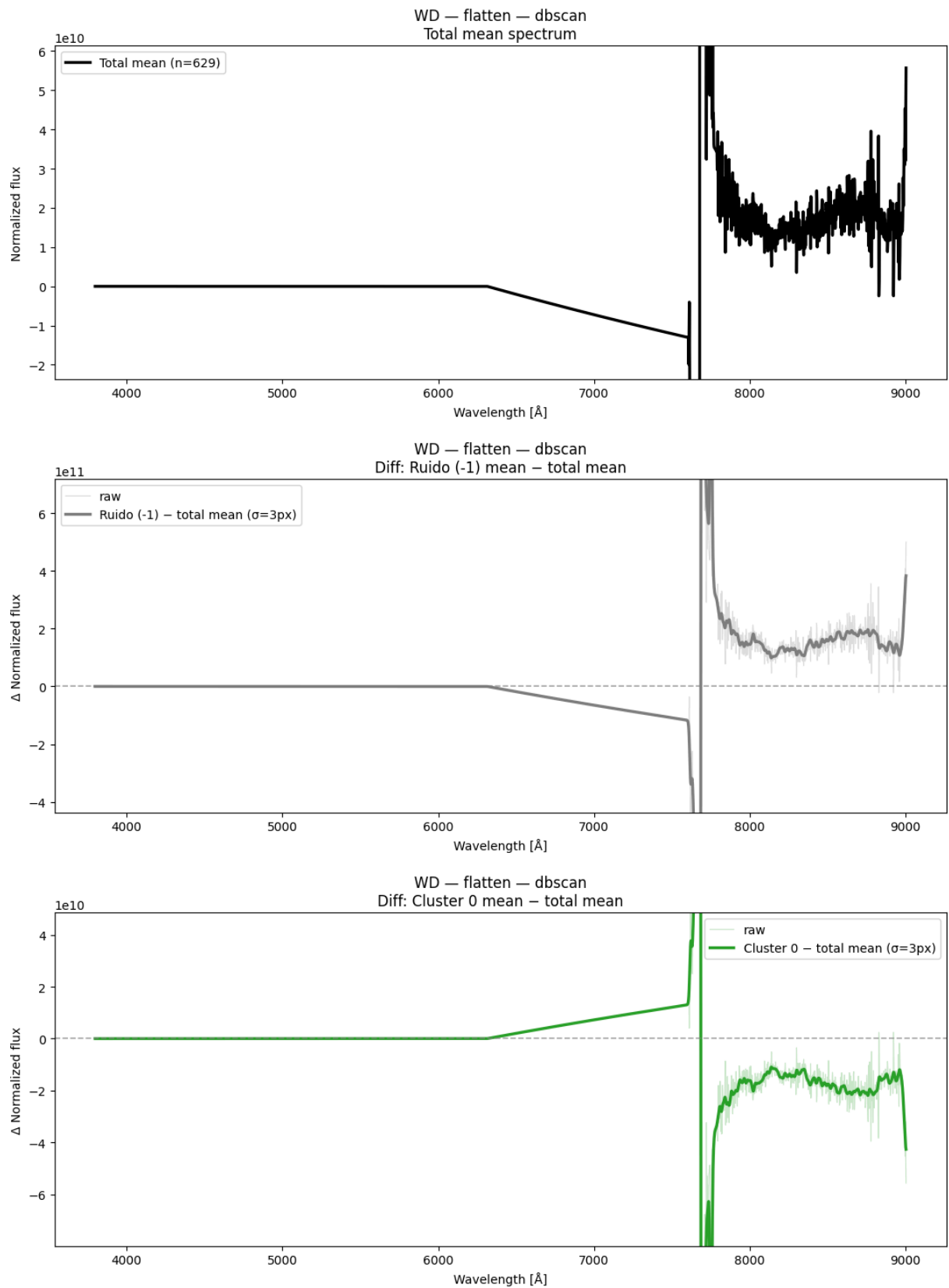
```
In [56]: plot_spectra_dbscan(
    X_wd_spec,
    labels_dbs_flatten_wd,
    W_wd[0],
    class_name="WD",
    layer_name="flatten",
    mark_balmer=False
)
```

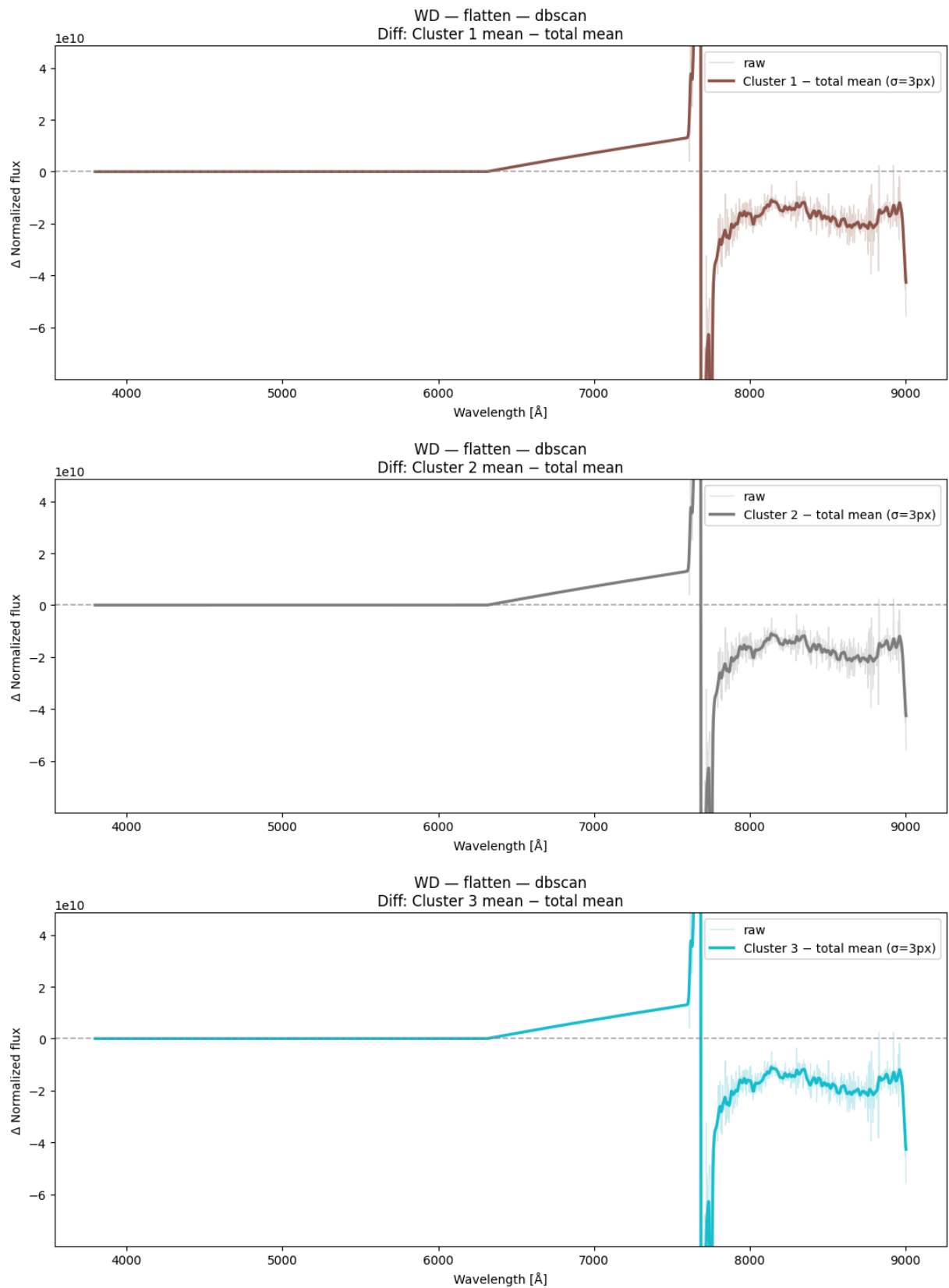
C:\Users\javip\AppData\Local\Temp\ipykernel_52368\531566132.py:28: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get\_cmap()`` or ``pyplot.get\_cmap()`` instead.

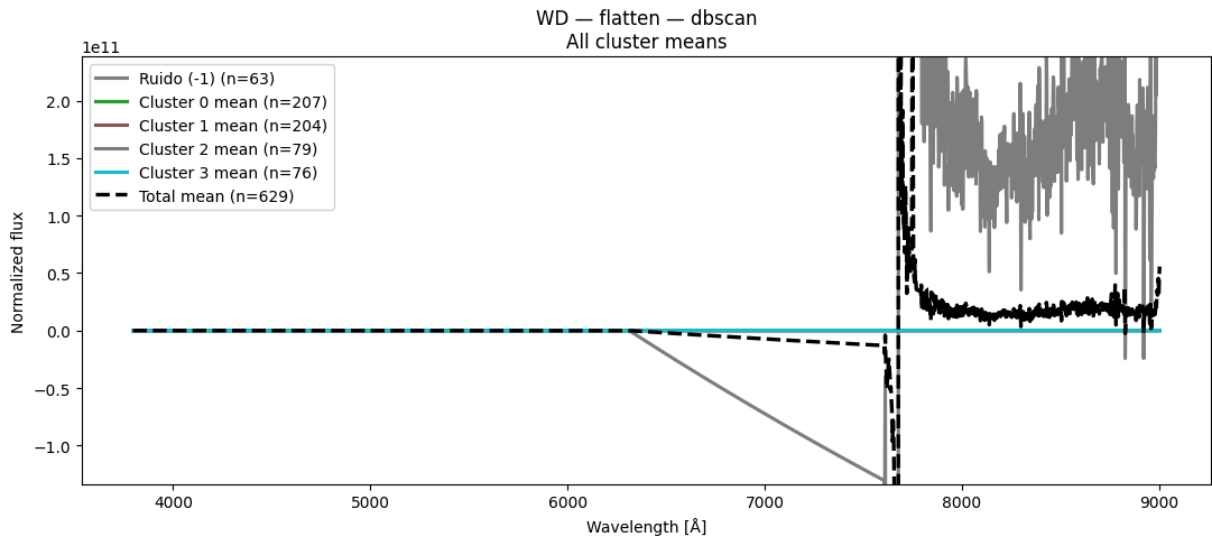
```
cmap = plt.cm.get_cmap("tab10", n_clusters)
```











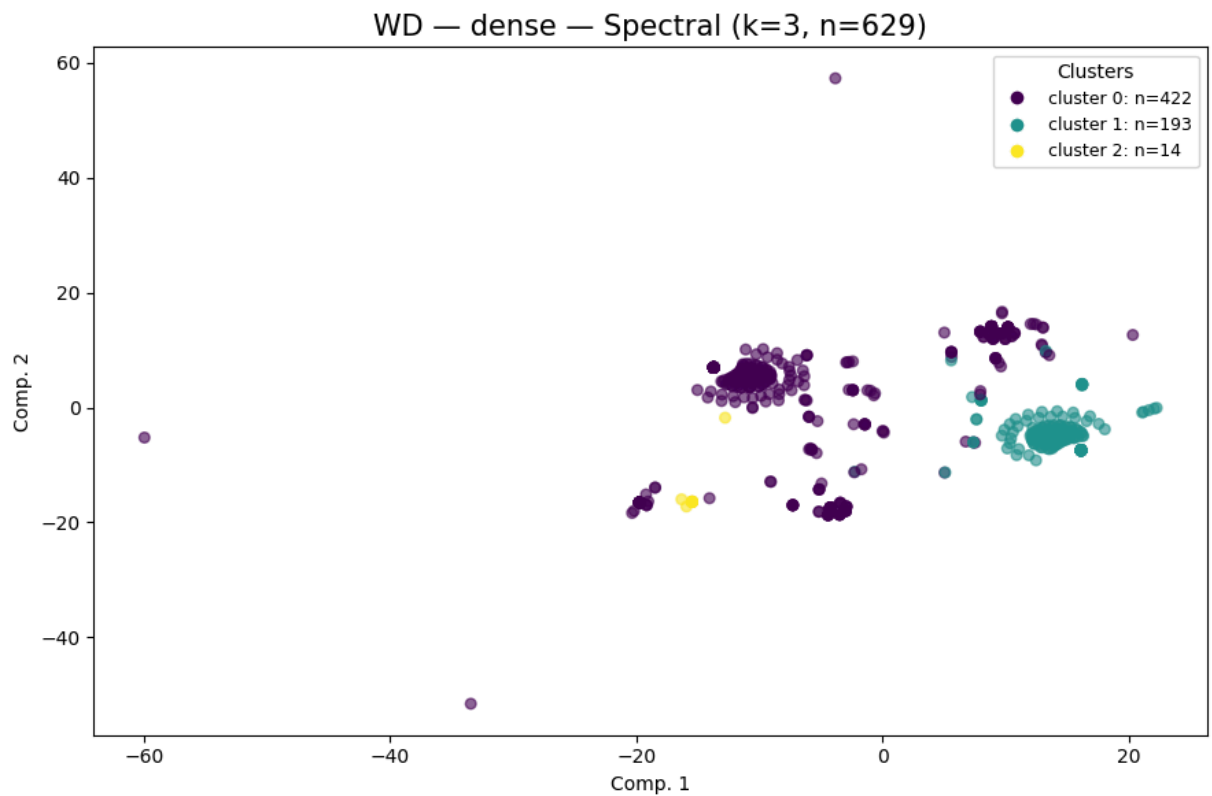
```
Out[56]: {'total_mean': array([1.8663167e+00, 1.4631674e+00, 2.3618588e+00, ..., 3.2000662e+10,
    4.3846500e+10, 5.5692263e+10], dtype=float32),
  'group_means': {-1: array([ 1.1101028e+00, -5.0965409e+00,  3.5101061e+00, ...,
    3.1949868e+11,  4.3776904e+11,  5.5603862e+11], dtype=float32),
    0: array([2.0063634 , 2.346046 , 2.2220476 , ..., 0.4563571 , 0.40479696,
    0.49997026], dtype=float32),
    1: array([1.902879 , 2.2018032 , 2.4125016 , ..., 0.6297253 , 0.65068763,
    0.5505564 ], dtype=float32),
    2: array([1.9950494, 2.0667143, 1.9471791, ..., 0.6844768, 0.5775485,
    0.5236292], dtype=float32),
    3: array([1.8797821 , 1.8861086 , 2.085941 , ..., 0.5124545 , 0.42313442,
    0.42786634], dtype=float32)},
  'unique_clusters': array([-1,  0,  1,  2,  3], dtype=int64)}
```

Resultados capa dense con Spectral Clustering + tSNE y PCA

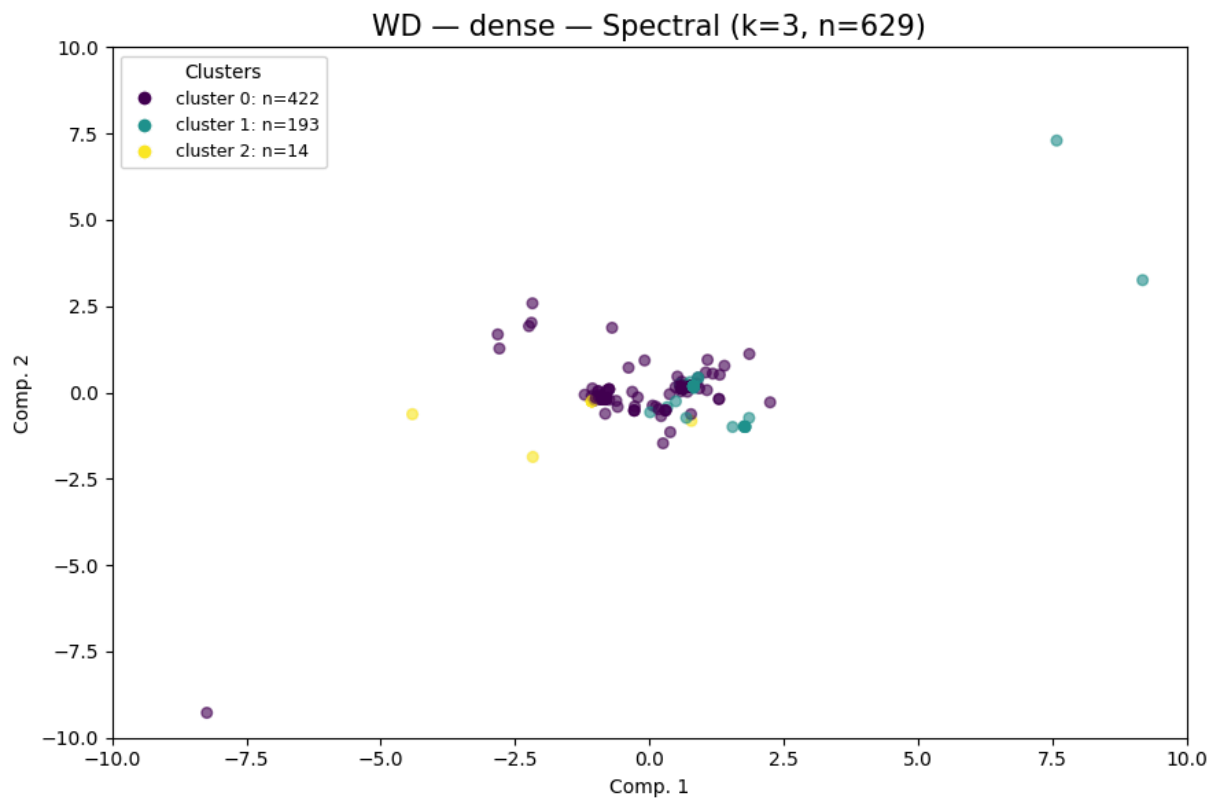
Spectral Clustering k = 3

```
In [57]: res_dense_k3 = cluster_plot_Spect(
    repr_wd_32["dense"],
    "WD",
    "dense",
    spectral_k=3,
    vis_method="tsne",
    container=cluster_labels_all
)

labels_dense_k3_wd = res_dense_k3["spectral"]
cluster_plot_Spect(
    repr_wd_32["dense"],
    "WD",
    "dense",
    spectral_k=3,
    vis_method="pca",
    xlim=(-10,10), ylim=(-10,10)
)
```



```
WD | dense | spectral | k=3  
clusters=3 noise=0 sil=0.1705 DB=2.4399 CH=12.71
```



WD		dense		spectral		k=3
clusters=3		noise=0		sil=0.1705		DB=2.4399 CH=12.71

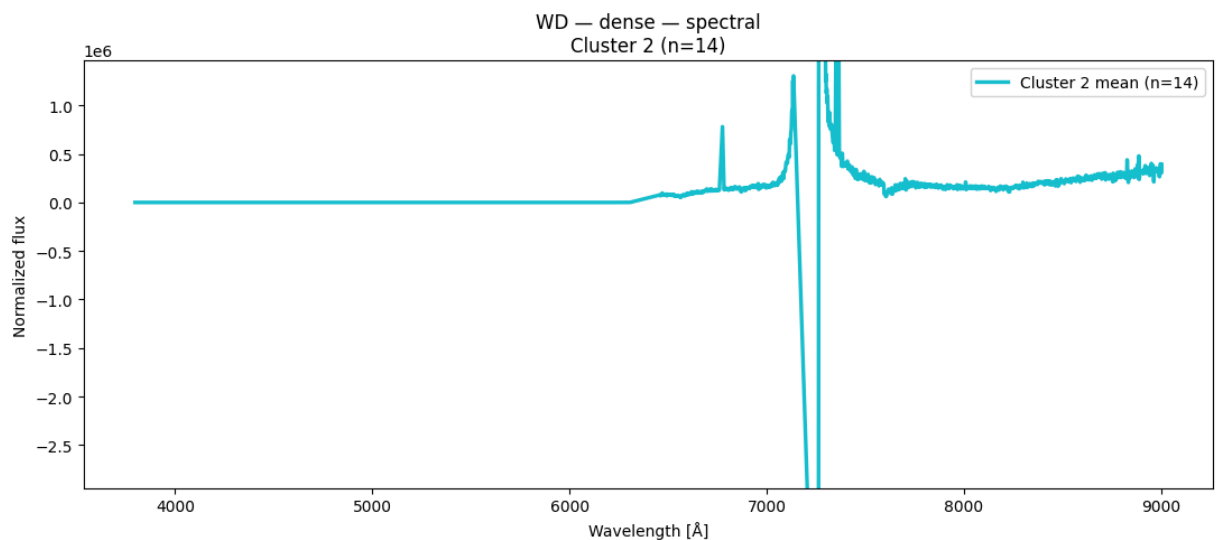
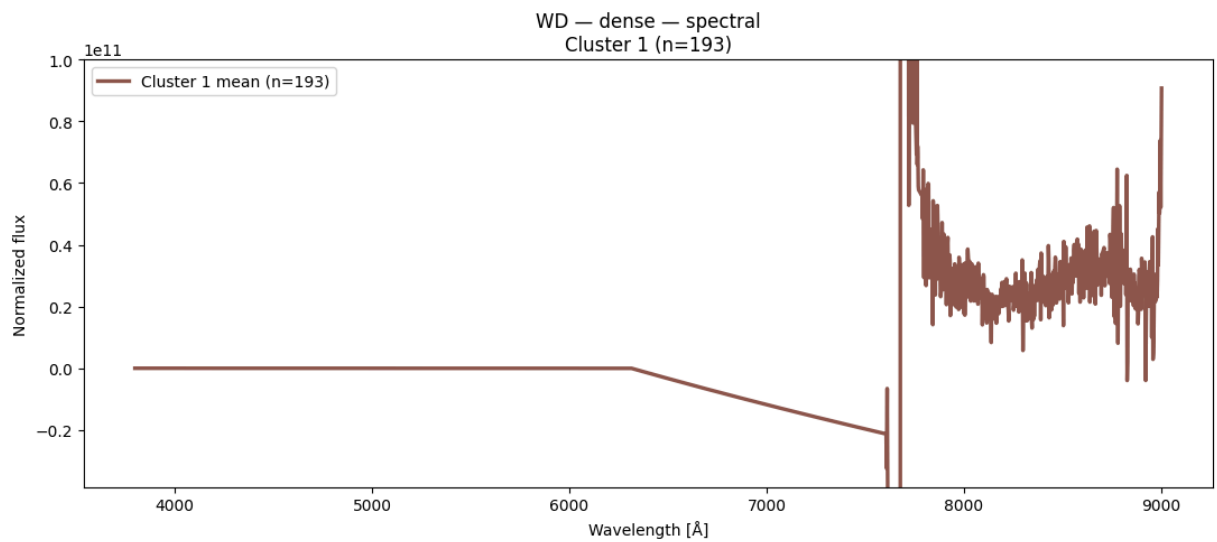
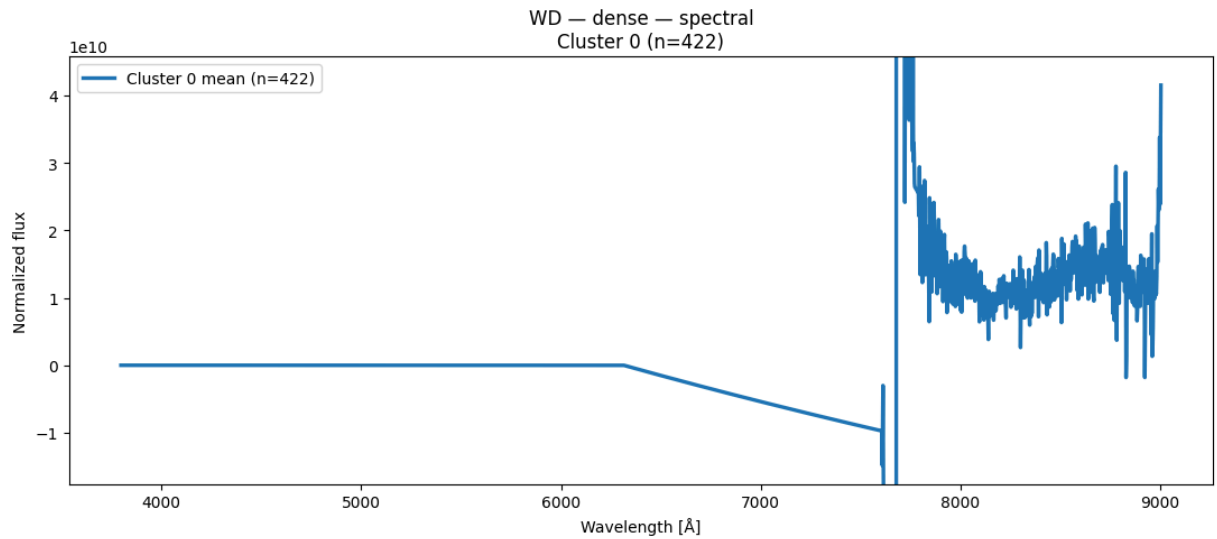
```
Out[57]: {'spectral': array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1,
0,
1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0,
1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1,
0, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 2, 0, 0, 0, 0,
0, 0, 0, 0, 2, 2, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 0,
0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 1, 0,
0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1,
0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0,
0, 0, 0, 0, 2, 2, 0, 2, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0,
0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 1, 2, 1, 1, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1,
1, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
2, 2, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1,
1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1,
1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 1, 1,
0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0,
0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1,
0, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 2, 0, 0, 0,
0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0,
0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1,
1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1,
1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 0, 1, 1, 1,
1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1]),
'metrics': {'n_clusters': 3,
'n_noise': 0,
'silhouette': 0.170522,
'davies_bouldin': 2.439853,
'calinski_harabasz': 12.7108}}
```

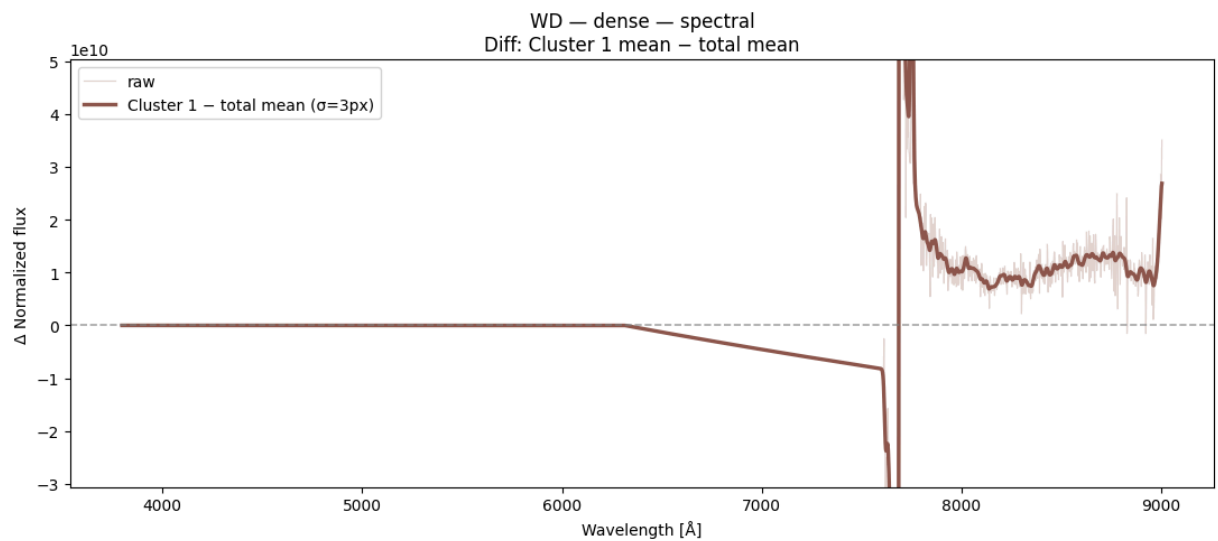
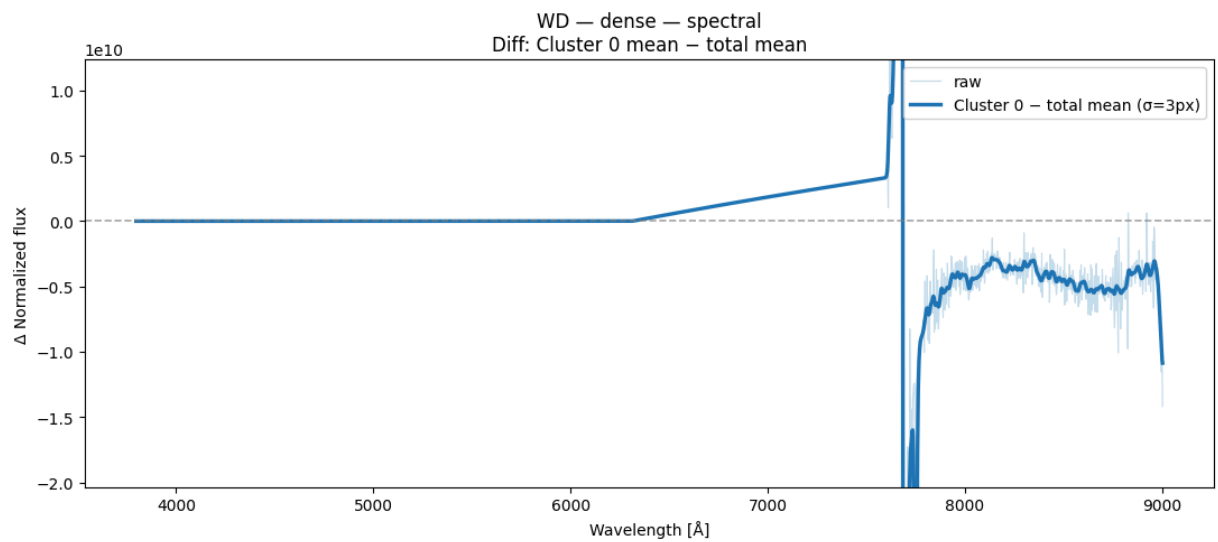
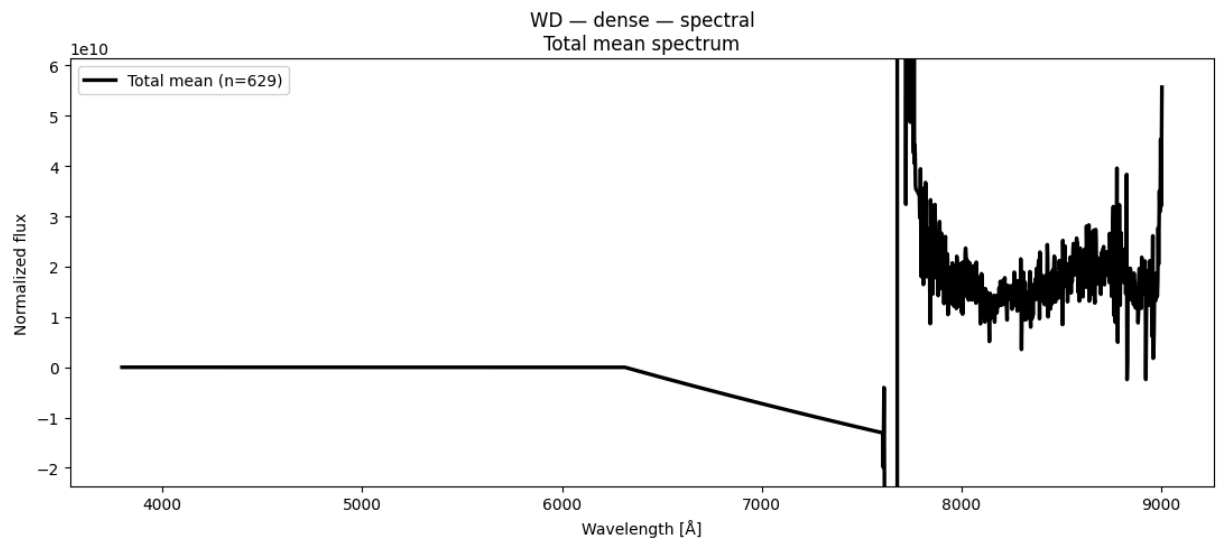
Espectros obtenidos para dense de Spectral Clustering k = 3 y n = 700

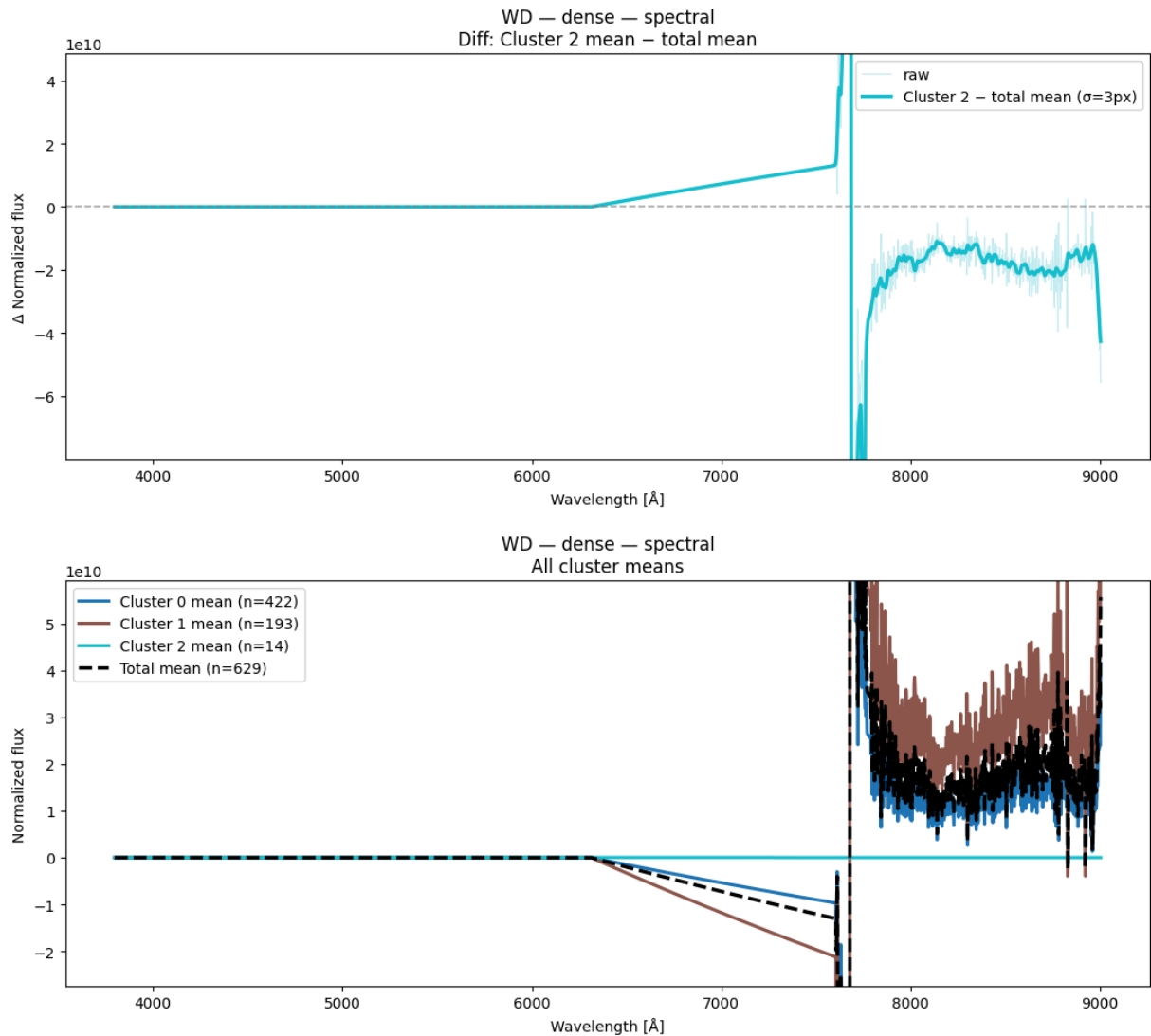
```
In [58]: plot_spectra_spectral(
X_wd_spec,
labels_dense_k3_wd,
W_wd[0],
class_name="WD",
layer_name="dense"
)
```

C:\Users\javip\AppData\Local\Temp\ipykernel_52368\531566132.py:28: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get\_cmap()`` or ``pyplot.get\_cmap()`` instead.

```
cmap = plt.cm.get_cmap("tab10", n_clusters)
```







```
Out[58]: {'total_mean': array([1.8663167e+00, 1.4631674e+00, 2.3618588e+00, ..., 3.2000662e
+10,
        4.3846500e+10, 5.5692263e+10], dtype=float32),
        'group_means': {0: array([2.4142790e+00, 2.3000607e+00, 2.4041824e+00, ..., 2.384
8835e+10,
        3.2677067e+10, 4.1505243e+10], dtype=float32),
        1: array([1.5825509e+00, 1.9135237e+00, 1.9653205e+00, ..., 5.2146139e+10,
        7.1449346e+10, 9.0752401e+10], dtype=float32),
        2: array([-1.0738884e+01, -2.9971674e+01, 6.5526977e+00, ...,
        3.9323906e+05, 3.0205644e+05, 3.9240656e+05], dtype=float32)},
        'unique_clusters': array([0, 1, 2])}
```

Spectral Clustering k = 4

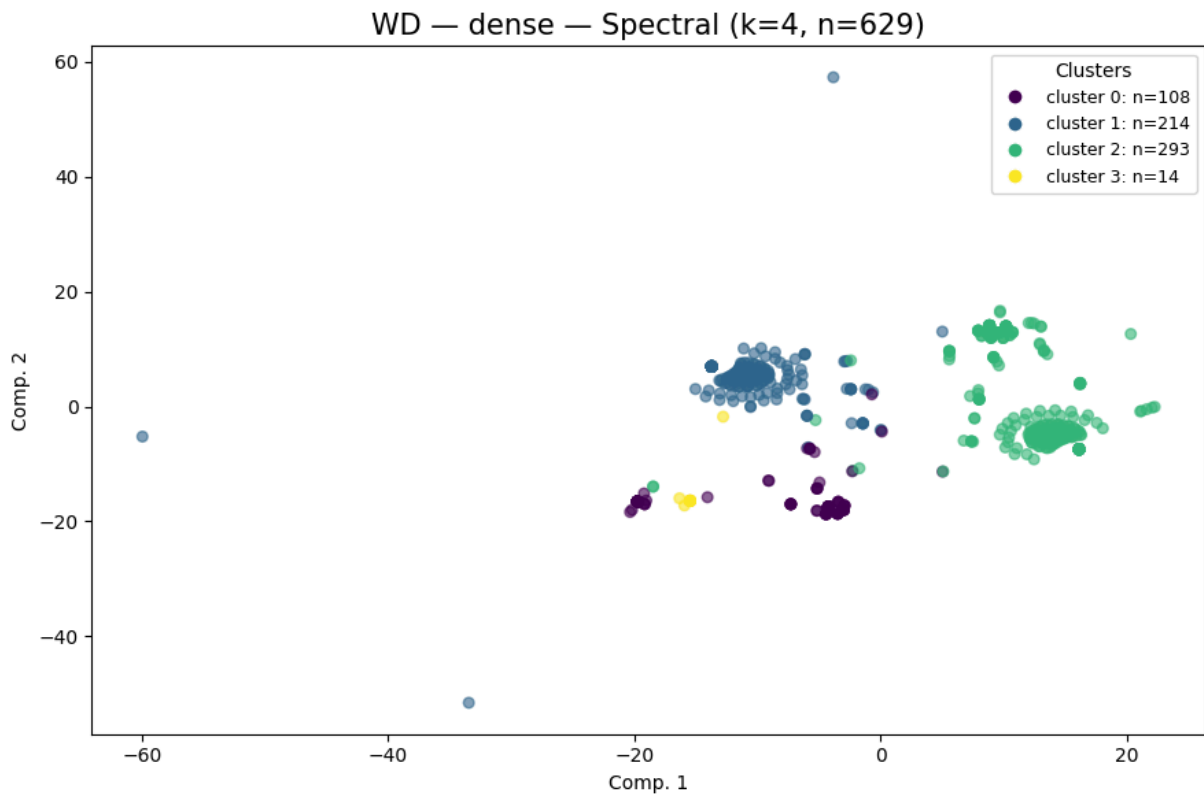
```
In [59]: res_dense_k4 = cluster_plot_Spect(
        repr_wd_32["dense"],
        "WD",
        "dense",
        spectral_k=4,
        vis_method="tsne",
        container=cluster_labels_all
    )
```



```

labels_dense_k4_wd = res_dense_k4["spectral"]
cluster_plot_Spect(
    repr_wd_32["dense"],
    "WD",
    "dense",
    spectral_k=4,
    vis_method="pca",
    xlim=(-10,10), ylim=(-10,10)
)

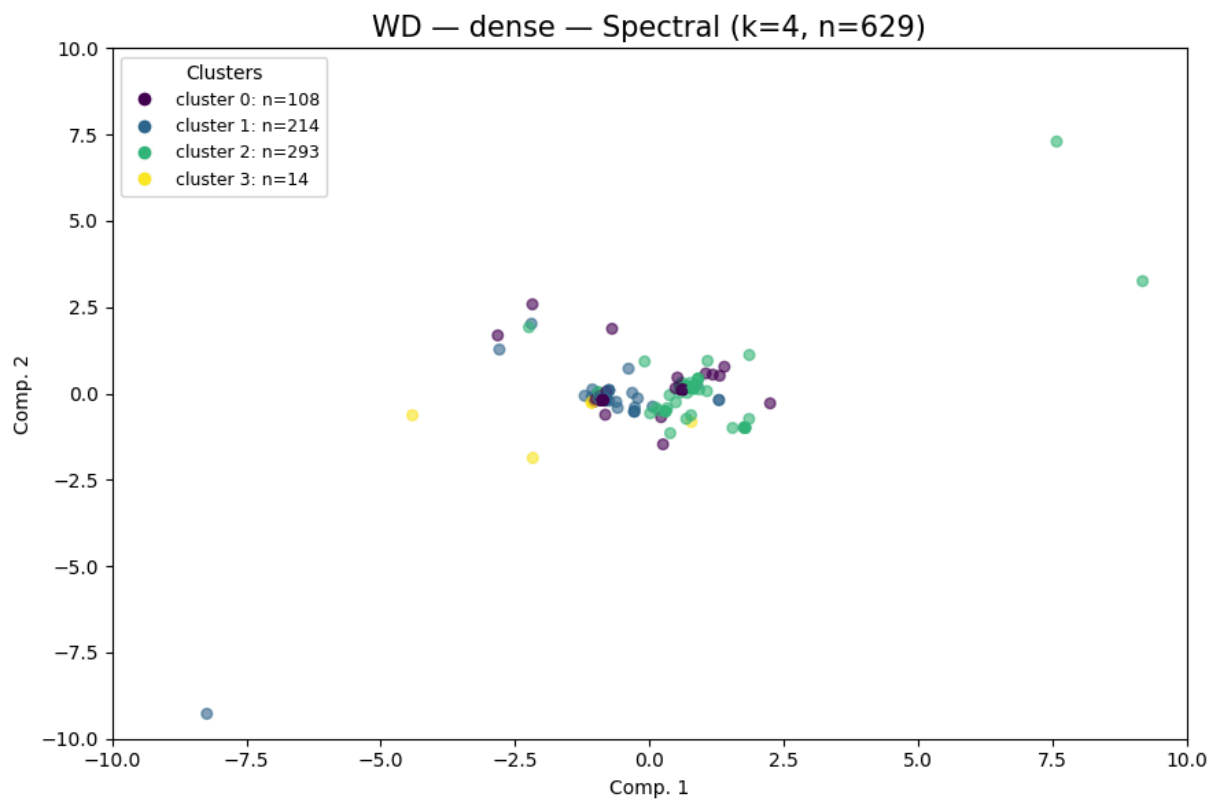
```



```

WD | dense | spectral | k=4
clusters=4 noise=0 sil=0.3000 DB=2.8175 CH=13.74

```



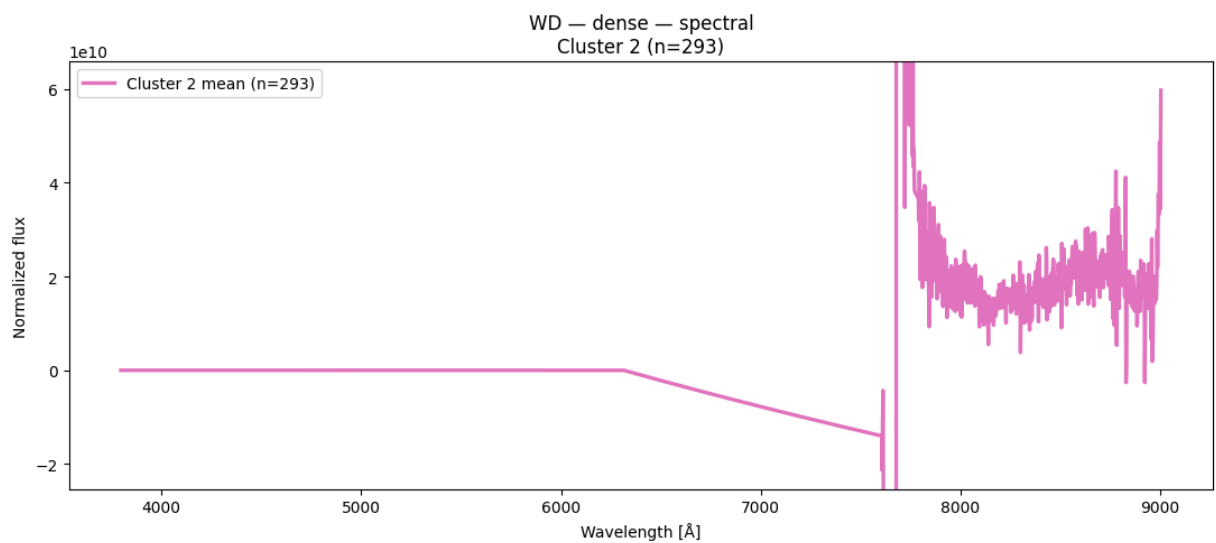
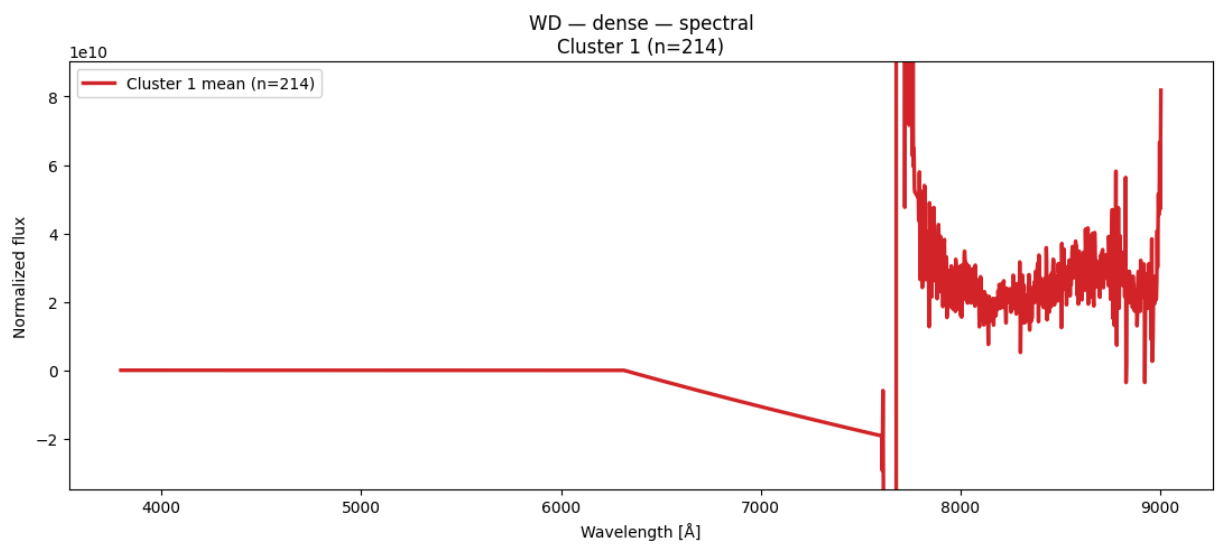
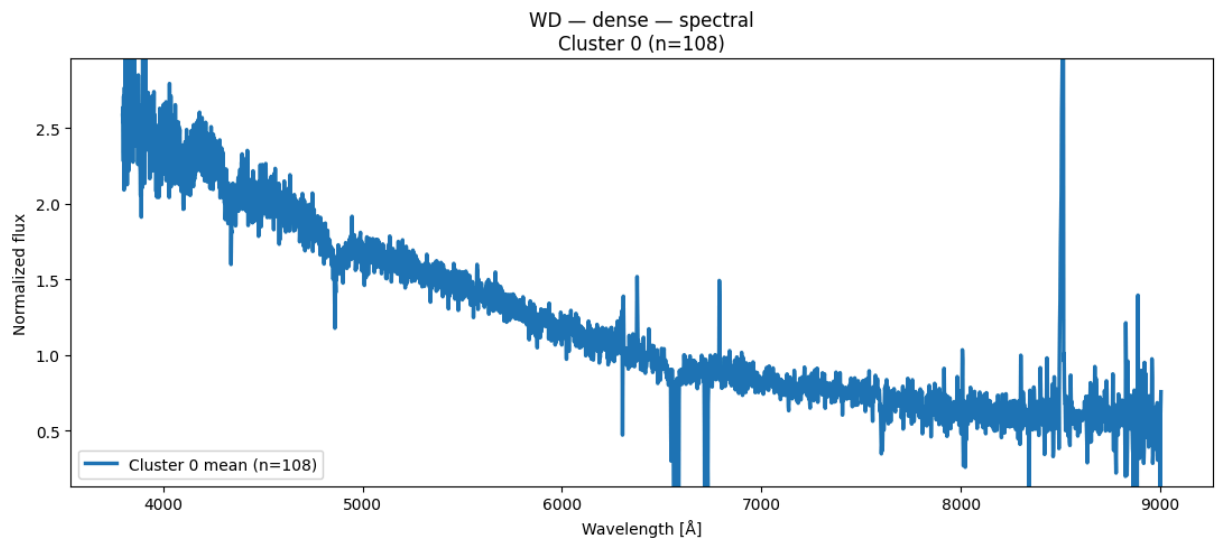
```
WD | dense | spectral | k=4  
clusters=4 noise=0 sil=0.3000 DB=2.8175 CH=13.74
```

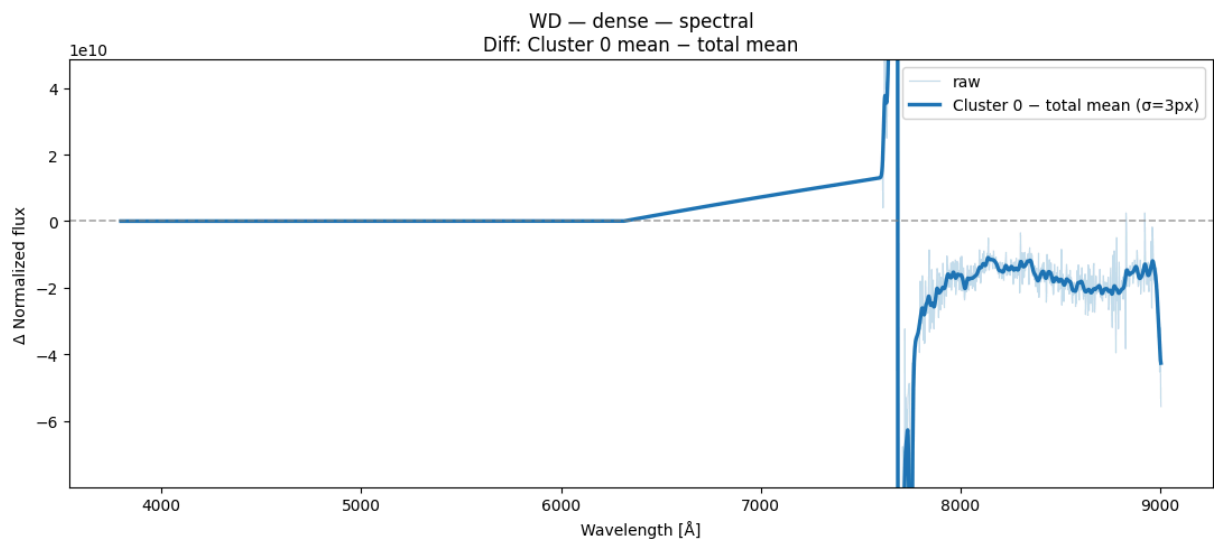
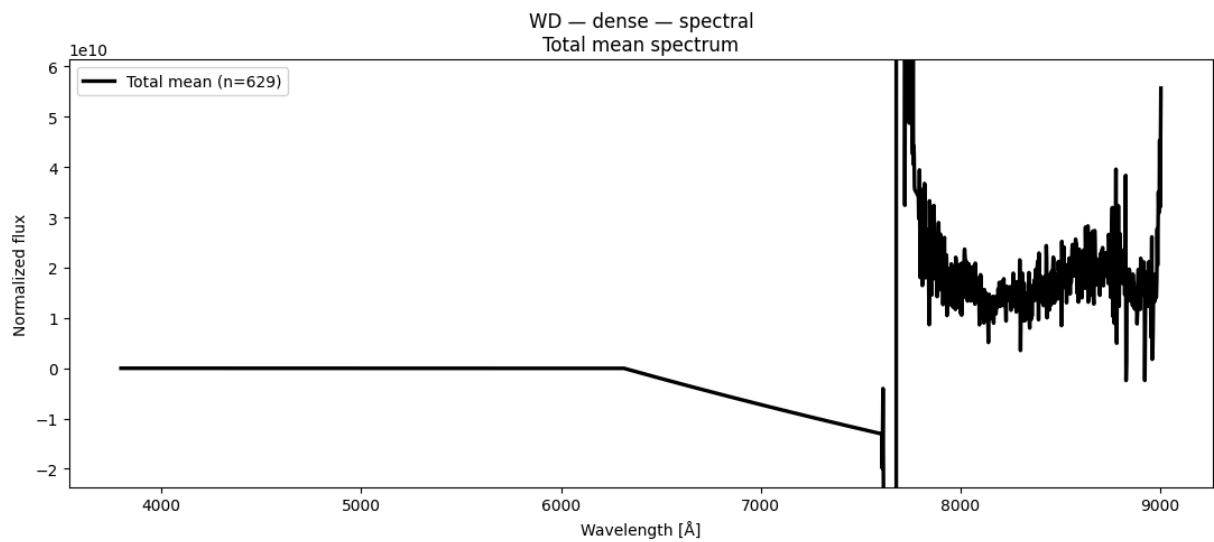
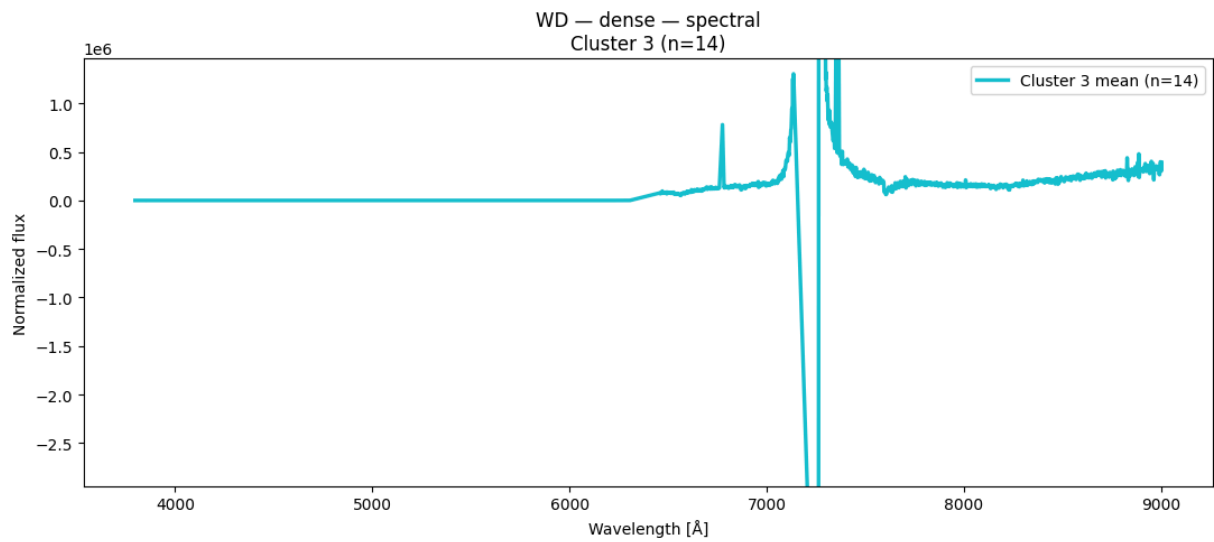
```
Out[59]: {'spectral': array([1, 1, 1, 0, 0, 2, 0, 0, 0, 2, 1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2,
2,
2, 0, 2, 2, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 0,
2, 2, 1, 1, 1, 1, 1, 1, 2, 0, 0, 1, 0, 0, 0, 0, 2, 2, 2, 2, 2, 2,
1, 1, 1, 1, 1, 1, 1, 0, 3, 0, 0, 0, 0, 0, 1, 0, 2, 2, 2, 2, 1, 1,
1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 0, 0, 2, 2, 2, 1, 1, 1, 1,
1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 0, 1, 3, 1, 2, 1, 1,
0, 0, 0, 0, 3, 3, 0, 0, 2, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 0, 0,
0, 0, 0, 0, 0, 0, 2, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2,
2, 2, 1, 2, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2,
2, 2, 0, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 3, 0,
0, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 2, 0, 2, 2, 1, 1, 1, 2, 2, 2,
0, 2, 2, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
0, 1, 1, 1, 1, 1, 1, 2, 0, 2, 2, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1,
1, 1, 1, 1, 3, 3, 0, 3, 0, 0, 0, 0, 2, 2, 2, 2, 2, 1, 1, 1, 1,
1, 1, 0, 2, 2, 2, 1, 1, 1, 0, 2, 2, 3, 2, 2, 1, 1, 1, 1, 1, 2,
2, 2, 2, 2, 2, 2, 0, 2, 0, 2, 2, 2, 2, 1, 1, 1, 1, 2, 2, 2,
2, 1, 1, 1, 2, 0, 2, 2, 2, 2, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0,
3, 3, 2, 2, 2, 2, 2, 2, 1, 2, 1, 1, 1, 1, 2, 0, 0, 2, 1, 1, 2,
2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 0, 2,
2, 2, 0, 1, 1, 1, 1, 1, 0, 2, 0, 2, 1, 1, 1, 2, 0, 2, 2, 2, 2,
1, 1, 1, 1, 1, 0, 0, 0, 3, 0, 0, 0, 0, 0, 2, 2, 2, 2, 2, 1, 1,
1, 1, 1, 1, 2, 2, 0, 2, 1, 1, 1, 2, 2, 2, 2, 0, 2, 1, 1, 1, 1,
1, 2, 2, 2, 2, 2, 2, 2, 2, 0, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 2,
1, 2, 2, 1, 1, 1, 2, 2, 2, 0, 0, 2, 1, 1, 1, 1, 0, 3, 0, 0, 0,
0, 0, 0, 0, 2, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 2, 2, 2, 0, 1, 1,
1, 2, 0, 2, 2, 2, 2, 1, 1, 0, 1, 1, 1, 2, 2, 2, 2, 2, 1, 2, 2,
2, 2, 2, 2, 2, 1, 1, 1, 1, 0, 2, 0, 0, 2, 1, 1, 1, 2, 2, 2, 2,
2, 2, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 3, 2, 2, 0, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]),
'metrics': {'n_clusters': 4,
'n_noise': 0,
'silhouette': 0.29995,
'davies_bouldin': 2.817505,
'calinski_harabasz': 13.7403}}
```

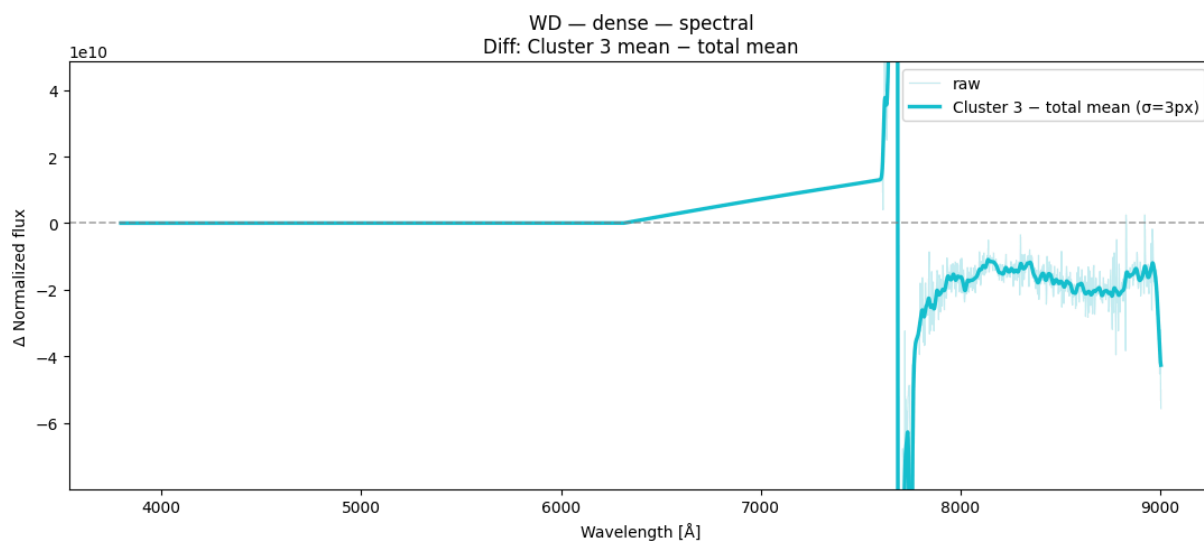
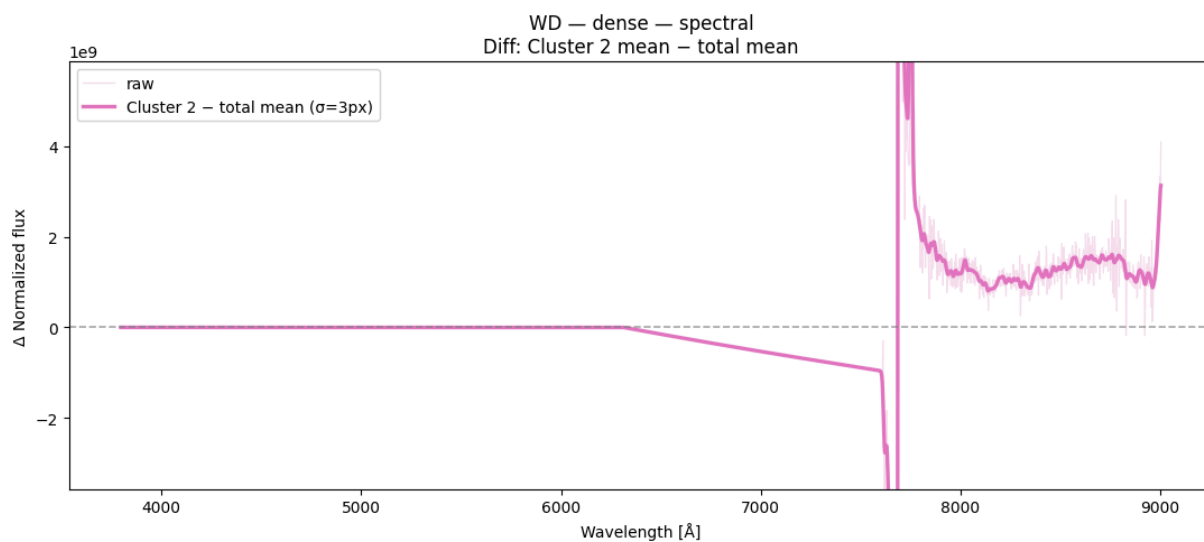
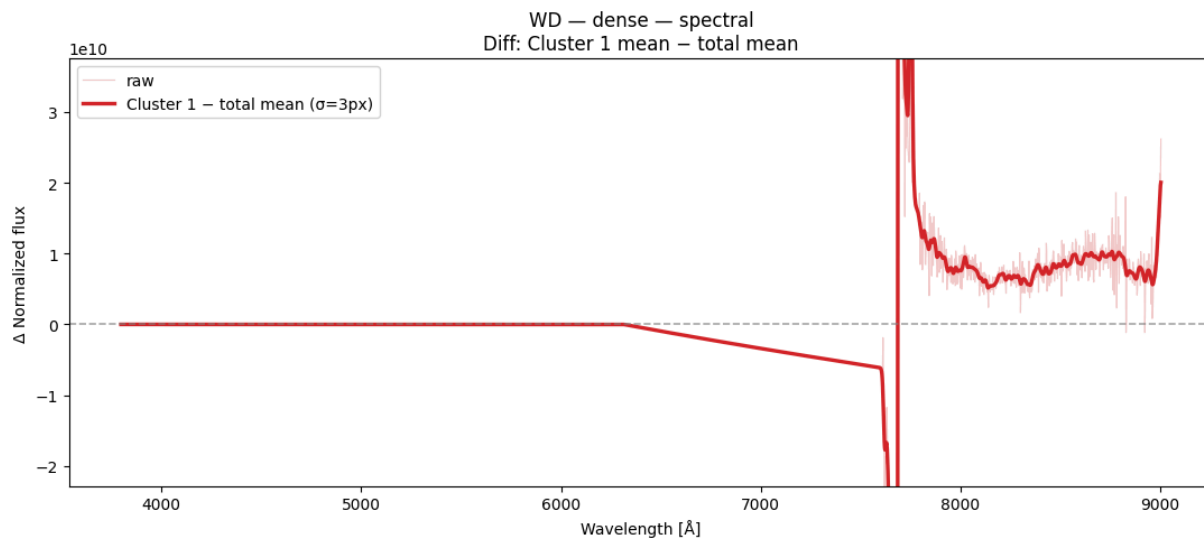
Espectros obtenidos para dense con Spectral Clustering $k = 4$ y $n = 700$

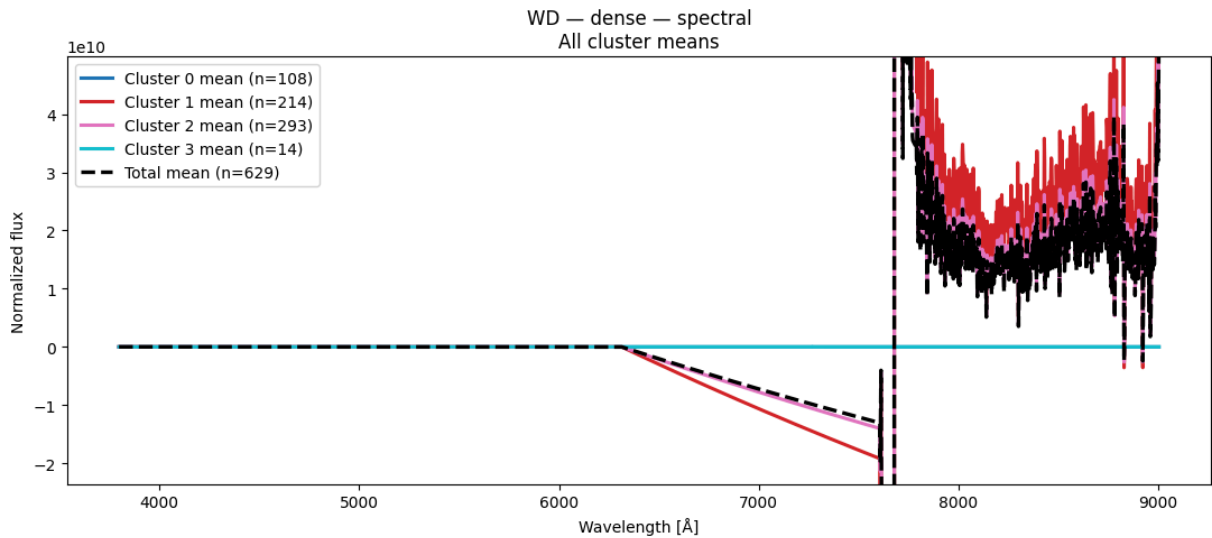
```
In [60]: plot_spectra_spectral(
            X_wd_spec,
            labels_dense_k4_wd,
            W_wd[0],
            class_name="WD",
            layer_name="dense"
        )
```

```
C:\Users\javip\AppData\Local\Temp\ipykernel_52368\531566132.py:28: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get_cmap()`` or ``pyplot.get_cmap()`` instead.
    cmap = plt.cm.get_cmap("tab10", n_clusters)
```







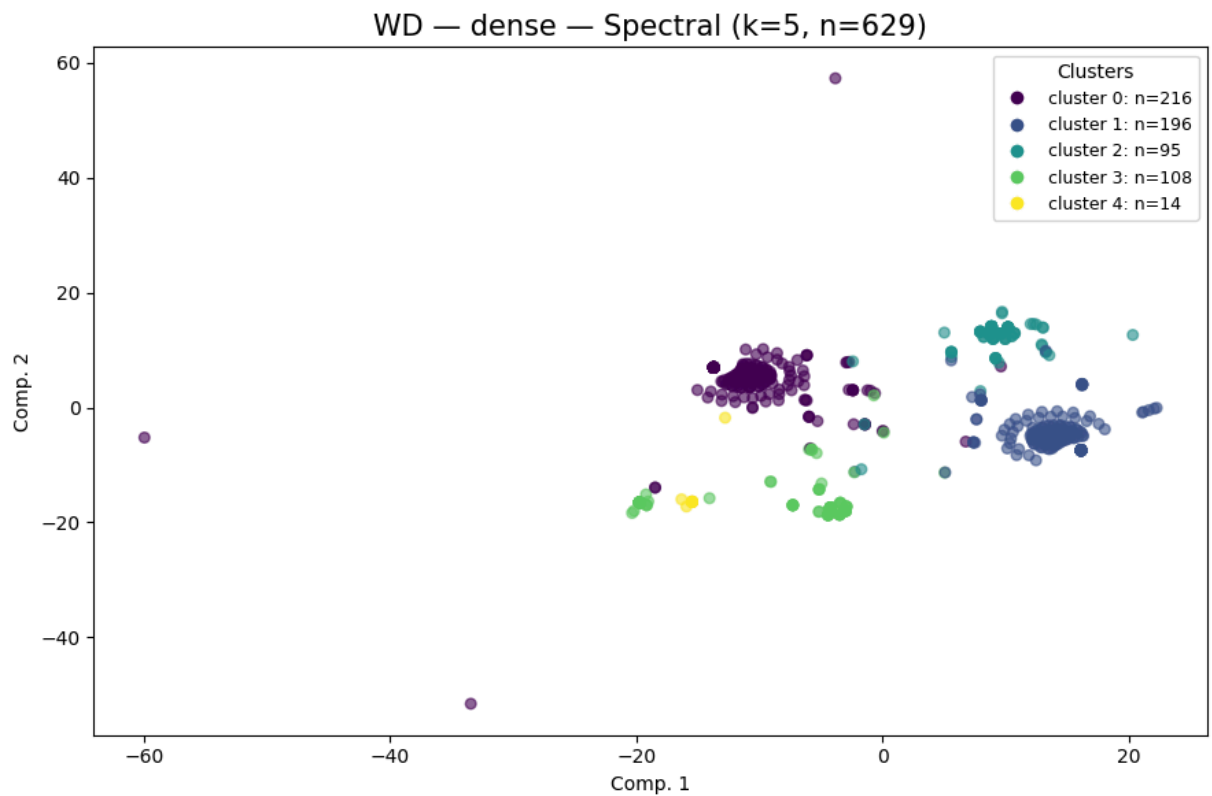


```
Out[60]: {'total_mean': array([1.8663167e+00, 1.4631674e+00, 2.3618588e+00, ..., 3.2000662e
+10,
      4.3846500e+10, 5.5692263e+10], dtype=float32),
      'group_means': {0: array([2.580767 , 2.5571568 , 2.537982 , ..., 0.11281608, 0.
61790997,
      0.7553298 ], dtype=float32),
      1: array([2.1614366e+00, 1.7890079e+00, 1.8644538e+00, ..., 4.7029010e+10,
      6.4437957e+10, 8.1846788e+10], dtype=float32),
      2: array([1.9897188e+00, 2.3239415e+00, 2.4599888e+00, ..., 3.4348823e+10,
      4.7063904e+10, 5.9778884e+10], dtype=float32),
      3: array([-1.0738884e+01, -2.9971674e+01, 6.5526977e+00, ...,
      3.9323906e+05, 3.0205644e+05, 3.9240656e+05], dtype=float32)},
      'unique_clusters': array([0, 1, 2, 3])}
```

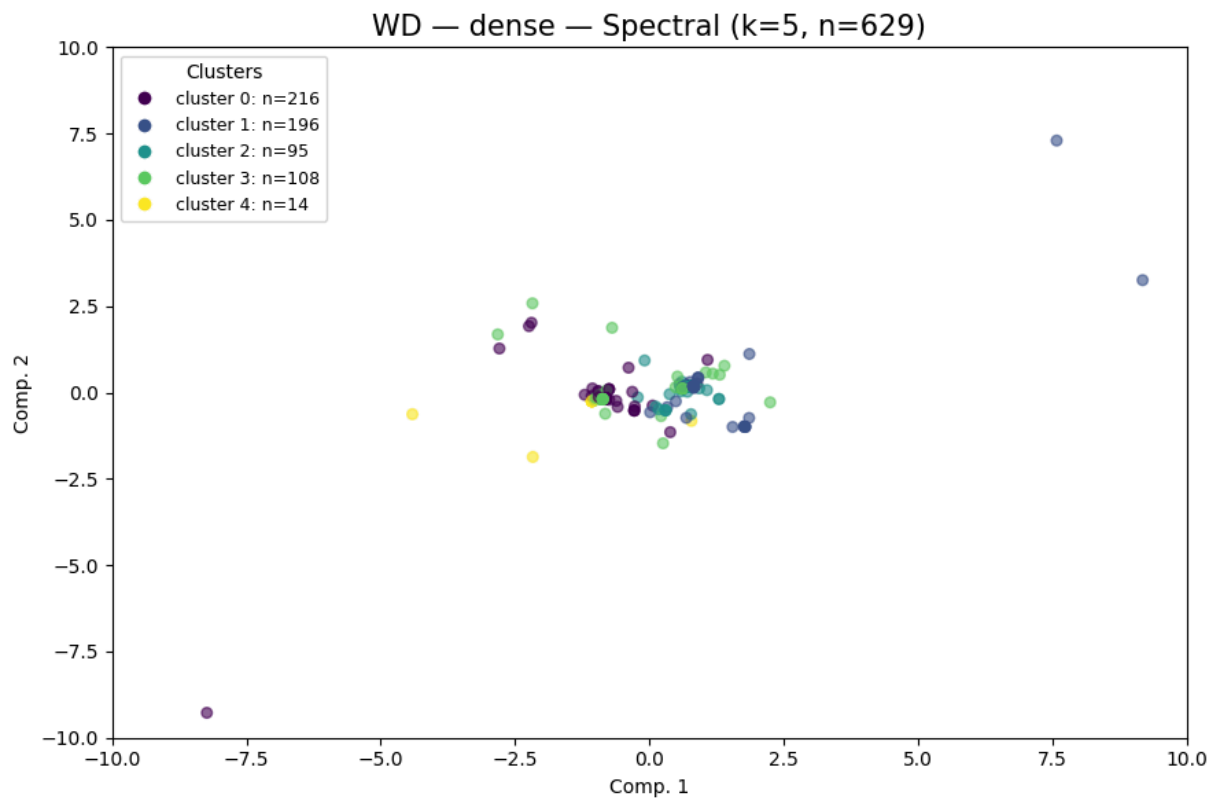
Spectral Clustering k = 5

```
In [61]: res_dense_k5 = cluster_plot_Spect(
      repr_wd_32["dense"],
      "WD",
      "dense",
      spectral_k=5,
      vis_method="tsne",
      container=cluster_labels_all
    )

labels_dense_k5_wd = res_dense_k5["spectral"]
cluster_plot_Spect(
      repr_wd_32["dense"],
      "WD",
      "dense",
      spectral_k=5,
      vis_method="pca",
      xlim=(-10,10), ylim=(-10,10)
    )
```



```
WD | dense | spectral | k=5  
clusters=5 noise=0 sil=0.3755 DB=2.5484 CH=14.72
```



WD	dense	spectral	k=5		
clusters=5	noise=0	sil=0.3755	DB=2.5484	CH=14.72	

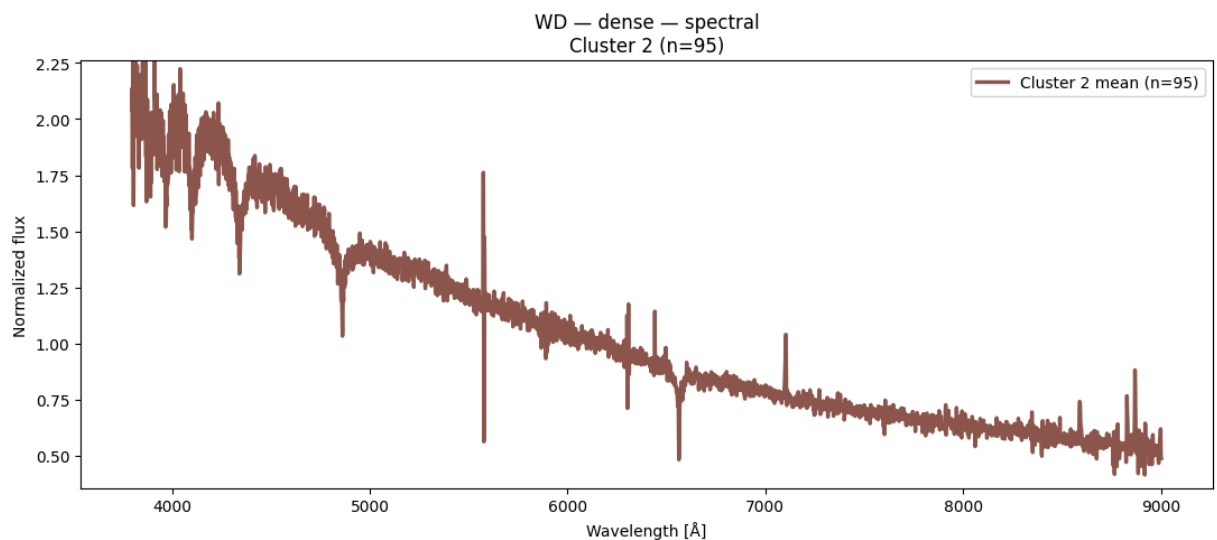
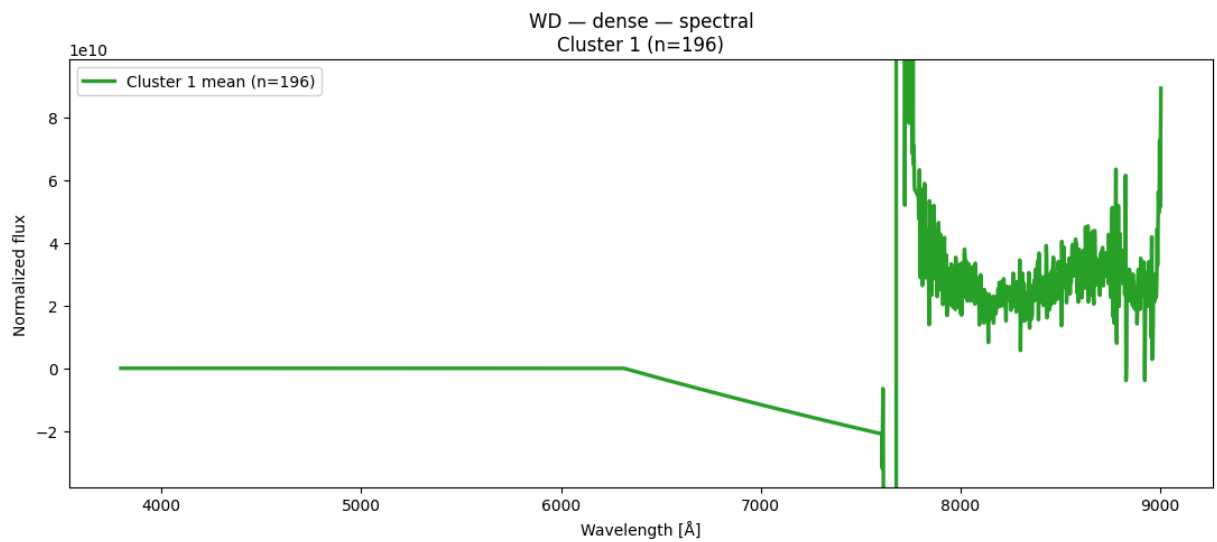
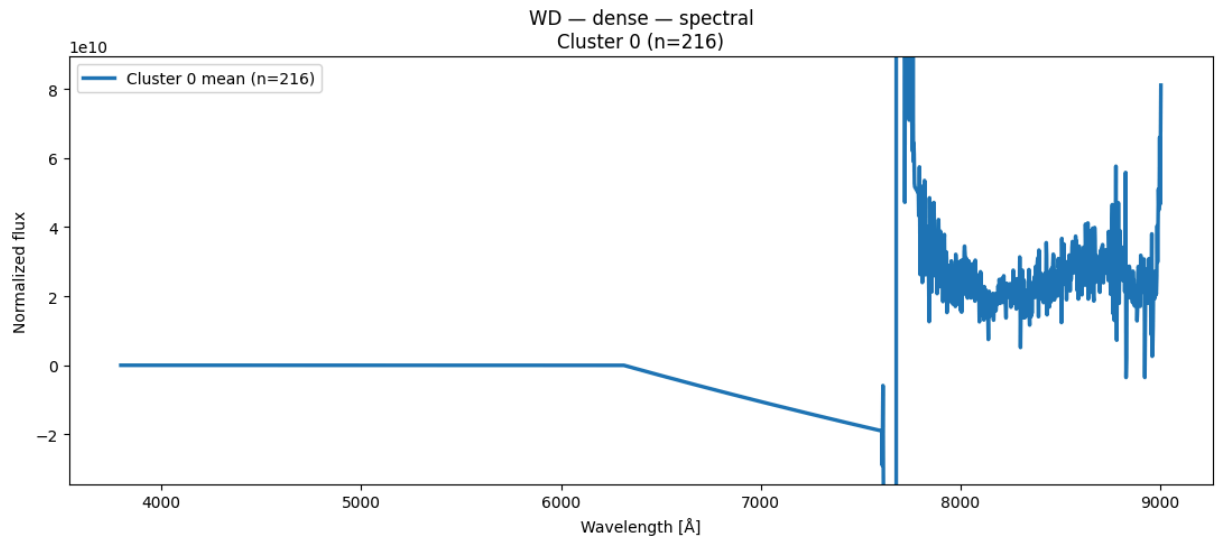
```
Out[61]: {'spectral': array([0, 0, 0, 3, 3, 2, 3, 3, 3, 0, 2, 0, 2, 2, 2, 2, 2, 2, 0, 2, 1,
2,
1, 3, 1, 1, 0, 0, 0, 0, 0, 0, 2, 2, 2, 2, 2, 2, 2, 2, 1, 1, 1, 3,
1, 1, 0, 0, 0, 0, 0, 0, 1, 3, 3, 0, 3, 3, 3, 3, 1, 1, 1, 1, 1, 1,
0, 0, 0, 0, 0, 0, 0, 3, 4, 3, 3, 3, 3, 3, 0, 3, 1, 1, 1, 1, 0, 0,
0, 0, 0, 0, 2, 2, 2, 2, 2, 2, 2, 2, 1, 3, 3, 1, 1, 1, 0, 0, 0, 0,
0, 0, 2, 2, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 3, 0, 4, 0, 0, 0, 0,
3, 3, 3, 3, 4, 4, 3, 3, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 3, 3,
3, 3, 3, 3, 3, 3, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 2, 2, 2, 2,
2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 2, 2, 1, 2, 2, 2,
2, 2, 3, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 3, 3, 3, 3, 3, 3, 4, 3,
3, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 3, 1, 1, 0, 0, 0, 1, 1, 1,
3, 1, 1, 0, 0, 0, 0, 0, 0, 2, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1,
3, 0, 0, 0, 0, 0, 0, 1, 3, 1, 1, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0,
0, 0, 0, 0, 4, 4, 3, 4, 3, 3, 3, 3, 3, 1, 1, 1, 1, 1, 0, 0, 0, 0,
0, 0, 3, 1, 1, 1, 0, 0, 0, 3, 1, 1, 4, 1, 1, 0, 0, 0, 0, 0, 2,
2, 2, 2, 2, 2, 2, 3, 1, 3, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1,
1, 0, 0, 0, 1, 3, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 3, 3, 3, 3, 3, 3,
4, 4, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 3, 3, 1, 0, 0, 0, 1,
1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 2, 2, 2, 2, 2, 2, 2, 1, 3, 1,
1, 1, 3, 0, 0, 0, 0, 0, 0, 3, 1, 3, 1, 0, 0, 0, 1, 3, 1, 1, 1, 1,
0, 0, 0, 0, 0, 0, 3, 3, 3, 4, 3, 3, 3, 3, 3, 1, 1, 1, 1, 1, 0, 0,
0, 0, 0, 0, 1, 1, 3, 1, 0, 0, 0, 1, 1, 1, 1, 3, 1, 0, 0, 0, 0, 0,
0, 2, 2, 2, 2, 2, 2, 2, 2, 3, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1,
0, 1, 1, 0, 0, 0, 1, 1, 1, 3, 3, 1, 0, 0, 0, 0, 0, 3, 4, 3, 3, 3,
3, 3, 3, 3, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 3, 0, 0,
0, 1, 3, 1, 1, 1, 1, 0, 0, 3, 0, 0, 0, 2, 2, 2, 2, 2, 2, 2, 1,
1, 1, 1, 1, 0, 0, 0, 0, 0, 3, 1, 3, 3, 1, 0, 0, 0, 1, 1, 1, 1,
1, 1, 0, 0, 0, 0, 0, 0, 3, 3, 3, 3, 3, 3, 4, 2, 2, 3, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 1, 1, 1])},
'metrics': {'n_clusters': 5,
'n_noise': 0,
'silhouette': 0.37552,
'davies_bouldin': 2.548383,
'calinski_harabasz': 14.7189}}
```

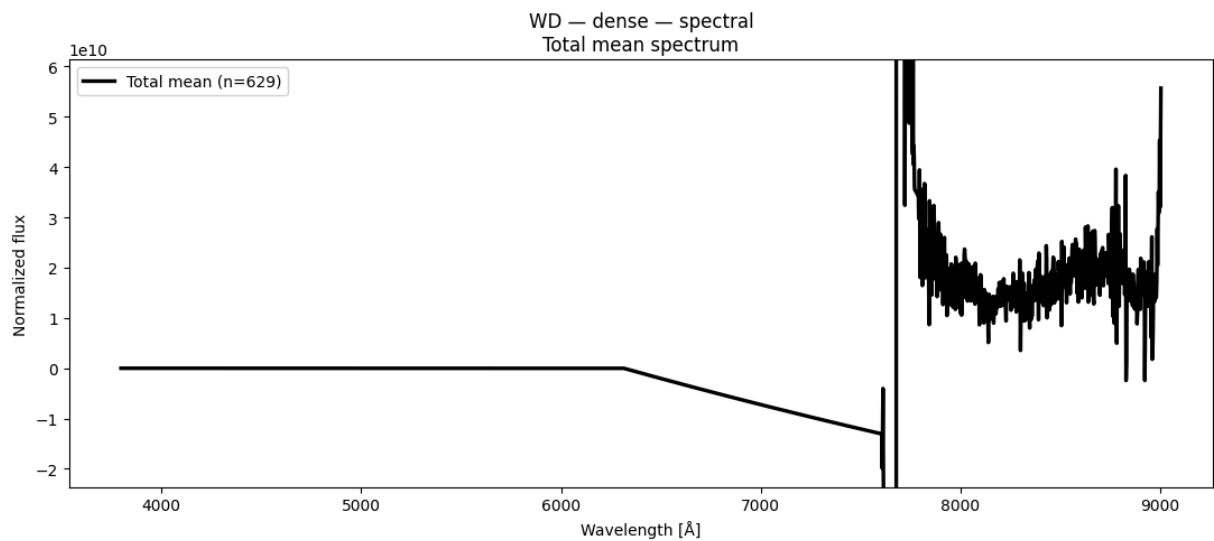
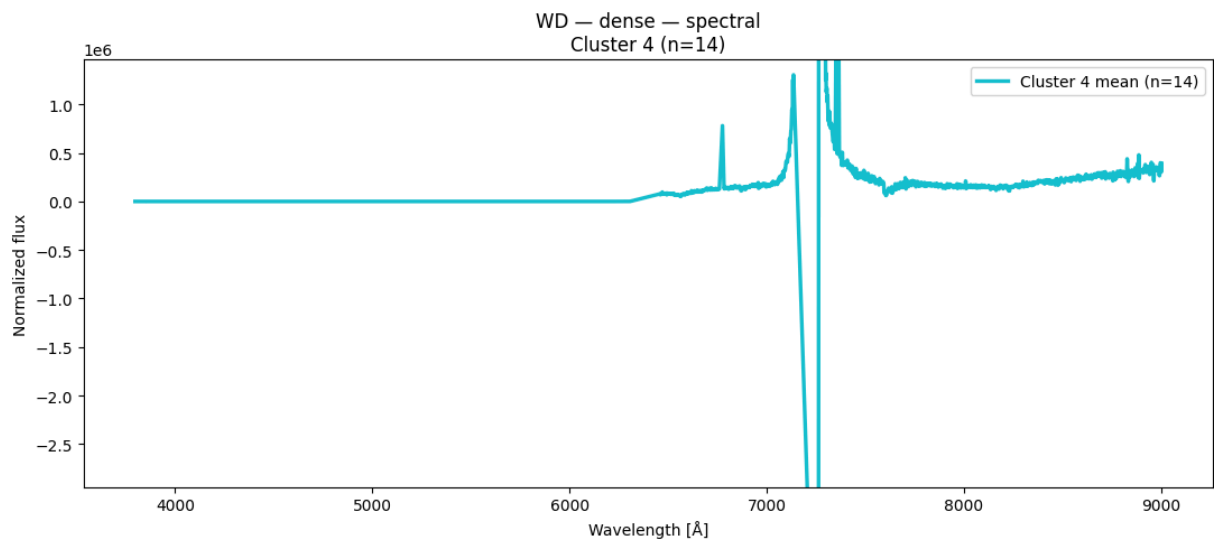
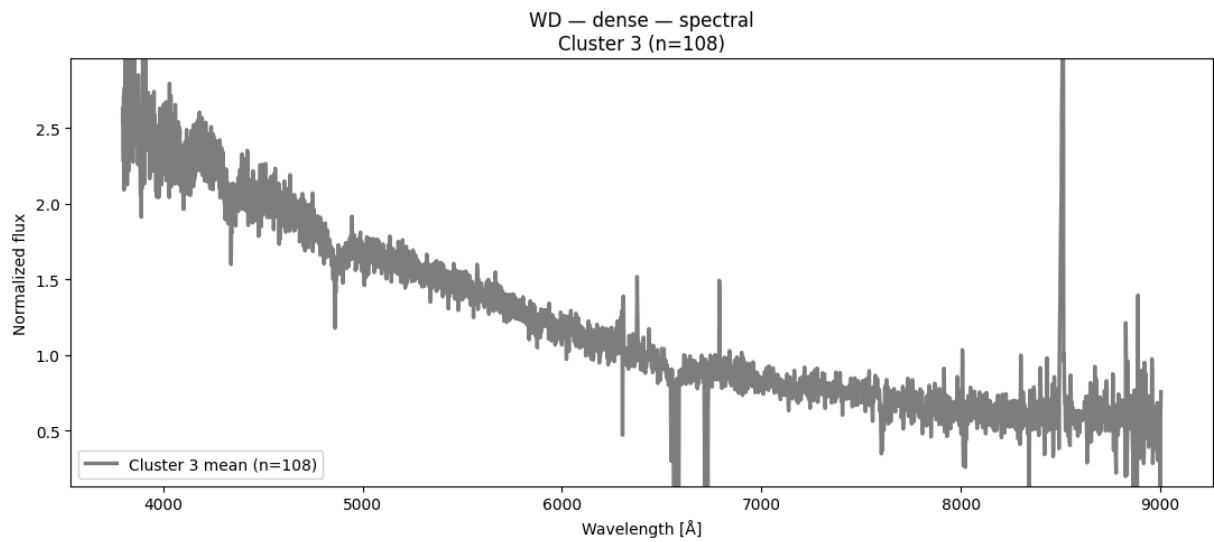
Espectros obtenidos para dense con Spectral Clustering k = 5 y n = 700

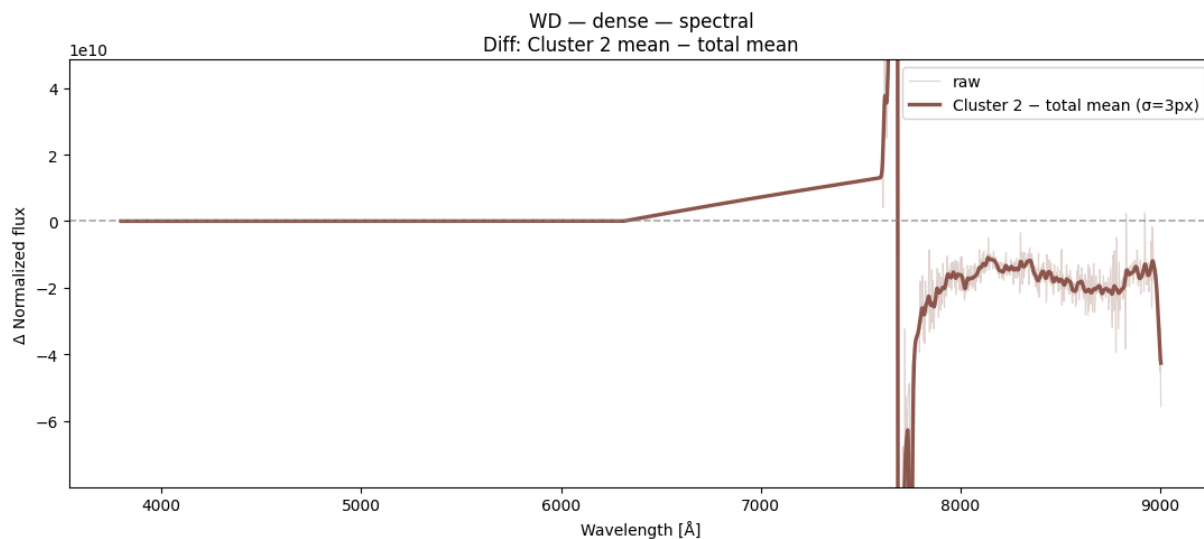
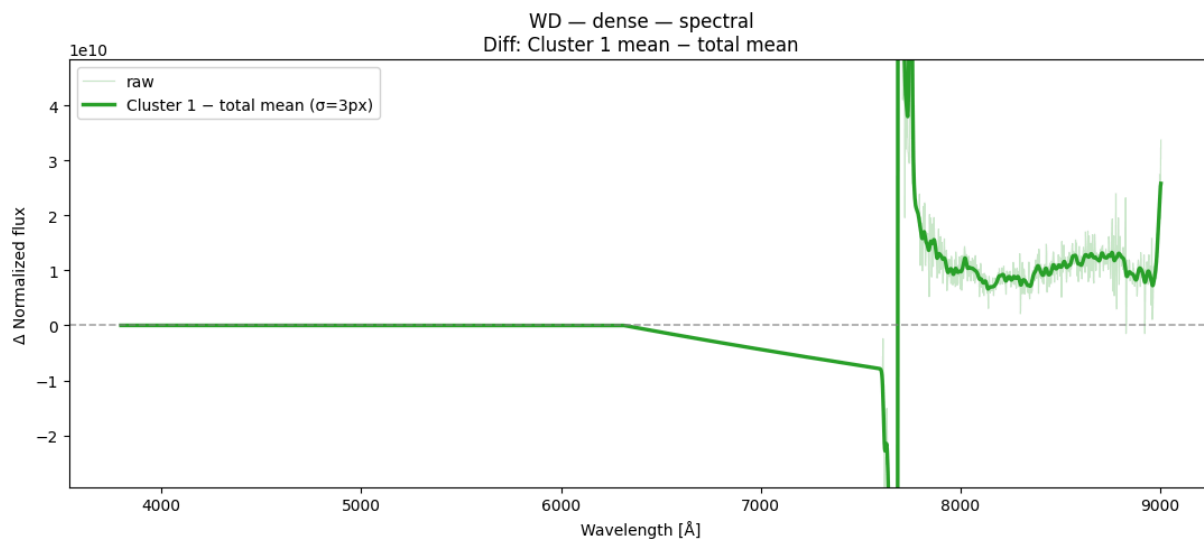
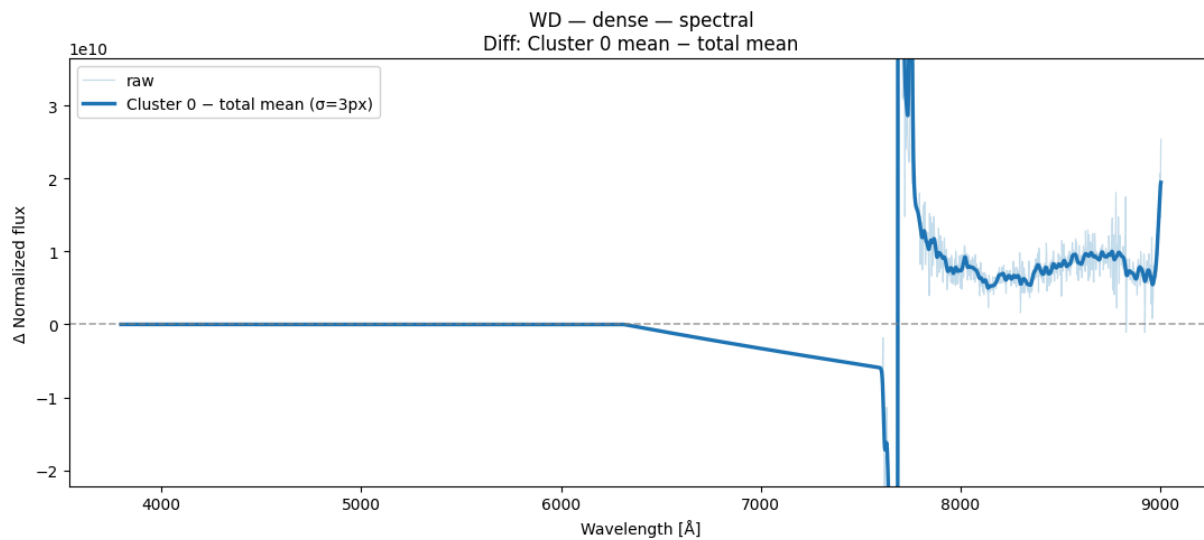
```
In [62]: plot_spectra_spectral(
X_wd_spec,
labels_dense_k5_wd,
W_wd[0],
class_name="WD",
layer_name="dense"
)
```

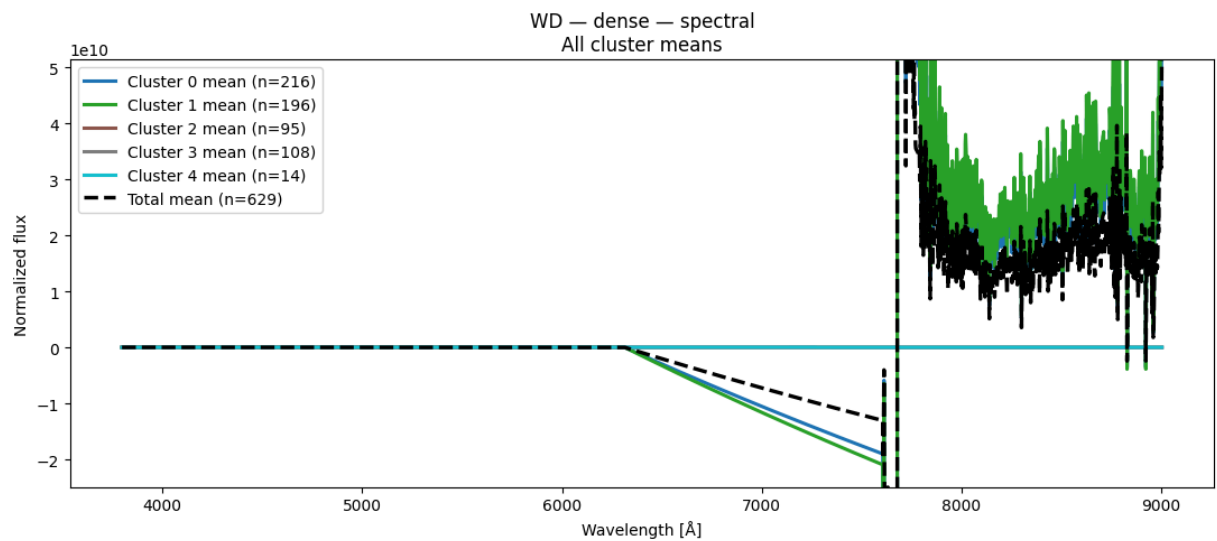
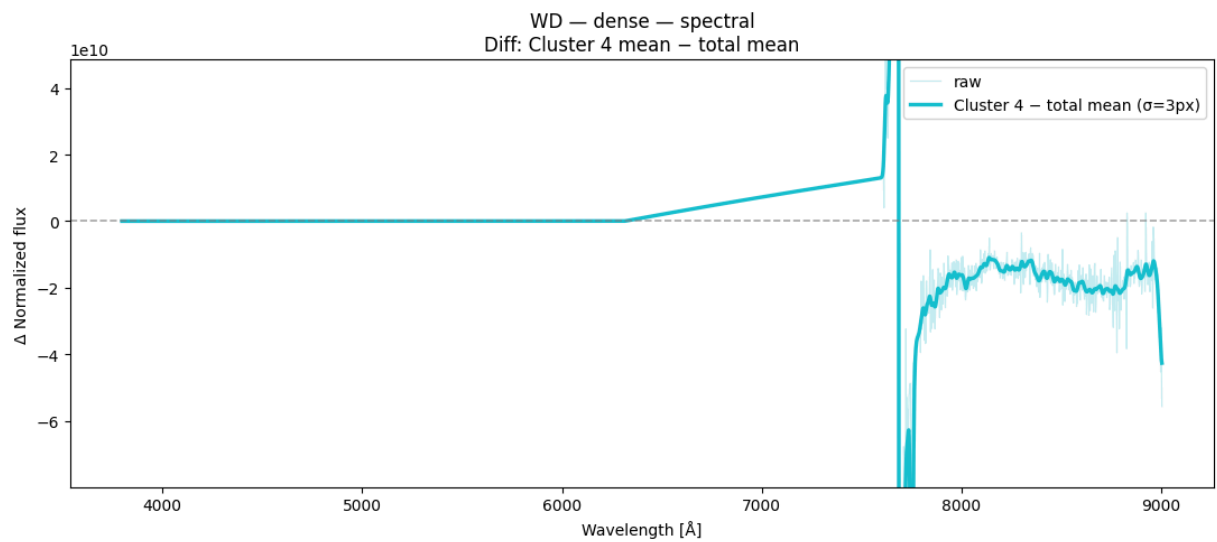
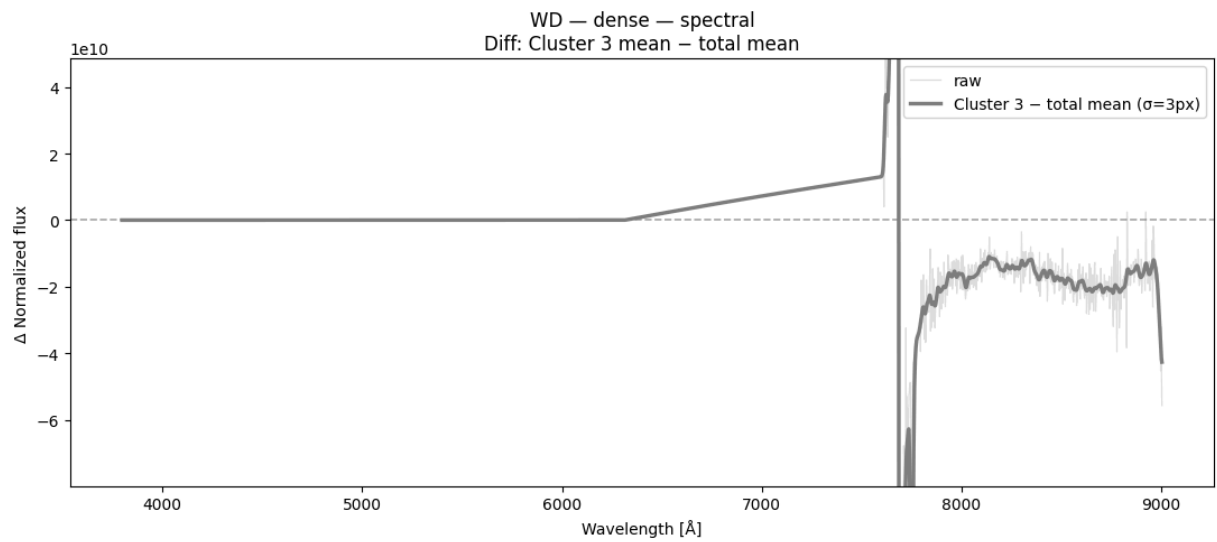
C:\Users\javip\AppData\Local\Temp\ipykernel_52368\531566132.py:28: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get\_cmap()`` or ``pyplot.get\_cmap()`` instead.

```
cmap = plt.cm.get_cmap("tab10", n_clusters)
```





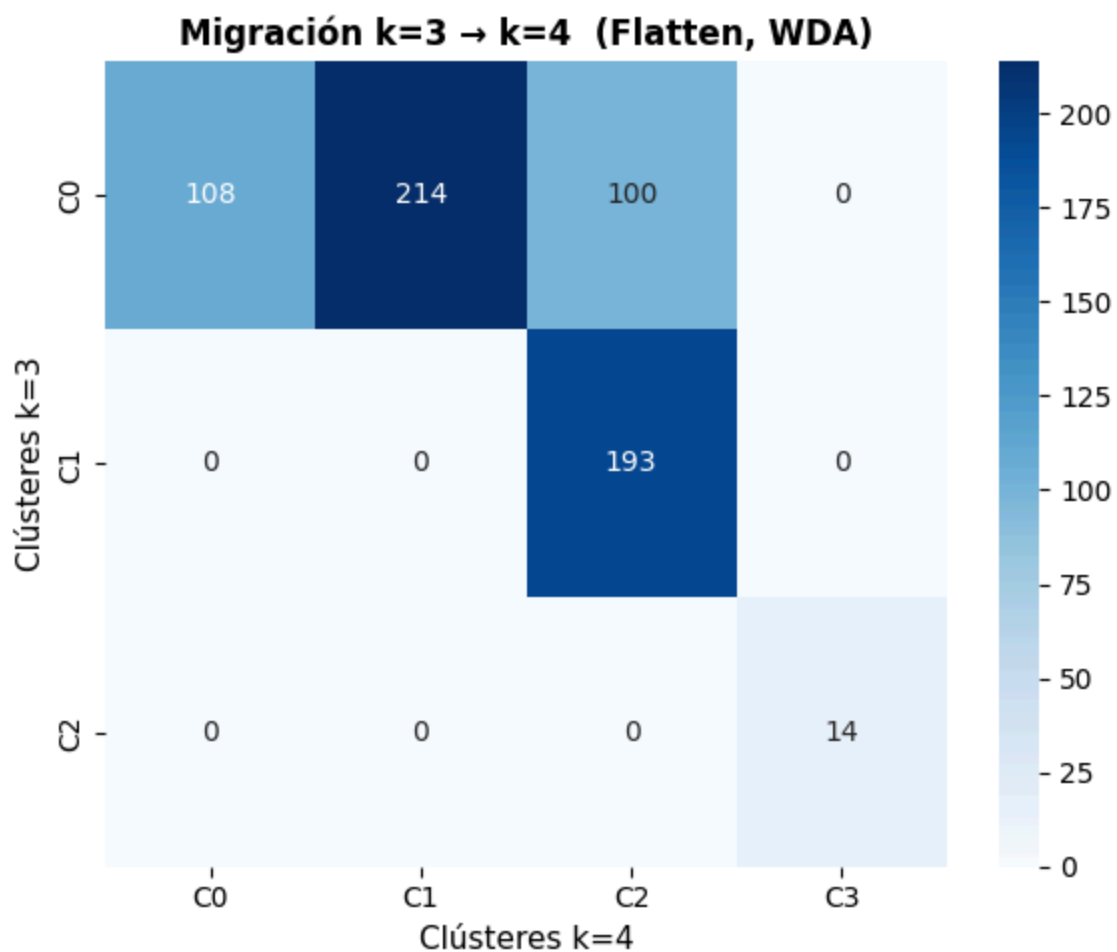


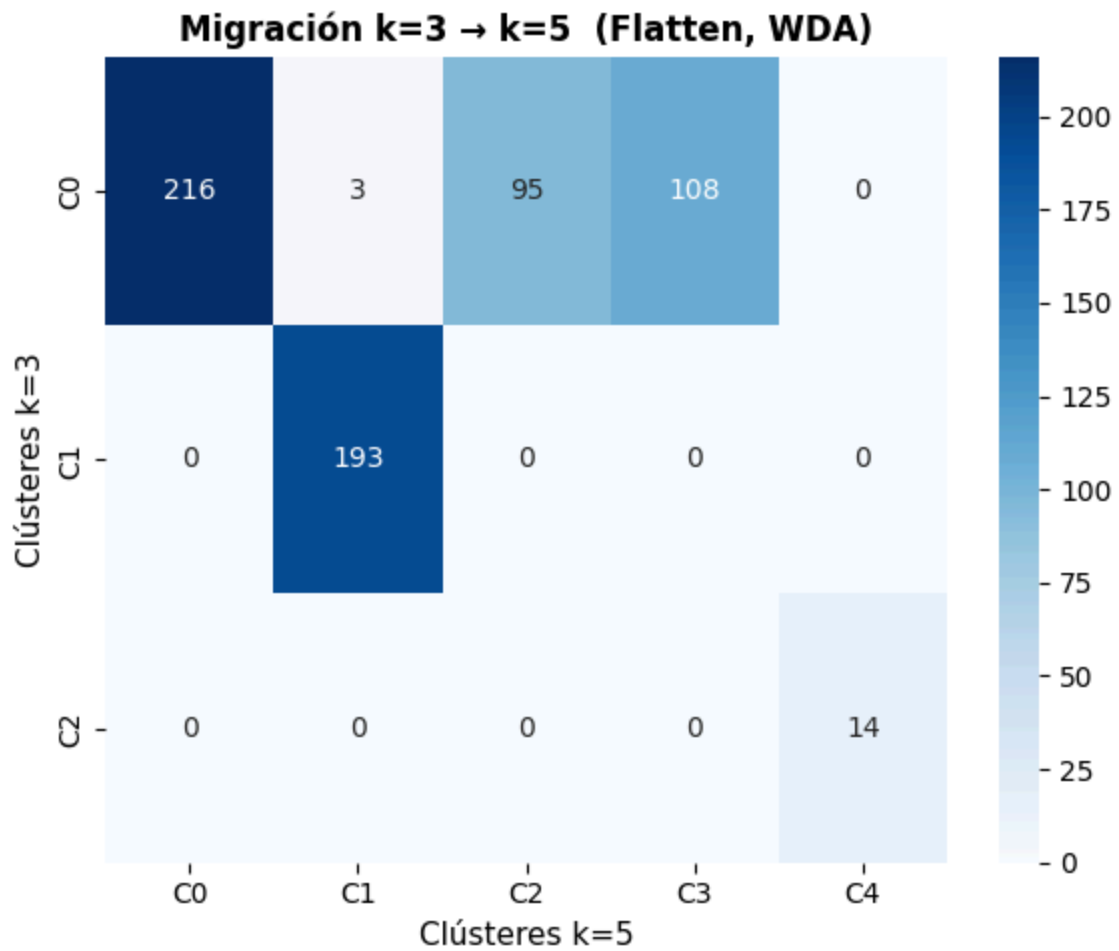


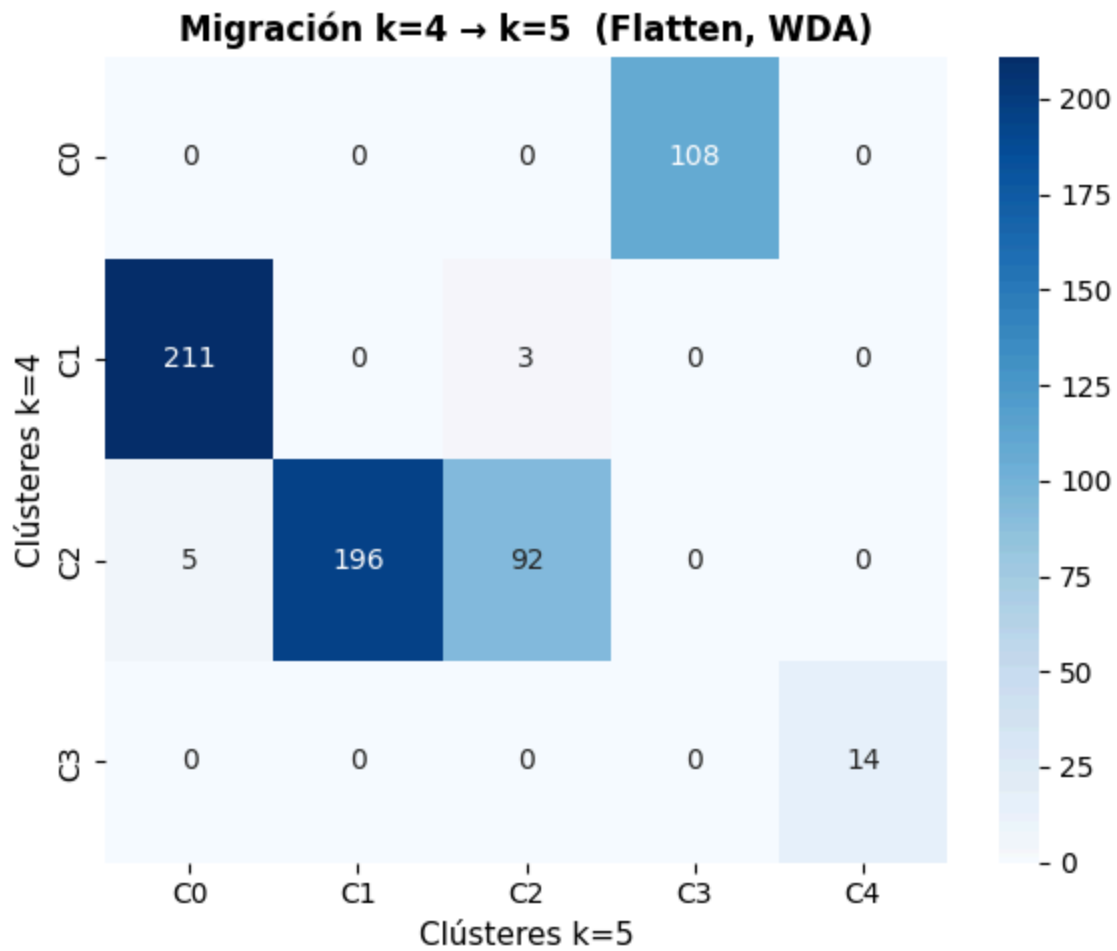
```
Out[62]: {'total_mean': array([1.8663167e+00, 1.4631674e+00, 2.3618588e+00, ..., 3.2000662e
+10,
        4.3846500e+10, 5.5692263e+10], dtype=float32),
'group_means': {0: array([2.5190859e+00, 2.2666886e+00, 2.5328262e+00, ..., 4.659
3556e+10,
        6.3841309e+10, 8.1088946e+10], dtype=float32),
1: array([1.5679045e+00, 1.8934050e+00, 1.9539561e+00, ..., 5.1347984e+10,
        7.0355730e+10, 8.9363333e+10], dtype=float32),
2: array([2.0431914 , 2.1373715 , 1.9968835 , ..., 0.6182493 , 0.4940375 ,
        0.48755923], dtype=float32),
3: array([2.580767 , 2.5571568 , 2.537982 , ..., 0.11281608, 0.61790997,
        0.7553298 ], dtype=float32),
4: array([-1.0738884e+01, -2.9971674e+01, 6.5526977e+00, ...,
        3.9323906e+05, 3.0205644e+05, 3.9240656e+05], dtype=float32)},
'unique_clusters': array([0, 1, 2, 3, 4])}
```

Análisis de migración

```
In [63]: plot_migration_matrix(labels_dense_k3_wd, labels_dense_k4_wd, ka=3, kb=4)
plot_migration_matrix(labels_dense_k3_wd, labels_dense_k5_wd, ka=3, kb=5)
plot_migration_matrix(labels_dense_k4_wd, labels_dense_k5_wd, ka=4, kb=5)
```

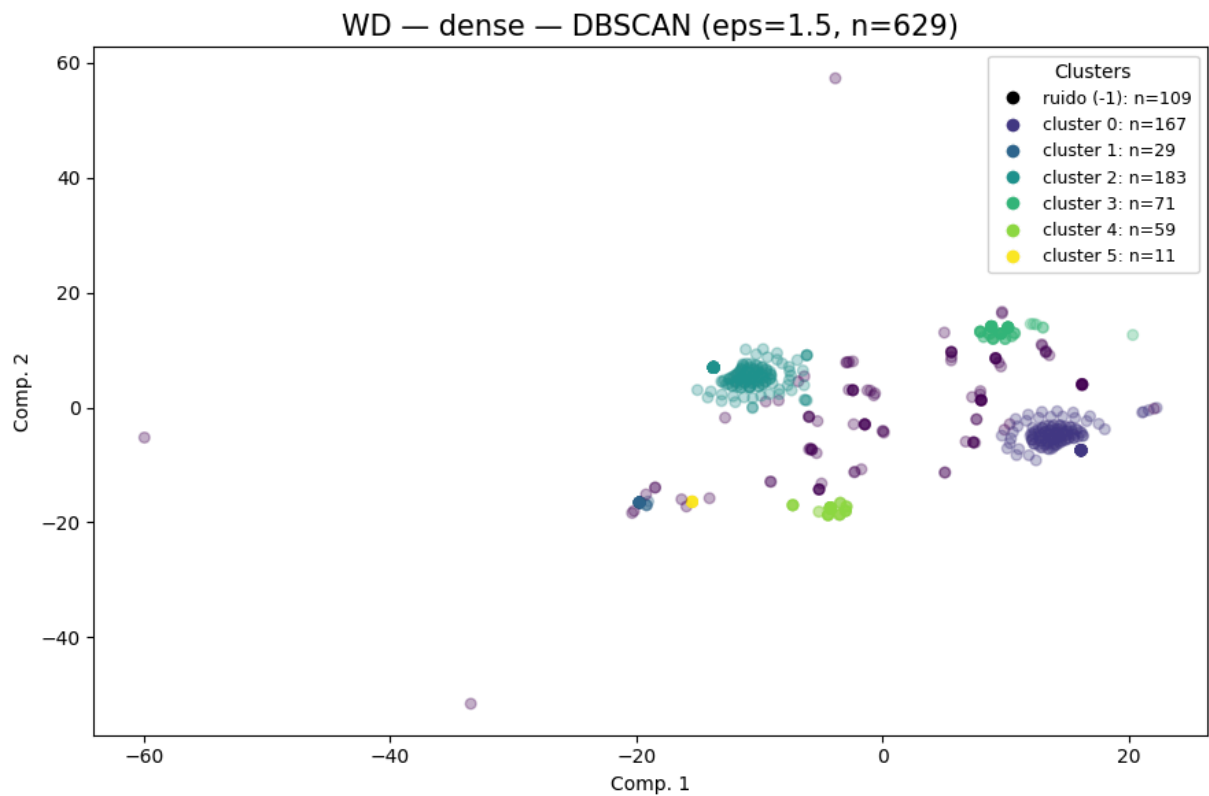




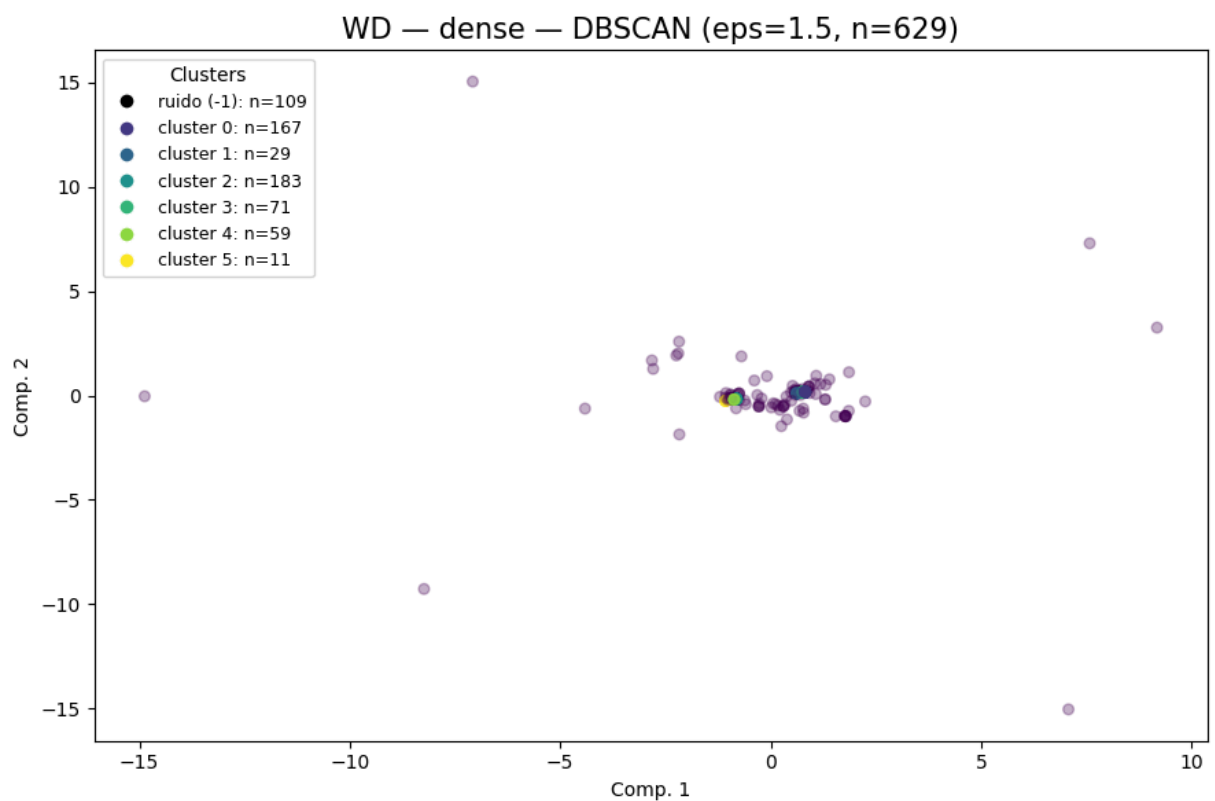


DBSCAN

```
In [64]: res_dbs_dense = cluster_plot_DBSCAN(
    repr_wd_32["dense"],
    "WD",
    "dense",
    dbscan_eps=1.5,
    dbscan_min_samples=10,
    vis_method="tsne",
    xlim=None,
    ylim=None,
    container=cluster_labels_all
)
labels_dbs_dense_wd = res_dbs_dense["dbscan"]
cluster_plot_DBSCAN(
    repr_wd_32["dense"],
    "WD",
    "dense",
    dbscan_eps=1.5,
    dbscan_min_samples=10,
    vis_method="pca",
    xlim=None,
    ylim=None
)
```

```
WD | dense | dbscan | eps=1.5
clusters=6 noise=109 sil=1.0000 DB=0.0000 CH=234263837673.74
DBCVC = 0.9999 (rango [-1,1], específico DBSCAN)
```



```
WD | dense | dbscan | eps=1.5
clusters=6 noise=109 sil=1.0000 DB=0.0000 CH=234263837673.74
DBCV = 0.9999 (rango [-1,1], específico DBSCAN)
```

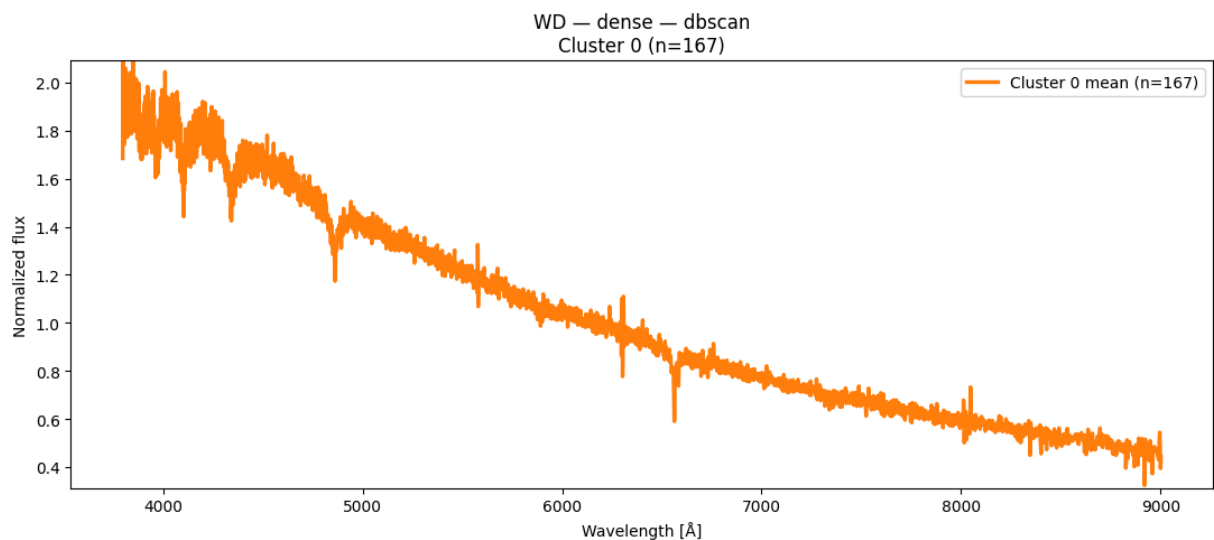
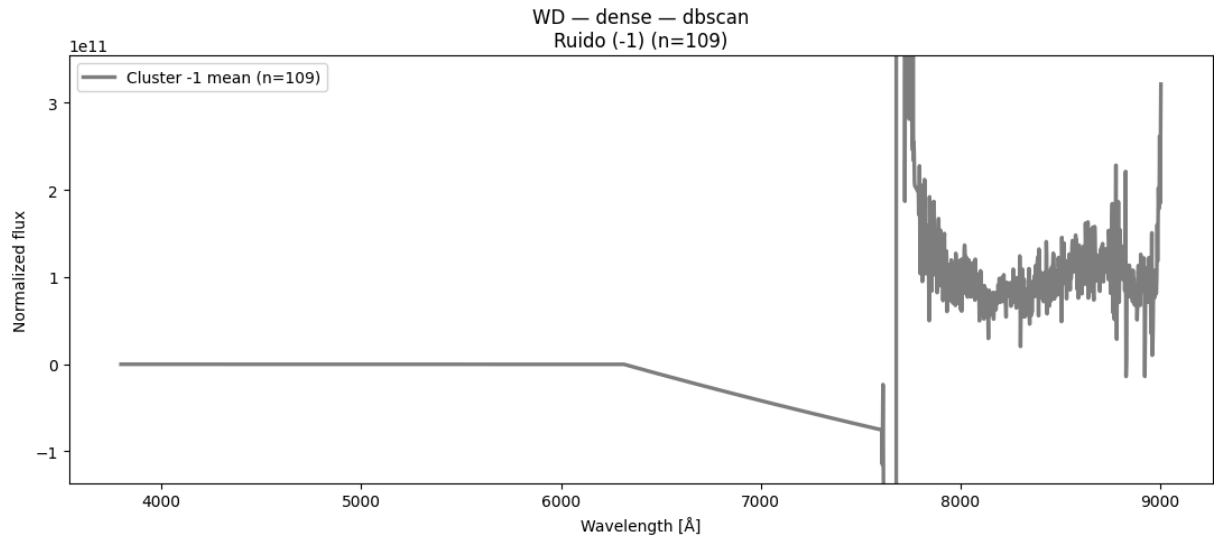
```
Out[64]: {'dbscan': array([-1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1,
-1,
-1, -1, -1, -1, -1, 0, 1, 0, 0, 2, 2, 2, 2, 2, 2, -1, 3,
3, -1, 3, 3, 3, 3, 0, -1, 0, 1, 0, 0, 2, 2, 2, 2, 2,
2, -1, 4, 4, -1, 4, 4, 4, 4, 0, 0, 0, 0, 0, 0, 2, 2,
2, 2, 2, 2, -1, 4, 5, 4, 4, 4, -1, 4, -1, 1, 0, 0, 0,
0, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 0, 1,
1, 0, 0, 0, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3,
3, 0, 0, 0, 0, 0, 1, 2, -1, 2, -1, 2, 2, 4, 4, 4, 4,
5, 5, 4, 4, 0, 0, 0, 0, -1, 0, 2, 2, 2, -1, 2, 2, 4,
-1, 4, 4, 4, 4, 4, -1, 0, 0, 0, 0, 0, 0, 2, 2, 2, 2,
2, 2, 3, 3, 3, 3, 3, 3, -1, 3, -1, 0, 0, 0, 0, 0, 2,
-1, 2, 2, 2, 2, 3, 3, -1, 3, 3, 3, 3, 3, 1, 0, 0, 0,
0, 0, 2, 2, 2, -1, 2, -1, 4, 4, 4, 4, 4, 4, 5, 4, 1,
0, 0, 0, 0, 0, 2, -1, 2, 2, 2, 2, 0, -1, 0, 0, 2, 2,
2, 0, 0, -1, -1, 0, 0, 2, 2, 2, 2, 2, 2, 3, 3, 3, -1,
3, -1, 3, 3, 0, 0, 0, 0, 0, 1, -1, 2, 2, -1, 2, 2, 0,
1, 0, 0, 2, 2, 2, -1, 0, 0, 0, 0, 0, -1, 2, 2, 2, 2,
2, 5, 5, 4, 5, 4, 4, 4, 4, 1, 0, 0, 0, -1, 0, 2, 2,
2, 2, -1, 2, 1, 0, 0, 0, 2, 2, 2, -1, 0, 0, -1, -1, 0,
2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 1, 0, 1,
-1, 0, 0, -1, 2, 2, 2, 2, 2, -1, 0, 0, 0, 2, 2, 2, 0,
1, 0, 0, 0, -1, 2, 2, 2, 2, 2, -1, 4, 4, 4, 4, 4, 4,
5, 5, 0, 0, 0, 0, 0, 0, 2, -1, 2, 2, 2, -1, 0, 1, 1,
0, 2, 2, 2, 0, 0, 0, -1, 0, 0, 2, 2, 2, 2, 2, 2, 3,
3, 3, 3, 3, -1, 3, 3, 0, 1, 0, 0, 0, 1, 2, 2, 2, 2,
2, -1, 1, 0, 1, 0, 2, -1, 2, 0, 1, 0, 0, 0, 0, 2, 2,
-1, 2, 2, 2, 4, 4, 4, 5, 4, 4, 4, 4, 1, 0, 0, 0, 0,
0, -1, 2, 2, 2, 2, 2, 0, 0, -1, 0, 2, 2, 2, 0, 0, 0,
0, 1, 0, 2, 2, 2, 2, 2, -1, 3, 3, 3, 3, -1, 3, 3, 3,
-1, 0, 0, 0, -1, 0, 2, 2, 2, 2, 2, 2, 0, -1, 0, 0, 2,
2, 2, 0, 0, 0, -1, 1, 0, -1, 2, 2, 2, 2, -1, 5, -1, 4,
-1, 4, 4, 4, 4, 0, 0, 0, 0, 0, 0, -1, 2, 2, 2, 2, 2,
0, 0, 0, 1, 2, 2, 2, 0, 1, 0, 0, 0, 0, 2, 2, -1, 2,
2, 2, 3, 3, 3, 3, 3, 3, -1, 3, 0, 0, 0, 0, 0, 0, 2,
2, 2, 2, -1, -1, -1, 1, 1, 0, 2, 2, 2, 0, 0, 0, 0, 0,
0, 2, 2, 2, 2, 2, -1, 4, 4, 4, 4, 4, 4, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1],
dtype=int64),
'metrics': {'n_clusters': 6,
'n_noise': 109,
'silhouette': 0.999998,
'davies_bouldin': 1.3e-05,
'calinski_harabasz': 234263837673.741},
'dbcv': 0.999904}
```

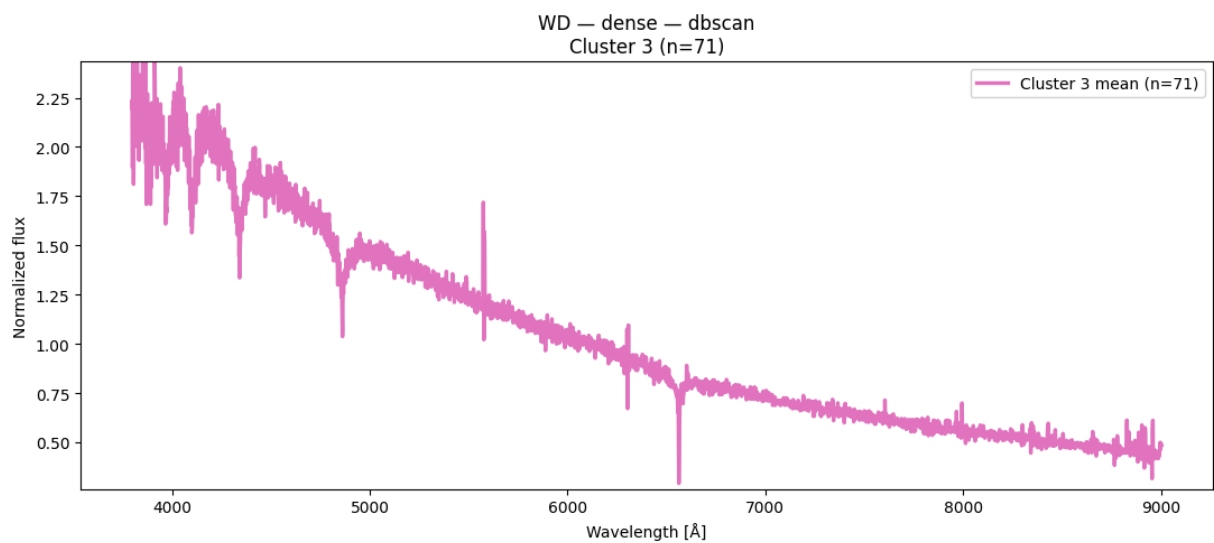
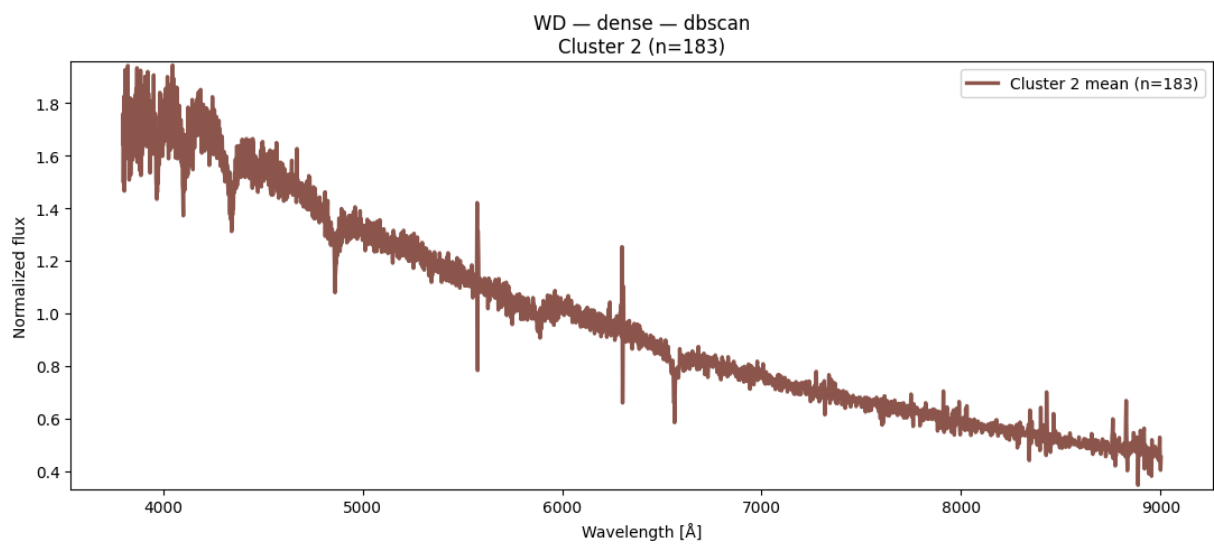
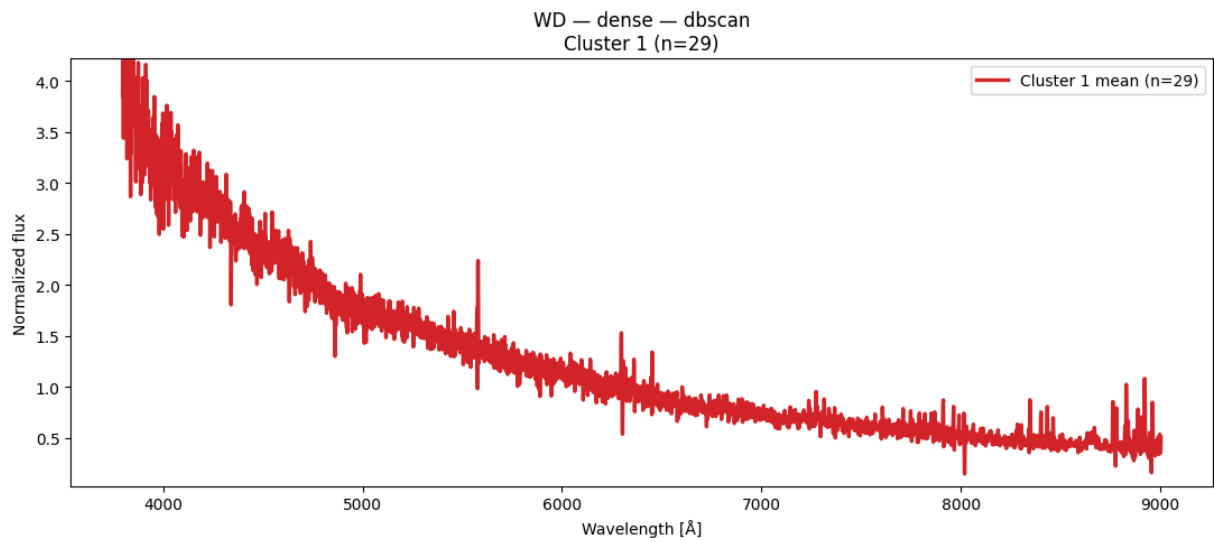
Espectros obtenidos para dense con DBSCAN y n = 700

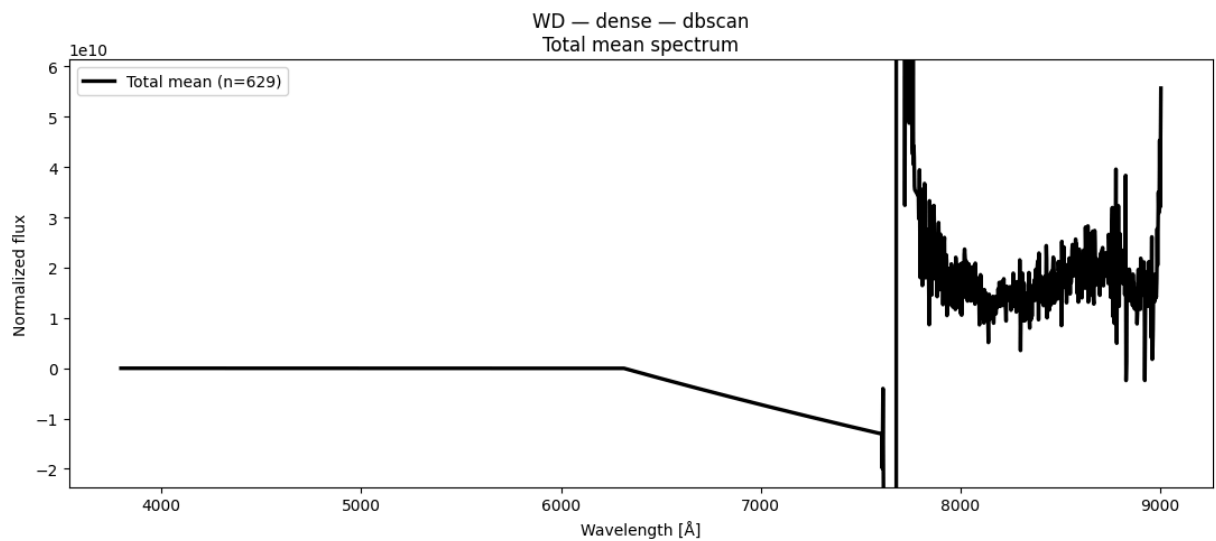
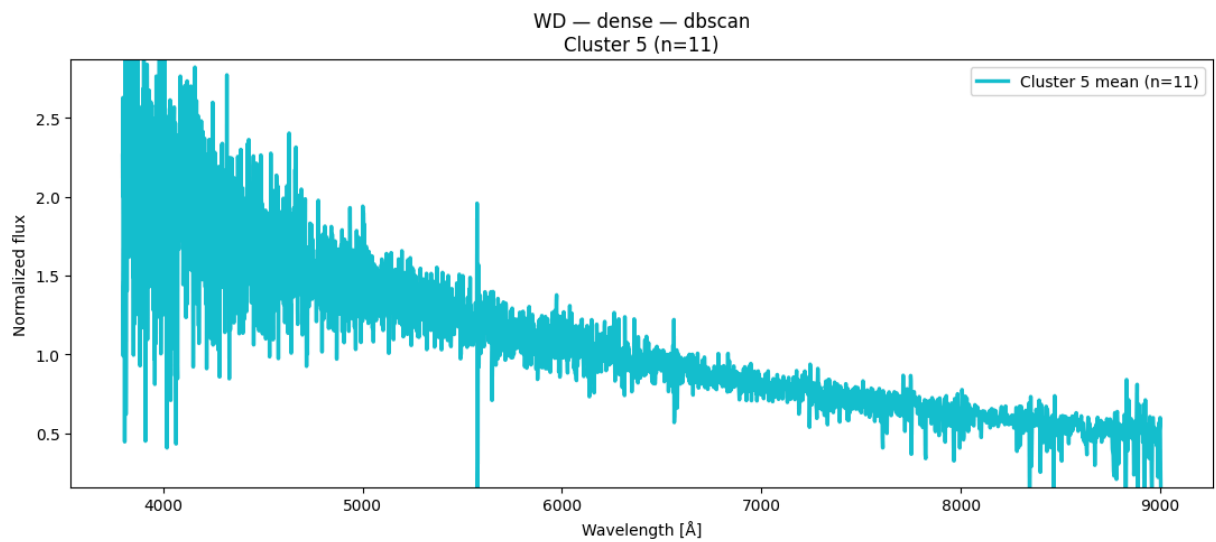
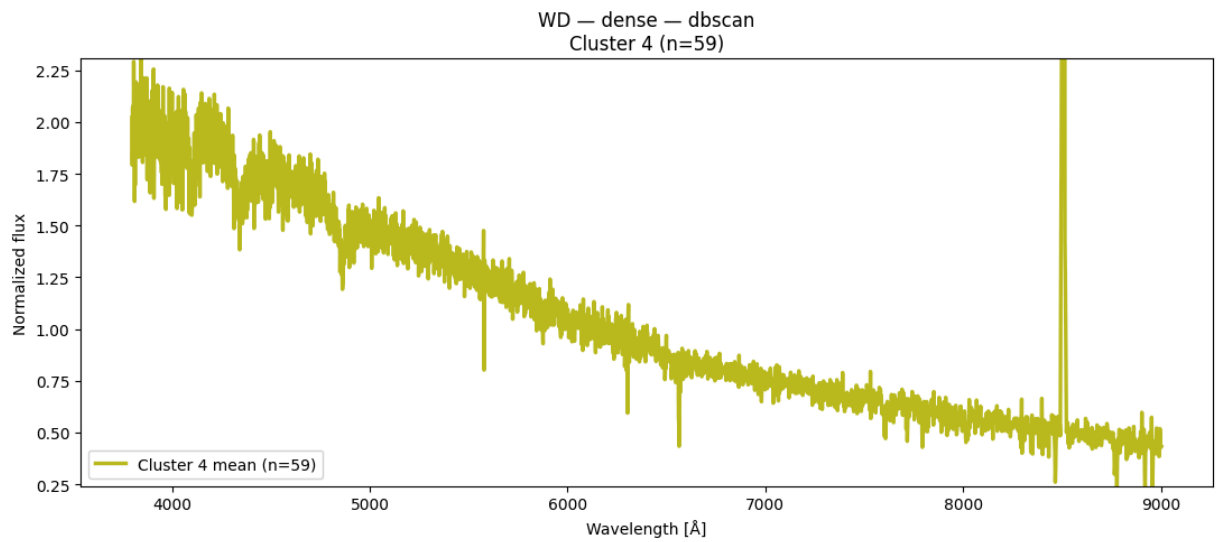
```
In [65]: plot_spectra_dbscan(
    X_wd_spec,
    labels_dbs_dense_wd,
    W_wd[0],
    class_name="WD",
    layer_name="dense"
)
```

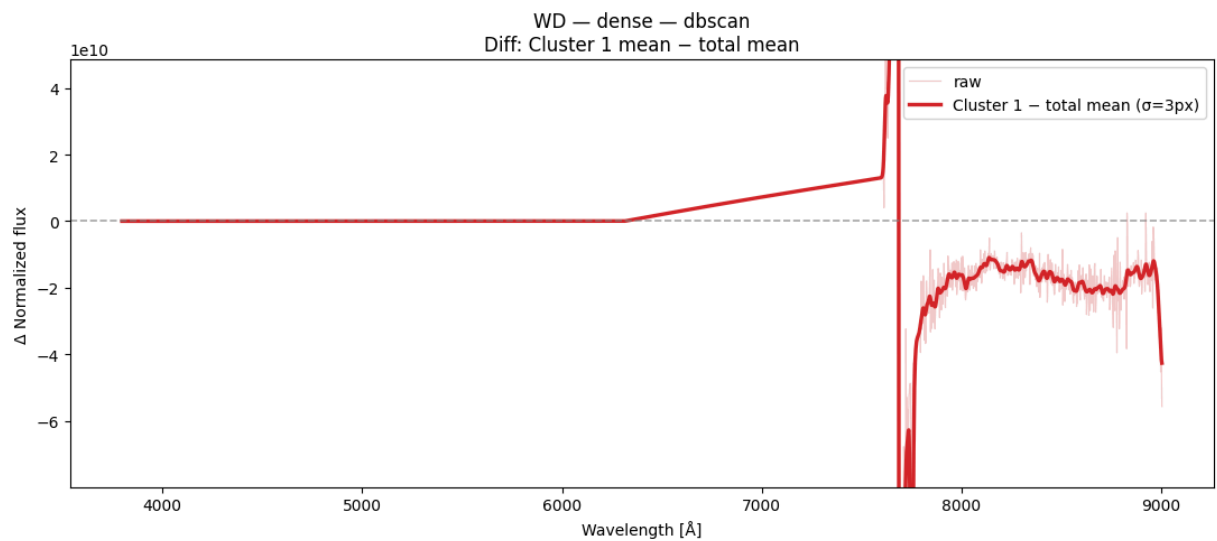
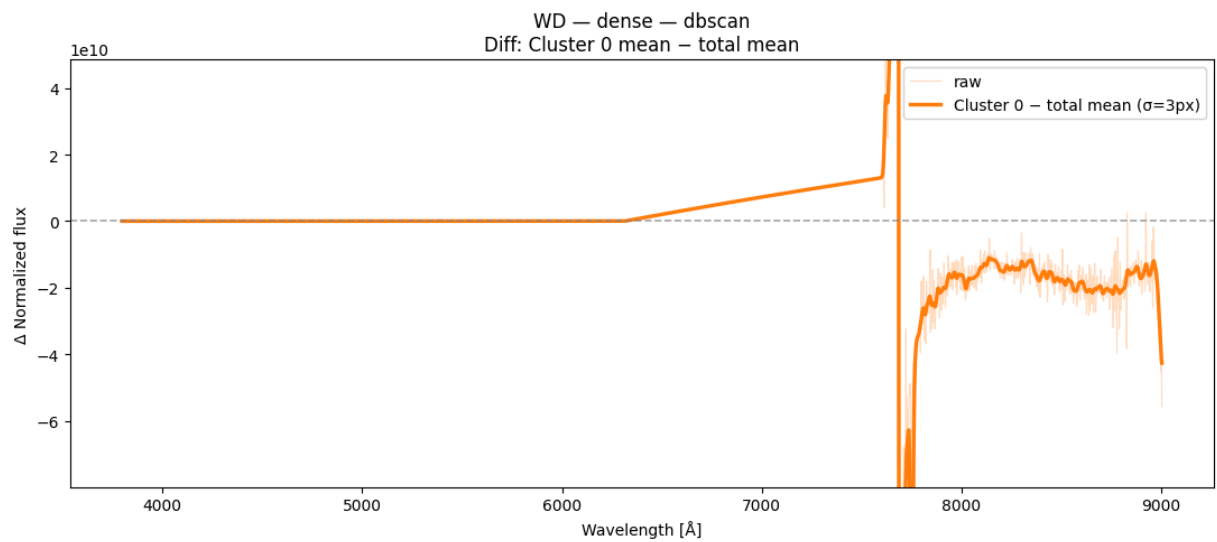
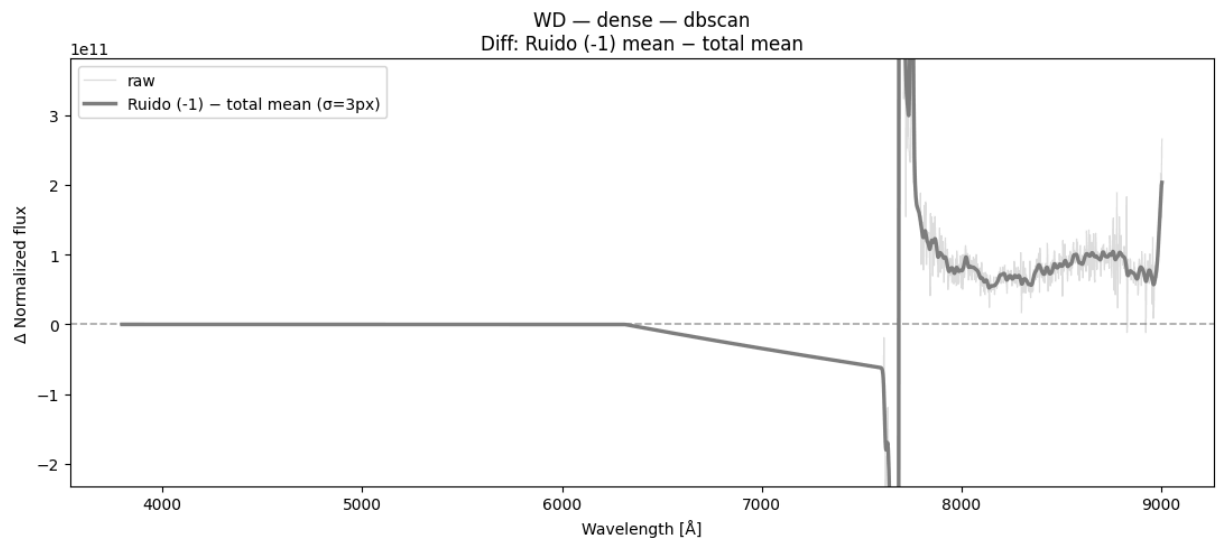
C:\Users\javip\AppData\Local\Temp\ipykernel_52368\531566132.py:28: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get\_cmap()`` or ``pyplot.get\_cmap()`` instead.

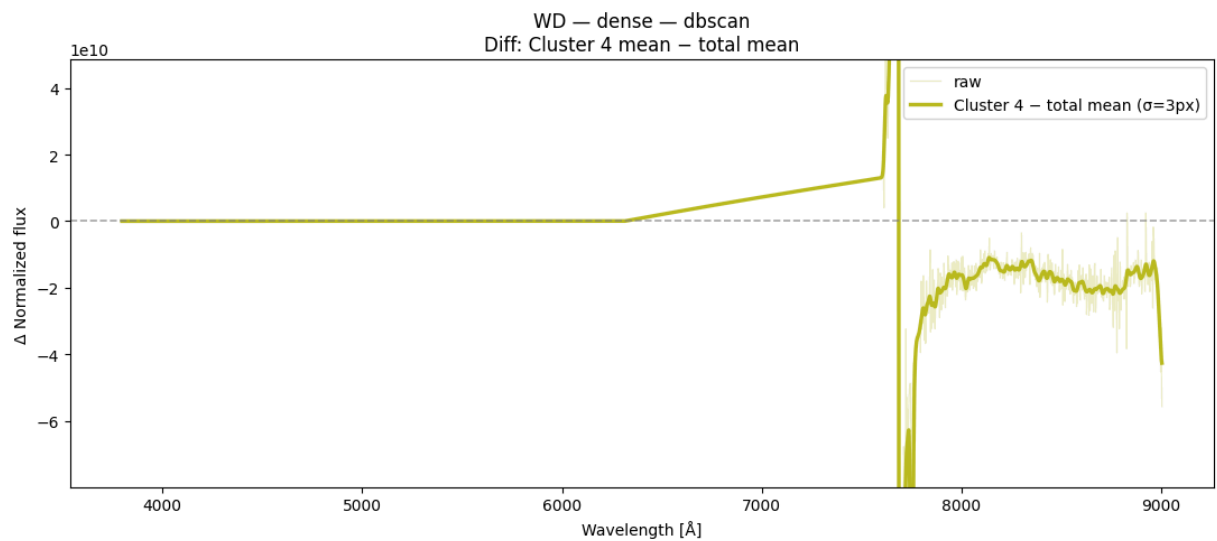
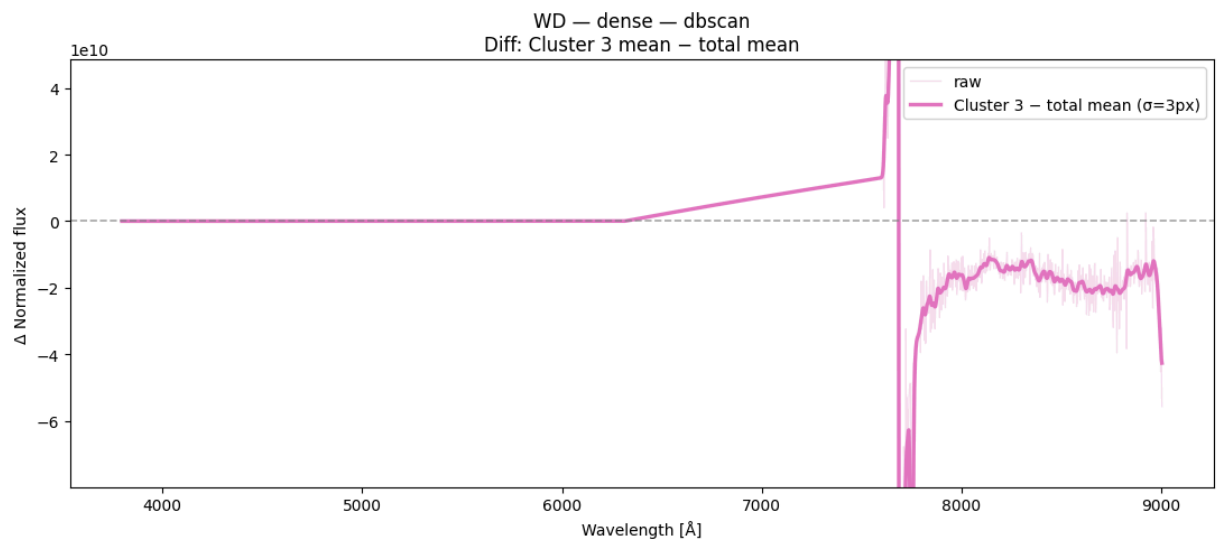
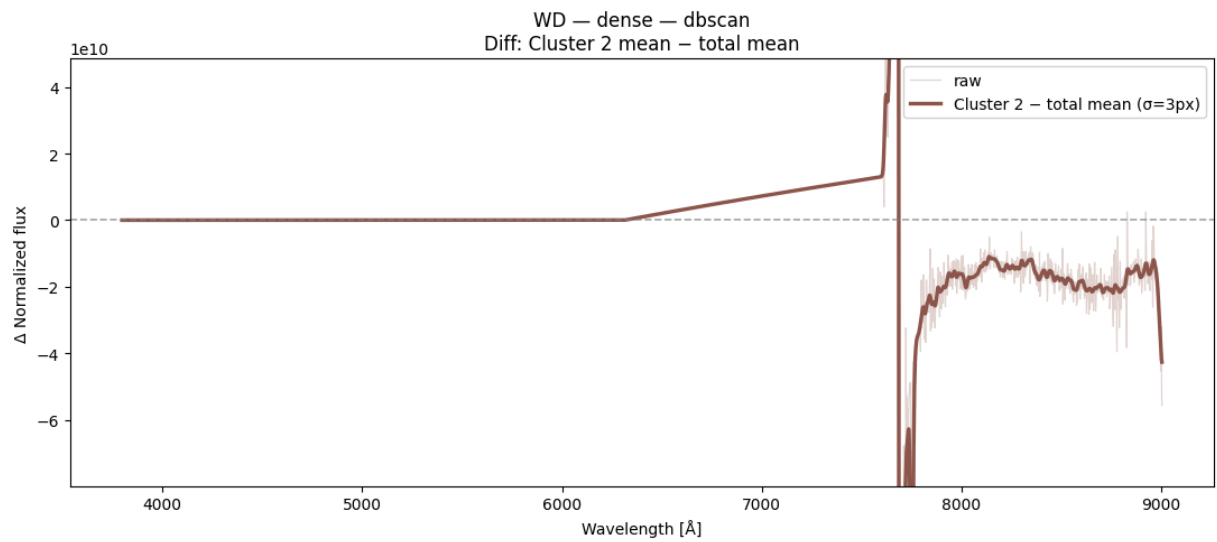
```
cmap = plt.cm.get_cmap("tab10", n_clusters)
```

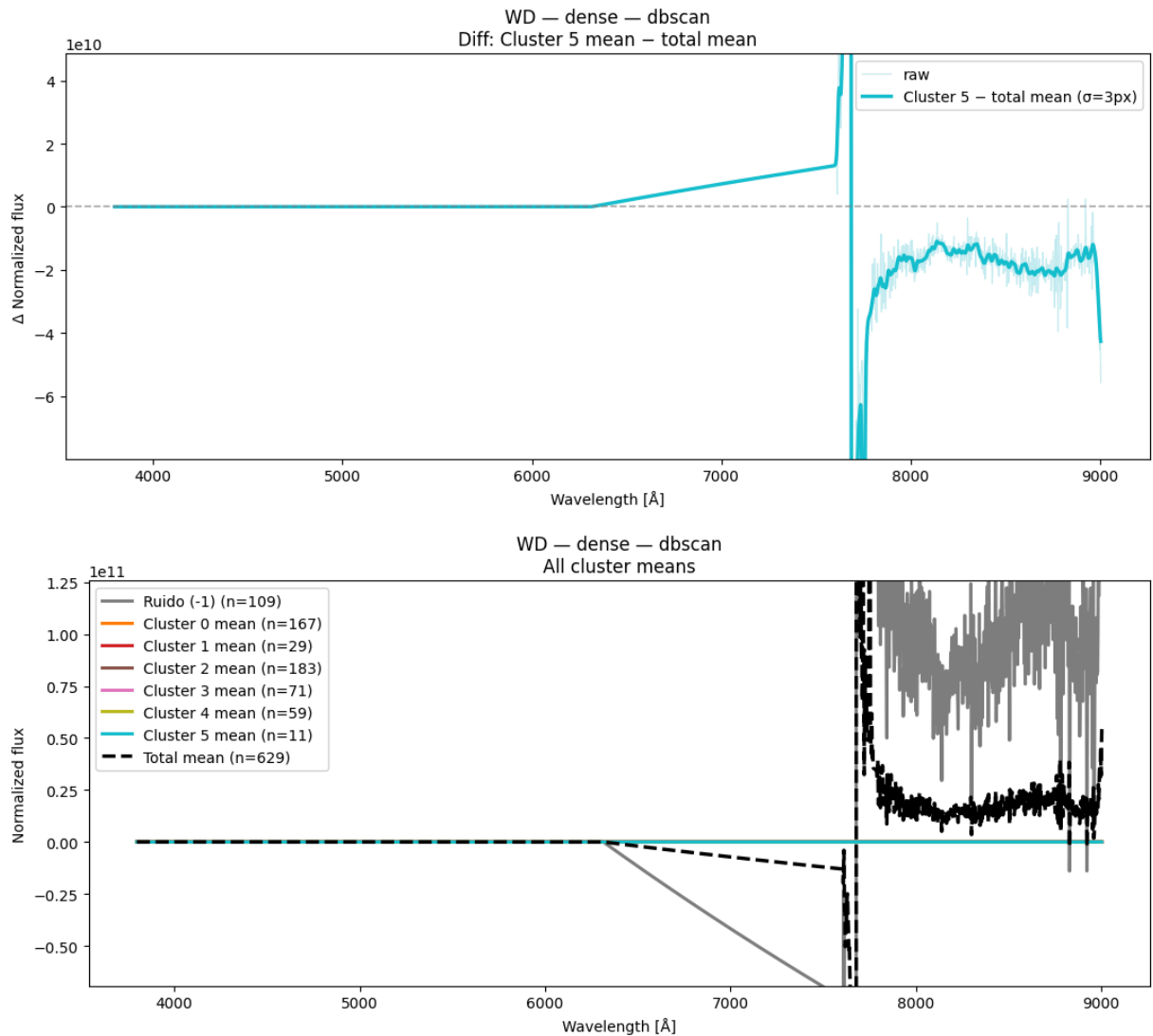












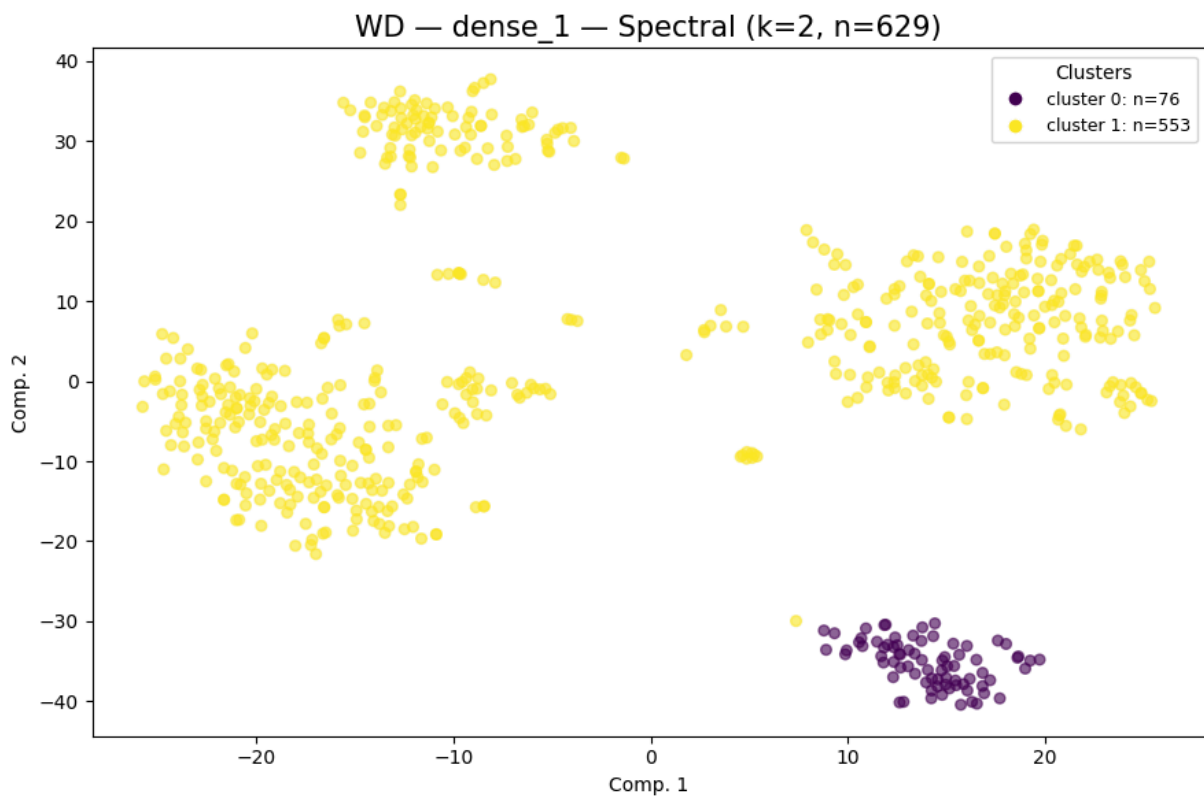
```
Out[65]: {'total_mean': array([1.8663167e+00, 1.4631674e+00, 2.3618588e+00, ..., 3.2000662e
+10,
        4.3846500e+10, 5.5692263e+10], dtype=float32),
'group_means': {-1: array([ 1.5653298e+00, -1.4698246e+00,  4.3070593e+00, ...,
        1.8466438e+11,  2.5302247e+11,  3.2138012e+11], dtype=float32),
 0: array([1.6844215 , 2.0973966 , 1.9249429 , ..., 0.46059665, 0.3942105 ,
        0.44315127], dtype=float32),
 1: array([3.8882143 , 4.1258554 , 3.8438573 , ..., 0.41760892, 0.35470298,
        0.5016809 ], dtype=float32),
 2: array([1.6883347 , 1.7588211 , 1.6414214 , ..., 0.44492832, 0.40326095,
        0.45199147], dtype=float32),
 3: array([2.1980743 , 2.2386997 , 2.086955 , ..., 0.4641504 , 0.49406648,
        0.4859111 ], dtype=float32),
 4: array([2.0229006 , 1.8264816 , 1.7919837 , ..., 0.43485004, 0.42773846,
        0.43204188], dtype=float32),
 5: array([2.2596838 , 2.0048544 , 2.62925 , ..., 0.5973191 , 0.26379913,
        0.03841503], dtype=float32)},
'unique_clusters': array([-1,  0,  1,  2,  3,  4,  5], dtype=int64)}
```

Resultados capa dense_1 con Spectral Clustering + tSNE y PCA

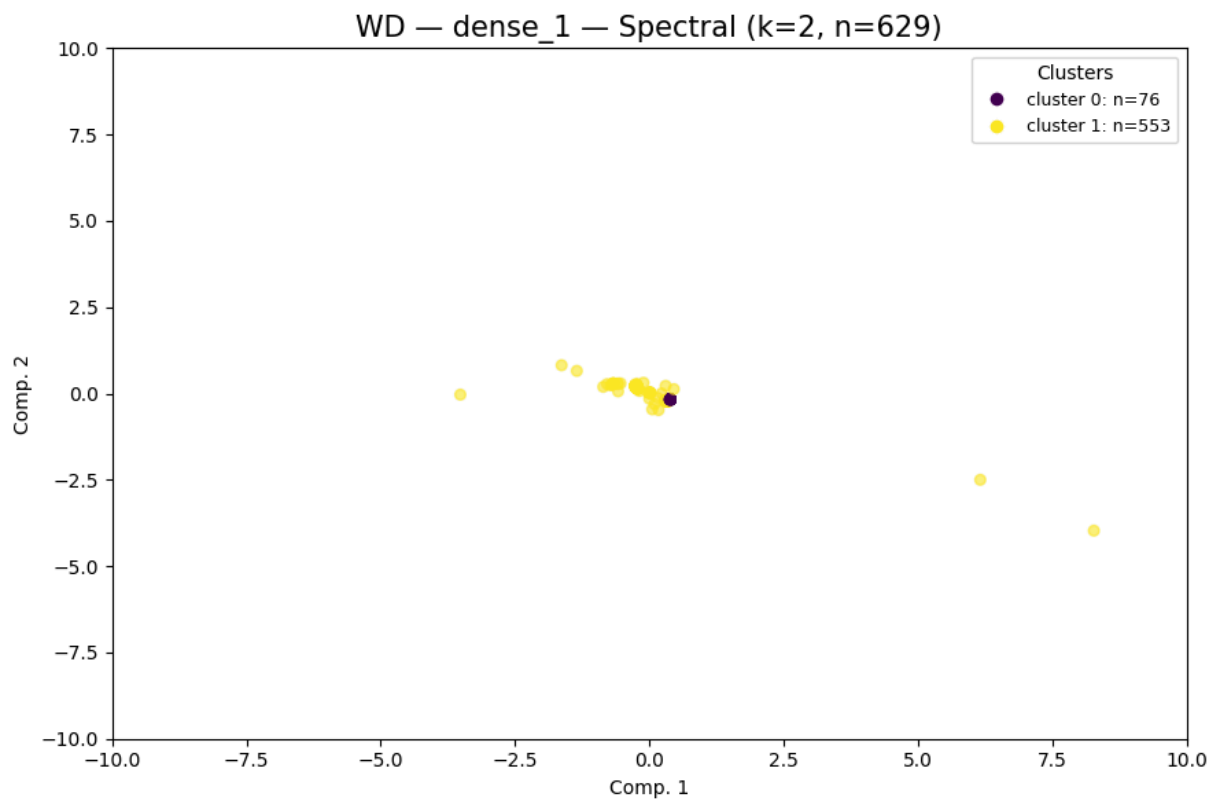
Spectral Clustering k = 2

```
In [66]: res_dense1_k2 = cluster_plot_Spect(
    repr_wd_32["dense_1"],
    "WD",
    "dense_1",
    spectral_k=2,
    vis_method="tsne",
    container=cluster_labels_all
)

labels_dense1_k2_wd = res_dense1_k2["spectral"]
cluster_plot_Spect(
    repr_wd_32["dense_1"],
    "WD",
    "dense_1",
    spectral_k=2,
    vis_method="pca",
    xlim=(-10,10), ylim=(-10,10)
)
```



WD	dense_1	spectral	k=2
clusters=2	noise=0	sil=0.0247	DB=1.5325 CH=18.52



```
WD | dense_1 | spectral | k=2  
clusters=2 noise=0 sil=0.0247 DB=1.5325 CH=18.52
```

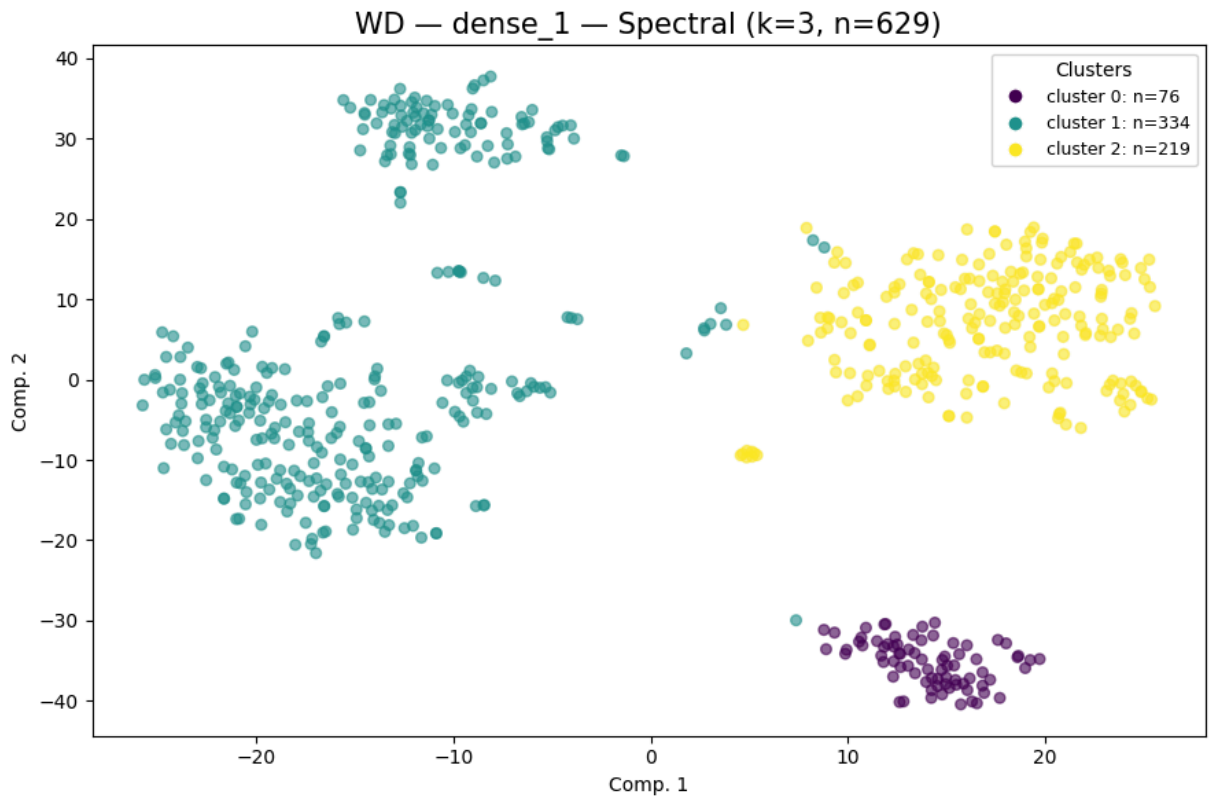
[illegible]

Spectral Clustering $k = 3$

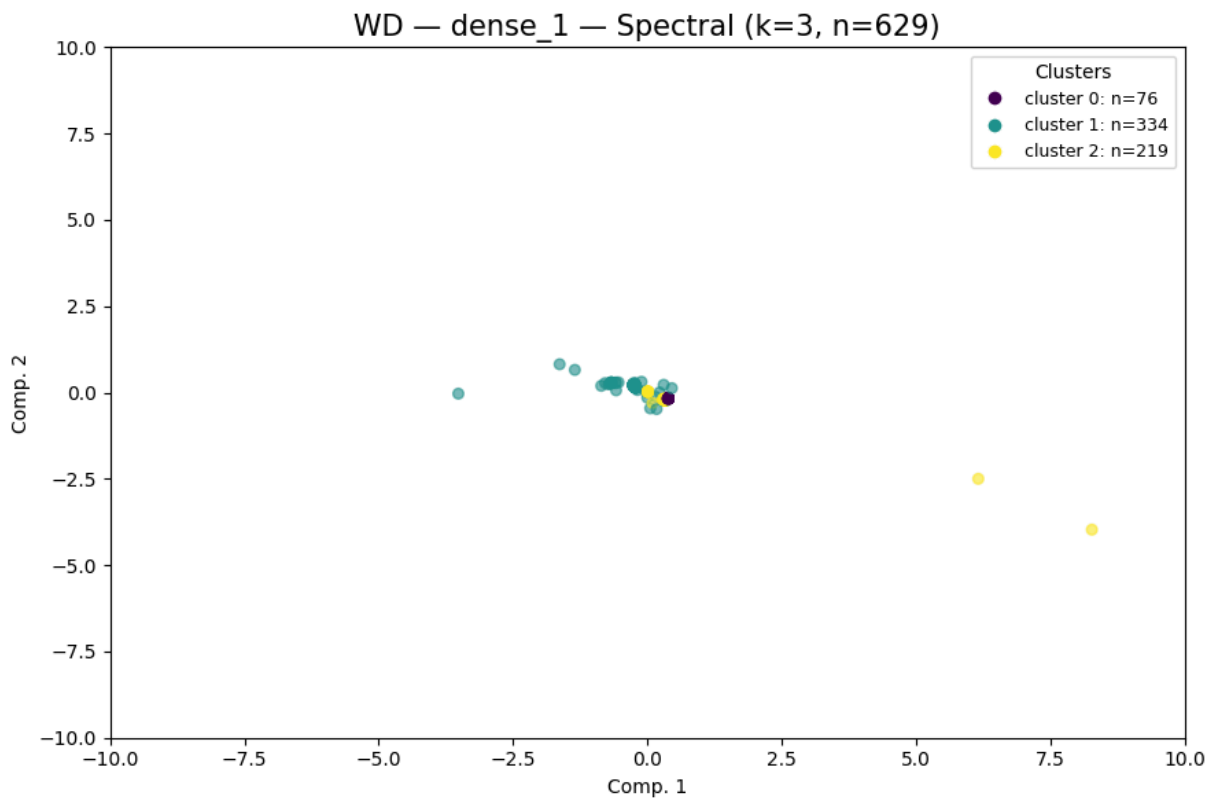
```
In [67]: res_dense1_k3 = cluster_plot_Spect(
    repr_wd_32["dense_1"],
    "WD",
    "dense_1",
    spectral_k=3,
    vis_method="tsne",
    container=cluster_labels_all
)

labels_dense1_k3_wd = res_dense1_k3["spectral"]
cluster_plot_Spect(
    repr_wd_32["dense_1"],
    "WD",
    "dense_1",
    spectral_k=3,
    vis_method="pca",
```

```
xlim=(-10,10), ylim=(-10,10)
)
```



```
WD | dense_1 | spectral | k=3
clusters=3 noise=0 sil=0.1289 DB=2.2536 CH=20.55
```



WD	dense_1	spectral	k=3
clusters=3	noise=0	sil=0.1289	DB=2.2536 CH=20.55

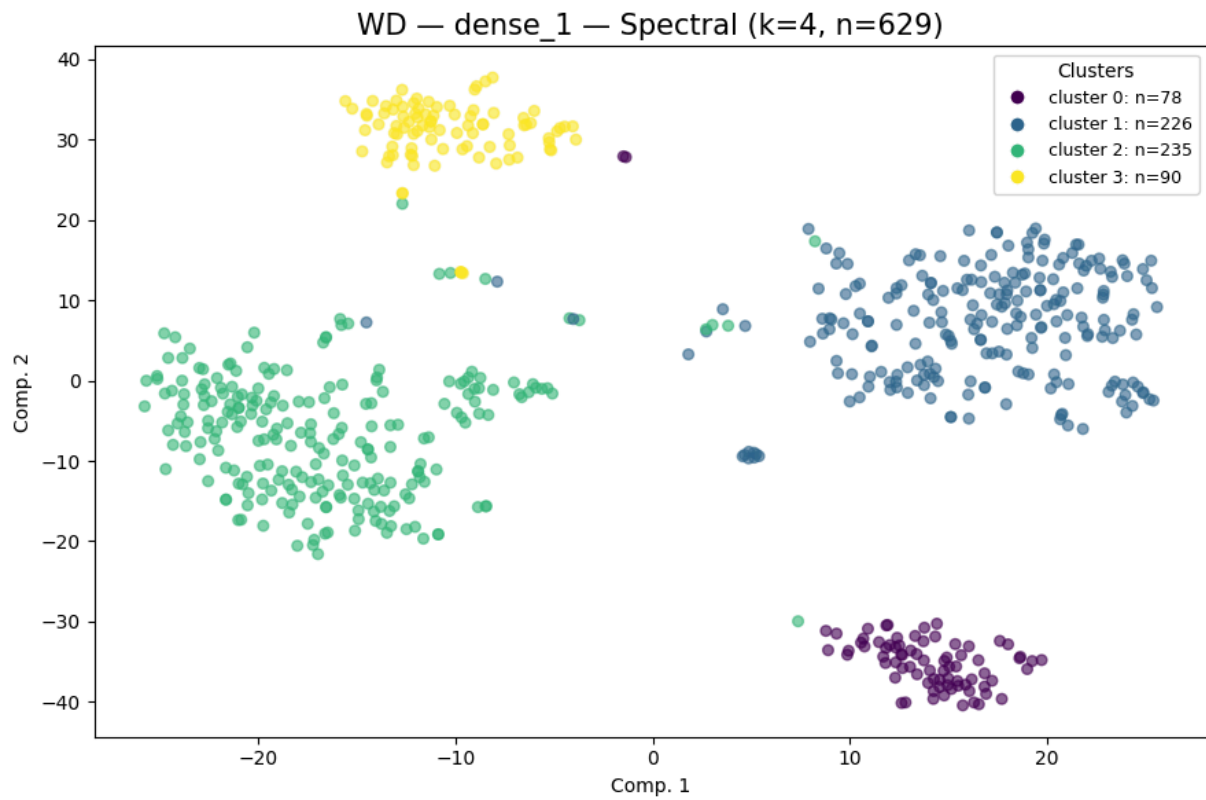
```
Out[67]: {'spectral': array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 1,
1,
1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 2, 2, 2, 2, 2, 2, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1,
2, 2, 2, 2, 2, 2, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2,
2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2,
2, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2,
0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 0, 0,
0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 2, 2, 2, 1, 1, 1,
1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 2, 2, 2, 2, 2, 1, 1, 1, 1, 2, 2, 2, 1, 1, 1, 1, 1, 1, 2, 2,
2, 2, 2, 2, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2,
2, 2, 1, 1, 1, 1, 2, 2, 2, 1, 1, 1, 2, 1, 1, 2, 2, 2, 2, 2, 2, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1, 1,
1, 2, 2, 2, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 0, 0, 0, 0, 0, 0,
0, 0, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 2, 2, 2, 1,
1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 2, 2, 2, 2, 2, 1, 1, 1, 1, 2, 1, 2, 1, 1, 1, 1, 1, 1,
2, 2, 1, 2, 2, 2, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2,
2, 2, 2, 2, 1, 1, 1, 1, 2, 2, 2, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2,
2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1,
2, 1, 1, 2, 2, 2, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 0, 0, 0, 0,
0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 2, 2,
2, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 2, 2, 2, 1, 1, 1, 1,
1, 1, 2, 2, 2, 2, 2, 2, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1,
2, 2, 1, 2, 2, 2, 1, 1, 1, 1, 2, 2, 2]),
'metrics': {'n_clusters': 3,
'n_noise': 0,
'silhouette': 0.128934,
'davies_bouldin': 2.253613,
'calinski_harabasz': 20.5546}}
```

Spectral Clustering k = 4

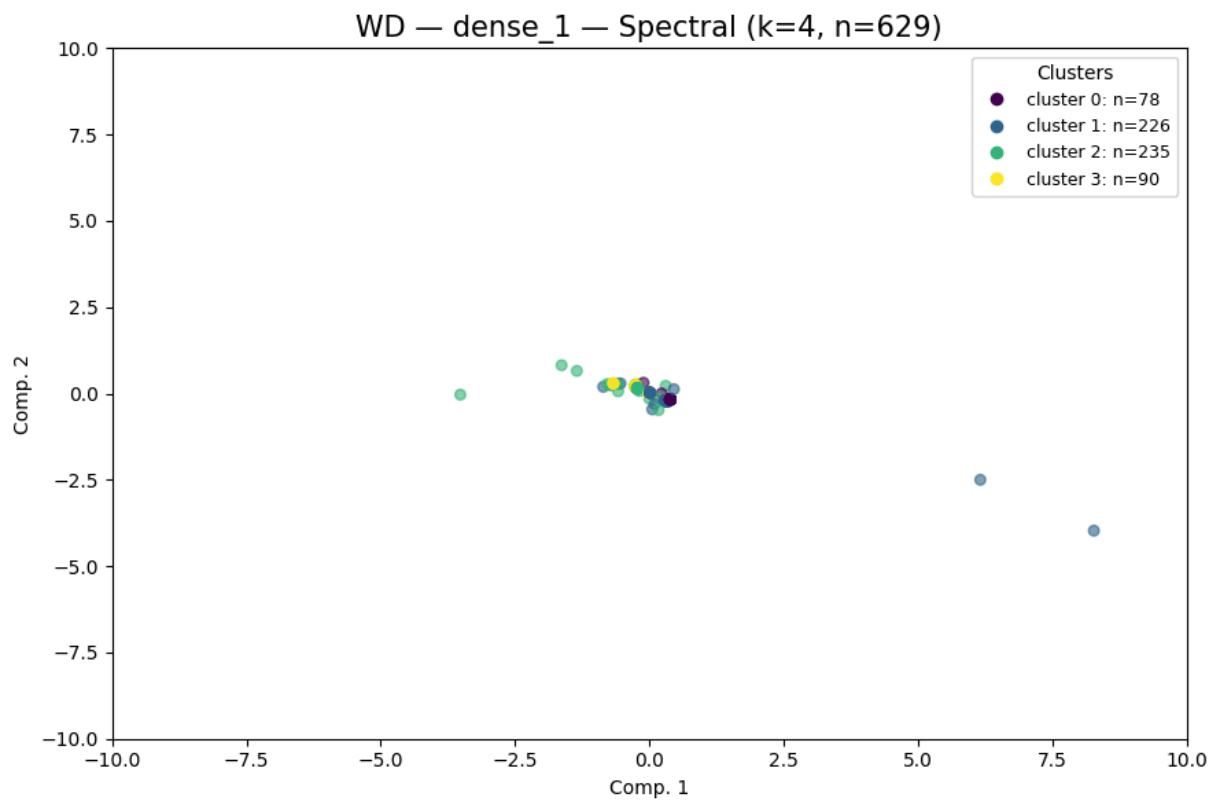
```
In [68]: res_dense1_k4 = cluster_plot_Spect(
    repr_wd_32["dense_1"],
    "WD",
    "dense_1",
    spectral_k=4,
    vis_method="tsne",
    container=cluster_labels_all
)

labels_dense1_k4_wd = res_dense1_k4["spectral"]
cluster_plot_Spect(
    repr_wd_32["dense_1"],
```

```
"WD",  
"dense_1",  
spectral_k=4,  
vis_method="pca",  
xlim=(-10,10), ylim=(-10,10)  
)
```



```
WD | dense_1 | spectral | k=4  
clusters=4 noise=0 sil=0.2468 DB=1.9645 CH=20.96
```



```
WD | dense_1 | spectral | k=4  
clusters=4 noise=0 sil=0.2468 DB=1.9645 CH=20.96
```

```

Out[68]: {'spectral': array([2, 1, 2, 2, 2, 1, 2, 2, 2, 0, 2, 1, 3, 3, 3, 3, 2, 3, 3, 3, 2,
0,
2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 3, 3, 3, 3, 3, 3, 3, 3, 2, 2, 2, 2,
2, 2, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 2, 2, 2, 2, 2, 2,
1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 2, 2, 2, 2, 2, 2, 1, 1,
1, 1, 1, 1, 3, 3, 3, 3, 3, 3, 3, 3, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1,
1, 1, 3, 3, 3, 3, 3, 3, 3, 3, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1,
0, 0, 0, 0, 0, 0, 0, 0, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 0, 0,
0, 0, 0, 0, 0, 0, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 3, 3, 3, 3,
3, 3, 3, 3, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 3, 3, 1, 3, 3, 3,
3, 3, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0,
2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 1, 1, 1, 2, 2,
2, 2, 2, 1, 1, 1, 1, 1, 1, 3, 3, 3, 3, 3, 3, 3, 3, 2, 2, 2, 2, 2,
2, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1,
1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1,
1, 1, 2, 2, 2, 2, 1, 1, 1, 2, 2, 2, 1, 2, 2, 1, 1, 1, 1, 1, 1, 3,
3, 3, 3, 3, 3, 3, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 2, 2, 2,
2, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0,
0, 0, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 1, 1, 1, 2,
2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 3, 3, 3, 3, 3, 3, 3, 3, 2, 2, 2,
2, 2, 2, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 1, 2, 2, 2, 2, 2,
1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 2, 2, 2, 2, 2, 2, 1, 1,
1, 1, 1, 1, 2, 2, 2, 2, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1,
1, 3, 3, 3, 3, 2, 3, 3, 3, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 2,
1, 2, 2, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0,
0, 0, 0, 0, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 1, 1,
1, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 3, 3, 3, 3, 3, 3, 3, 3, 2,
2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 1, 1, 1, 2, 2, 2, 2,
2, 2, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 3, 2, 2, 1, 2, 2, 2,
1, 1, 2, 1, 1, 1, 3, 3, 3, 3, 1, 1, 1])),
'metrics': {'n_clusters': 4,
'n_noise': 0,
'silhouette': 0.246754,
'davies_bouldin': 1.964491,
'calinski_harabasz': 20.956}}

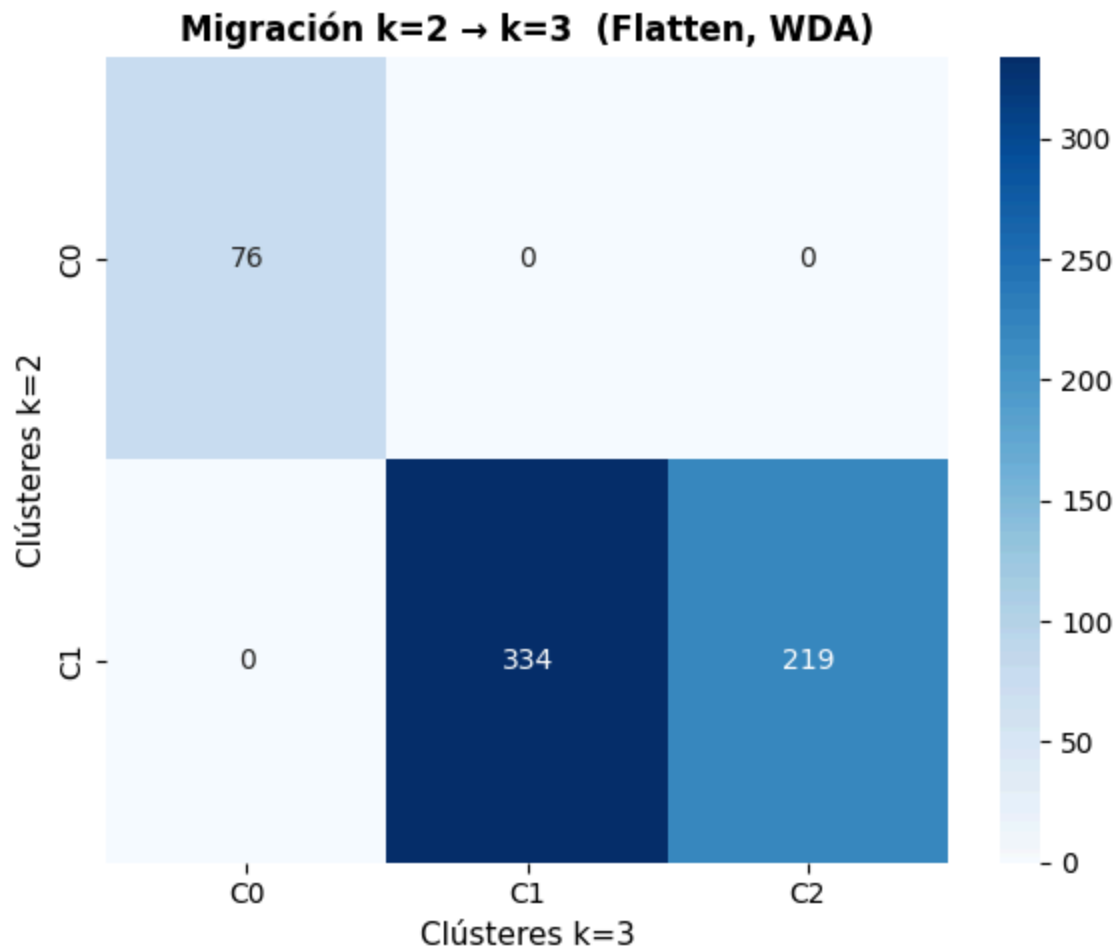
```

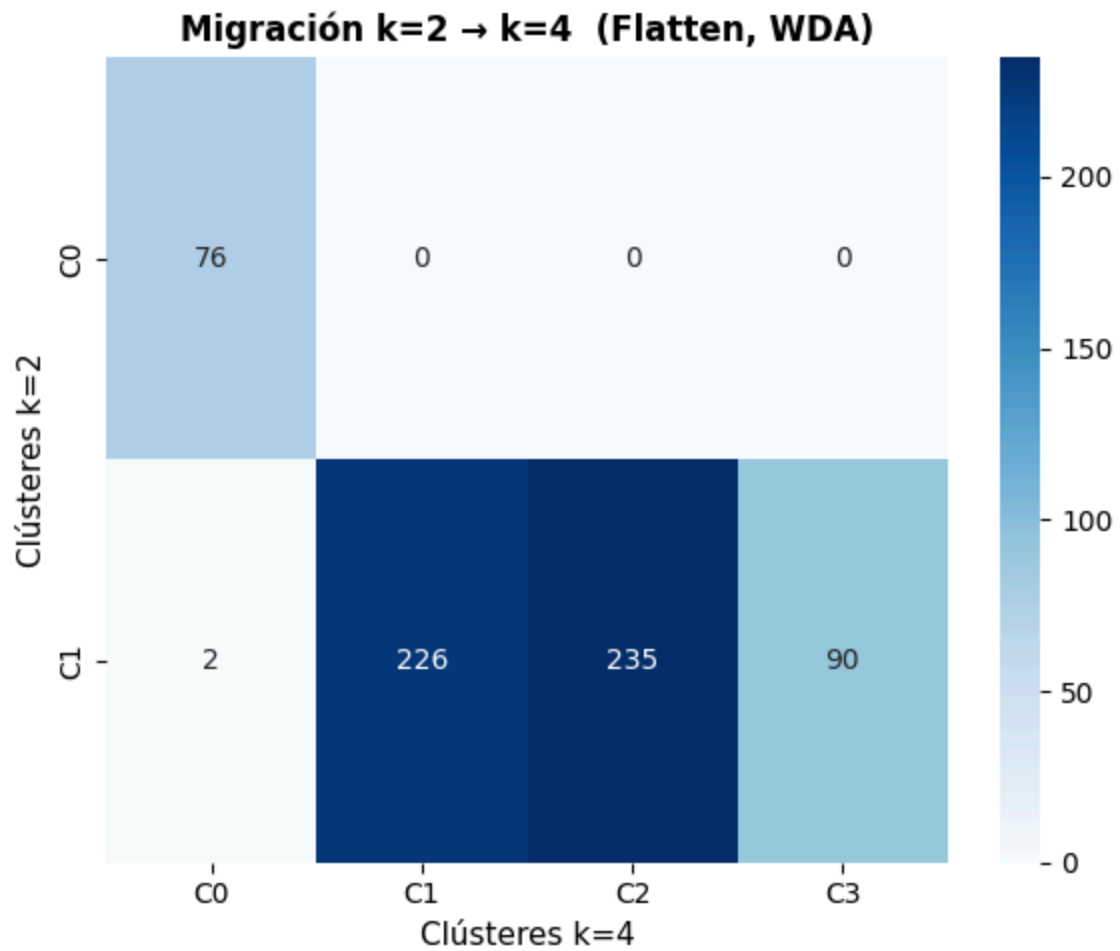
Análisis de migración

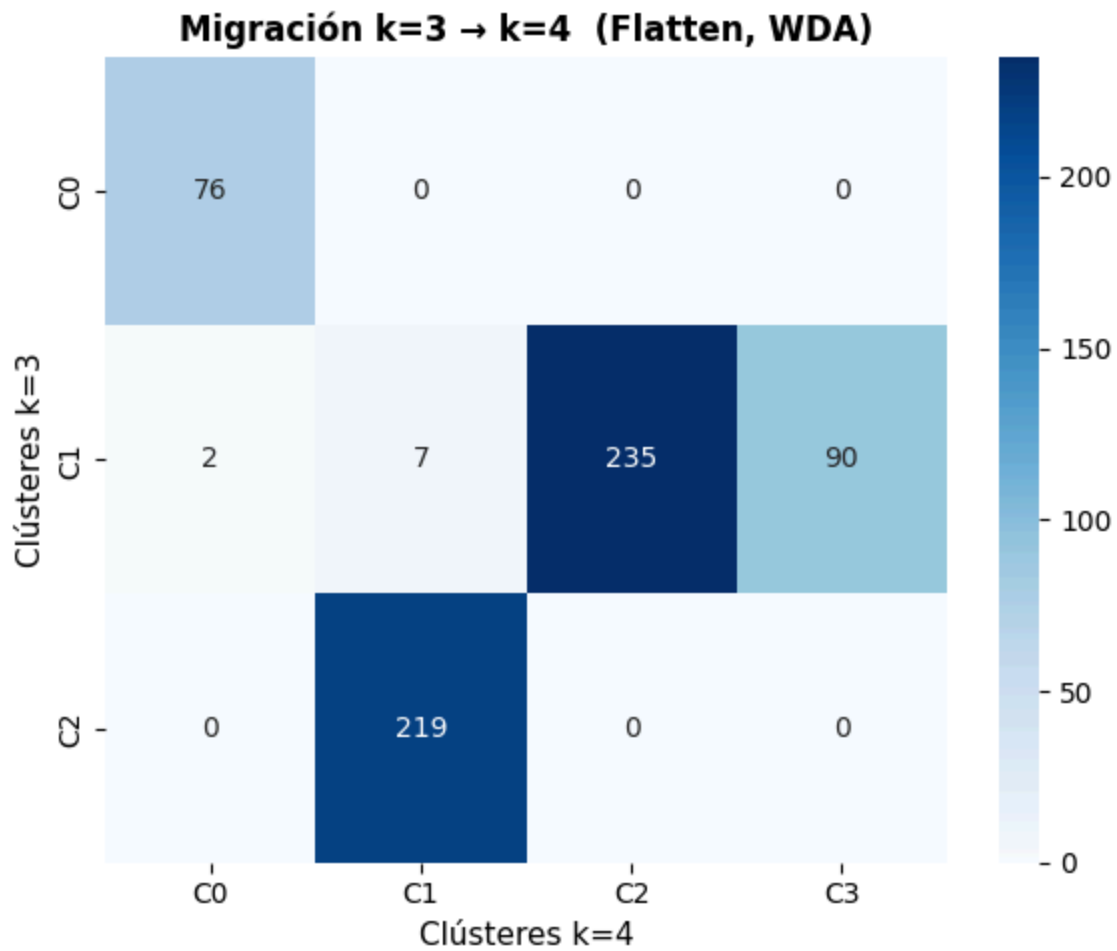
```

In [69]: plot_migration_matrix(labels_dense1_k2_wd, labels_dense1_k3_wd, ka=2, kb=3)
plot_migration_matrix(labels_dense1_k2_wd, labels_dense1_k4_wd, ka=2, kb=4)
plot_migration_matrix(labels_dense1_k3_wd, labels_dense1_k4_wd, ka=3, kb=4)

```



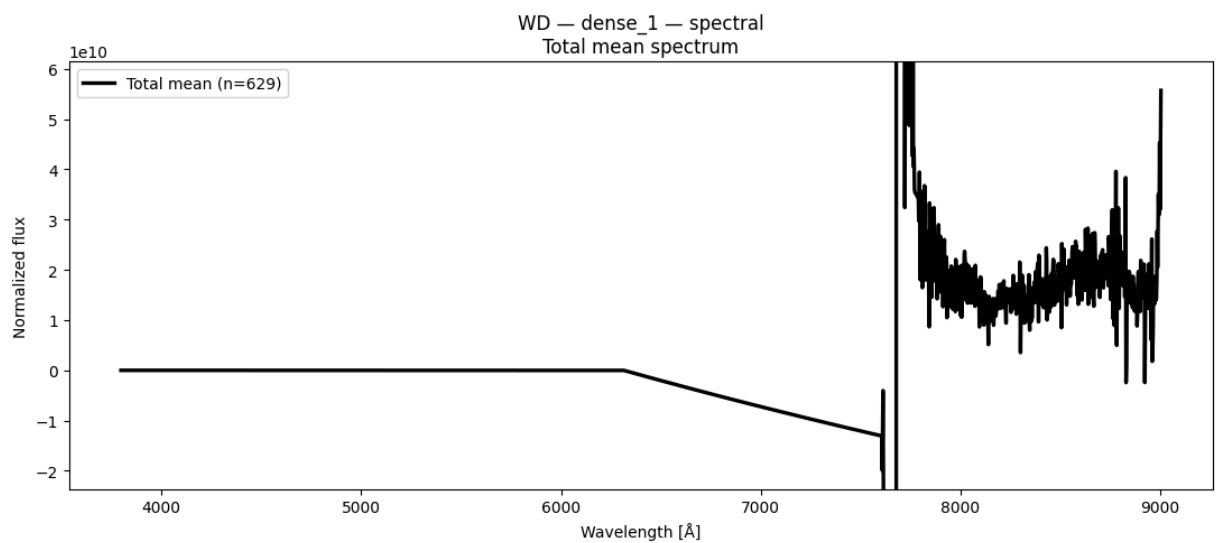
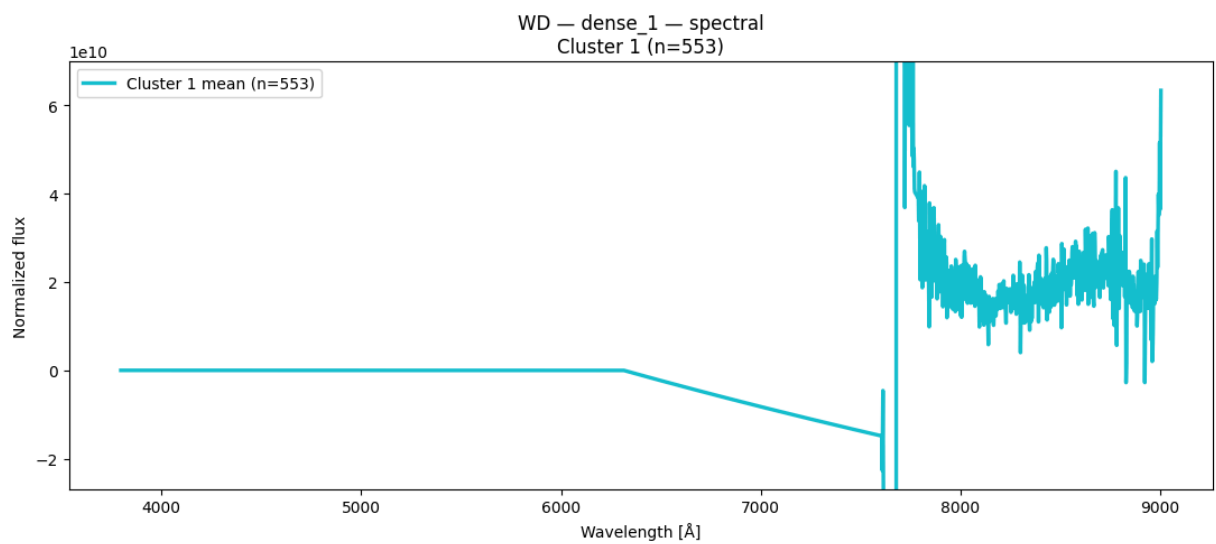
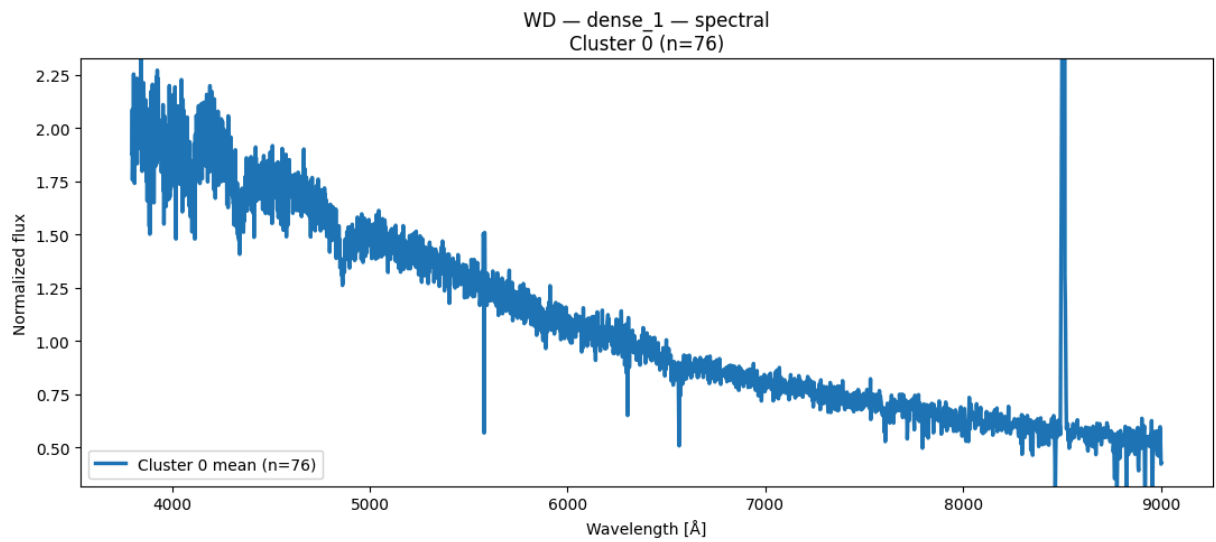


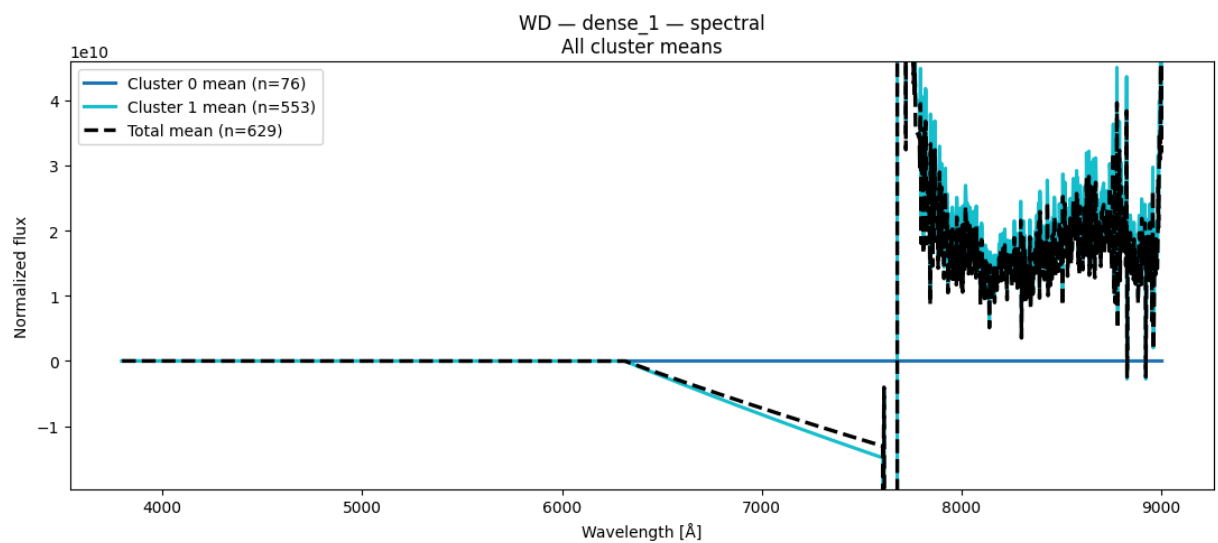
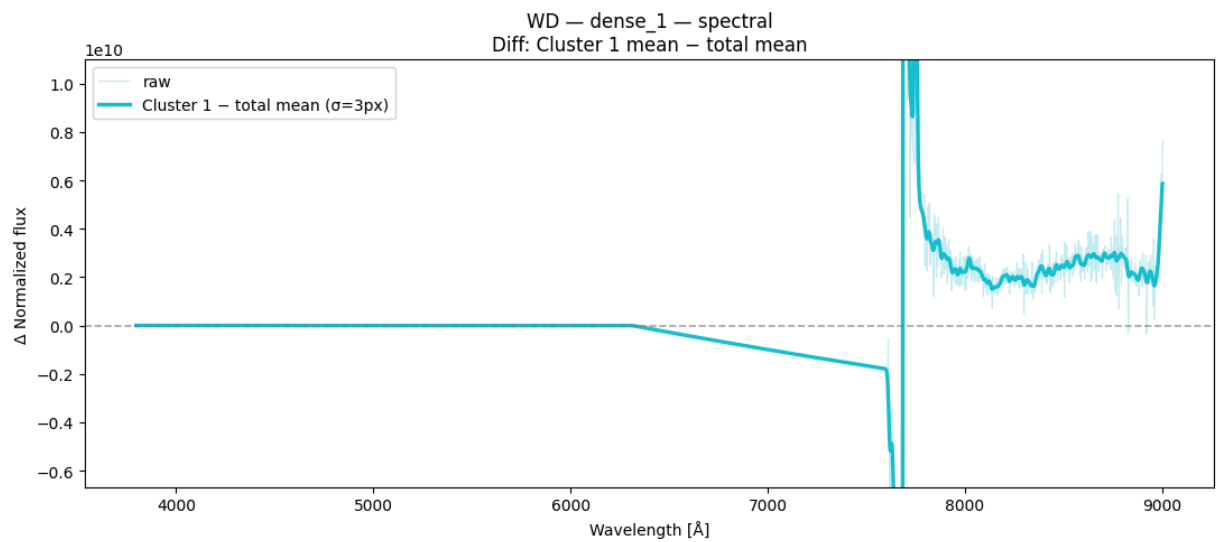
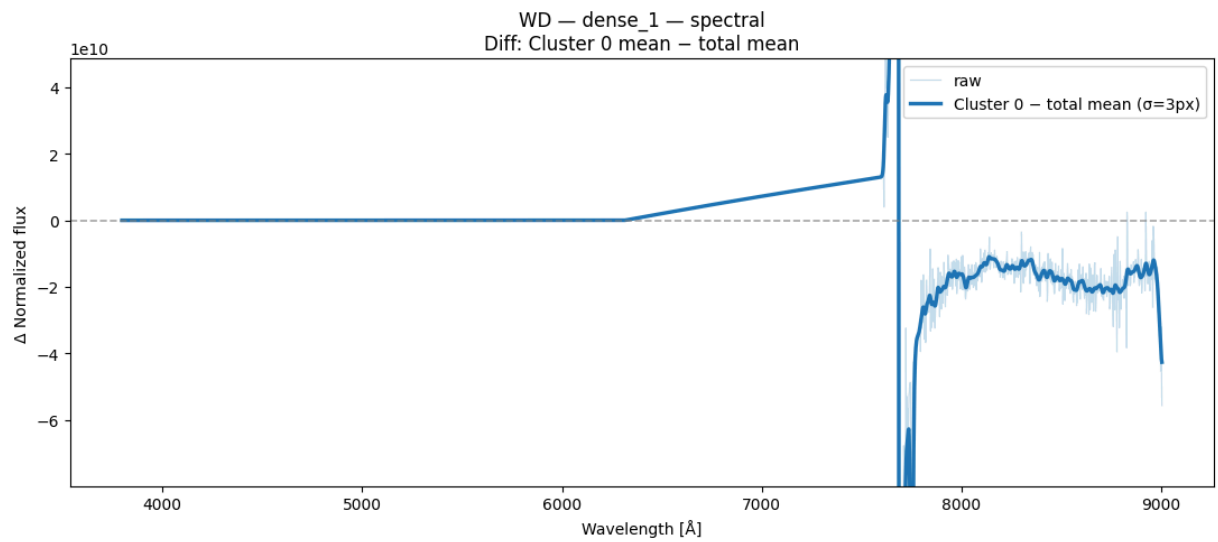
Espectros obtenidos de Spectral Clustering para k=2 y n=700

```
In [70]: plot_spectra_spectral(
    X_wd_spec,
    labels_dense1_k2_wd,
    W_wd[0],
    class_name="WD",
    layer_name="dense_1"
)
```

C:\Users\javip\AppData\Local\Temp\ipykernel_52368\531566132.py:28: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get\_cmap()`` or ``pyplot.get\_cmap()`` instead.

```
cmap = plt.cm.get_cmap("tab10", n_clusters)
```





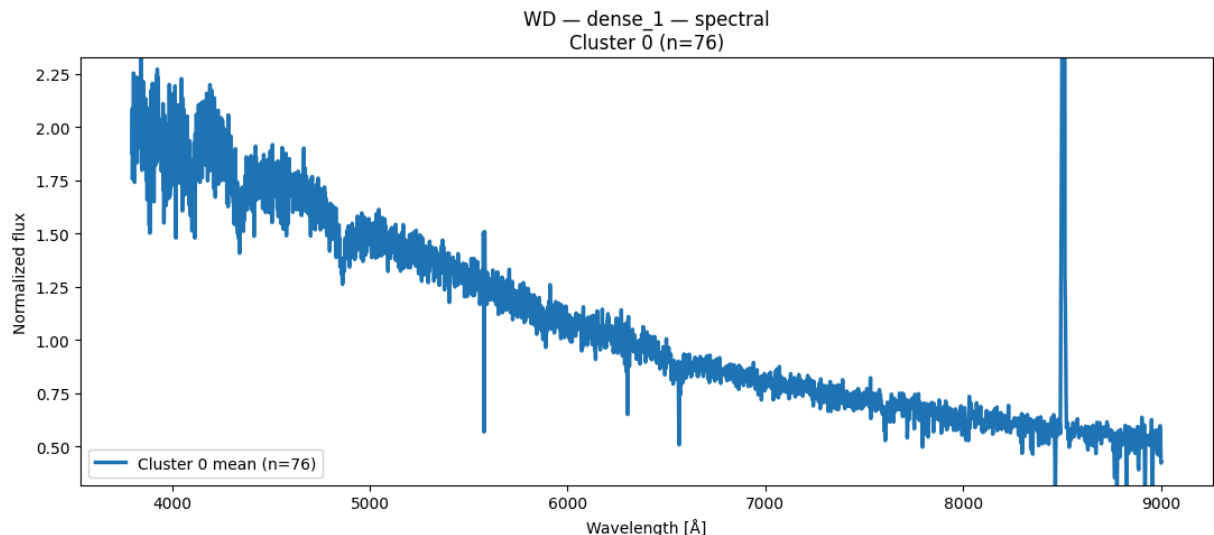
```
Out[70]: {'total_mean': array([1.8663167e+00, 1.4631674e+00, 2.3618588e+00, ..., 3.2000662e
+10,
        4.3846500e+10, 5.5692263e+10], dtype=float32),
        'group_means': {0: array([1.8797821 , 1.8861086 , 2.085941 , ..., 0.5124545 , 0.
42313442,
        0.42786634], dtype=float32),
        1: array([1.8644673e+00, 1.4050417e+00, 2.3997786e+00, ..., 3.6398584e+10,
        4.9872421e+10, 6.3346176e+10], dtype=float32)},
        'unique_clusters': array([0, 1])}
```

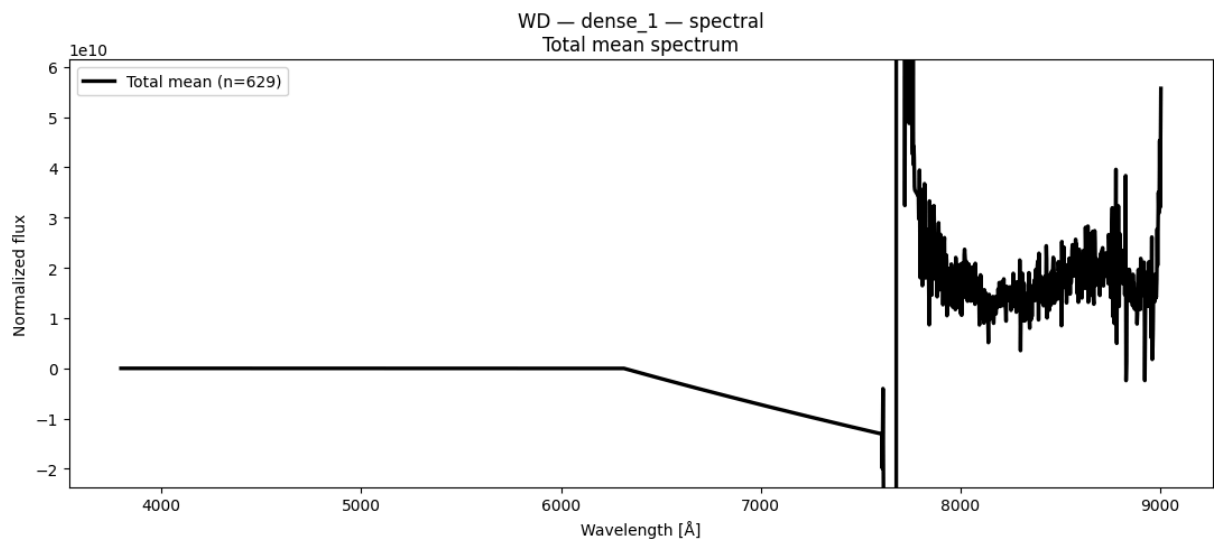
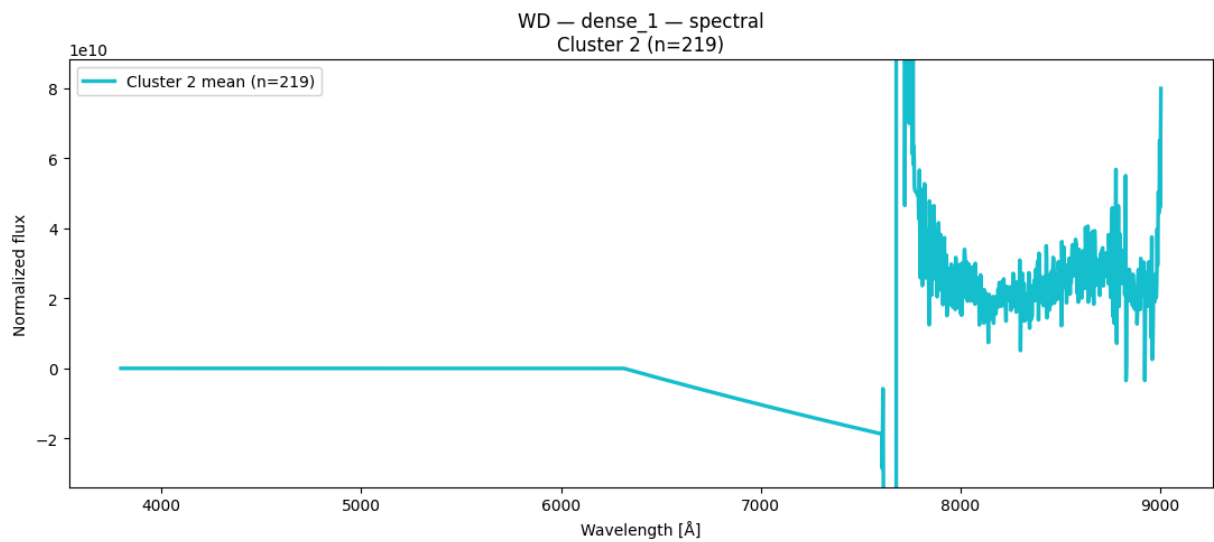
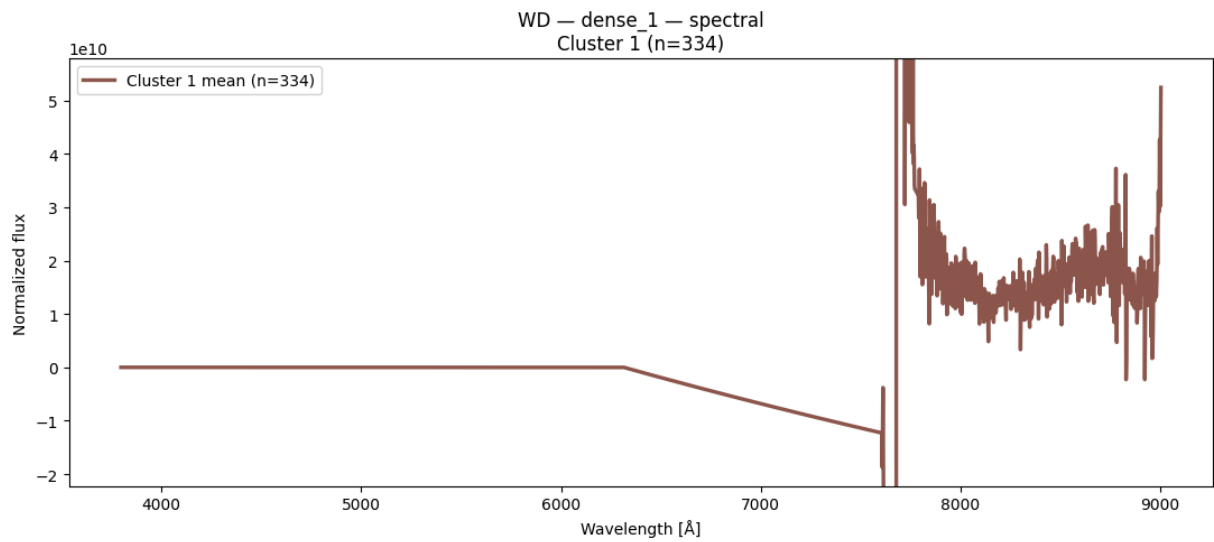
Espectros obtenidos de Spectral Clustering para k=3 y n=700

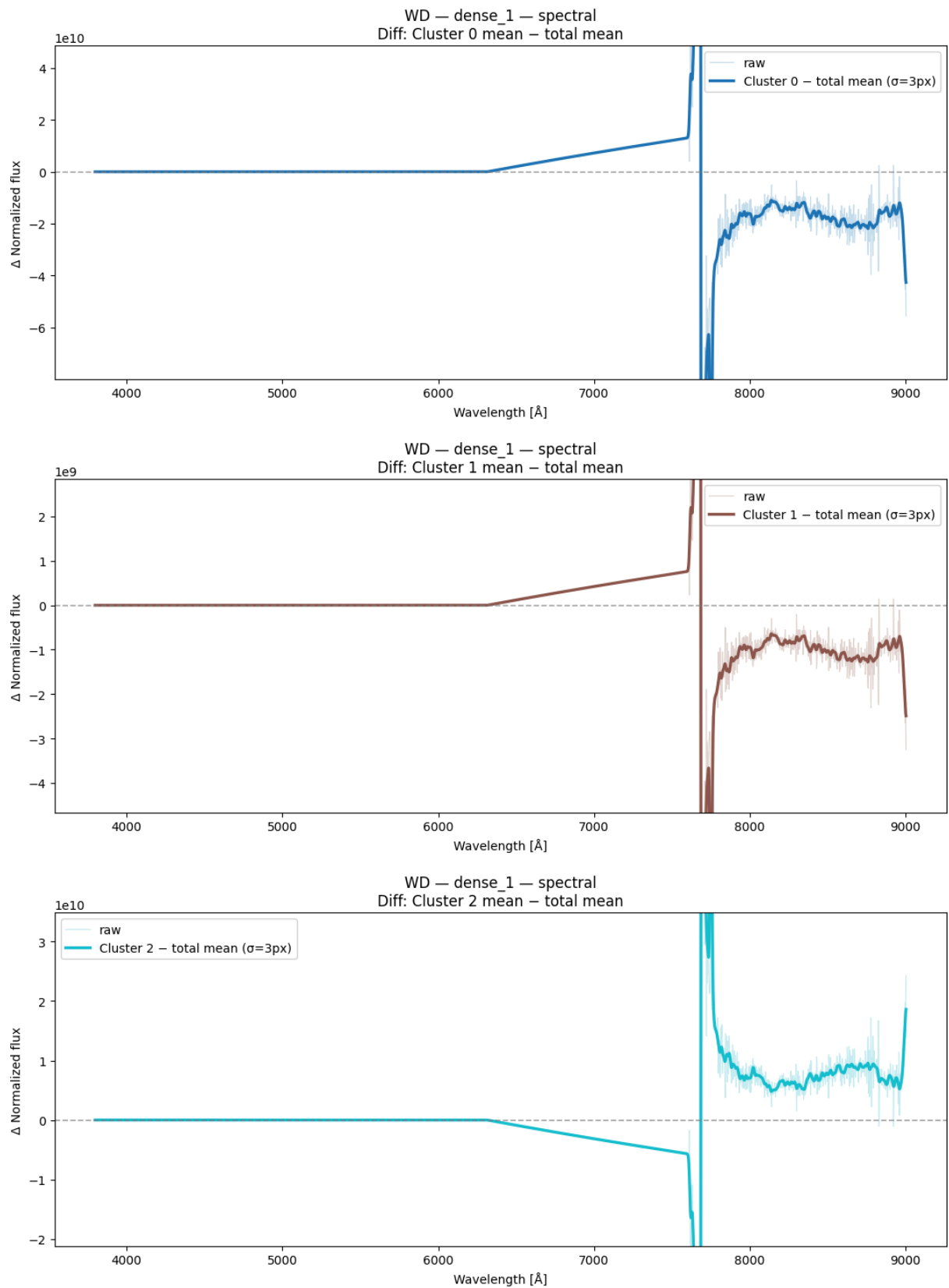
```
In [71]: plot_spectra_spectral(
        X_wd_spec,
        labels_dense1_k3_wd,
        W_wd[0],
        class_name="WD",
        layer_name="dense_1"
    )
```

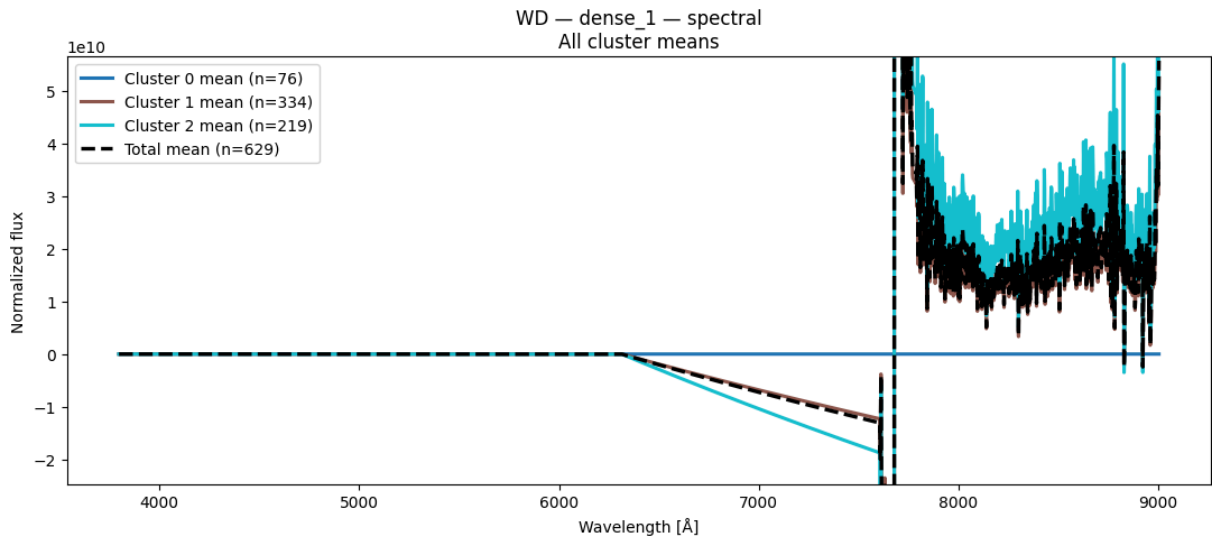
C:\Users\javip\AppData\Local\Temp\ipykernel_52368\531566132.py:28: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get\_cmap()`` or ``pyplot.get\_cmap()`` instead.

```
cmap = plt.cm.get_cmap("tab10", n_clusters)
```









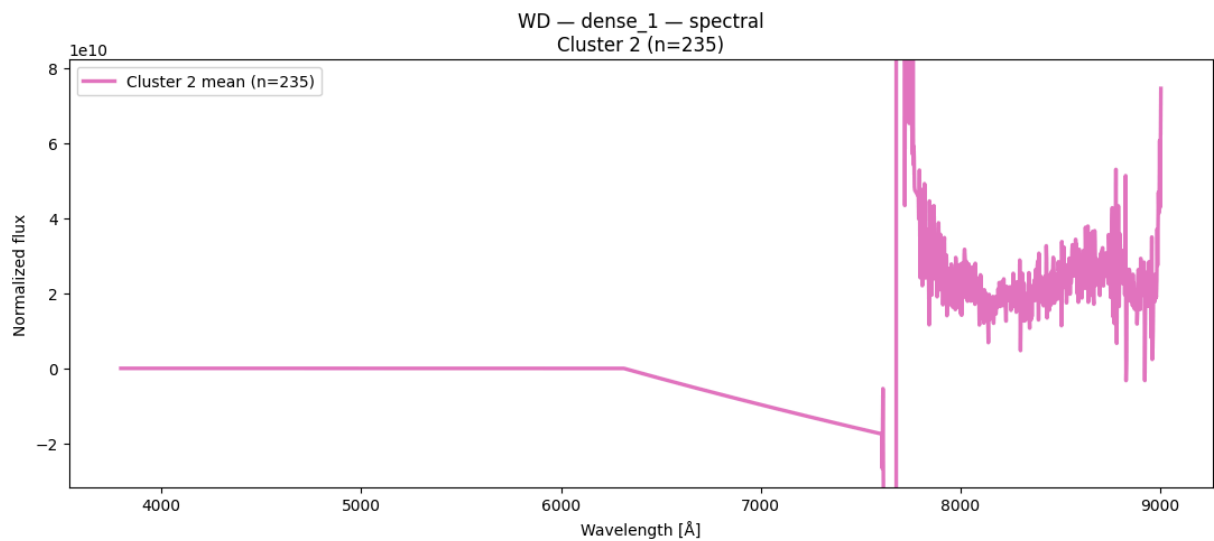
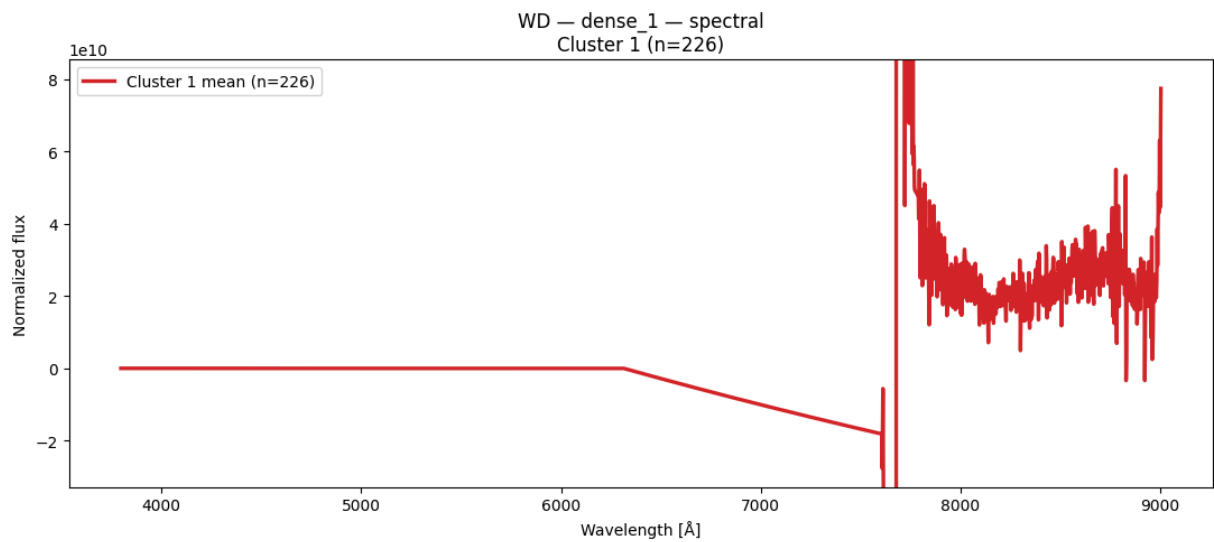
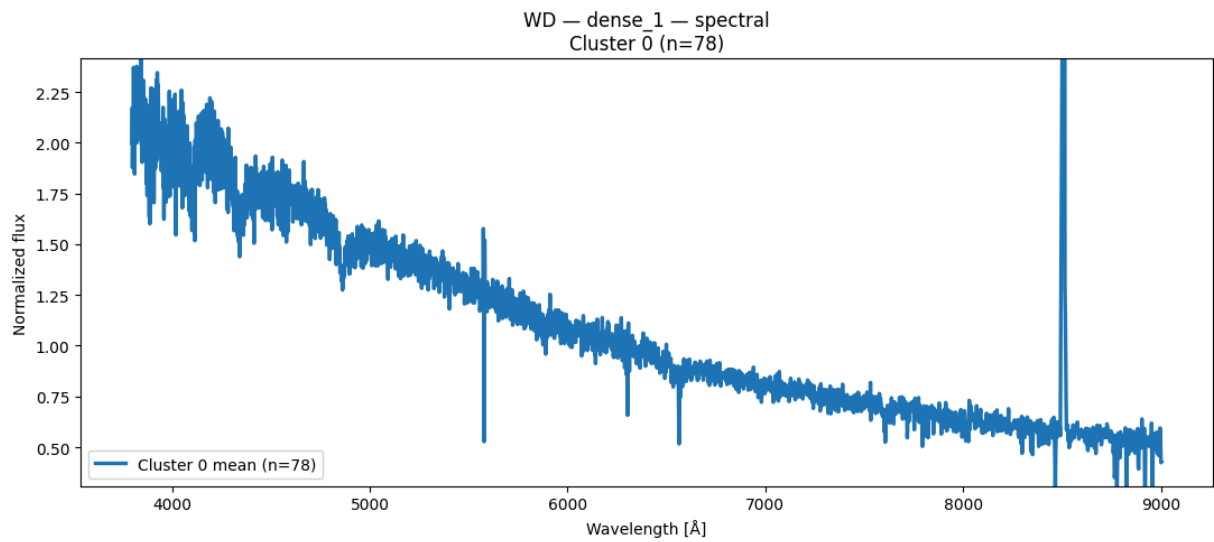
```
Out[71]: {'total_mean': array([1.8663167e+00, 1.4631674e+00, 2.3618588e+00, ..., 3.2000662e
+10,
      4.3846500e+10, 5.5692263e+10], dtype=float32),
      'group_means': {0: array([1.8797821 , 1.8861086 , 2.085941 , ..., 0.5124545 , 0.
42313442,
      0.42786634], dtype=float32),
      1: array([2.3334751e+00, 2.2147918e+00, 2.2229018e+00, ..., 3.0132351e+10,
      4.1286599e+10, 5.2440756e+10], dtype=float32),
      2: array([1.1491760e+00, 1.7008002e-01, 2.6695373e+00, ..., 4.5955314e+10,
      6.2966788e+10, 7.9978168e+10], dtype=float32)},
      'unique_clusters': array([0, 1, 2])}
```

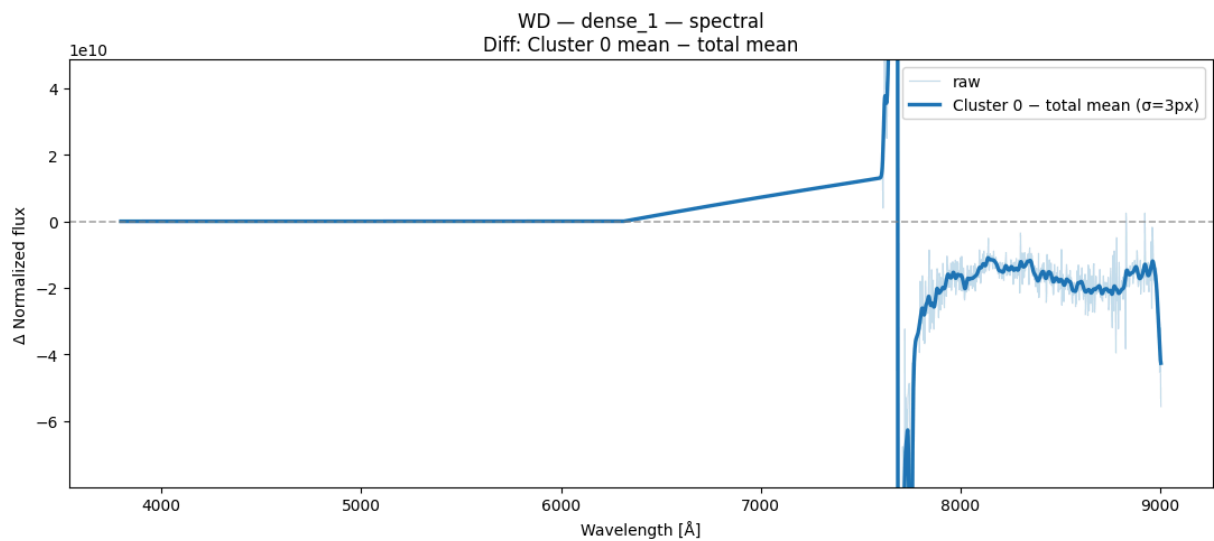
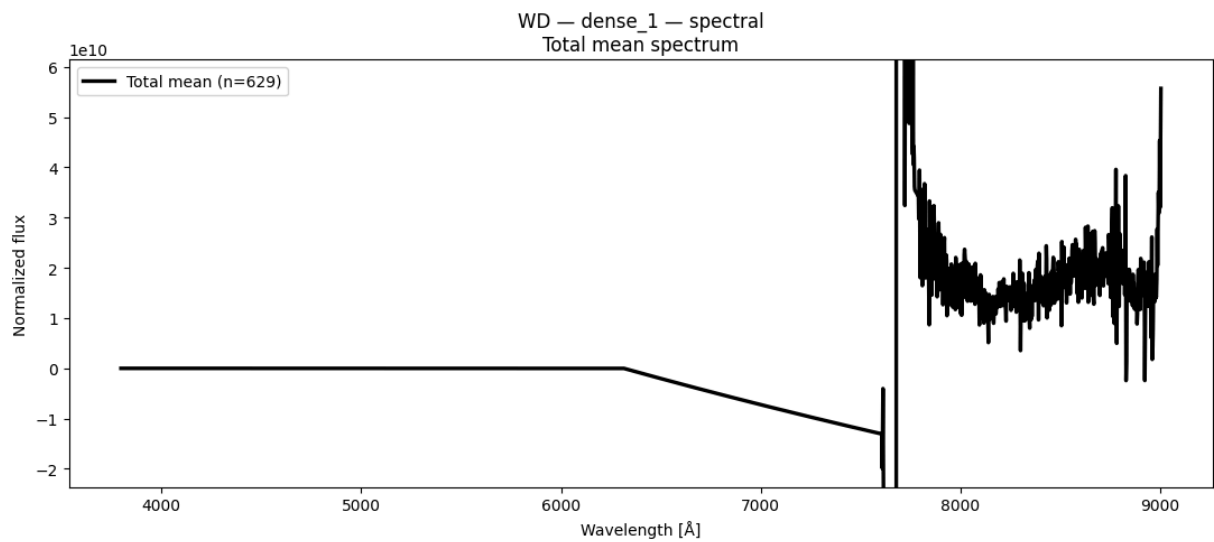
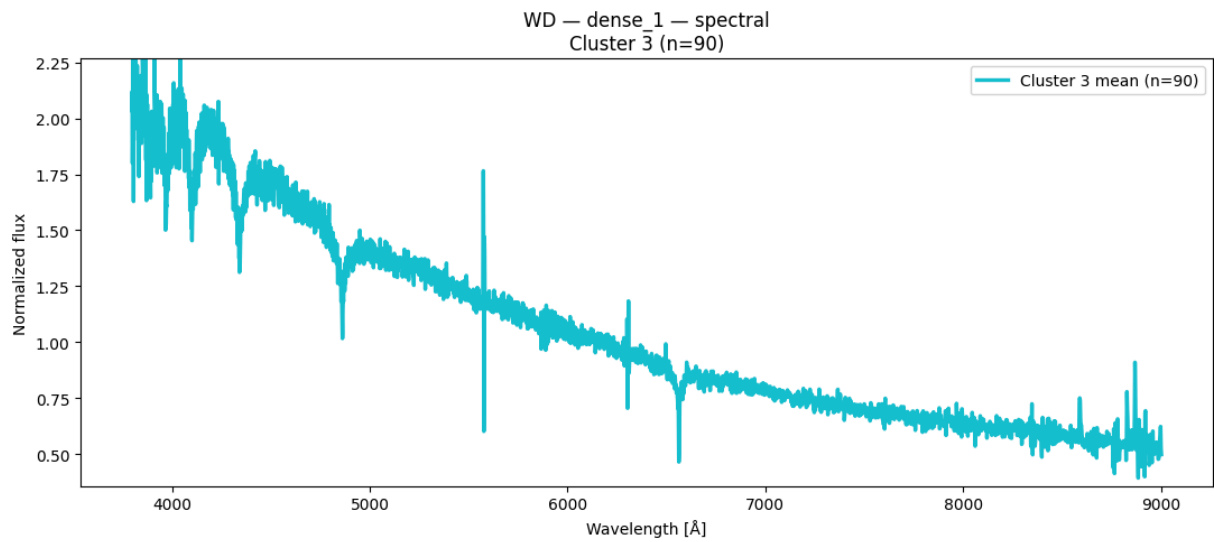
Espectros obtenidos de Spectral Clustering para k=4 y n=700

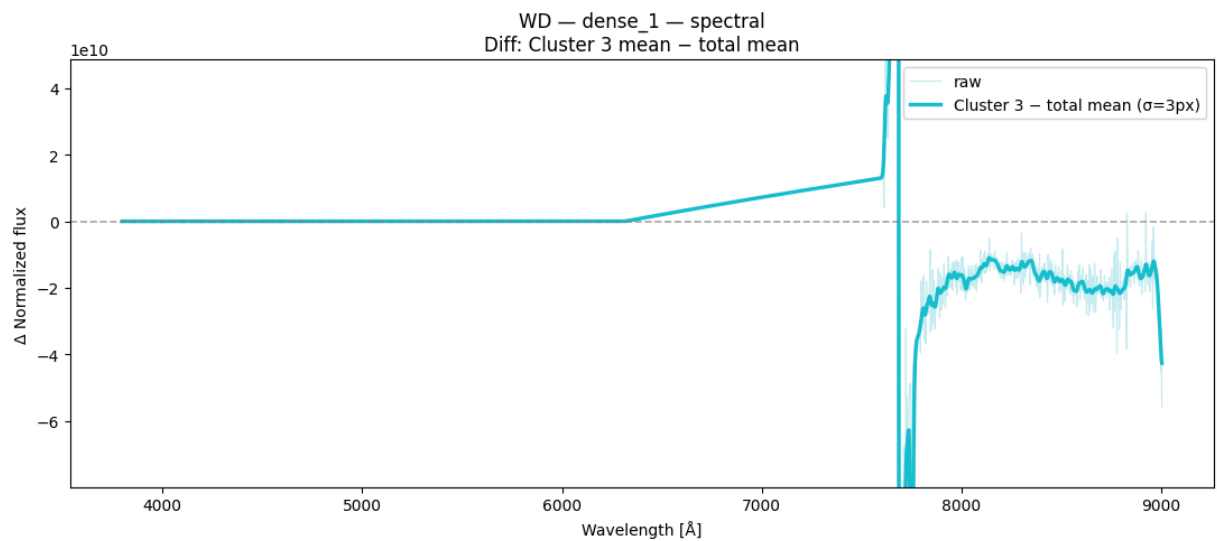
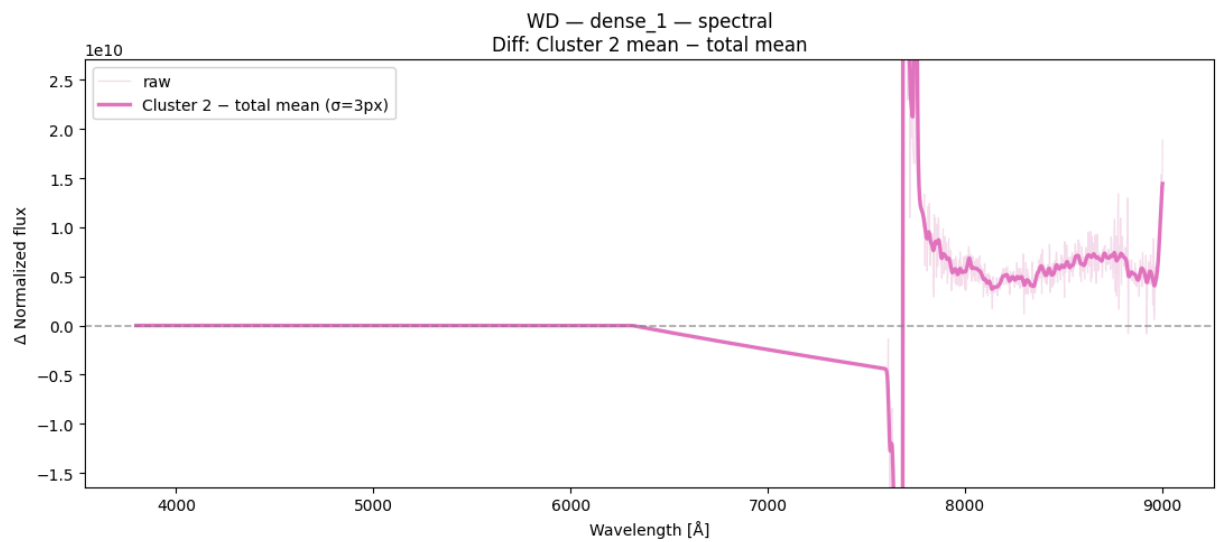
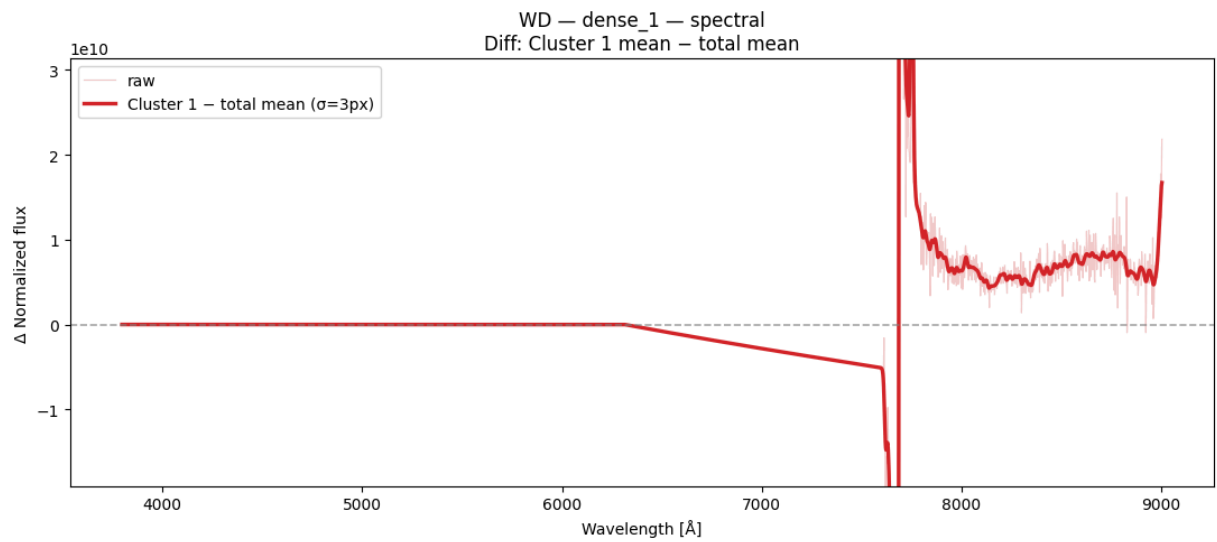
```
In [72]: plot_spectra_spectral(
      X_wd_spec,
      labels_dense1_k4_wd,
      W_wd[0],
      class_name="WD",
      layer_name="dense_1"
    )
```

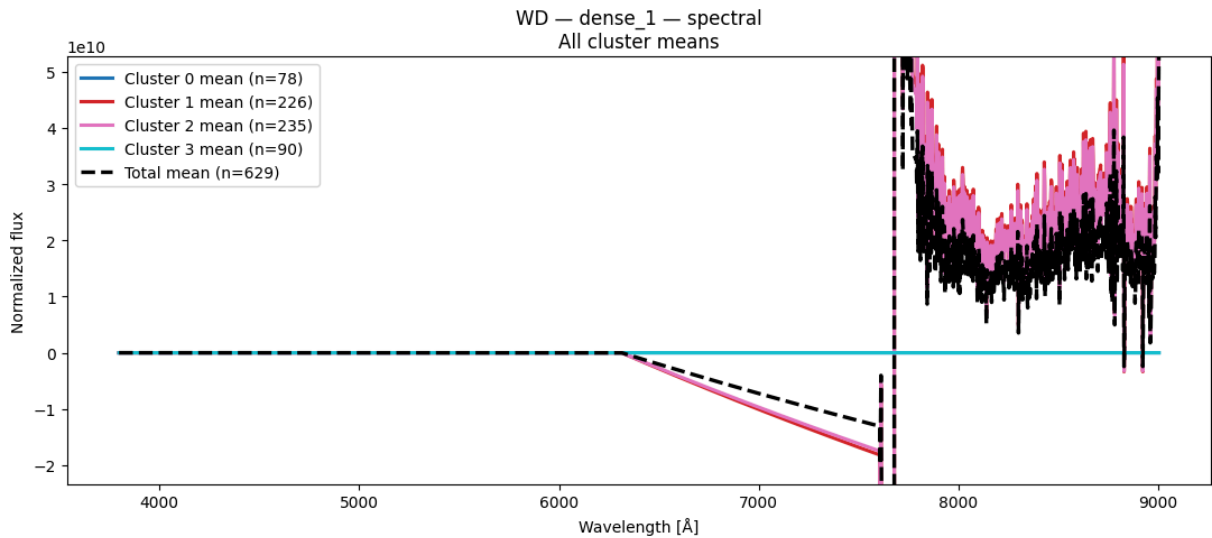
C:\Users\javip\AppData\Local\Temp\ipykernel_52368\531566132.py:28: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get\_cmap()`` or ``pyplot.get\_cmap()`` instead.

```
cmap = plt.cm.get_cmap("tab10", n_clusters)
```





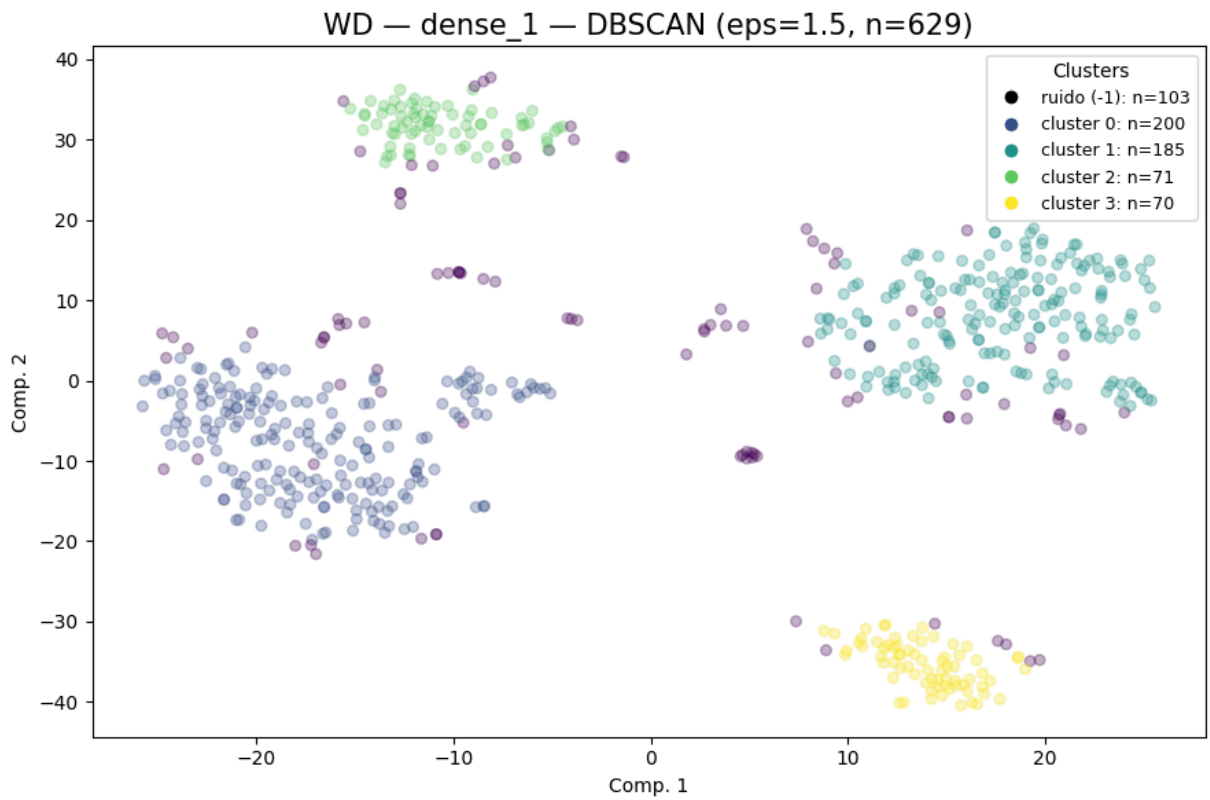




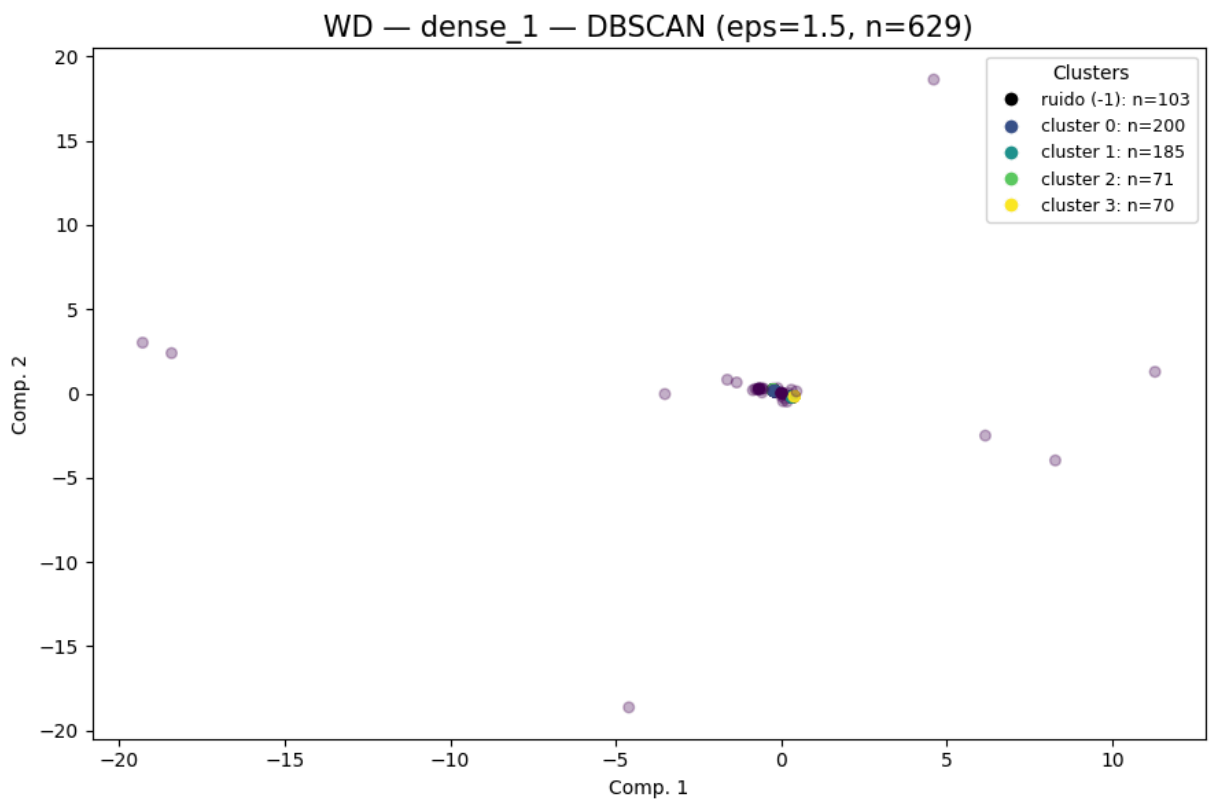
```
Out[72]: {'total_mean': array([1.8663167e+00, 1.4631674e+00, 2.3618588e+00, ..., 3.2000662e
+10,
      4.3846500e+10, 5.5692263e+10], dtype=float32),
      'group_means': {0: array([2.0025396 , 1.9914504 , 2.1715734 , ..., 0.5077039 , 0.
42377695,
      0.42661774], dtype=float32),
      1: array([1.1957948e+00, 2.3185274e-01, 2.6597047e+00, ..., 4.4531917e+10,
      6.1016490e+10, 7.7500965e+10], dtype=float32),
      2: array([2.4013567e+00, 2.2205632e+00, 2.2692659e+00, ..., 4.2826404e+10,
      5.8679673e+10, 7.4532823e+10], dtype=float32),
      3: array([2.0349662, 2.1196446, 2.0206237, ..., 0.6232097, 0.5278435,
      0.4985768], dtype=float32)},
      'unique_clusters': array([0, 1, 2, 3])}
```

DBSCAN n = 700

```
In [73]: res_dbs_dense1 = cluster_plot_DBSCAN(
      repr_wd_32["dense_1"],
      "WD",
      "dense_1",
      dbscan_eps=1.5,
      dbscan_min_samples=10,
      vis_method="tsne",
      xlim=None,
      ylim=None,
      container=cluster_labels_all
    )
labels_dbs_dense1_wd = res_dbs_dense1["dbscan"]
cluster_plot_DBSCAN(
      repr_wd_32["dense_1"],
      "WD",
      "dense_1",
      dbscan_eps=1.5,
      dbscan_min_samples=10,
      vis_method="pca",
      xlim=None,
      ylim=None
    )
```



```
WD | dense_1 | dbscan | eps=1.5  
clusters=4 noise=103 sil=0.5241 DB=0.7465 CH=401.98  
DBCV = -0.1156 (rango [-1,1], específico DBSCAN)
```



```
WD | dense_1 | dbscan | eps=1.5
clusters=4 noise=103 sil=0.5241 DB=0.7465 CH=401.98
DBCV = -0.1156 (rango [-1,1], específico DBSCAN)
```

```
Out[73]: {'dbscan': array([-1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, 2, -1, -1, -1,
-1,
```

```
-1, 2, -1, 0, -1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2,
-1, 2, 2, 2, 2, 2, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1,
1, -1, 3, 3, -1, 3, 3, 3, 3, 0, 0, 0, 0, 0, 0, 1, 1,
1, 1, 1, 1, -1, -1, 3, 3, 3, 3, 3, 3, -1, 0, 0, 0, 0,
0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 0, 0,
0, 0, 0, 0, 1, 1, 1, 1, 1, 1, -1, 2, -1, 2, 2, 2, -1,
2, 0, 0, 0, 0, 0, 0, 0, 1, -1, 1, -1, 1, 1, 3, 3, 3, 3,
3, -1, 3, 3, -1, 0, 0, 0, -1, 0, 1, 1, 1, 1, 1, 1, 3,
3, 3, 3, 3, 3, 3, 3, 0, 0, 0, 0, 0, 1, 1, 1, 1,
1, 1, 2, 2, 2, 2, 2, -1, -1, 2, -1, 0, 0, 0, 0, 0, 1,
-1, 1, 1, -1, 1, 2, 2, -1, 2, 2, 2, 2, 2, 0, 0, 0, 0,
0, 0, 1, 1, 1, -1, 1, 1, 3, 3, 3, 3, 3, 3, 3, 3, -1,
0, -1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1,
1, 0, 0, 0, -1, 0, 0, 1, 1, 1, 1, 1, 1, -1, 2, 2, 2,
2, 2, 2, 2, -1, 0, 0, 0, 0, 0, 0, -1, 1, 1, -1, 1, -1, 0,
0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, -1, 1, 1, 1, 1,
1, 3, 3, 3, 3, 3, 3, 3, 3, -1, 0, 0, 0, 0, 0, 1, 1,
1, 1, -1, 1, 0, 0, 0, 0, 1, 1, 1, -1, 0, 0, -1, 0, 0,
1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 0, 0, 0,
-1, 0, 0, -1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 0,
0, 0, 0, 0, -1, 1, 1, 1, 1, 1, -1, 3, -1, 3, 3, 3, 3,
3, 3, 0, 0, 0, 0, 0, -1, 1, -1, 1, 1, 1, -1, 0, 0, 0,
0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 2,
2, 2, 2, 2, -1, 2, 2, 0, 0, 0, 0, 0, 0, 1, -1, 1, 1,
1, -1, 0, 0, 0, 0, 1, -1, 1, 0, 0, 0, 0, 0, 0, 1, 1,
-1, 1, 1, 1, 3, 3, 3, 3, 3, 3, 3, 3, 0, 0, 0, 0, 0,
0, 1, 1, 1, -1, 1, 1, 0, 0, -1, 0, 1, 1, 1, 0, 0, -1,
0, 0, 0, 1, 1, 1, 1, 1, -1, -1, 2, 2, 2, -1, 2, 2, 2,
-1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, -1, 0, 0, 1,
1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, -1, 3, 3, -1,
-1, 3, 3, 3, 3, 0, 0, 0, 0, -1, 0, -1, 1, 1, 1, 1, 1,
0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, -1, 1,
1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 0, 0, 0, 0, 0, 1,
1, 1, 1, 1, -1, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0,
0, -1, 1, 1, 1, 1, 1, 3, 3, 3, 3, 3, 3, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1],
dtype=int64),
```

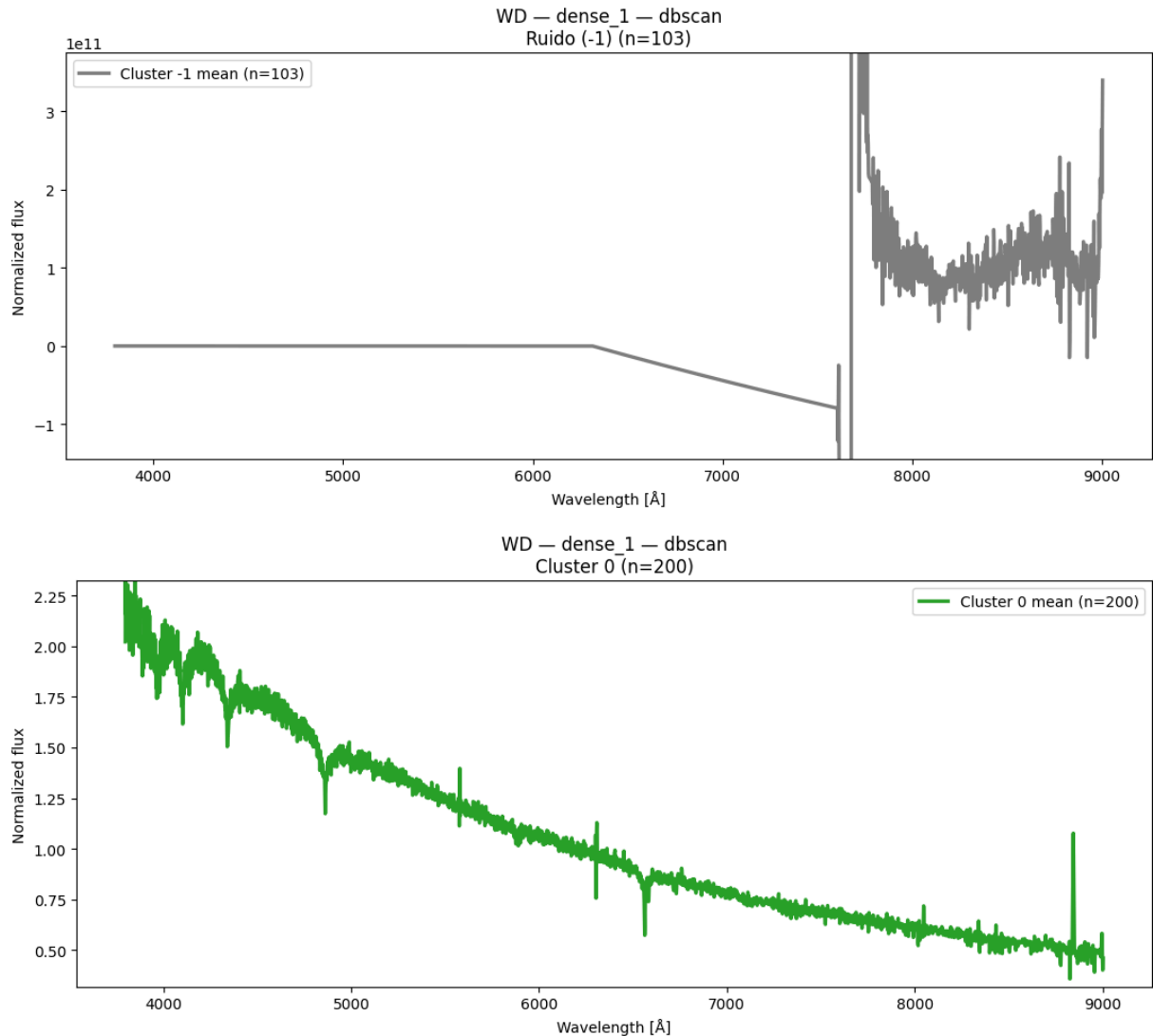
```
'metrics': {'n_clusters': 4,
'n_noise': 103,
'silhouette': 0.524051,
'davies_bouldin': 0.746543,
'calinski_harabasz': 401.9843},
'dbcv': -0.115633}
```

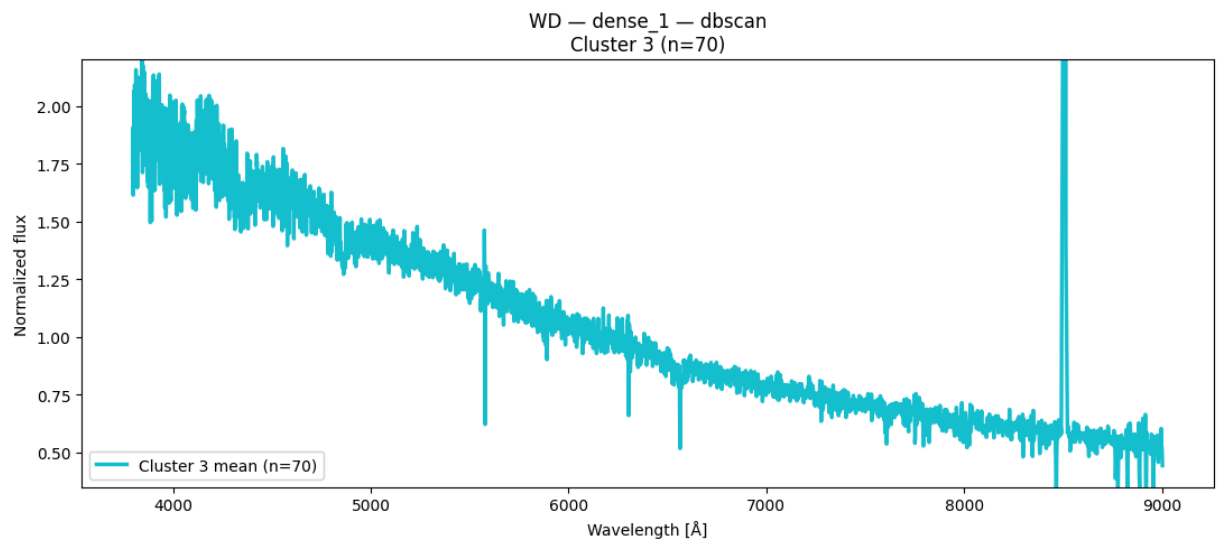
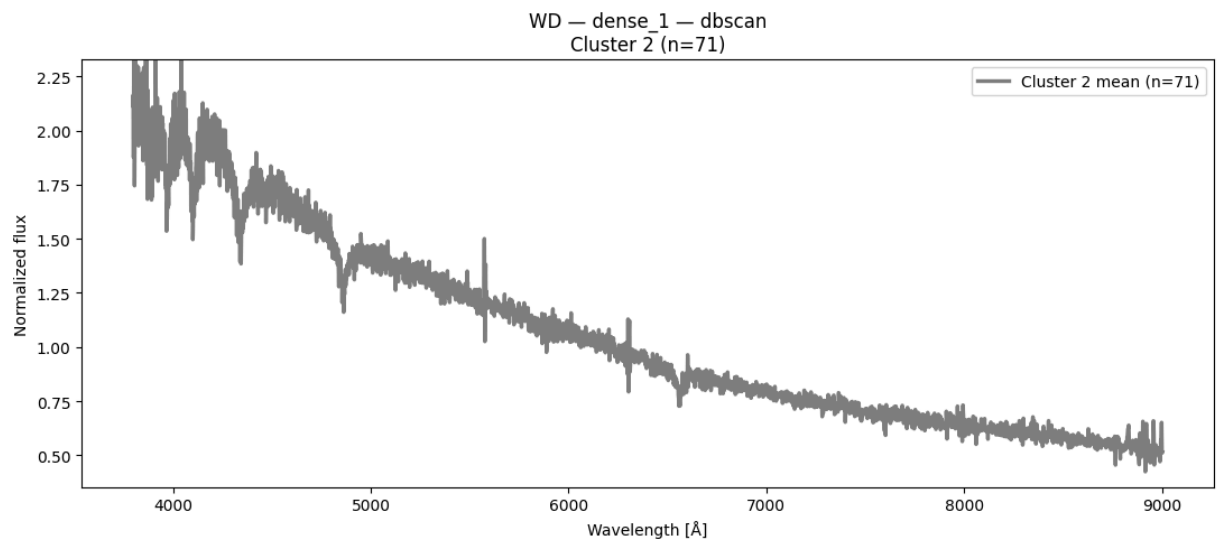
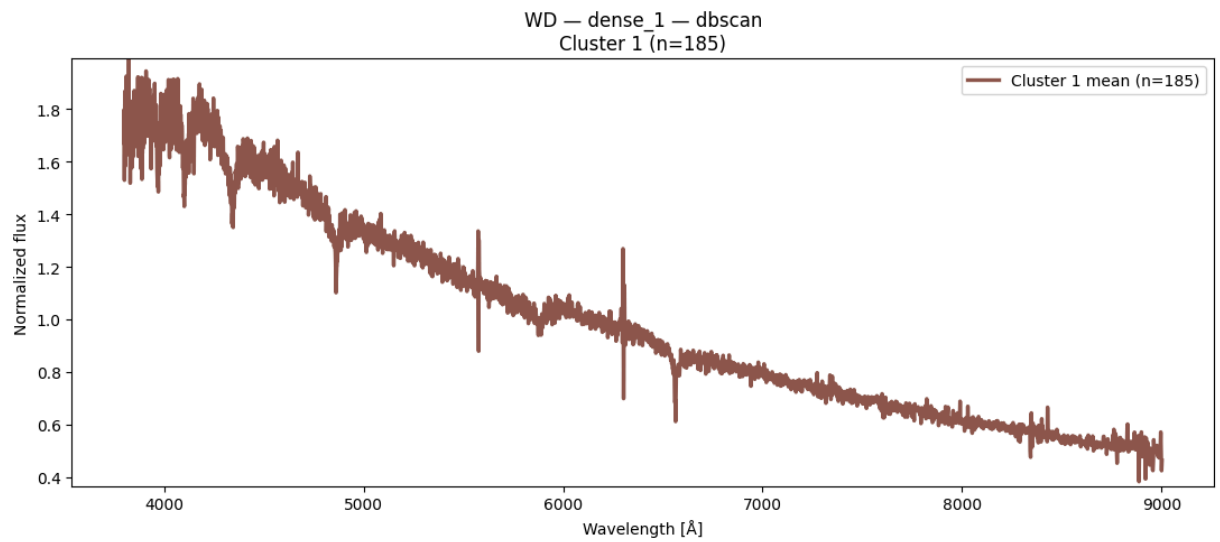
Espectros obtenidos de DBSCAN

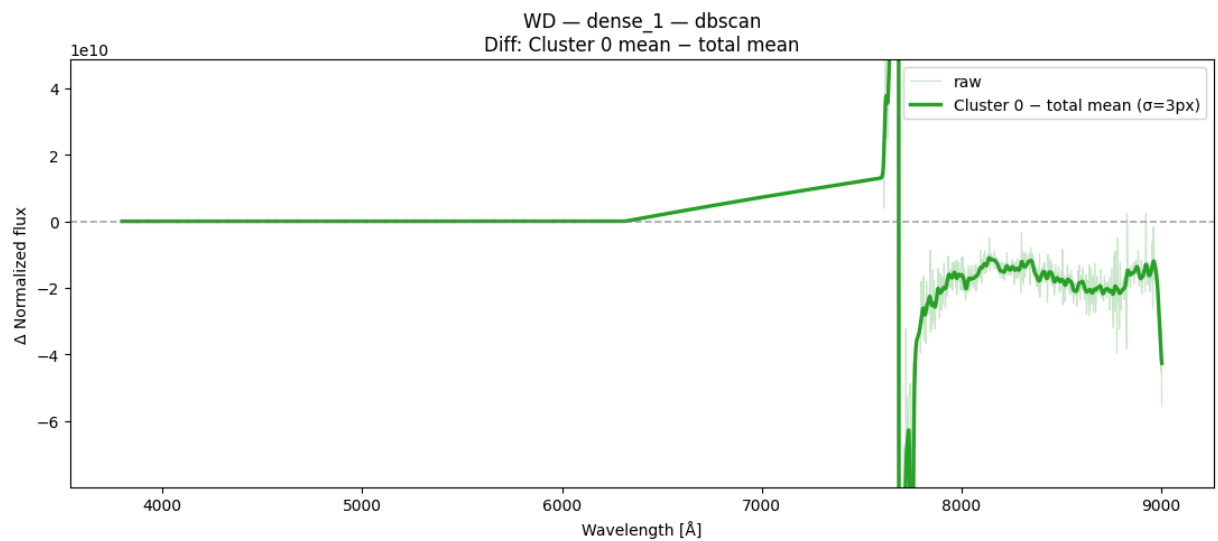
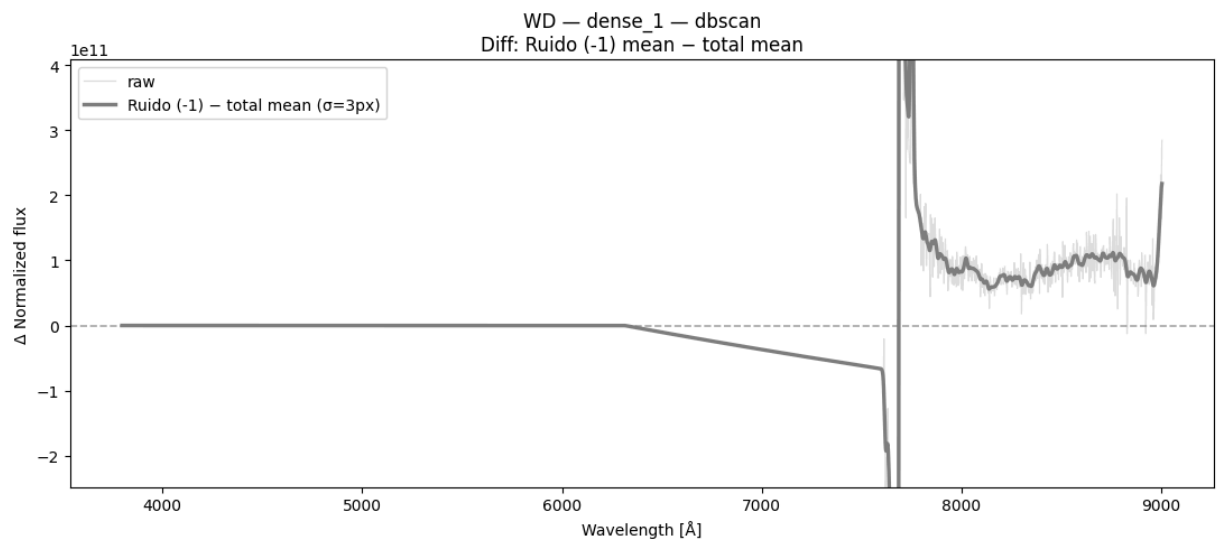
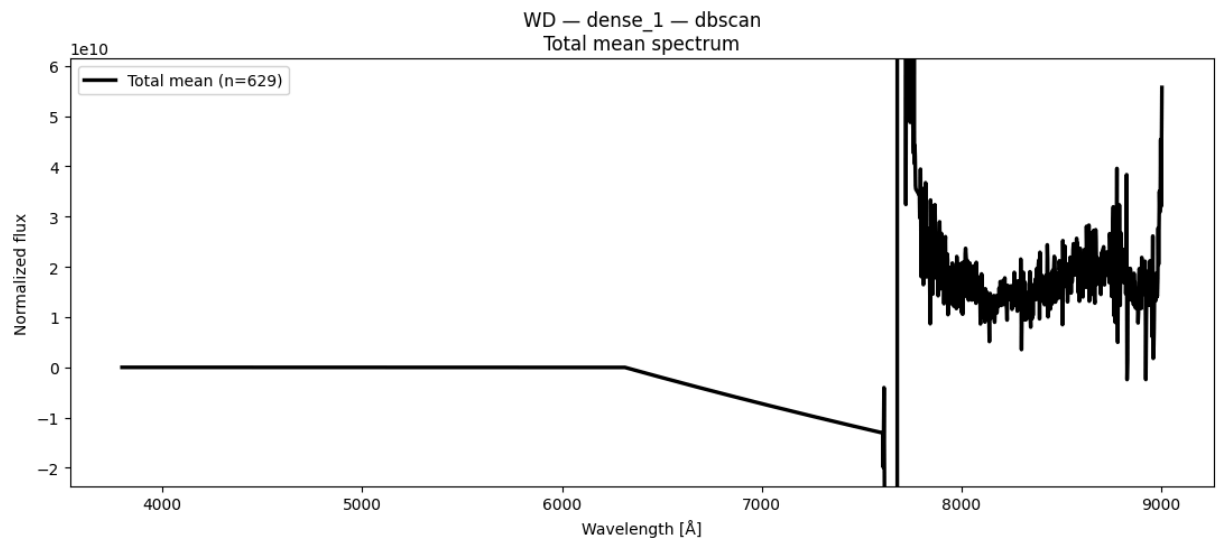
```
In [74]: plot_spectra_dbscan(
    X_wd_spec,
    labels_dbs_dense1_wd,
    W_wd[0],
    class_name="WD",
    layer_name="dense_1"
)
```

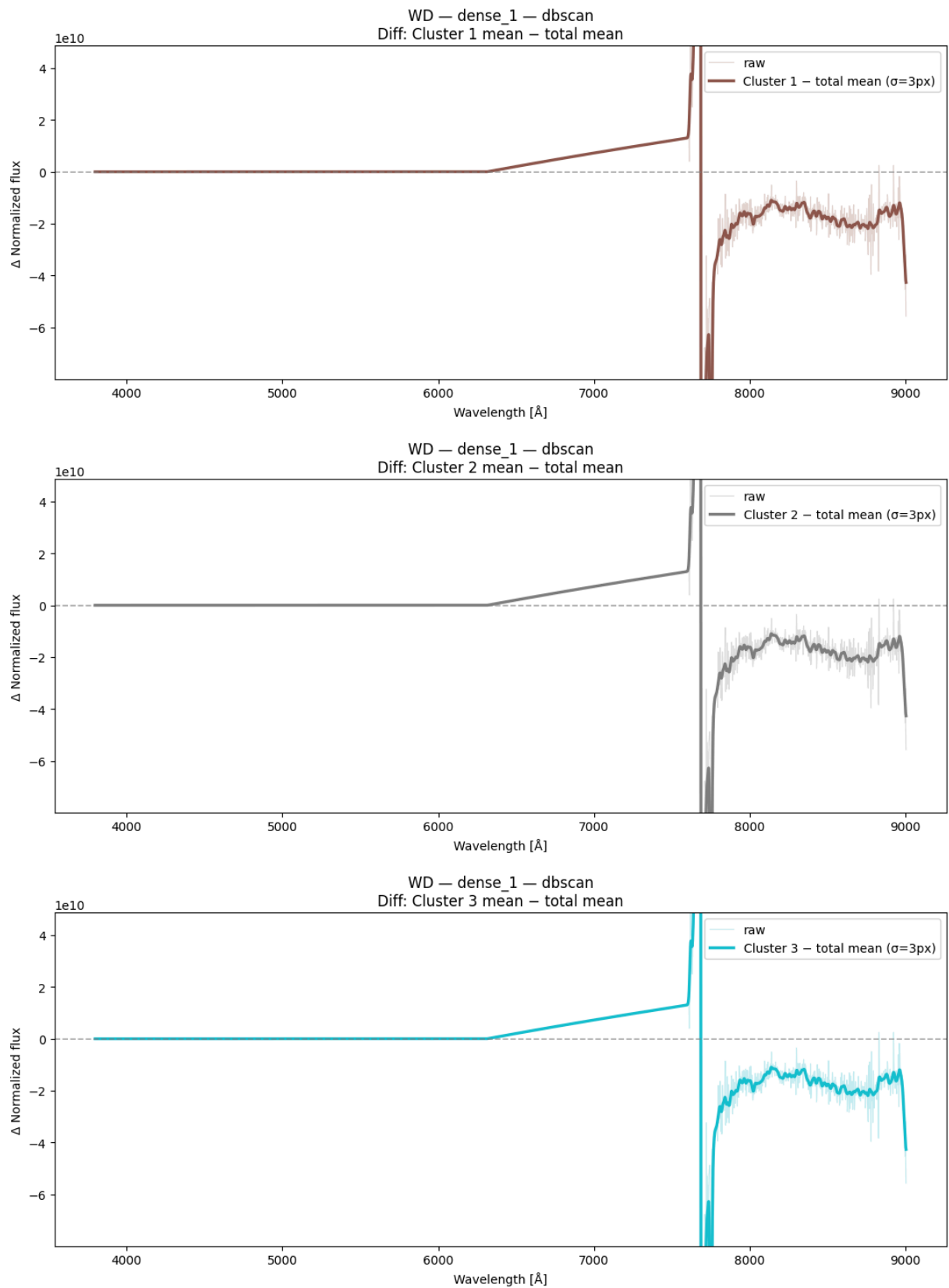
C:\Users\javip\AppData\Local\Temp\ipykernel_52368\531566132.py:28: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get\_cmap()`` or ``pyplot.get\_cmap()`` instead.

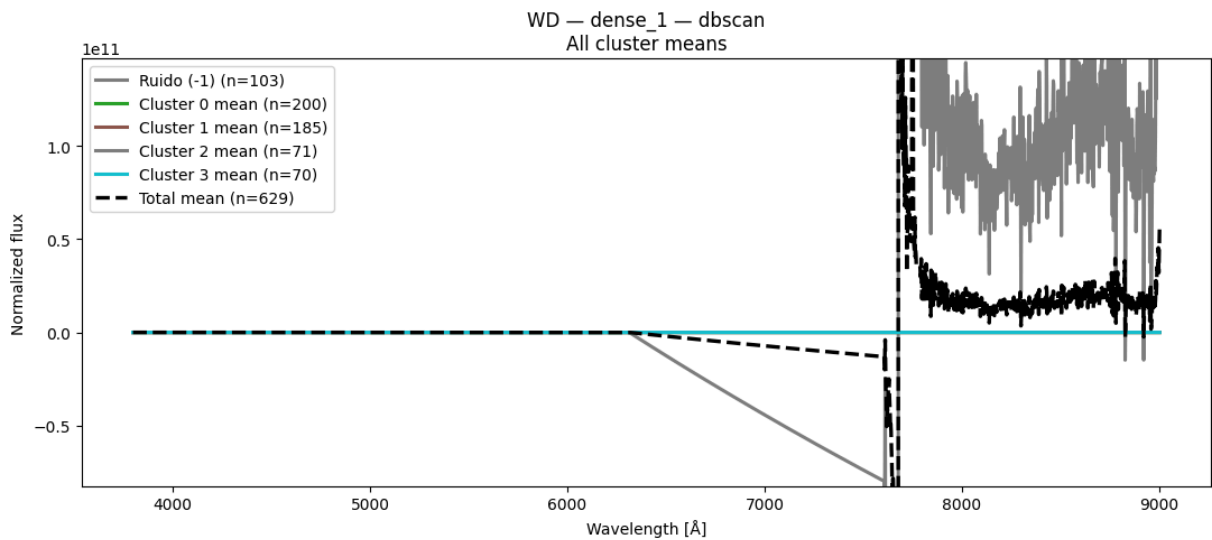
```
cmap = plt.cm.get_cmap("tab10", n_clusters)
```











```
Out[74]: {'total_mean': array([1.8663167e+00, 1.4631674e+00, 2.3618588e+00, ..., 3.2000662e
+10,
      4.3846500e+10, 5.5692263e+10], dtype=float32),
'group_means': {-1: array([ 1.6449013e+00, -1.3947980e+00,  4.4945350e+00, ...,
      1.9542152e+11,  2.6776163e+11,  3.4010130e+11], dtype=float32),
      0: array([2.0213327 , 2.3249326 , 2.1583364 , ..., 0.48786697, 0.40204364,
      0.46102732], dtype=float32),
      1: array([1.7133136 , 1.7967008 , 1.6679933 , ..., 0.50308114, 0.4245657 ,
      0.46374   ], dtype=float32),
      2: array([2.1135368 , 2.1634521 , 2.1047983 , ..., 0.651553 , 0.5104529 ,
      0.51593614], dtype=float32),
      3: array([1.9028294 , 1.6145003 , 1.8997971 , ..., 0.5276388 , 0.5165965 ,
      0.44271836], dtype=float32)},
'unique_clusters': array([-1,  0,  1,  2,  3], dtype=int64)}
```

Resultados capa dense_2 con Spectral Clustering + tSNE y PCA

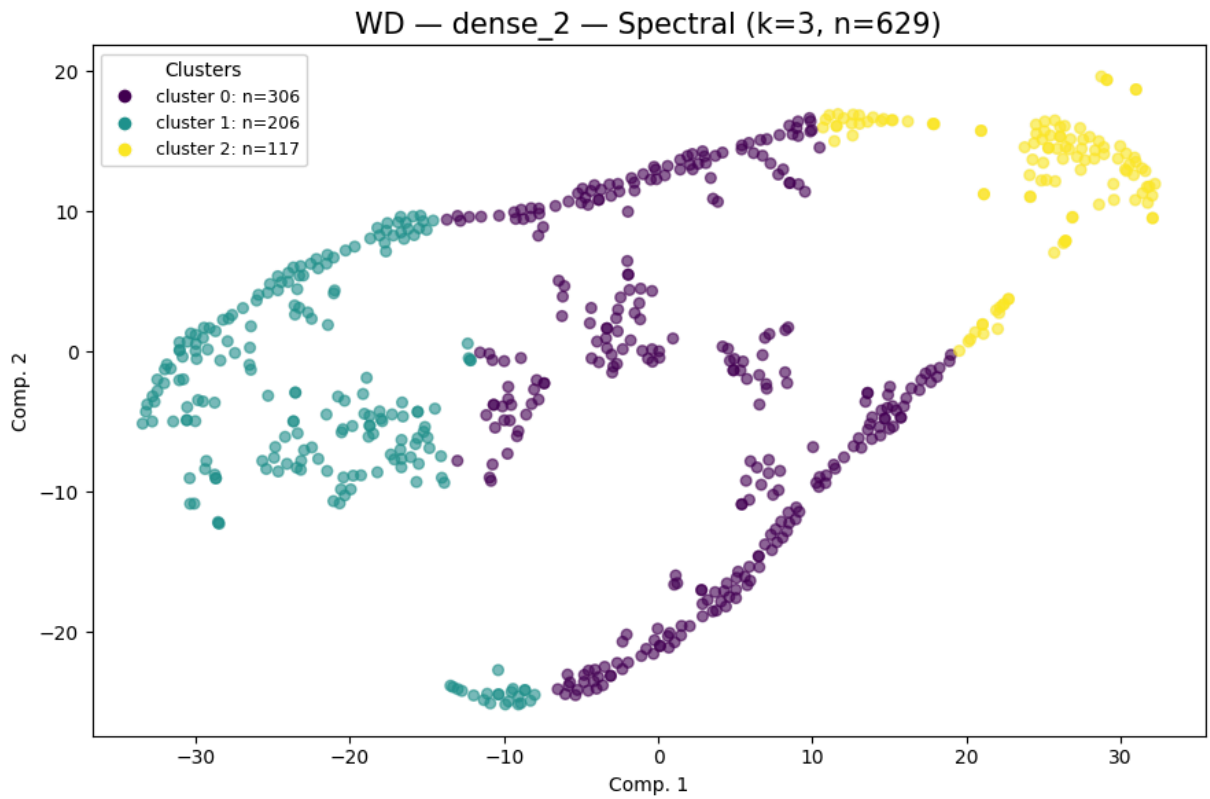
Spectral Clustering k=3

```
In [75]: res_dense2_k3 = cluster_plot_Spect(
        repr_wd_32["dense_2"],
        "WD",
        "dense_2",
        spectral_k=3,
        vis_method="tsne",
        container=cluster_labels_all
    )

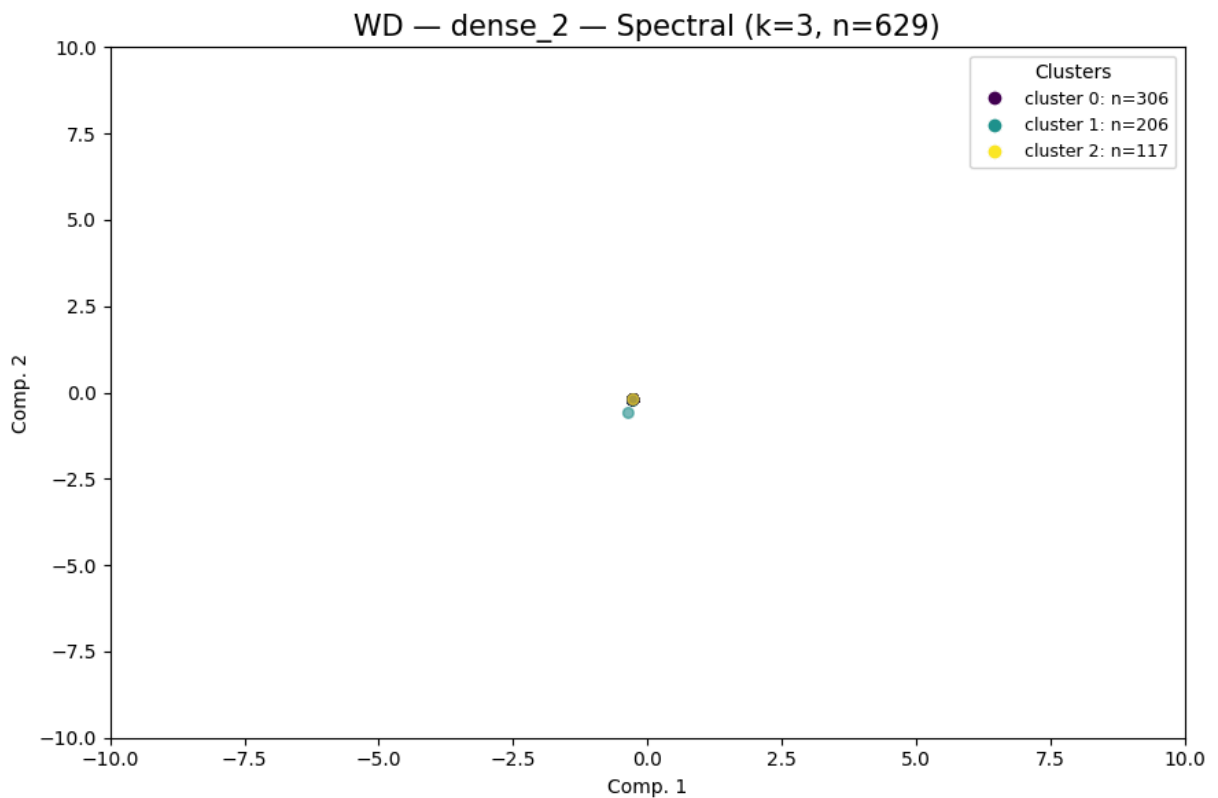
labels_dense2_k3_wd = res_dense2_k3["spectral"]
cluster_plot_Spect(
    repr_wd_32["dense_2"],
    "WD",
    "dense_2",
    spectral_k=3,
    vis_method="pca",

```

```
xlim=(-10,10), ylim=(-10,10)
)
```



```
WD | dense_2 | spectral | k=3
clusters=3 noise=0 sil=0.1018 DB=2.5561 CH=5.27
```



WD	dense_2	spectral	k=3
clusters=3	noise=0	sil=0.1018	DB=2.5561 CH=5.27

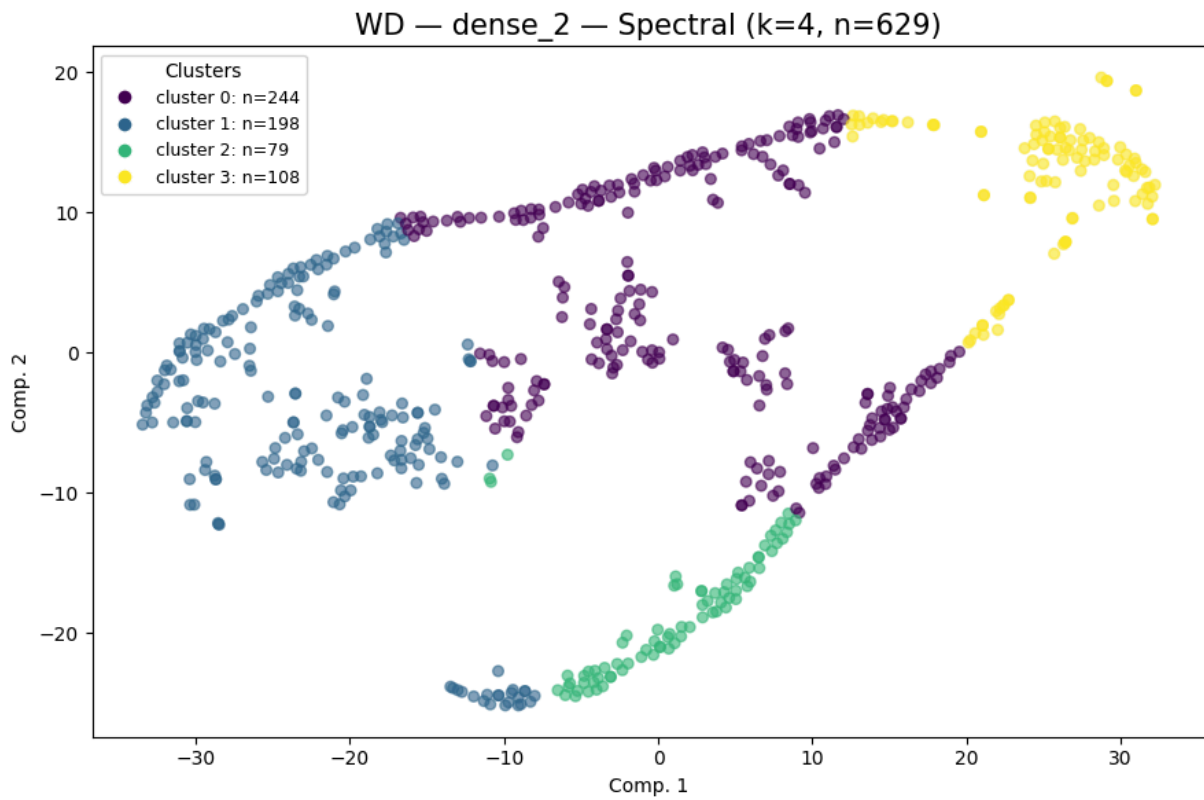
```
Out[75]: {'spectral': array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 2, 1, 1, 1,
1,
1, 0, 0, 0, 0, 2, 1, 2, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1, 2, 0, 2, 0,
0, 0, 2, 1, 0, 2, 0, 0, 2, 0, 1, 2, 0, 1, 0, 2, 1, 2, 0, 0, 2, 1,
0, 0, 0, 2, 0, 2, 2, 1, 0, 0, 2, 0, 2, 1, 2, 0, 2, 0, 1, 2, 0, 0,
1, 1, 1, 1, 1, 1, 2, 0, 2, 1, 1, 2, 1, 0, 2, 1, 0, 2, 1, 1, 0, 2,
0, 1, 1, 0, 1, 0, 0, 2, 1, 1, 2, 0, 0, 0, 1, 0, 1, 1, 0, 1, 2, 0,
2, 0, 2, 0, 2, 1, 0, 1, 1, 1, 2, 0, 2, 0, 1, 2, 0, 0, 0, 2, 0, 2,
2, 2, 1, 1, 0, 2, 0, 0, 2, 0, 0, 0, 0, 1, 0, 2, 2, 0, 2, 1, 2, 1,
0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 2, 1, 0, 0, 1, 2, 2, 2, 1, 0, 2, 2,
0, 0, 2, 0, 0, 0, 2, 1, 1, 1, 0, 1, 0, 2, 1, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 0, 0, 2, 2, 0, 1, 2, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1,
1, 2, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0,
0, 2, 0, 2, 2, 0, 1, 1, 0, 0, 1, 0, 1, 0, 2, 0, 2, 1, 1, 0, 2, 1,
0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 2, 1, 0, 0, 1, 0, 0, 0, 2,
2, 0, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0, 2, 0, 0, 2, 0, 0, 1, 1, 1, 0,
2, 0, 1, 2, 0, 1, 0, 0, 2, 0, 0, 2, 1, 1, 0, 0, 0, 0, 0, 2, 2, 0,
1, 1, 1, 1, 2, 0, 1, 1, 0, 0, 1, 0, 2, 1, 0, 1, 0, 0, 2, 1, 0, 0,
0, 1, 1, 1, 0, 2, 0, 1, 1, 1, 0, 2, 1, 1, 2, 0, 0, 0, 1, 2, 2, 1,
1, 0, 0, 1, 2, 0, 0, 2, 0, 0, 1, 1, 0, 2, 2, 0, 0, 0, 0, 0, 1, 0,
0, 2, 0, 1, 1, 2, 0, 0, 2, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0,
0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 0, 1, 0, 0, 0, 2,
1, 1, 0, 2, 0, 0, 2, 2, 1, 1, 1, 0, 0, 1, 1, 1, 1, 2, 2, 0, 1, 1,
2, 1, 1, 1, 1, 1, 2, 2, 1, 2, 1, 0, 1, 1, 0, 1, 0, 1, 1, 2, 0, 0,
2, 1, 1, 0, 0, 0, 0, 1, 2, 0, 0, 0, 0, 0, 2, 0, 0, 0, 1, 0, 1, 1,
2, 1, 1, 0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0,
0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 2,
1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 2, 0, 1, 0,
1, 2, 1, 2, 2, 1, 0, 0, 2, 0, 1, 0, 0, 2, 1, 1, 0, 1, 1, 2, 1, 0,
1, 0, 2, 0, 2, 0, 0, 0, 0, 1, 1, 0, 2]),
'metrics': {'n_clusters': 3,
'n_noise': 0,
'silhouette': 0.101788,
'davies_bouldin': 2.556064,
'calinski_harabasz': 5.2671}}
```

Spectral Clustering k = 4

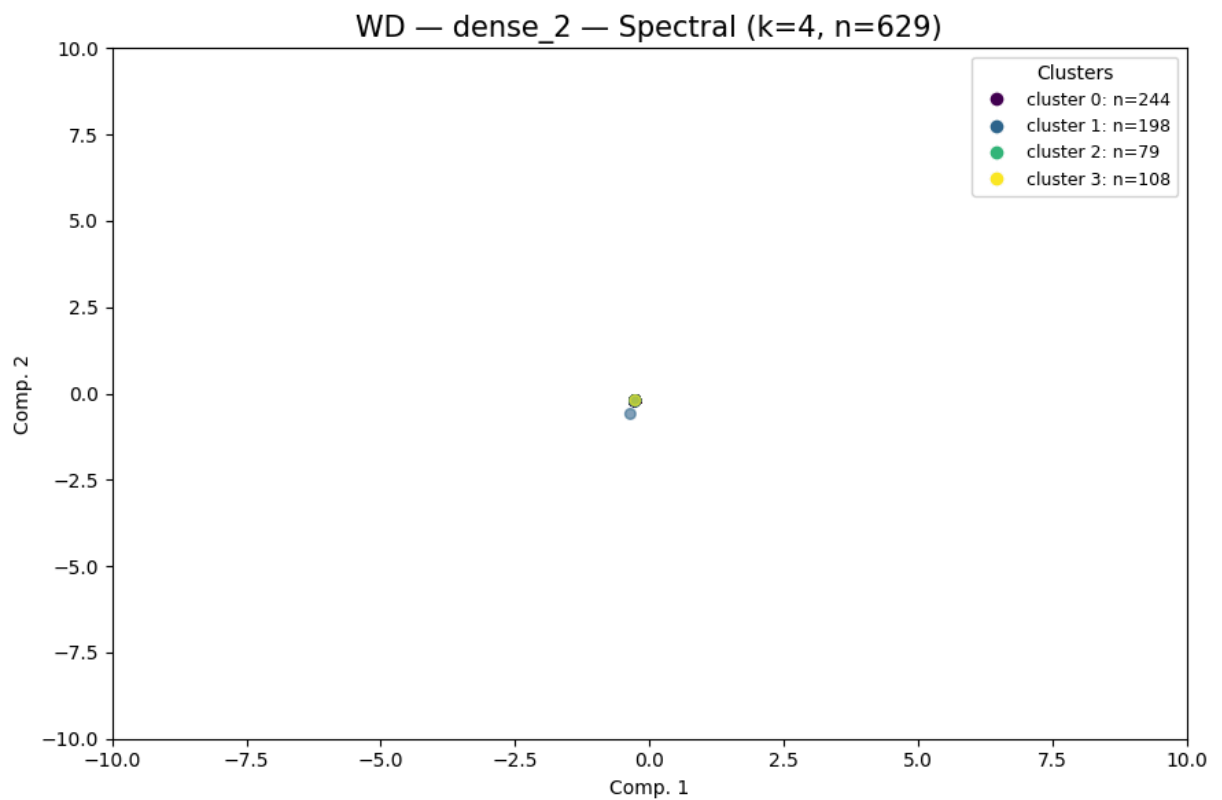
```
In [76]: res_dense2_k4 = cluster_plot_Spect(
    repr_wd_32["dense_2"],
    "WD",
    "dense_2",
    spectral_k=4,
    vis_method="tsne",
    container=cluster_labels_all
)

labels_dense2_k4_wd = res_dense2_k4["spectral"]
cluster_plot_Spect(
    repr_wd_32["dense_2"],
```

```
"WD",  
"dense_2",  
spectral_k=4,  
vis_method="pca",  
xlim=(-10,10), ylim=(-10,10)  
)
```



```
WD | dense_2 | spectral | k=4  
clusters=4 noise=0 sil=-0.0556 DB=2.5136 CH=3.86
```



```
WD | dense_2 | spectral | k=4  
clusters=4 noise=0 sil=-0.0556 DB=2.5136 CH=3.86
```



```

Out[76]: {'spectral': array([2, 0, 0, 0, 0, 2, 0, 1, 2, 0, 0, 1, 0, 1, 0, 1, 0, 3, 1, 1, 1,
1,
1, 0, 0, 0, 0, 3, 1, 3, 0, 1, 0, 2, 1, 2, 1, 1, 1, 1, 0, 0, 3, 0,
0, 0, 3, 1, 0, 3, 0, 0, 3, 0, 1, 3, 0, 1, 2, 3, 1, 3, 0, 0, 3, 1,
0, 0, 0, 3, 0, 3, 3, 1, 0, 2, 3, 2, 3, 1, 3, 0, 3, 0, 1, 3, 0, 0,
1, 1, 1, 1, 1, 1, 3, 0, 3, 0, 1, 3, 1, 0, 3, 1, 0, 3, 1, 1, 2, 3,
2, 1, 1, 0, 1, 2, 0, 3, 1, 1, 3, 2, 0, 0, 1, 0, 1, 1, 0, 1, 3, 2,
3, 0, 3, 0, 0, 1, 0, 1, 1, 1, 3, 0, 3, 0, 1, 3, 0, 0, 0, 3, 2, 3,
3, 3, 1, 1, 0, 3, 0, 0, 3, 0, 2, 0, 0, 1, 0, 3, 3, 0, 3, 1, 3, 1,
2, 1, 1, 0, 2, 0, 2, 2, 1, 0, 3, 1, 0, 0, 1, 3, 3, 3, 1, 0, 3, 3,
0, 0, 3, 0, 2, 0, 3, 1, 1, 1, 0, 1, 2, 3, 1, 0, 0, 0, 0, 2, 0,
1, 1, 1, 1, 2, 0, 3, 3, 0, 1, 3, 1, 0, 1, 2, 0, 1, 0, 0, 0, 1, 0,
1, 3, 1, 0, 0, 0, 2, 2, 0, 1, 0, 0, 0, 2, 0, 0, 0, 1, 1, 2, 0, 2,
2, 3, 0, 3, 3, 0, 1, 1, 0, 0, 1, 2, 1, 0, 3, 2, 0, 1, 1, 0, 3, 1,
0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 2, 0, 1, 3, 1, 0, 2, 1, 0, 0, 0, 3,
3, 0, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0, 3, 0, 0, 3, 2, 0, 1, 1, 1, 0,
3, 0, 1, 3, 0, 1, 0, 2, 0, 2, 2, 0, 1, 1, 0, 0, 2, 0, 0, 3, 3, 0,
1, 1, 1, 1, 3, 0, 1, 1, 0, 2, 1, 2, 3, 1, 0, 1, 2, 0, 3, 1, 0, 0,
2, 1, 0, 1, 0, 3, 2, 1, 1, 1, 0, 3, 1, 1, 3, 0, 0, 0, 1, 3, 3, 1,
1, 0, 0, 1, 3, 0, 0, 3, 0, 0, 1, 1, 0, 3, 3, 0, 2, 0, 2, 0, 1, 2,
0, 0, 0, 0, 1, 3, 0, 0, 3, 0, 0, 1, 1, 1, 1, 0, 0, 1, 2, 0, 0, 0,
2, 2, 1, 1, 2, 1, 2, 2, 2, 0, 0, 2, 0, 3, 0, 2, 2, 1, 0, 0, 0, 3,
1, 1, 0, 3, 0, 0, 3, 3, 1, 1, 1, 0, 2, 1, 1, 1, 1, 3, 3, 0, 1, 1,
3, 1, 1, 1, 1, 1, 3, 3, 1, 3, 0, 0, 1, 1, 2, 1, 2, 1, 1, 3, 0, 0,
3, 1, 1, 0, 2, 0, 0, 1, 0, 2, 0, 0, 0, 0, 3, 0, 2, 2, 1, 0, 1, 1,
3, 0, 1, 0, 2, 1, 0, 0, 1, 0, 1, 0, 2, 0, 1, 1, 0, 2, 0, 1, 0, 0,
2, 0, 1, 0, 2, 0, 2, 1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 2, 0, 2, 3,
0, 0, 0, 1, 0, 0, 0, 2, 1, 0, 1, 1, 1, 0, 0, 0, 0, 2, 3, 1, 1, 2,
1, 3, 1, 0, 3, 1, 0, 2, 3, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 3, 1, 0,
1, 0, 3, 0, 3, 2, 2, 0, 0, 1, 1, 2, 3]),
'metrics': {'n_clusters': 4,
'n_noise': 0,
'silhouette': -0.055605,
'davies_bouldin': 2.513604,
'calinski_harabasz': 3.861}}

```

Spectral Clustering k = 5

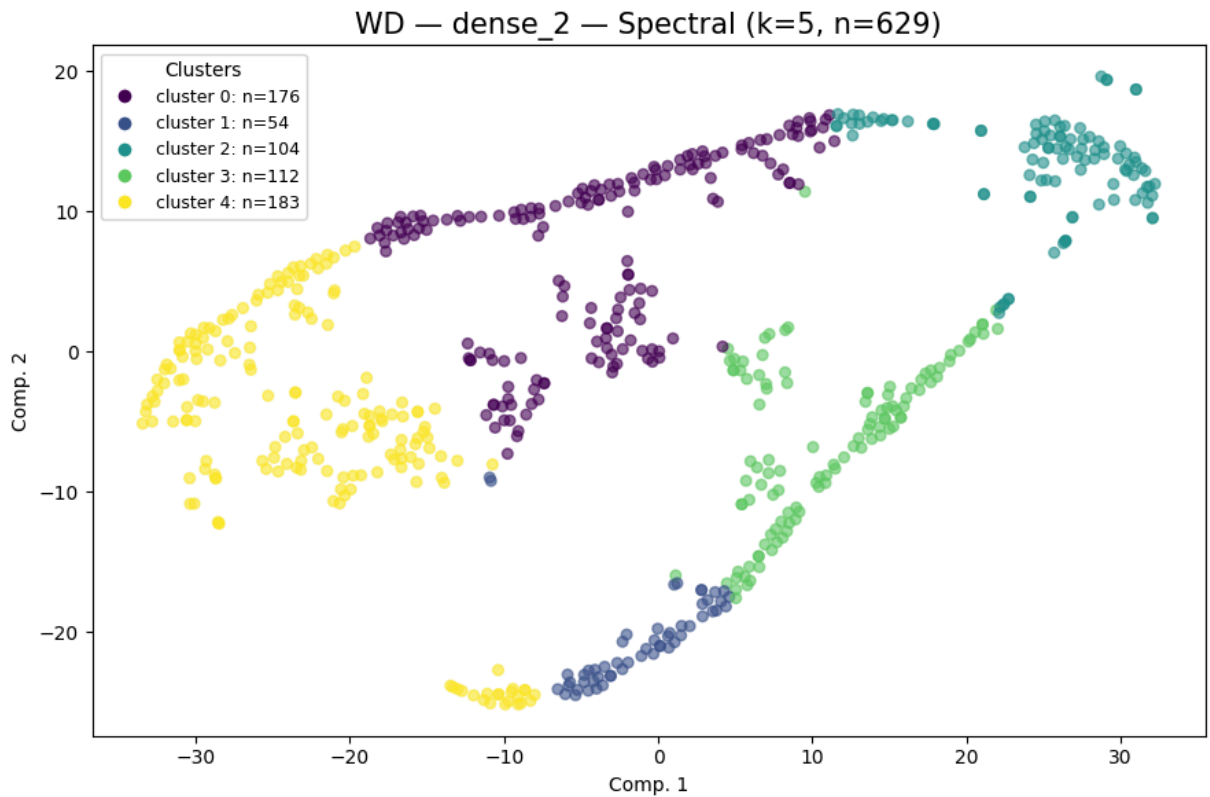
```

In [77]: res_dense2_k5 = cluster_plot_Spect(
    repr_wd_32["dense_2"],
    "WD",
    "dense_2",
    spectral_k=5,
    vis_method="tsne",
    container=cluster_labels_all
)

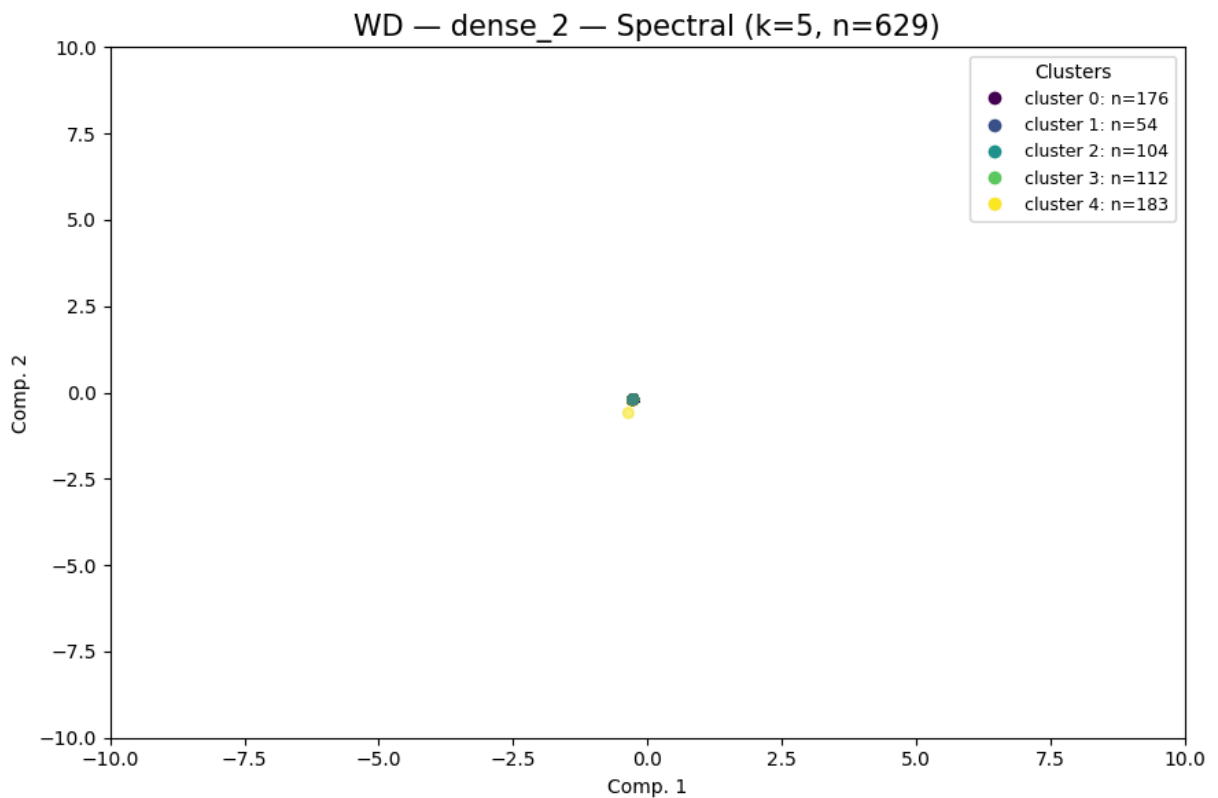
labels_dense2_k5_wd = res_dense2_k5["spectral"]
cluster_plot_Spect(
    repr_wd_32["dense_2"],
    "WD",
    "dense_2",
    spectral_k=5,
    vis_method="pca",

```

```
xlim=(-10,10), ylim=(-10,10)
)
```



```
WD | dense_2 | spectral | k=5
clusters=5 noise=0 sil=-0.1175 DB=2.4924 CH=3.04
```



```
WD | dense_2 | spectral | k=5
clusters=5 noise=0 sil=-0.1175 DB=2.4924 CH=3.04
```

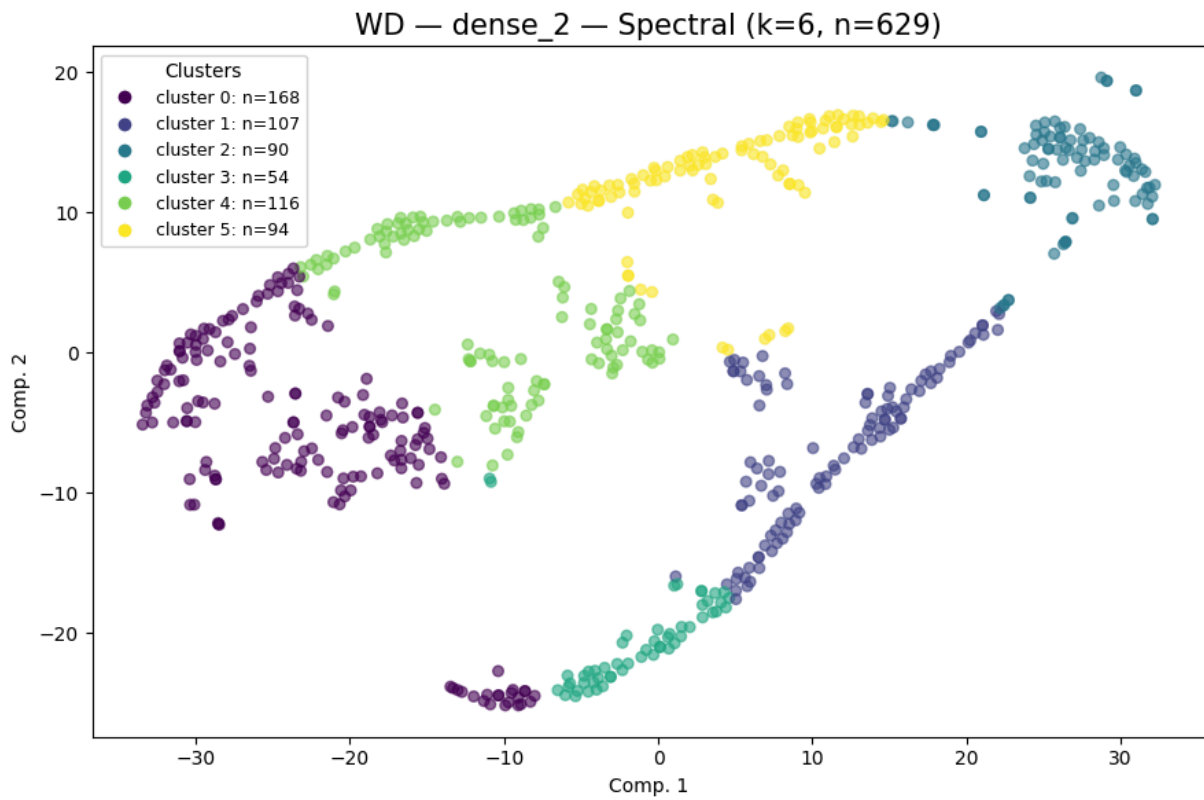
```
Out[77]: {'spectral': array([1, 3, 0, 0, 3, 3, 0, 4, 1, 3, 0, 4, 3, 4, 3, 4, 0, 2, 4, 4, 4,
4,
4, 3, 0, 3, 3, 2, 4, 2, 3, 4, 3, 3, 4, 3, 4, 4, 0, 4, 0, 3, 2, 0,
3, 3, 2, 4, 0, 2, 0, 0, 2, 0, 4, 2, 0, 4, 3, 2, 4, 2, 0, 0, 2, 4,
3, 3, 0, 2, 3, 2, 2, 4, 3, 1, 2, 1, 2, 4, 2, 0, 2, 0, 4, 2, 3, 0,
0, 4, 4, 4, 4, 0, 2, 3, 2, 0, 4, 2, 4, 3, 2, 4, 0, 2, 4, 0, 1, 2,
3, 4, 4, 0, 4, 1, 0, 2, 4, 4, 2, 3, 0, 3, 4, 0, 4, 4, 0, 4, 2, 3,
2, 0, 3, 0, 0, 4, 0, 4, 4, 4, 2, 0, 2, 0, 4, 3, 3, 3, 3, 2, 1, 2,
2, 2, 4, 4, 0, 2, 0, 0, 2, 0, 3, 3, 0, 0, 3, 3, 2, 0, 2, 4, 2, 4,
1, 4, 4, 0, 3, 0, 1, 1, 0, 0, 2, 4, 3, 3, 4, 2, 2, 2, 4, 0, 2, 2,
0, 0, 2, 0, 1, 3, 3, 4, 4, 4, 3, 4, 1, 2, 4, 0, 3, 0, 3, 0, 1, 0,
4, 4, 4, 4, 1, 0, 2, 2, 0, 0, 2, 4, 0, 4, 3, 0, 4, 3, 3, 3, 4, 0,
4, 2, 4, 0, 0, 0, 1, 3, 0, 4, 3, 3, 3, 1, 3, 0, 3, 4, 4, 1, 3, 1,
1, 2, 3, 2, 2, 0, 4, 4, 0, 0, 4, 1, 4, 0, 2, 3, 2, 0, 4, 3, 2, 4,
3, 3, 0, 3, 0, 0, 0, 0, 4, 0, 1, 3, 4, 2, 4, 3, 3, 4, 0, 0, 0, 2,
2, 0, 4, 4, 3, 4, 4, 4, 0, 3, 0, 3, 2, 3, 0, 2, 1, 3, 4, 4, 4, 3,
3, 0, 4, 2, 0, 4, 0, 1, 2, 1, 1, 0, 4, 4, 3, 0, 1, 0, 0, 2, 2, 0,
4, 4, 4, 4, 2, 3, 4, 4, 0, 3, 4, 1, 2, 4, 0, 4, 1, 0, 2, 4, 0, 0,
1, 4, 0, 4, 0, 2, 3, 4, 4, 4, 0, 2, 4, 0, 2, 0, 3, 0, 4, 3, 2, 4,
4, 3, 0, 4, 2, 3, 0, 2, 0, 0, 4, 4, 0, 2, 2, 3, 1, 0, 1, 0, 4, 3,
0, 0, 0, 0, 4, 2, 0, 0, 2, 0, 3, 4, 4, 4, 4, 0, 0, 4, 1, 3, 0, 0,
3, 3, 4, 4, 1, 4, 1, 1, 1, 3, 0, 1, 0, 2, 0, 1, 0, 4, 0, 0, 3, 2,
4, 4, 0, 2, 3, 0, 2, 3, 4, 4, 0, 3, 1, 4, 0, 4, 4, 2, 2, 0, 4, 4,
2, 4, 4, 0, 4, 4, 2, 2, 0, 2, 0, 3, 4, 4, 1, 4, 1, 4, 4, 2, 0, 0,
2, 4, 0, 0, 1, 0, 0, 4, 3, 3, 0, 0, 0, 0, 2, 0, 3, 1, 4, 3, 4, 4,
3, 0, 4, 0, 1, 4, 0, 0, 4, 0, 4, 0, 3, 0, 4, 4, 0, 1, 0, 4, 0, 3,
1, 0, 4, 0, 3, 0, 1, 4, 3, 4, 3, 4, 4, 4, 4, 3, 4, 3, 1, 0, 1, 2,
0, 0, 0, 4, 0, 0, 3, 1, 4, 0, 4, 4, 4, 0, 3, 0, 0, 1, 2, 4, 4, 1,
4, 2, 4, 2, 2, 4, 3, 3, 2, 3, 4, 0, 0, 2, 0, 4, 3, 4, 4, 2, 4, 0,
4, 0, 2, 3, 2, 3, 1, 0, 0, 4, 4, 1, 2]]),
'metrics': {'n_clusters': 5,
'n_noise': 0,
'silhouette': -0.117476,
'davies_bouldin': 2.492377,
'calinski_harabasz': 3.0358}}
```

Spectral Clustering k = 6

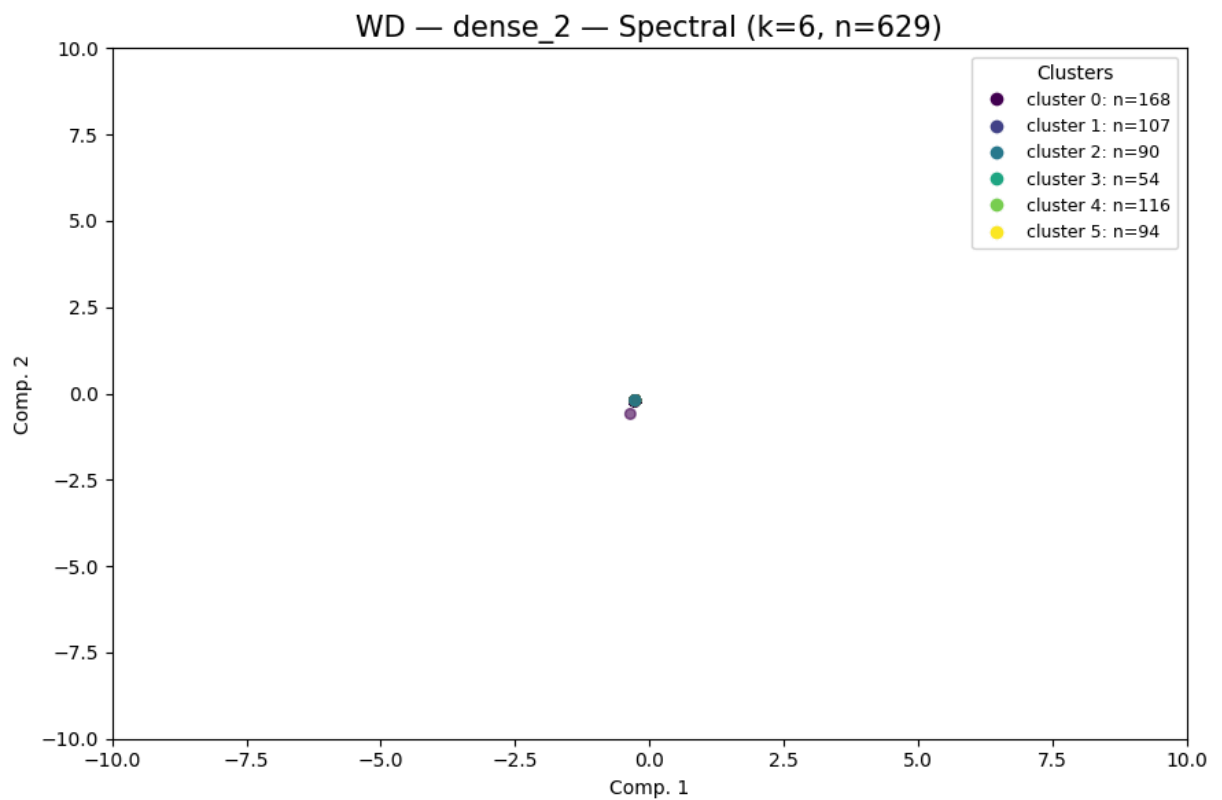
```
In [78]: res_dense2_k6 = cluster_plot_Spect(
    repr_wd_32["dense_2"],
    "WD",
    "dense_2",
    spectral_k=6,
    vis_method="tsne",
    container=cluster_labels_all
)

labels_dense2_k6_wd = res_dense2_k6["spectral"]
cluster_plot_Spect(
    repr_wd_32["dense_2"],
```

```
"WD",  
"dense_2",  
spectral_k=6,  
vis_method="pca",  
xlim=(-10,10), ylim=(-10,10)  
)
```



```
WD | dense_2 | spectral | k=6  
clusters=6 noise=0 sil=-0.0764 DB=2.4610 CH=2.86
```



```
WD | dense_2 | spectral | k=6  
clusters=6 noise=0 sil=-0.0764 DB=2.4610 CH=2.86
```

```

Out[78]: {'spectral': array([3, 1, 5, 5, 1, 1, 4, 0, 3, 1, 5, 0, 1, 0, 1, 0, 5, 2, 4, 0, 0,
0,
0, 1, 4, 1, 1, 2, 0, 2, 1, 0, 1, 1, 0, 1, 0, 0, 4, 0, 5, 1, 2, 4,
5, 1, 5, 0, 4, 2, 4, 5, 2, 4, 0, 2, 4, 0, 1, 2, 0, 2, 4, 4, 2, 0,
1, 1, 5, 2, 1, 2, 2, 0, 1, 3, 2, 3, 2, 4, 2, 4, 2, 5, 0, 2, 1, 5,
4, 0, 0, 0, 0, 4, 5, 1, 2, 4, 0, 2, 0, 1, 2, 0, 5, 2, 0, 4, 3, 2,
1, 0, 0, 5, 0, 3, 5, 2, 0, 0, 2, 1, 4, 1, 0, 4, 0, 0, 5, 0, 2, 1,
2, 4, 1, 5, 5, 0, 4, 0, 0, 0, 2, 4, 2, 5, 0, 1, 1, 1, 5, 2, 3, 2,
2, 2, 0, 0, 5, 2, 4, 5, 2, 5, 1, 1, 5, 4, 1, 1, 2, 5, 2, 0, 2, 0,
3, 0, 0, 4, 1, 4, 3, 3, 4, 4, 2, 0, 1, 1, 0, 2, 2, 2, 0, 5, 2, 2,
5, 4, 5, 4, 3, 1, 1, 0, 0, 0, 5, 0, 3, 2, 0, 4, 1, 4, 1, 5, 3, 4,
0, 0, 0, 0, 3, 5, 2, 2, 4, 4, 2, 0, 4, 0, 1, 5, 0, 5, 1, 1, 0, 4,
0, 2, 0, 5, 4, 4, 3, 1, 4, 0, 1, 1, 1, 3, 1, 4, 1, 0, 0, 3, 1, 3,
3, 2, 1, 2, 2, 4, 0, 0, 5, 5, 0, 3, 0, 4, 2, 1, 5, 4, 0, 1, 2, 0,
1, 1, 5, 1, 4, 5, 5, 4, 4, 4, 3, 1, 0, 2, 4, 5, 1, 0, 4, 4, 4, 2,
2, 4, 0, 0, 1, 0, 0, 0, 4, 1, 4, 1, 2, 1, 5, 1, 3, 1, 4, 0, 0, 1,
1, 4, 0, 2, 4, 0, 5, 3, 5, 3, 3, 5, 0, 0, 1, 5, 3, 4, 5, 2, 2, 4,
0, 0, 0, 0, 2, 1, 0, 0, 5, 1, 0, 3, 2, 0, 5, 4, 3, 5, 2, 0, 4, 5,
3, 0, 4, 4, 4, 2, 1, 0, 0, 4, 5, 2, 0, 4, 2, 4, 1, 4, 0, 1, 2, 0,
0, 1, 5, 0, 5, 1, 5, 2, 4, 4, 0, 0, 5, 2, 2, 1, 3, 5, 3, 4, 0, 1,
4, 5, 4, 4, 0, 2, 4, 5, 2, 5, 1, 0, 0, 0, 0, 5, 5, 0, 3, 1, 5, 4,
1, 1, 0, 4, 3, 0, 3, 3, 3, 1, 4, 3, 4, 5, 5, 3, 4, 0, 4, 5, 1, 5,
0, 0, 4, 2, 1, 4, 2, 1, 0, 0, 4, 1, 3, 0, 4, 0, 0, 5, 5, 5, 0, 0,
2, 4, 0, 4, 0, 0, 2, 2, 4, 2, 4, 1, 0, 0, 3, 0, 3, 0, 0, 2, 4, 5,
2, 0, 4, 5, 3, 5, 4, 4, 1, 1, 5, 4, 4, 5, 2, 5, 1, 3, 4, 1, 0, 0,
1, 4, 0, 4, 3, 0, 5, 5, 0, 5, 4, 4, 1, 4, 0, 0, 5, 3, 4, 0, 5, 1,
3, 5, 0, 4, 1, 5, 3, 0, 1, 0, 5, 0, 0, 0, 0, 1, 0, 1, 3, 4, 3, 2,
4, 4, 4, 0, 4, 5, 1, 3, 0, 5, 0, 0, 0, 4, 1, 5, 5, 3, 2, 4, 4, 3,
0, 2, 0, 5, 2, 0, 1, 1, 2, 1, 0, 4, 5, 5, 4, 0, 1, 0, 0, 5, 0, 4,
0, 5, 2, 1, 2, 1, 3, 5, 4, 0, 0, 3, 2]),
'metrics': {'n_clusters': 6,
'n_noise': 0,
'silhouette': -0.076407,
'davies_bouldin': 2.461002,
'calinski_harabasz': 2.8645}}

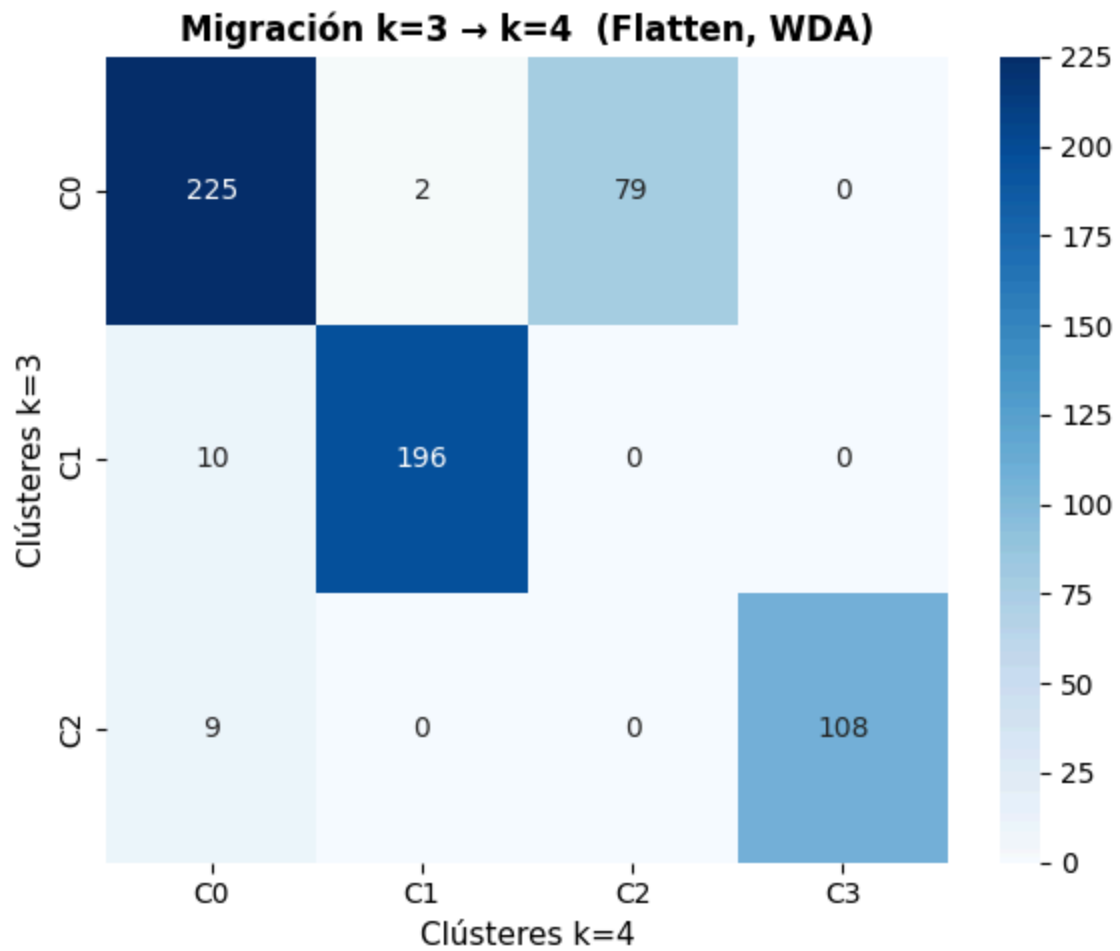
```

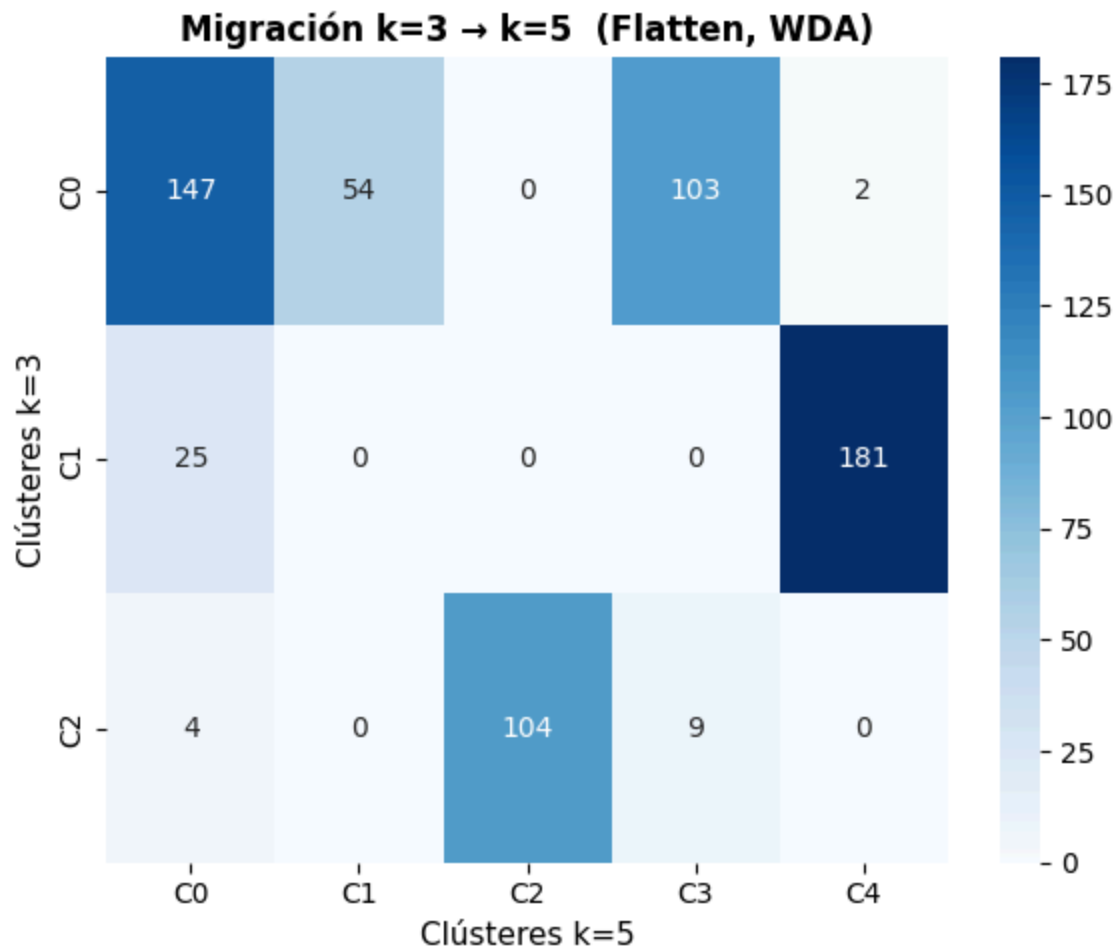
Análisis de migración

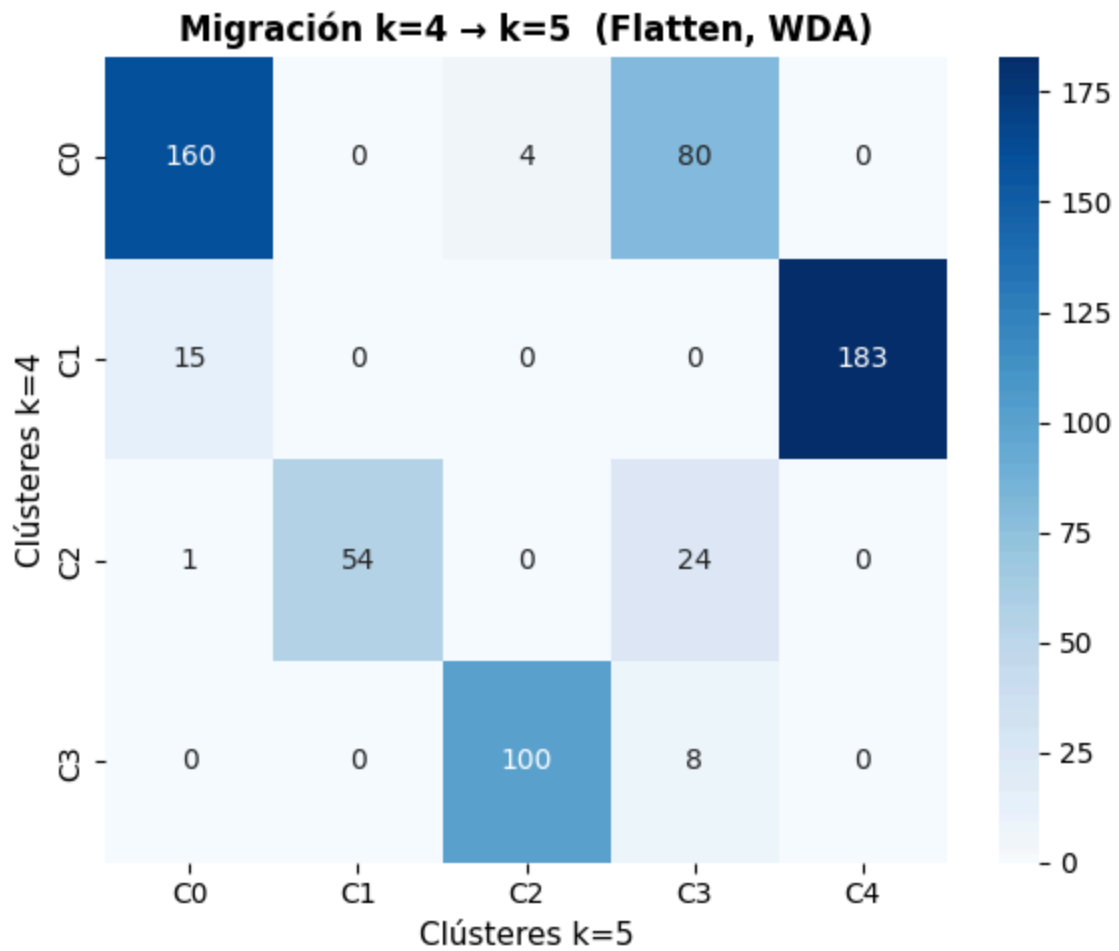
```

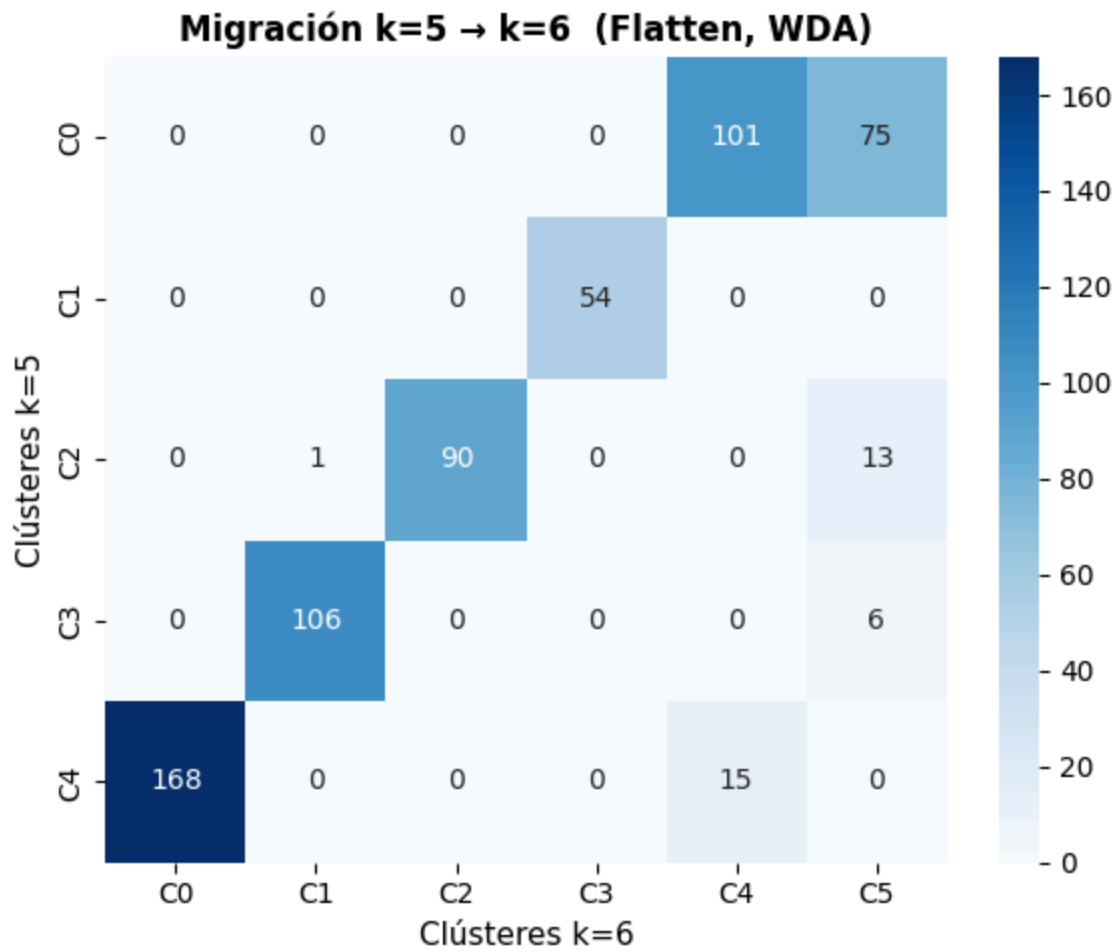
In [79]: plot_migration_matrix(labels_dense2_k3_wd, labels_dense2_k4_wd, ka=3, kb=4)
plot_migration_matrix(labels_dense2_k3_wd, labels_dense2_k5_wd, ka=3, kb=5)
plot_migration_matrix(labels_dense2_k4_wd, labels_dense2_k5_wd, ka=4, kb=5)
plot_migration_matrix(labels_dense2_k5_wd, labels_dense2_k6_wd, ka=5, kb=6)

```







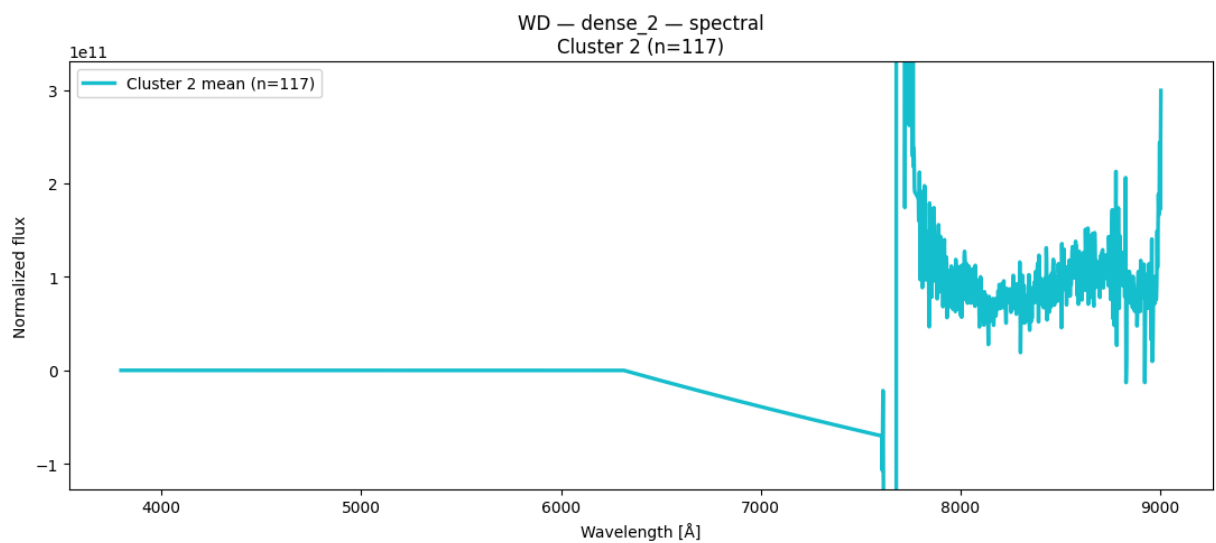
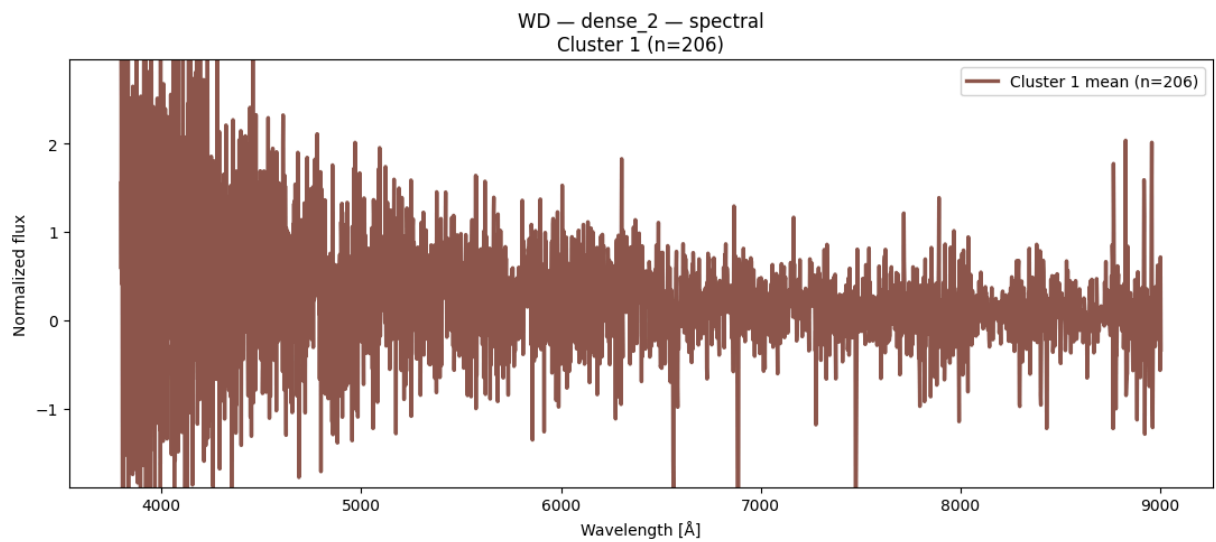
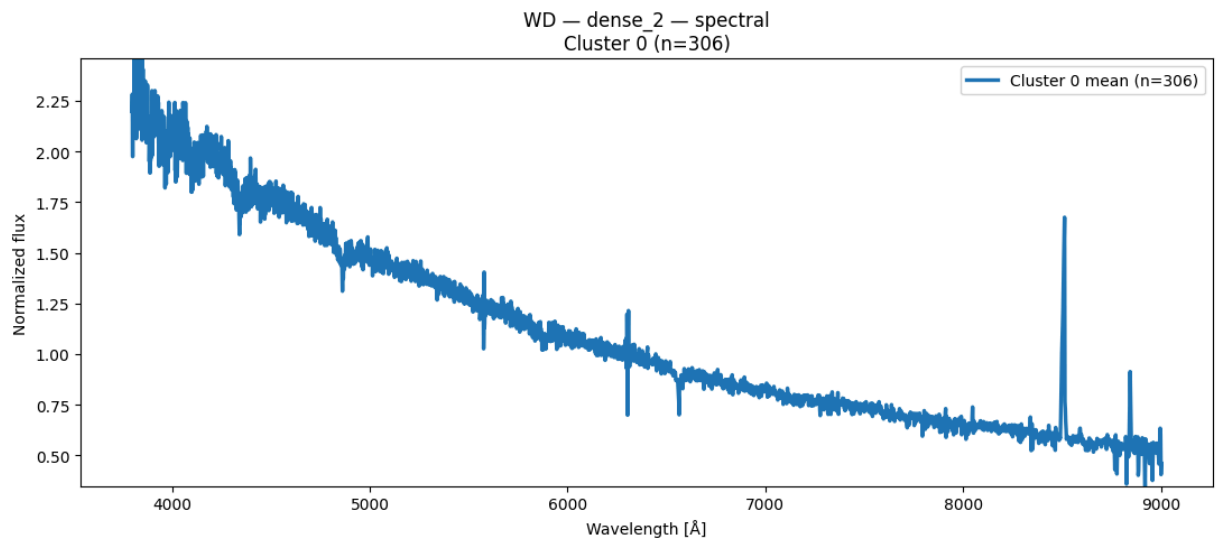


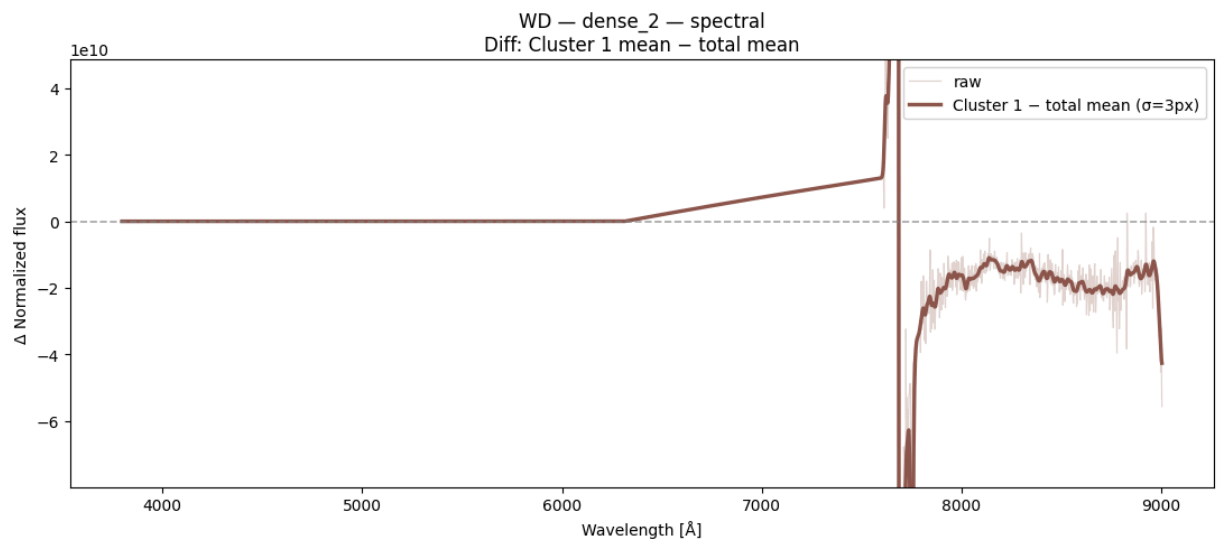
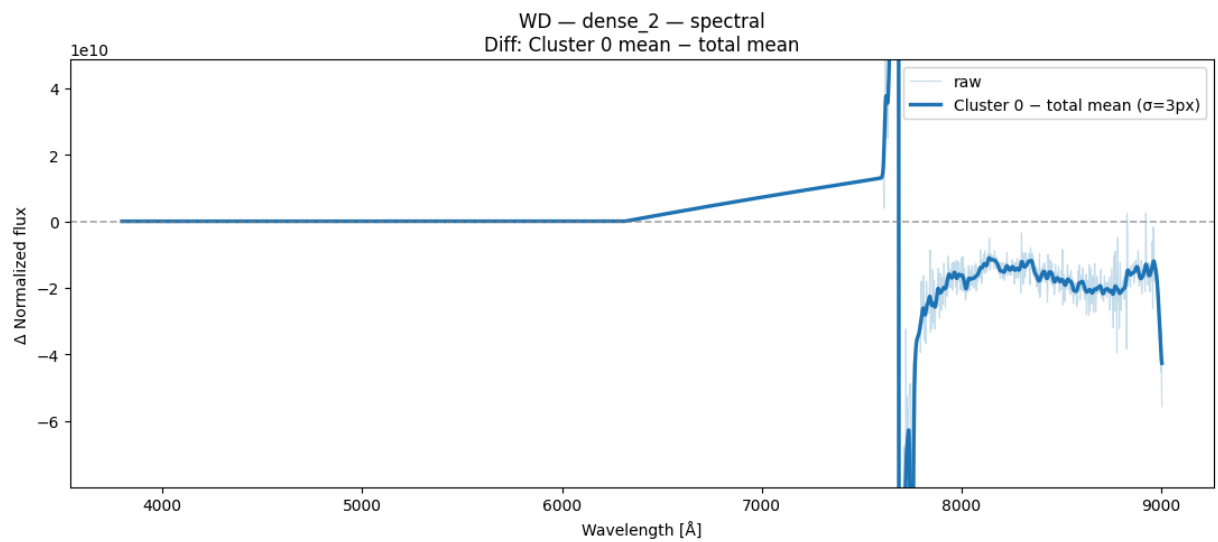
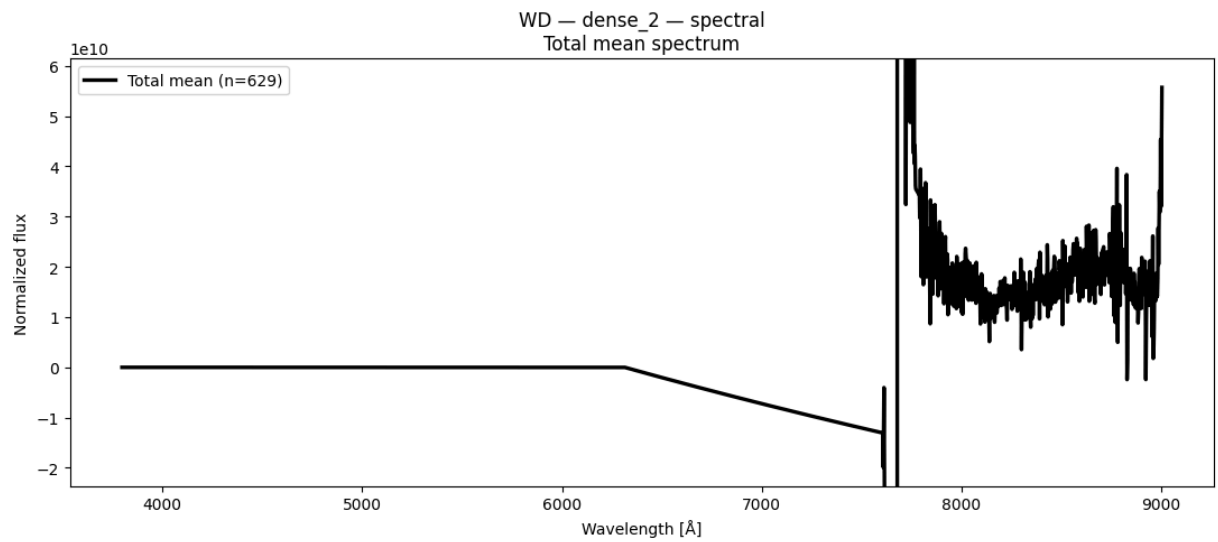
Espectros obtenidos para dense_2 con Spectral Clustering k = 3 y n = 700

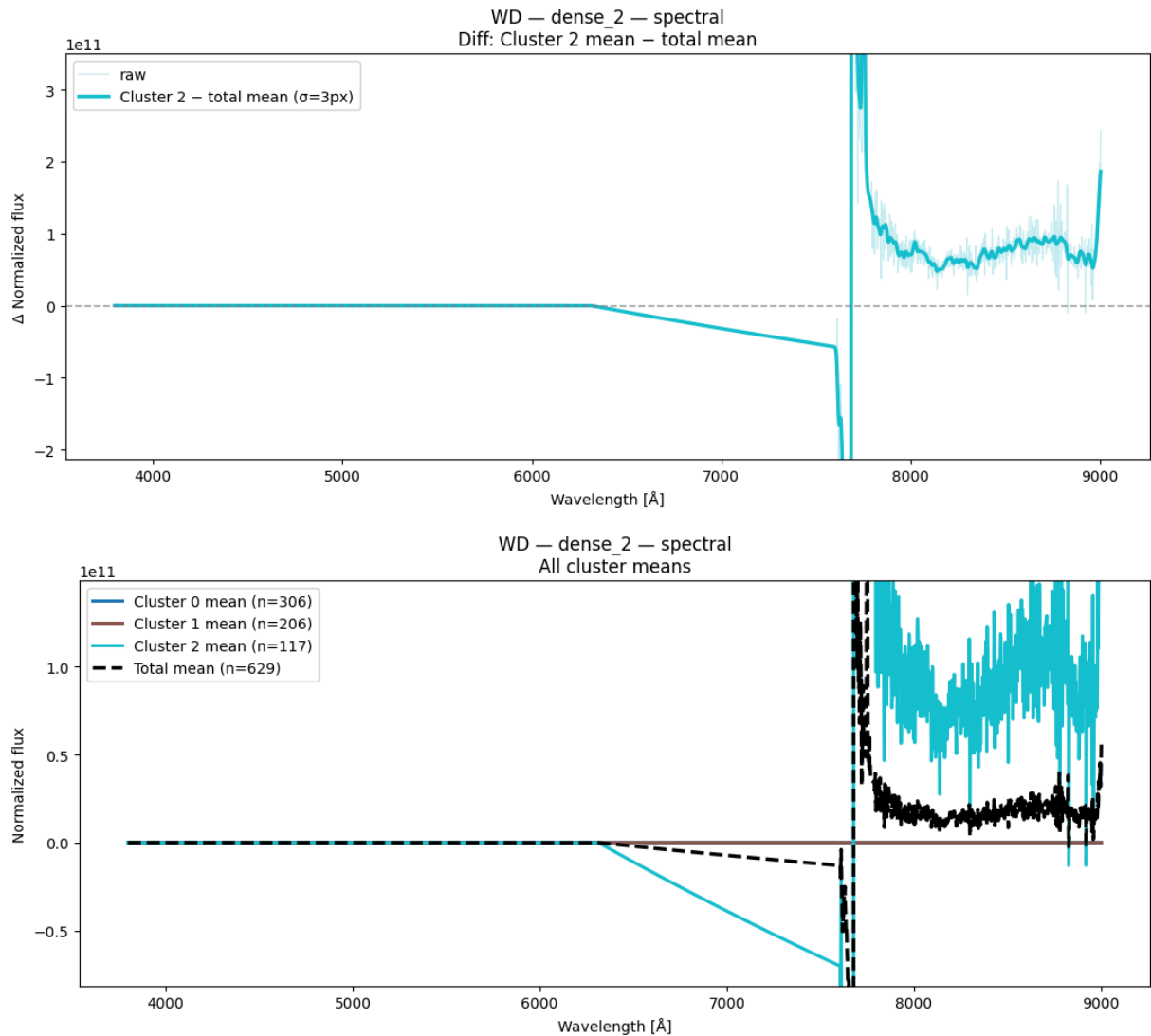
```
In [80]: plot_spectra_spectral(
    X_wd_spec,
    labels_dense2_k3_wd,
    W_wd[0],
    class_name="WD",
    layer_name="dense_2"
)
```

C:\Users\javip\AppData\Local\Temp\ipykernel_52368\531566132.py:28: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get\_cmap()`` or ``pyplot.get\_cmap()`` instead.

```
cmap = plt.cm.get_cmap("tab10", n_clusters)
```





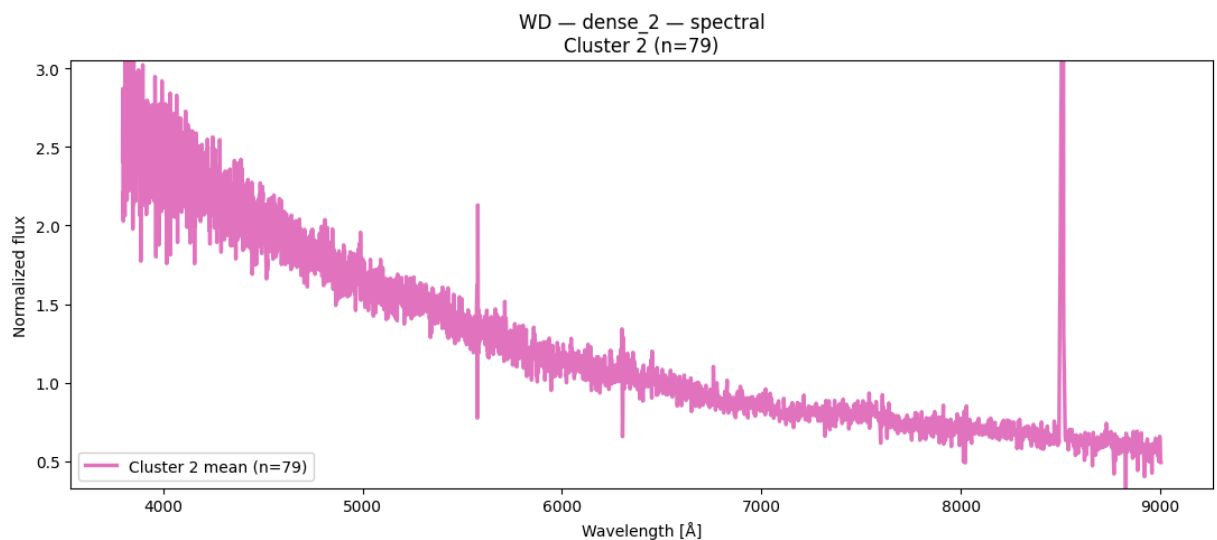
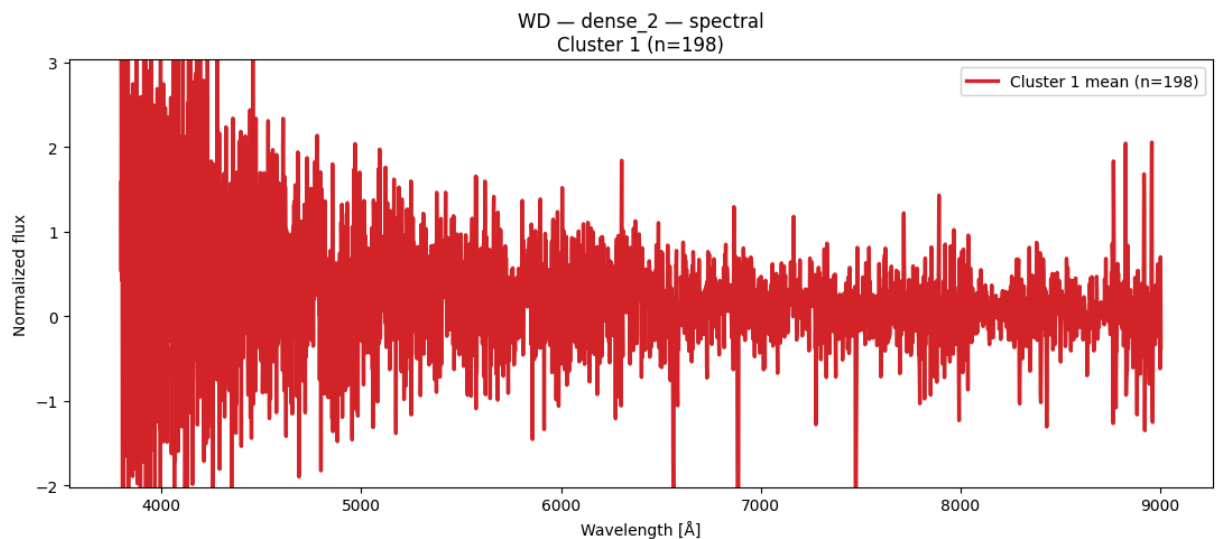
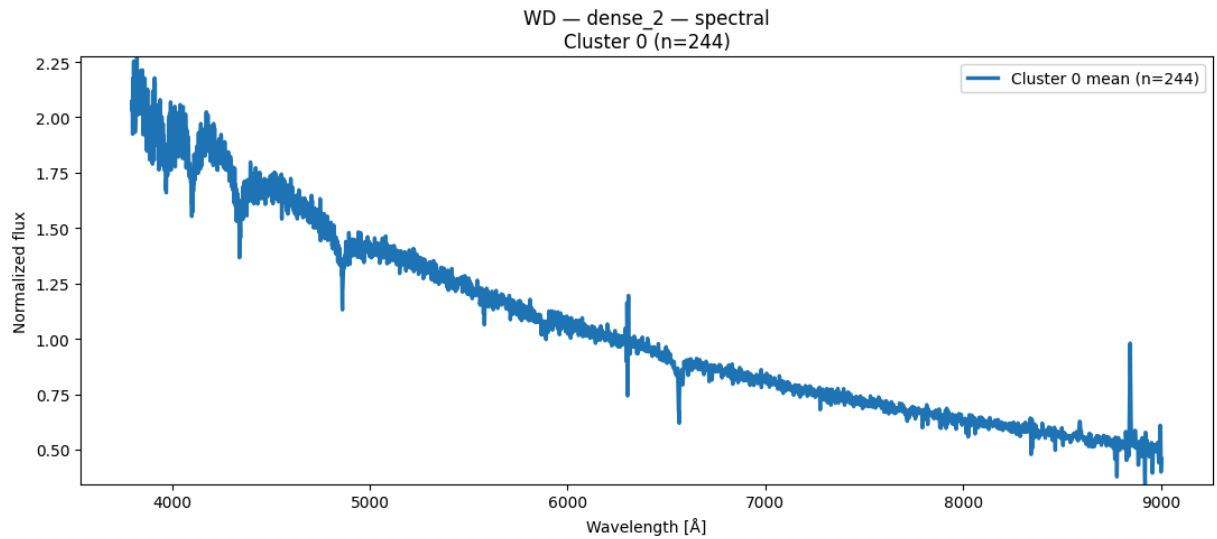


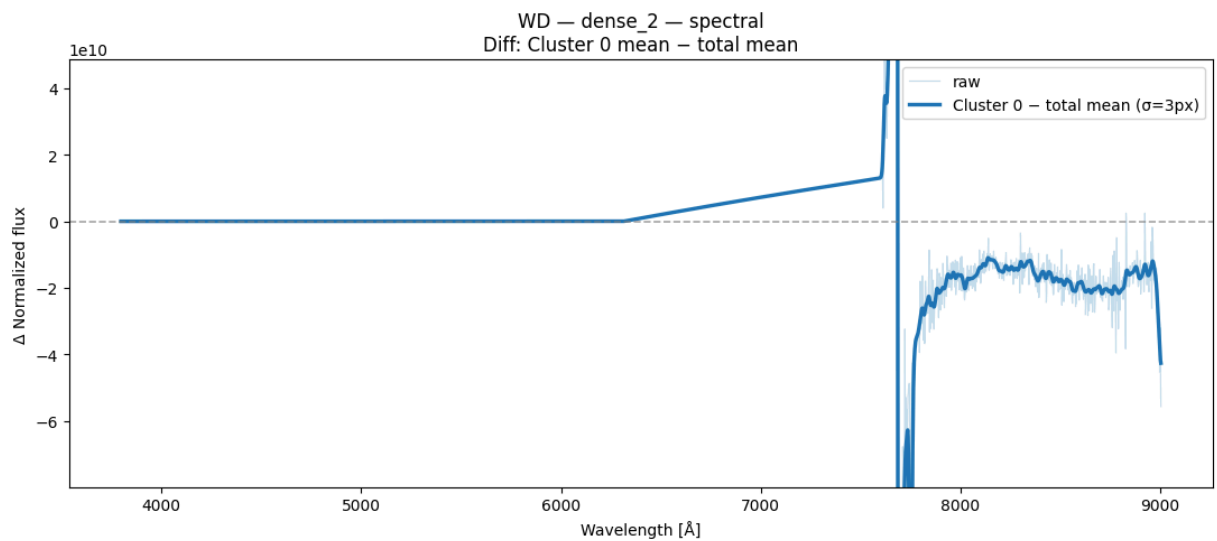
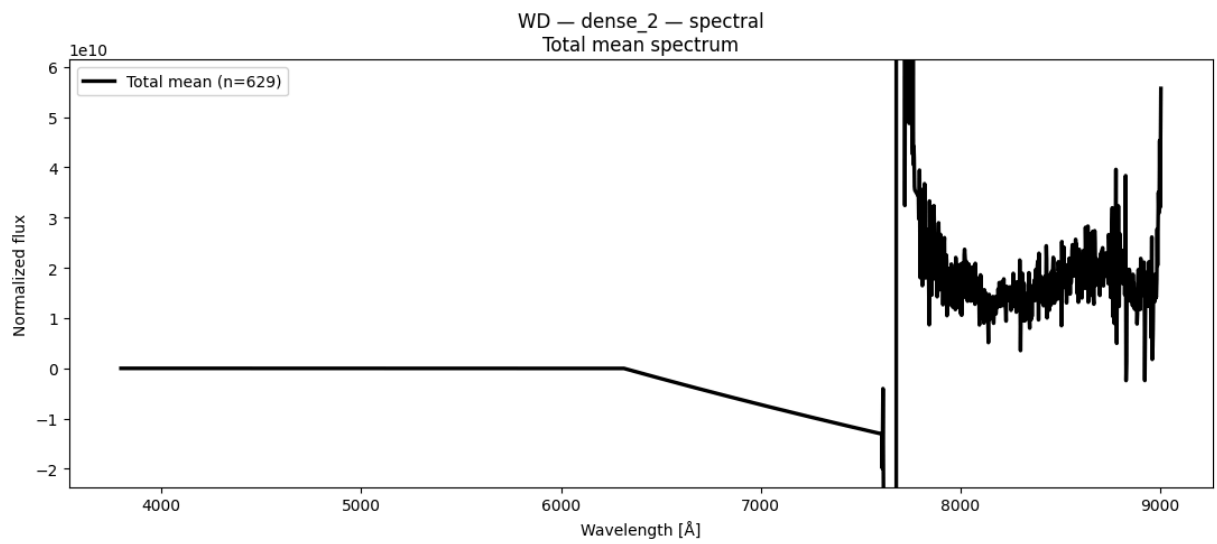
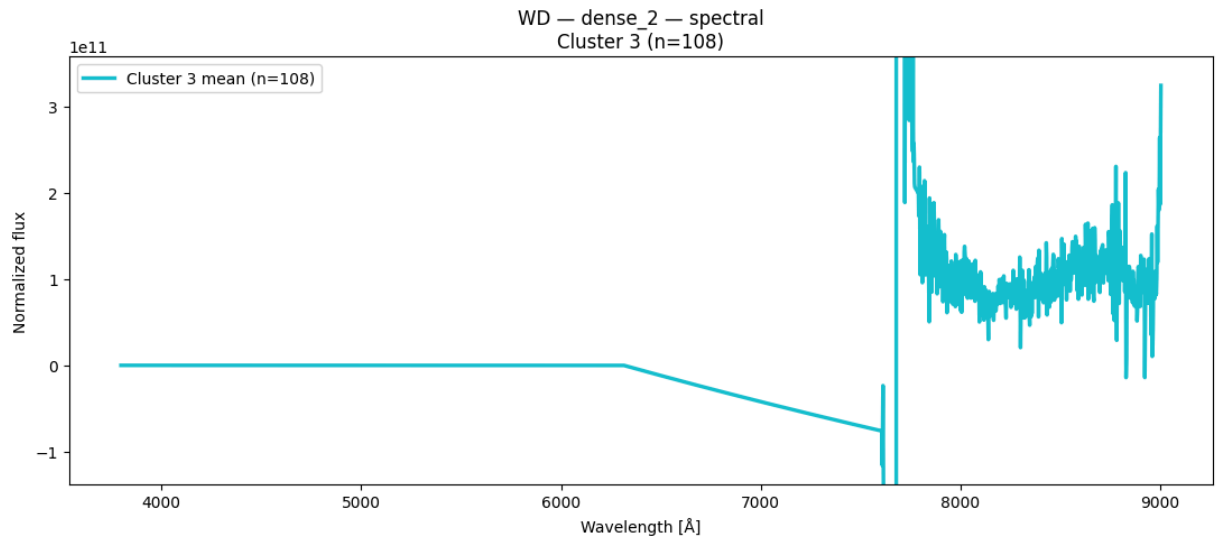
```
Out[80]: {'total_mean': array([1.8663167e+00, 1.4631674e+00, 2.3618588e+00, ..., 3.2000662e
+10,
        4.3846500e+10, 5.5692263e+10], dtype=float32),
        'group_means': {0: array([2.1980844 , 2.2563398 , 2.282918 , ..., 0.5986235 , 0.
4045057 ,
        0.45944786], dtype=float32),
        1: array([ 1.5506324 ,  0.58019656,  3.1469386 , ..., -0.5682053 ,
        0.7131882 , -0.3357689 ], dtype=float32),
        2: array([1.5544395e+00, 9.4334781e-01, 1.1860451e+00, ..., 1.7203775e+11,
        2.3572179e+11, 2.9940541e+11], dtype=float32)},
        'unique_clusters': array([0, 1, 2])}
```

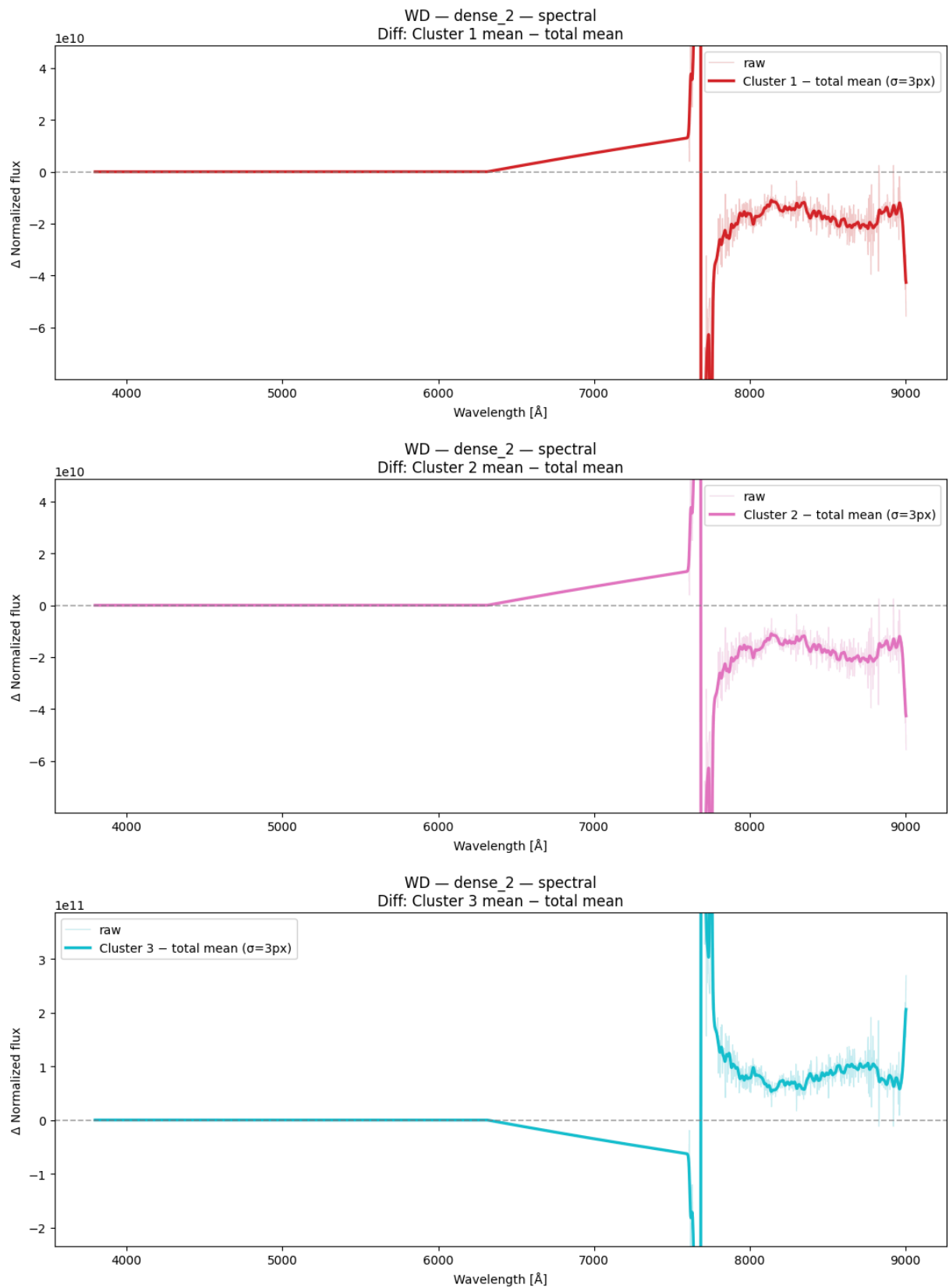
Espectros obtenidos para dense_2 con Spectral Clustering k = 4 y n = 700

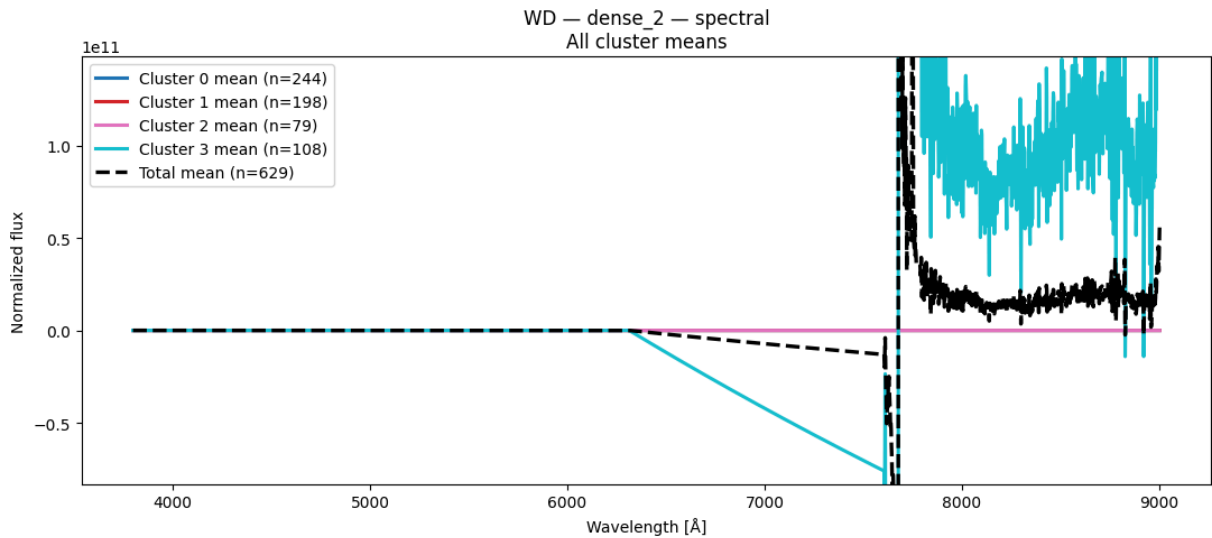
```
In [81]: plot_spectra_spectral(
        X_wd_spec,
        labels_dense2_k4_wd,
        W_wd[0],
        class_name="WD",
        layer_name="dense_2"
    )
```

```
C:\Users\javip\AppData\Local\Temp\ipykernel_52368\531566132.py:28: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get_cmap()`` or ``pyplot.get_cmap()`` instead.
cmap      = plt.cm.get_cmap("tab10", n_clusters)
```









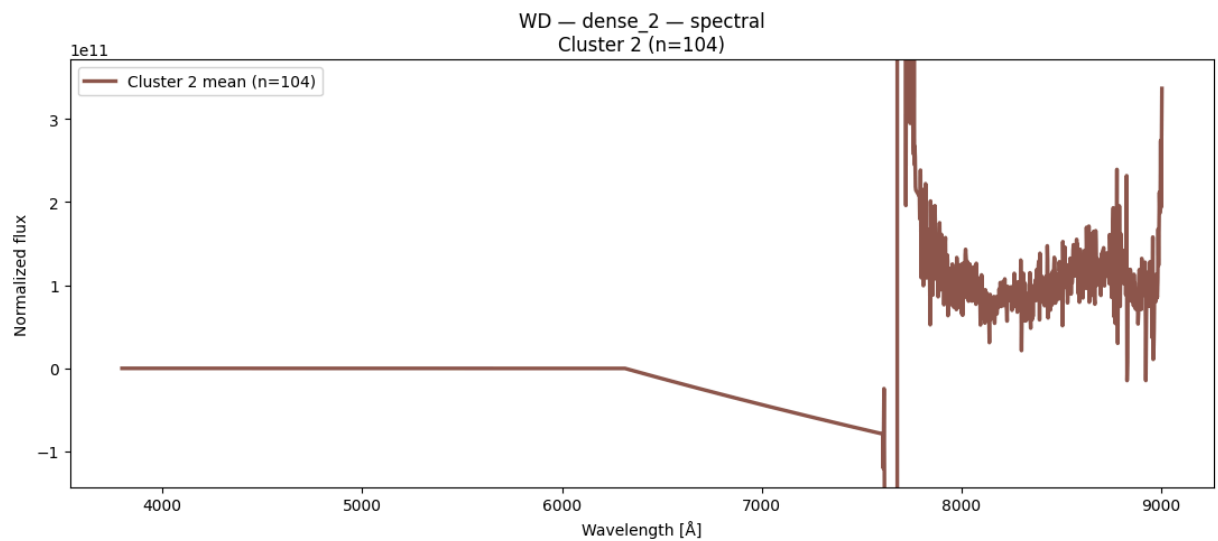
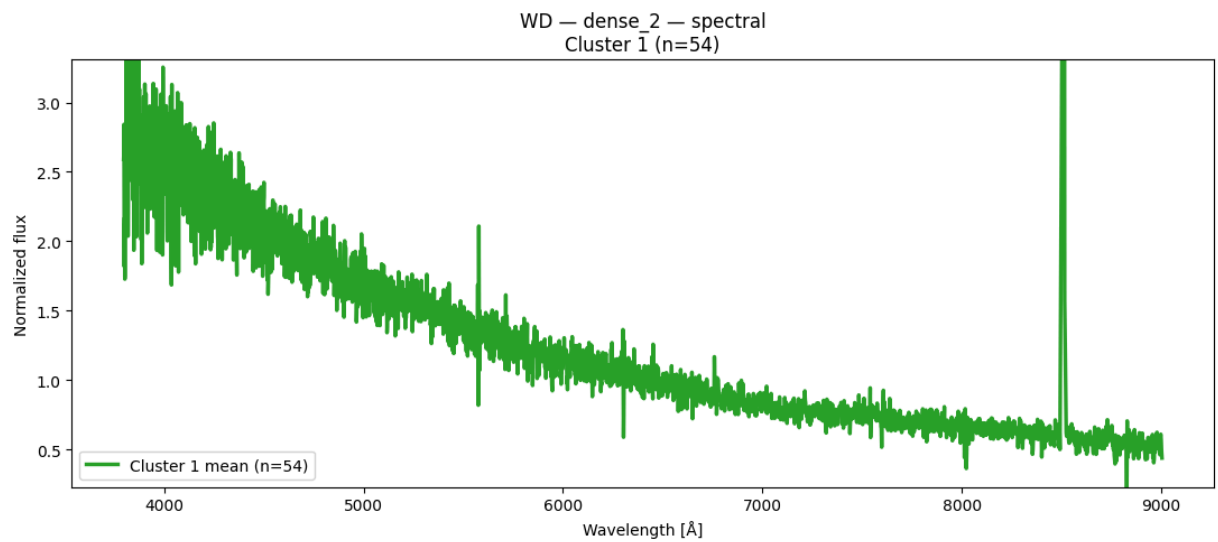
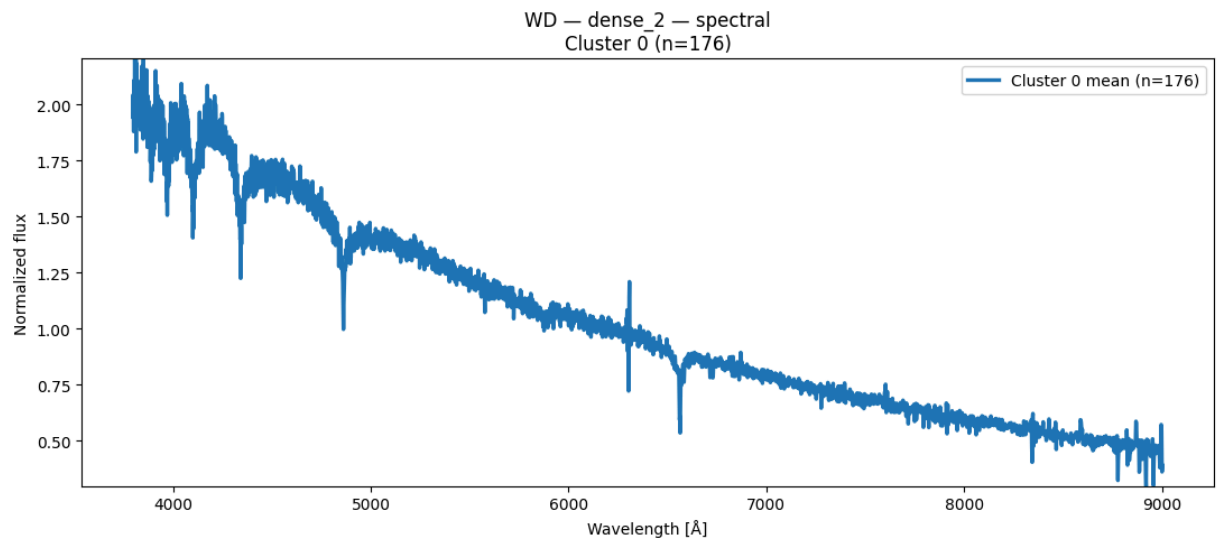
```
Out[81]: {'total_mean': array([1.8663167e+00, 1.4631674e+00, 2.3618588e+00, ..., 3.2000662e
+10,
      4.3846500e+10, 5.5692263e+10], dtype=float32),
  'group_means': {0: array([2.0684507 , 2.0284925 , 2.0636415 , ..., 0.5867345 , 0.
39876187,
      0.45834965], dtype=float32),
    1: array([ 1.5793062 ,  0.52783245,  3.2117832 , ..., -0.6184997 ,
      0.69940394, -0.37148324], dtype=float32),
    2: array([2.4092348 , 2.8705668 , 2.7922056 , ..., 0.63705254, 0.5100119 ,
      0.49269283], dtype=float32),
    3: array([1.5386974e+00, 8.7124205e-01, 1.1626288e+00, ..., 1.8637423e+11,
      2.5536527e+11, 3.2435588e+11], dtype=float32)},
  'unique_clusters': array([0, 1, 2, 3])}
```

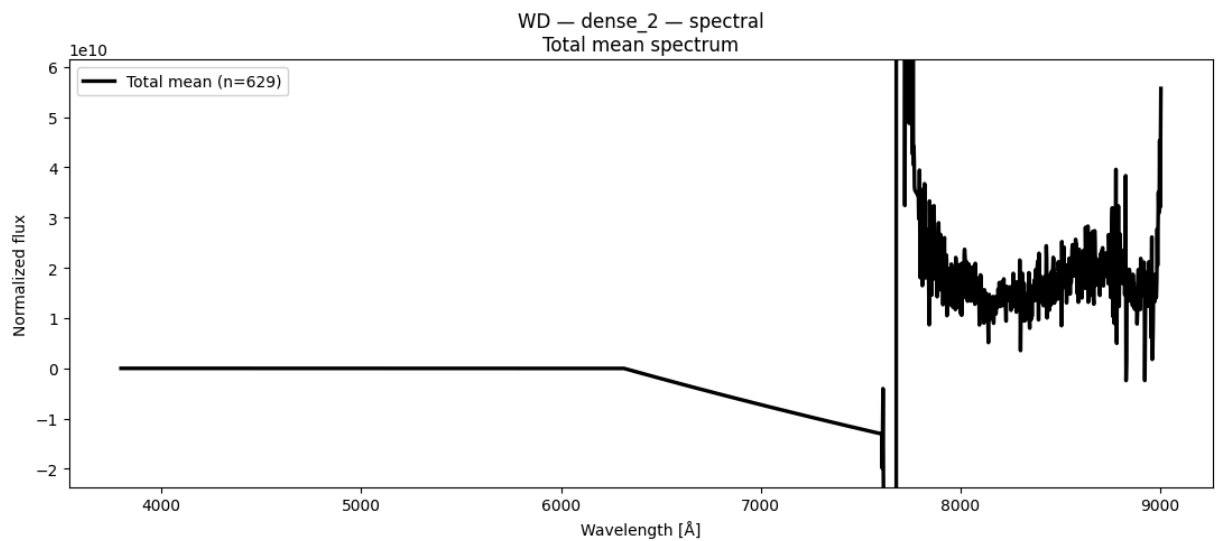
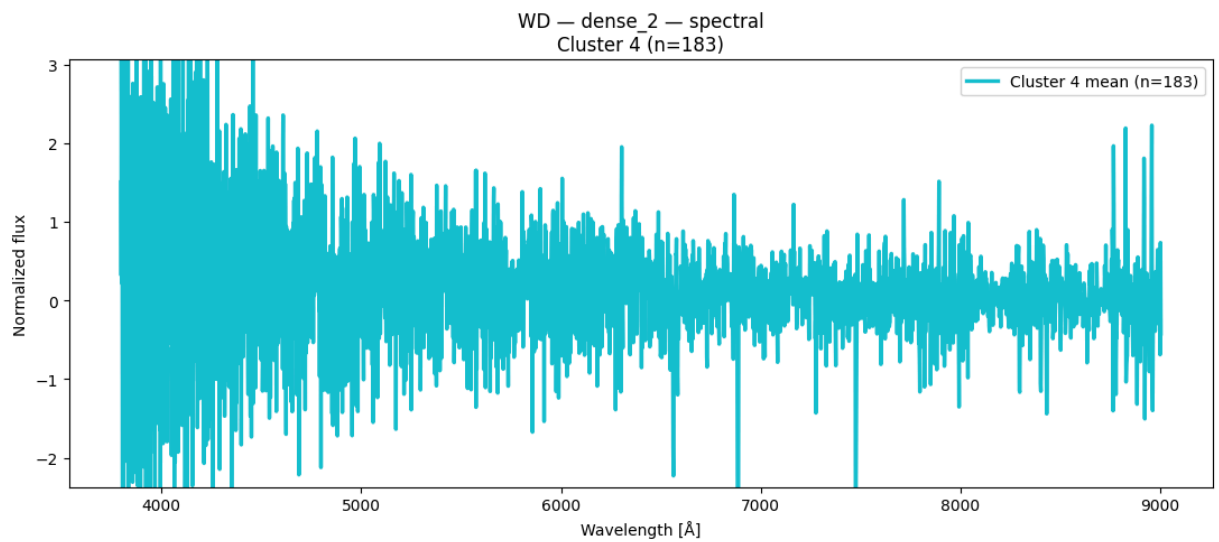
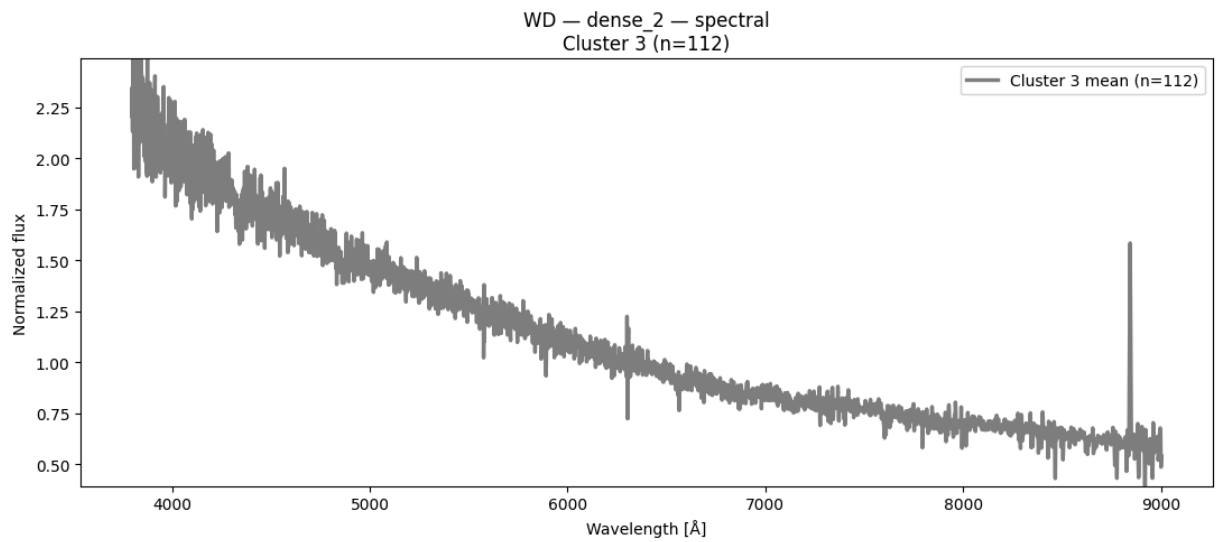
Espectros obtenidos para dense_2 con Spectral Clustering k = 5 y n = 700

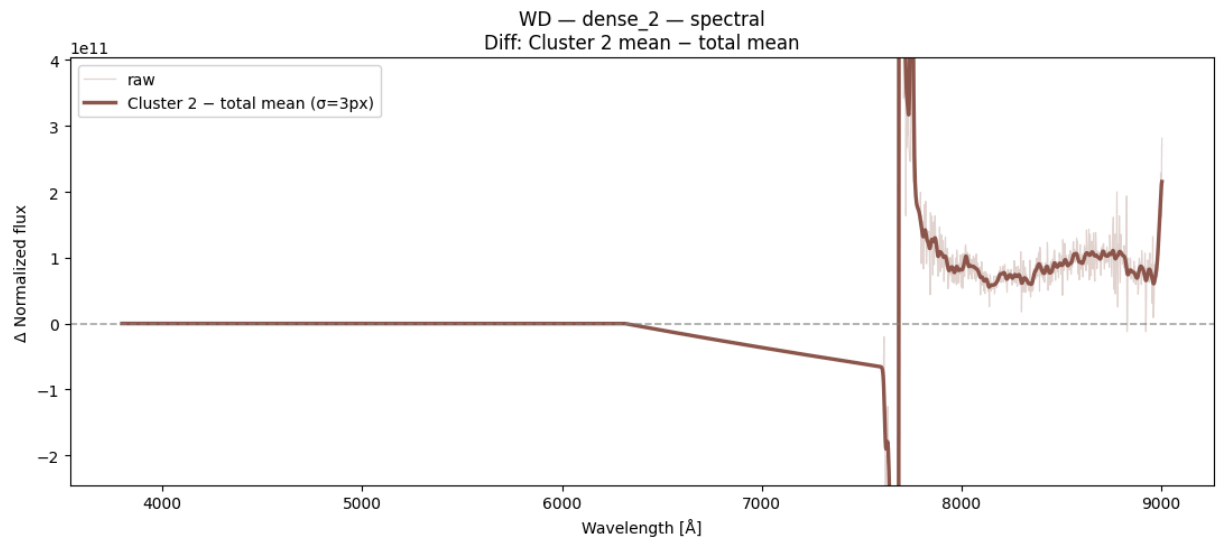
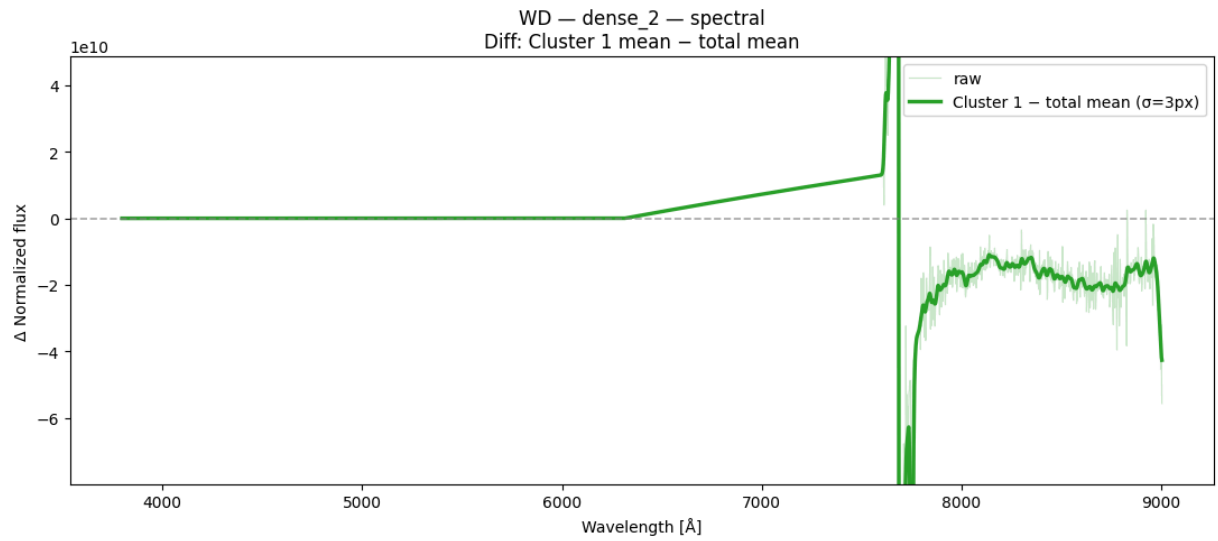
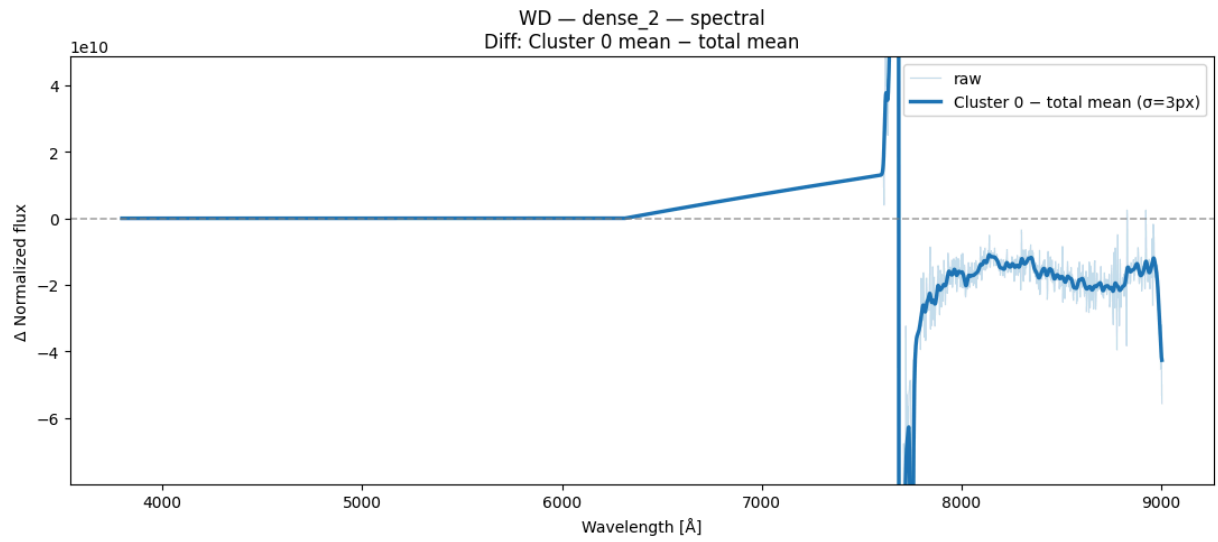
```
In [82]: plot_spectra_spectral(
    X_wd_spec,
    labels_dense2_k5_wd,
    W_wd[0],
    class_name="WD",
    layer_name="dense_2"
)
```

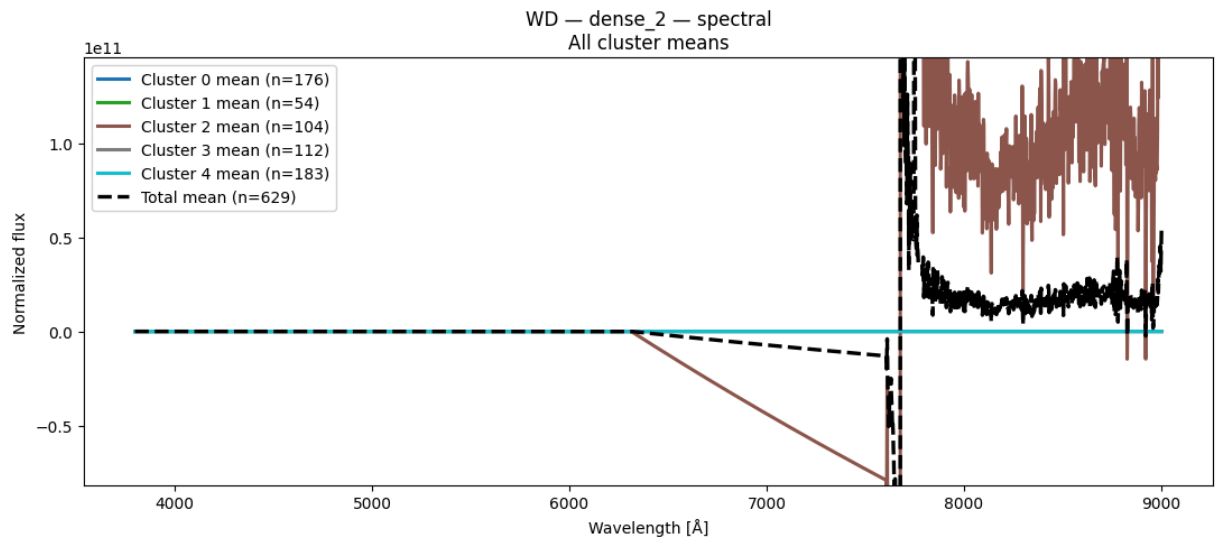
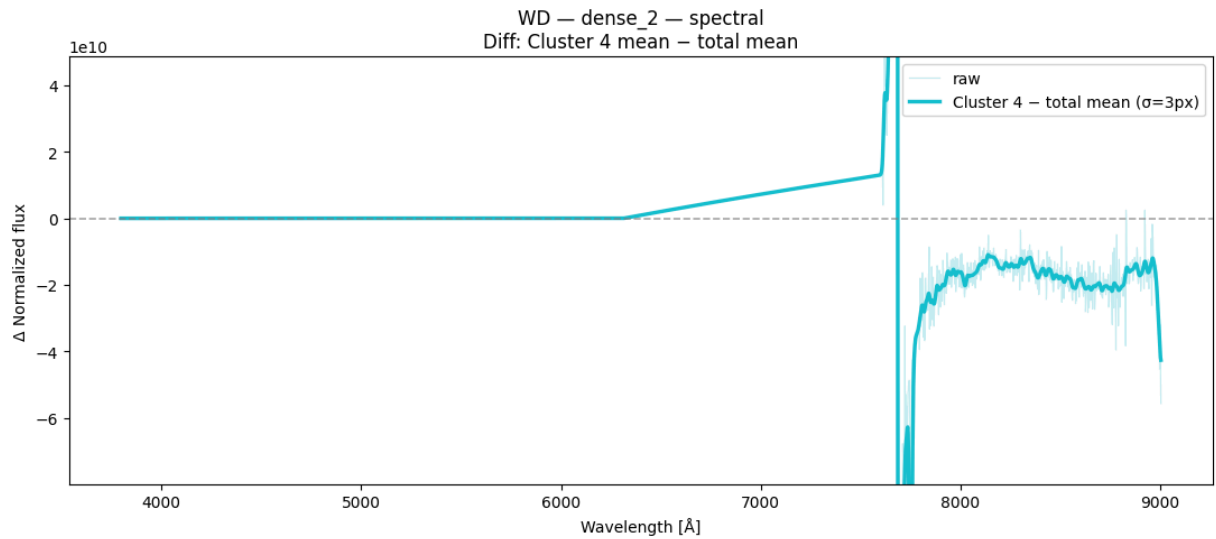
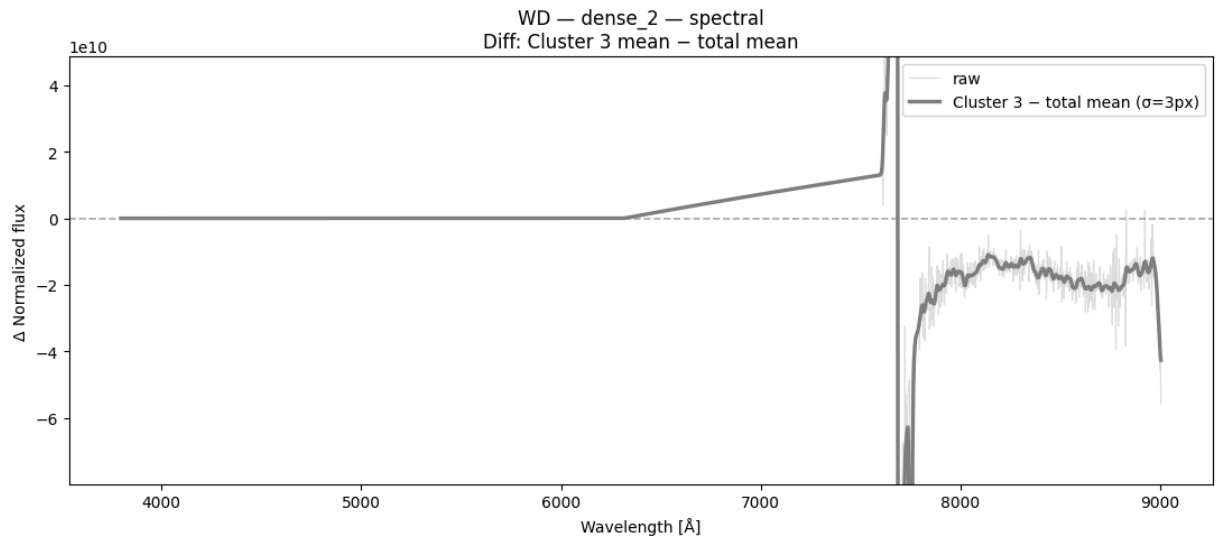
C:\Users\javip\AppData\Local\Temp\ipykernel_52368\531566132.py:28: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get\_cmap()`` or ``pyplot.get\_cmap()`` instead.

```
cmap = plt.cm.get_cmap("tab10", n_clusters)
```









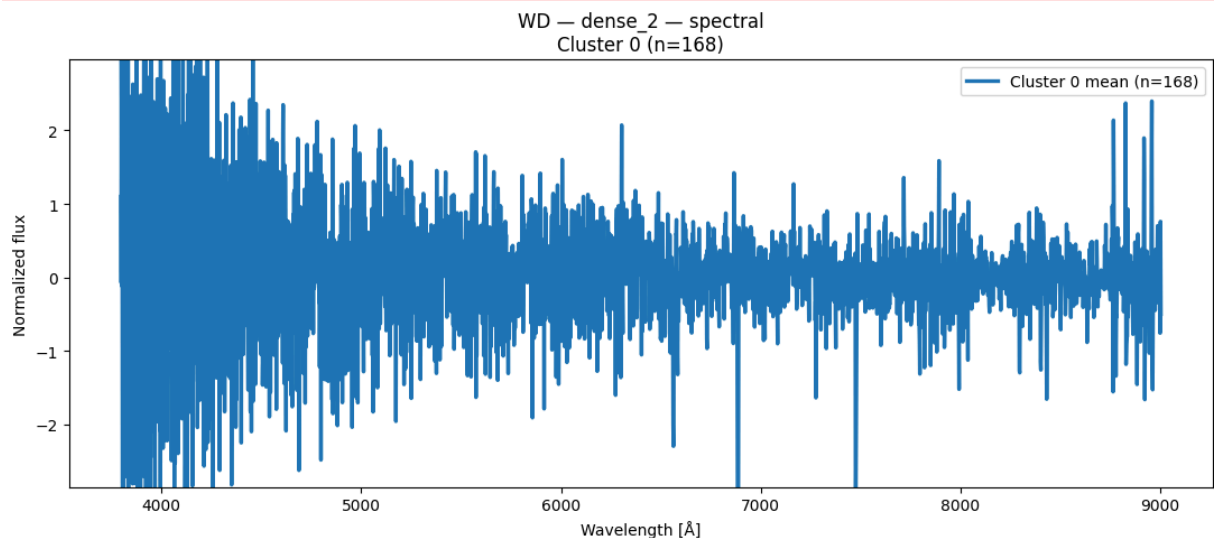
```
Out[82]: {'total_mean': array([1.8663167e+00, 1.4631674e+00, 2.3618588e+00, ..., 3.2000662e
+10,
        4.3846500e+10, 5.5692263e+10], dtype=float32),
        'group_means': {0: array([1.9433925 , 2.0424745 , 1.9844741 , ..., 0.5384327 , 0.
36029717,
        0.3891551 ], dtype=float32),
        1: array([2.5901697 , 2.6194887 , 2.842158 , ..., 0.57188344, 0.48596615,
        0.43764782], dtype=float32),
        2: array([1.6316242e+00, 9.4384980e-01, 1.2436826e+00, ..., 1.9354247e+11,
        2.6518700e+11, 3.3683112e+11], dtype=float32),
        3: array([2.2055428 , 2.3477066 , 2.3422046 , ..., 0.64878243, 0.48572806,
        0.53914434], dtype=float32),
        4: array([ 1.5043591 ,  0.31858248,  3.2305777 , ..., -0.68915164,
        0.7344912 , -0.42178848], dtype=float32)},
        'unique_clusters': array([0, 1, 2, 3, 4])}
```

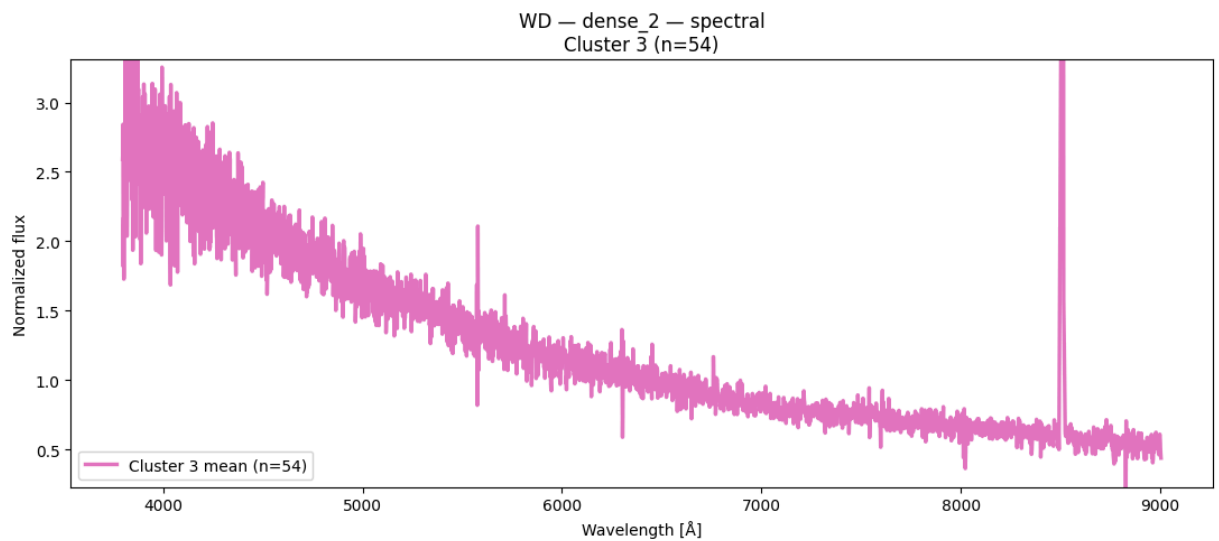
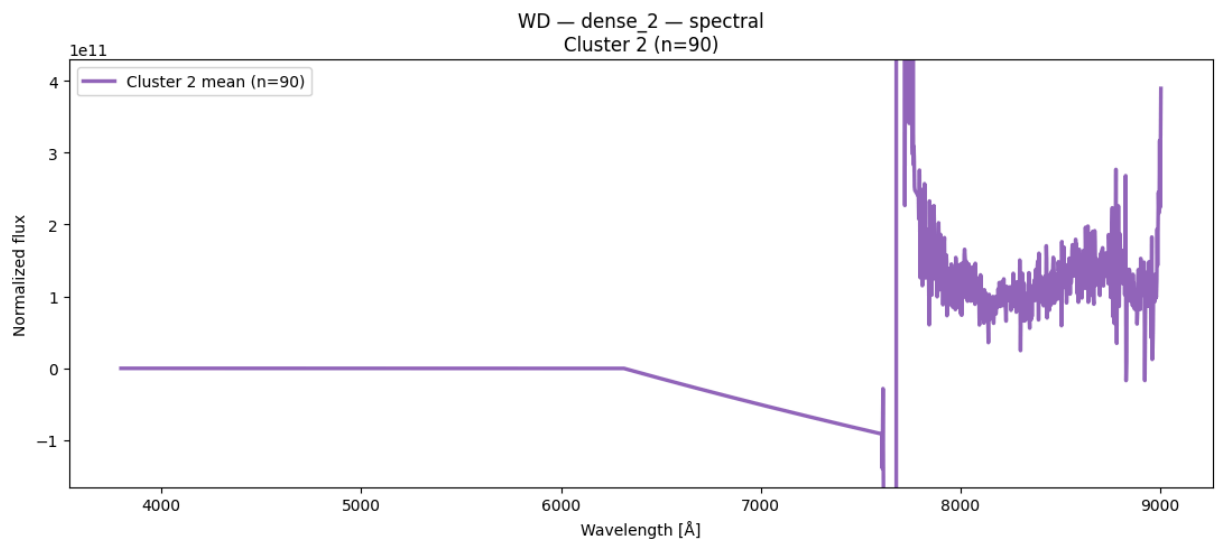
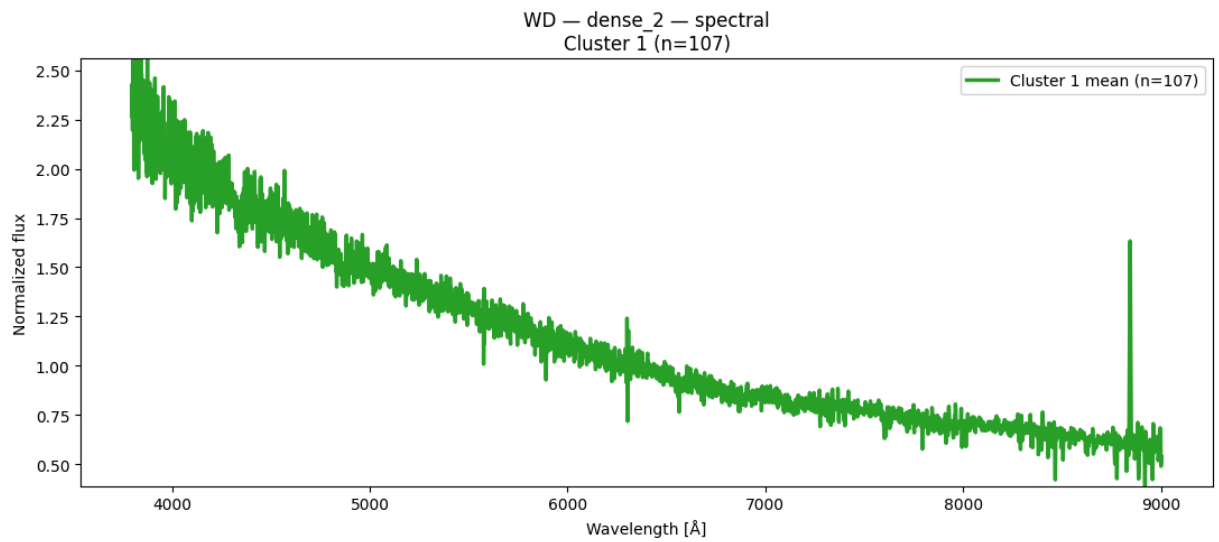
Espectros obtenidos para dense_2 con Spectral Clustering k = 6 y n = 700

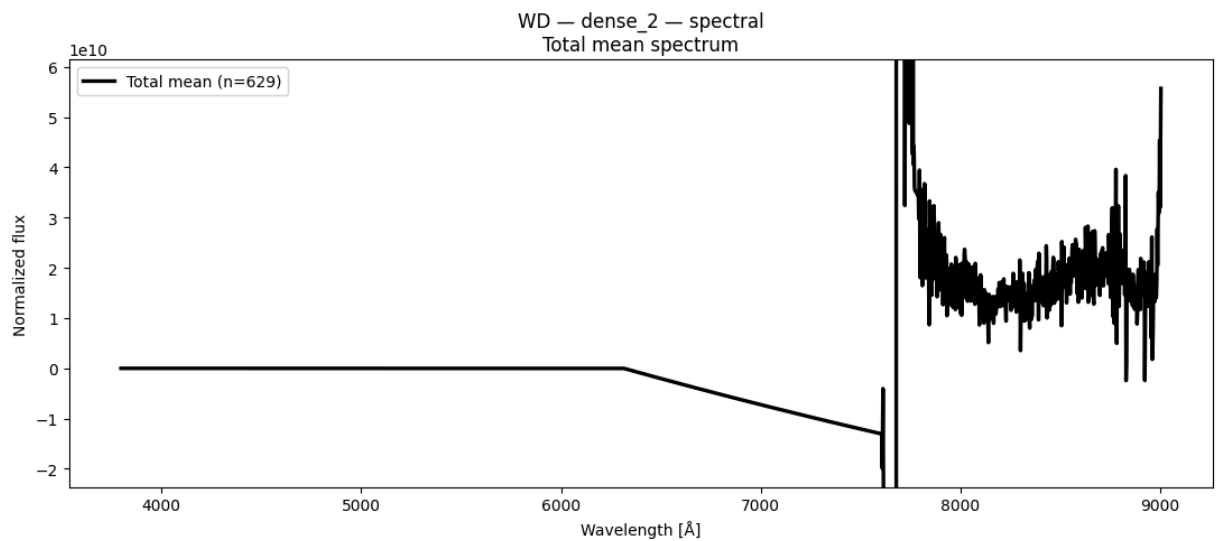
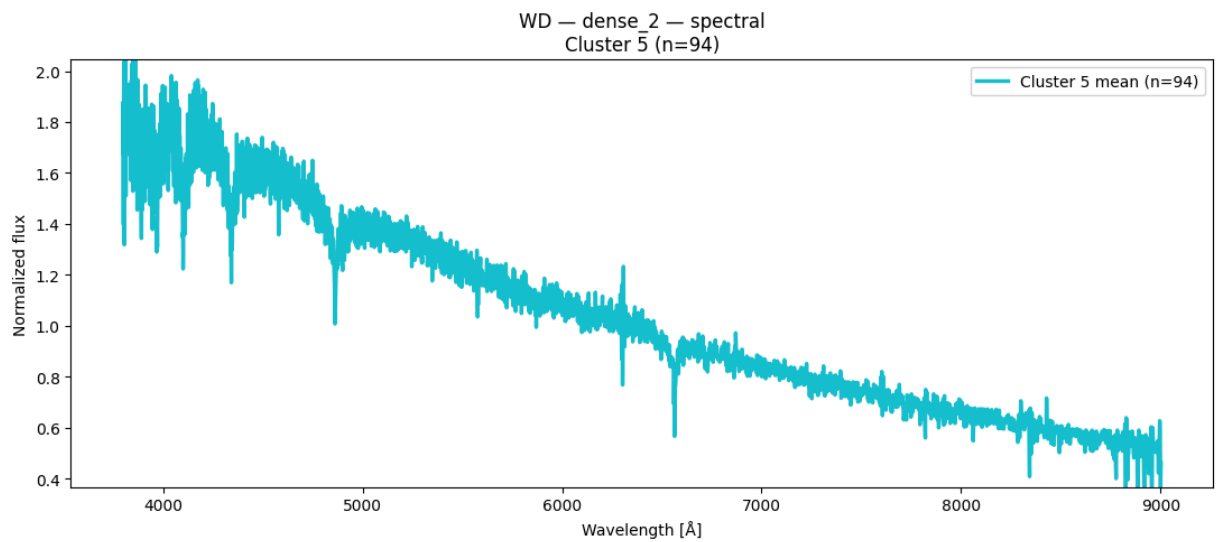
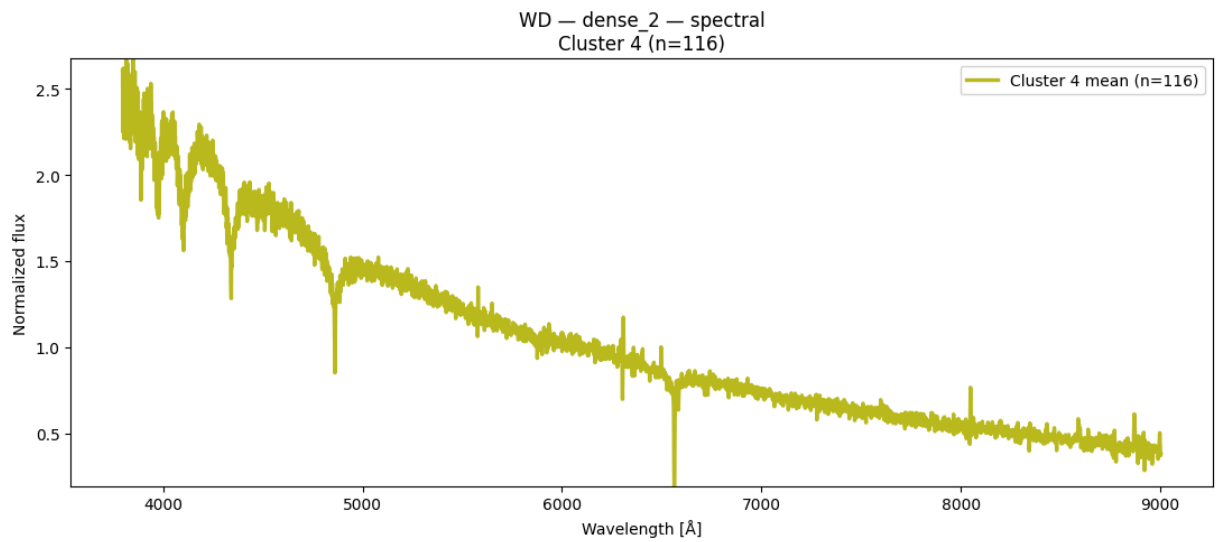
```
In [83]: plot_spectra_spectral(
        X_wd_spec,
        labels_dense2_k6_wd,
        W_wd[0],
        class_name="WD",
        layer_name="dense_2"
    )
```

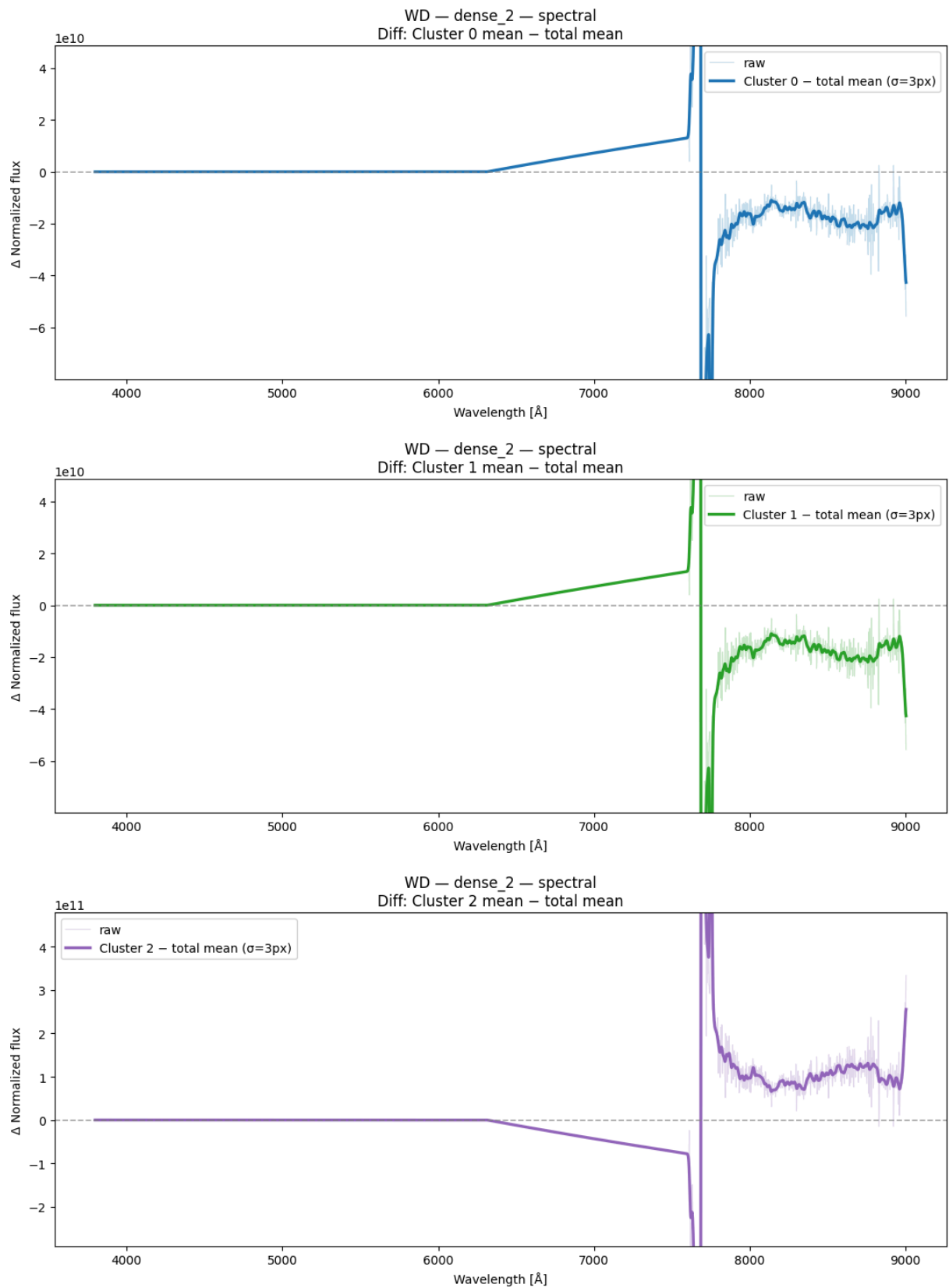
C:\Users\javip\AppData\Local\Temp\ipykernel_52368\531566132.py:28: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get\_cmap()`` or ``pyplot.get\_cmap()`` instead.

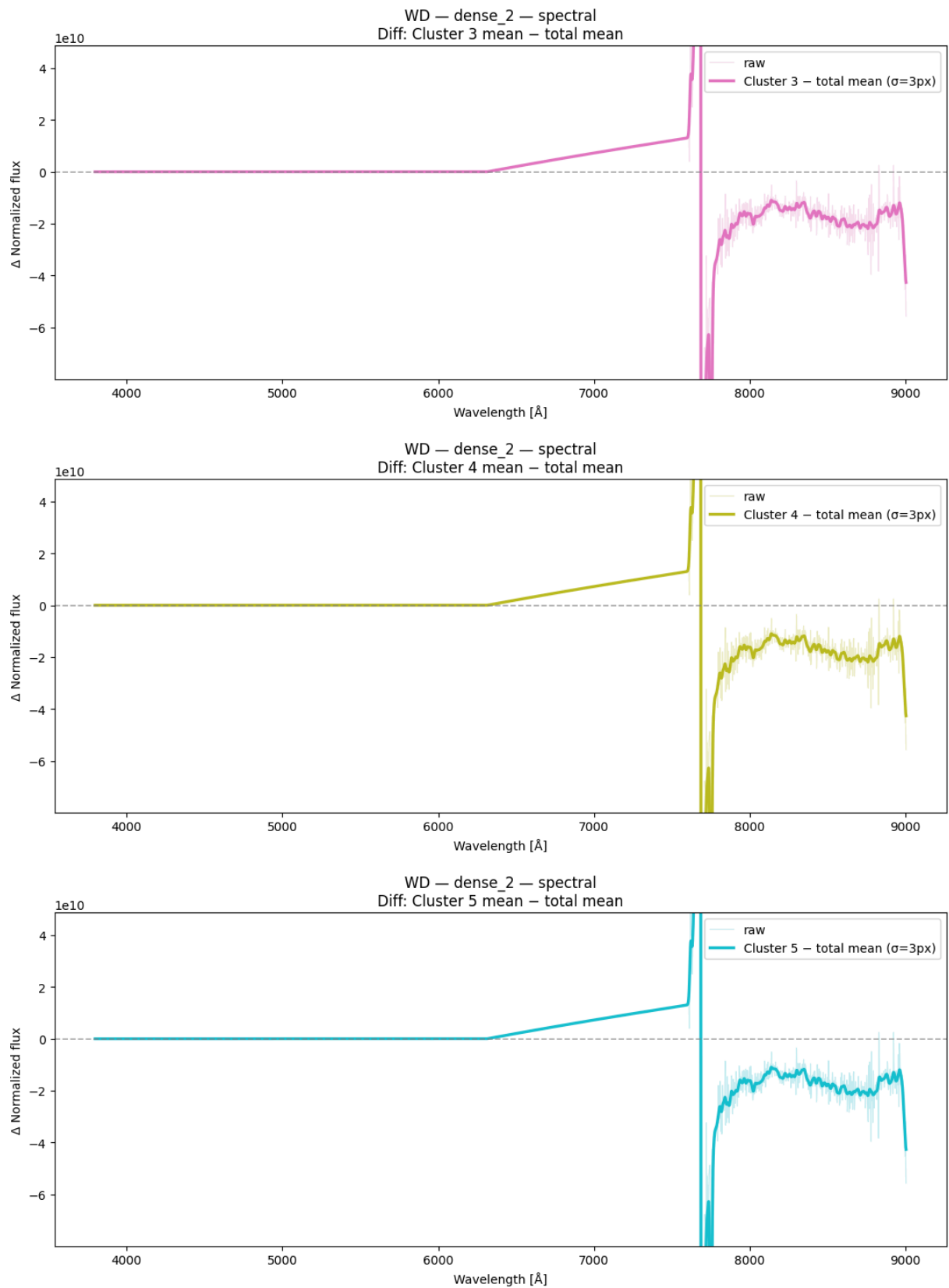
```
cmap = plt.cm.get_cmap("tab10", n_clusters)
```

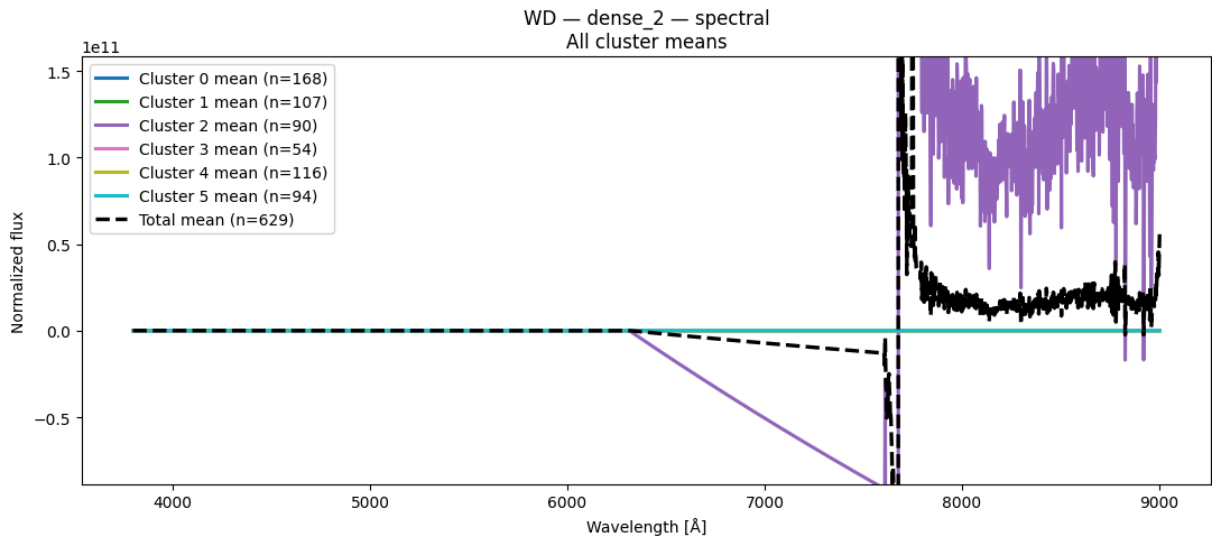










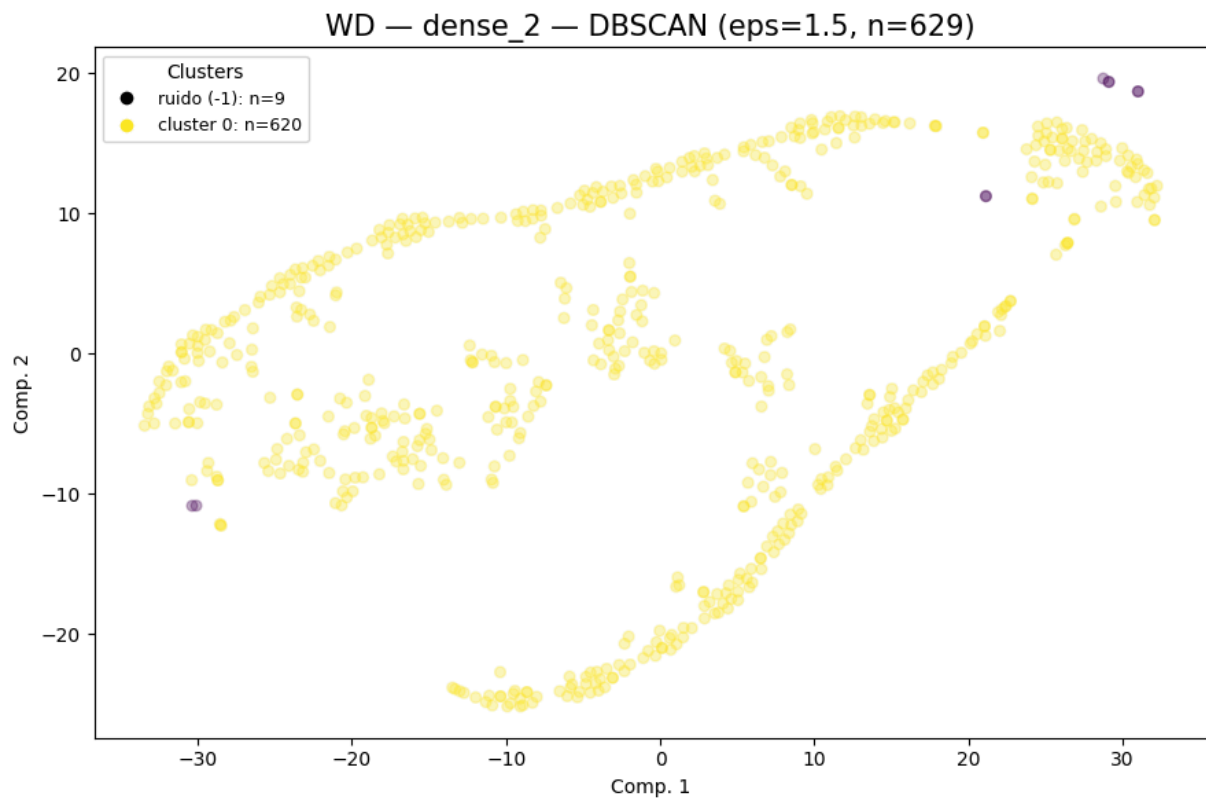


```
Out[83]: {'total_mean': array([1.8663167e+00, 1.4631674e+00, 2.3618588e+00, ..., 3.2000662e
+10,
      4.3846500e+10, 5.5692263e+10], dtype=float32),
'group_means': {0: array([ 1.099338 , -0.07267553,  3.1754603 , ..., -0.7593704
,
      0.75973034, -0.500535  ], dtype=float32),
1: array([2.2677174 , 2.4317443 , 2.4024372 , ..., 0.65337765, 0.4887532 ,
      0.5361671 ], dtype=float32),
2: array([1.6272856e+00, 8.2969749e-01, 1.2017322e+00, ..., 2.2364907e+11,
      3.0643831e+11, 3.8922705e+11], dtype=float32),
3: array([2.5901697 , 2.6194887 , 2.842158 , ..., 0.57188344, 0.48596615,
      0.43764782], dtype=float32),
4: array([2.6096642 , 2.4118953 , 2.249153 , ..., 0.42663196, 0.3693656 ,
      0.37887698], dtype=float32),
5: array([1.6758854 , 1.8770192 , 1.8355026 , ..., 0.5756999 , 0.36532292,
      0.461931  ], dtype=float32)},
'unique_clusters': array([0, 1, 2, 3, 4, 5])}
```

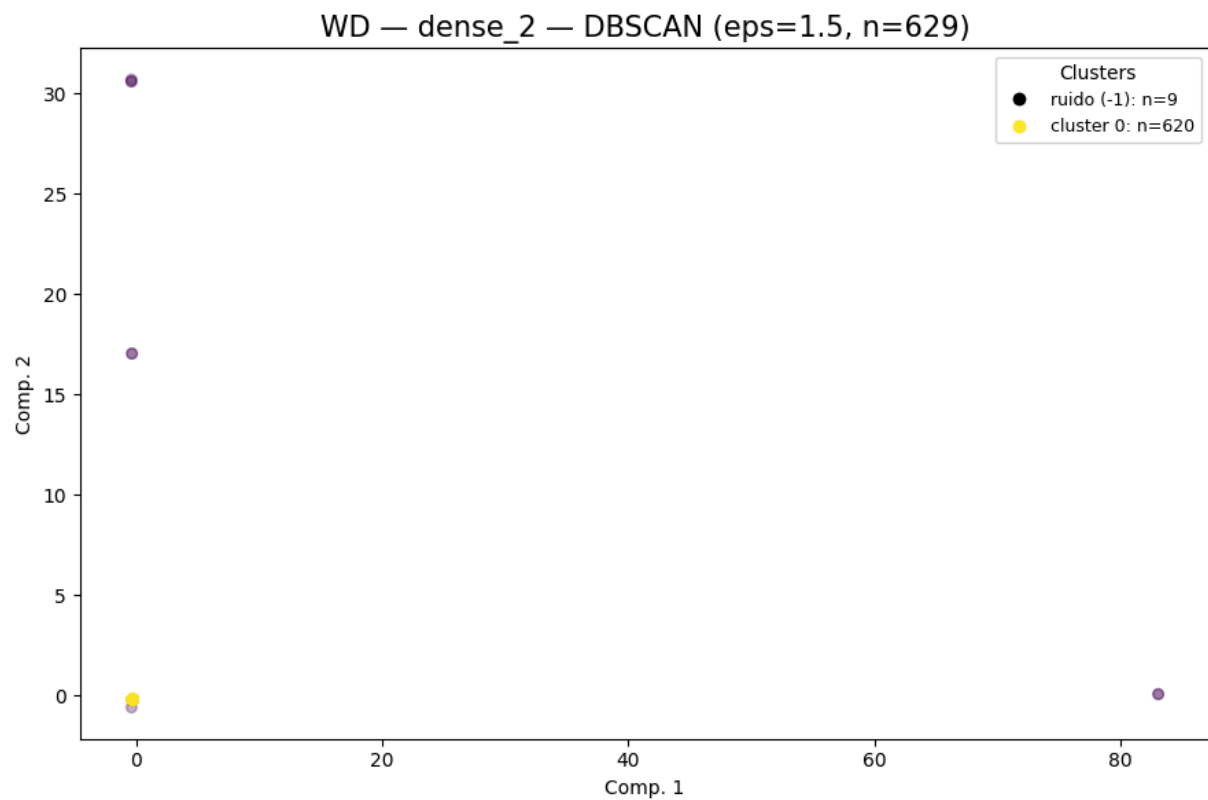
DBSCAN n=700

```
In [84]: res_dbs_dense2 = cluster_plot_DBSCAN(
    repr_wd_32["dense_2"],
    "WD",
    "dense_2",
    dbscan_eps=1.5,
    dbscan_min_samples=10,
    vis_method="tsne",
    xlim=None,
    ylim=None,
    container=cluster_labels_all
)
labels_dbs_dense2_wd = res_dbs_dense2["dbscan"]
cluster_plot_DBSCAN(
    repr_wd_32["dense_2"],
    "WD",
    "dense_2",
    dbscan_eps=1.5,
    dbscan_min_samples=10,
    vis_method="pca",
```

```
xlim=None,  
ylim=None  
)
```



```
WD | dense_2 | dbscan | eps=1.5  
clusters=1 noise=9 sil=N/A DB=N/A CH=N/A  
DBCV = N/A (rango [-1,1], específico DBSCAN)
```



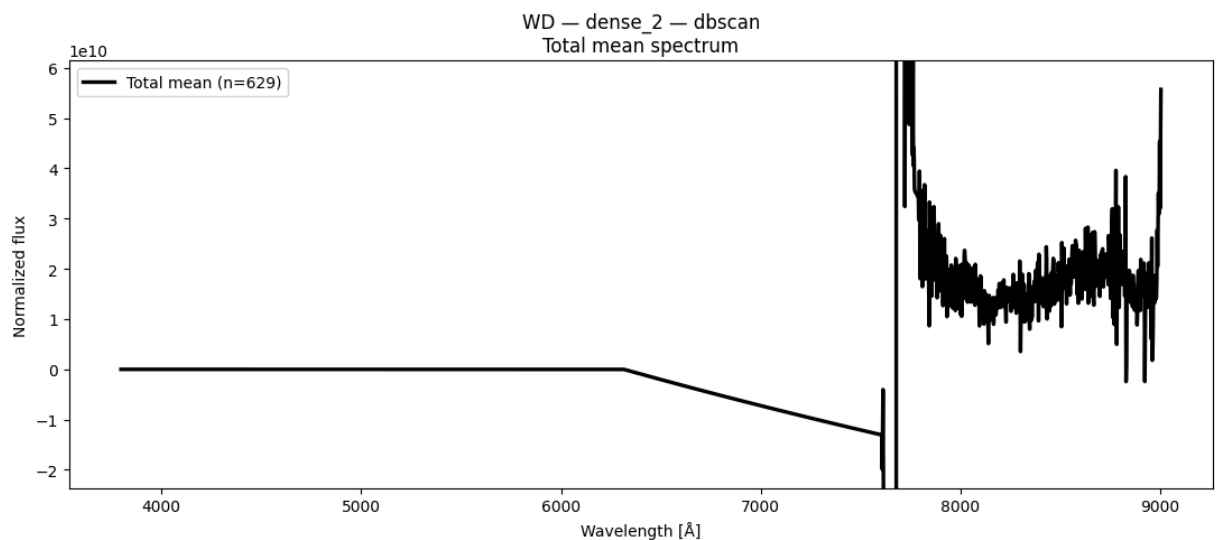
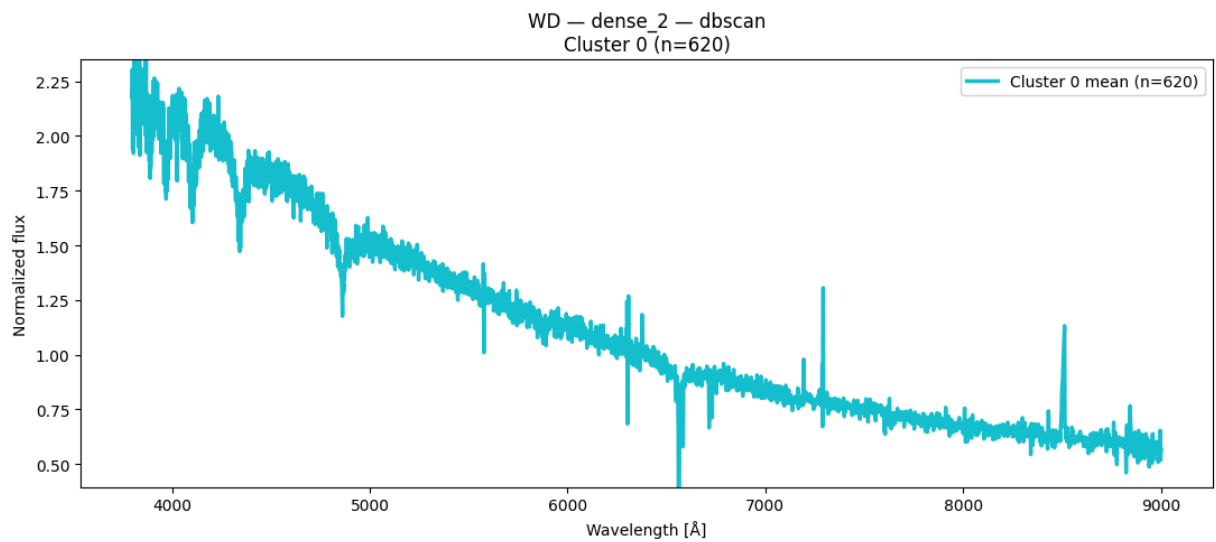
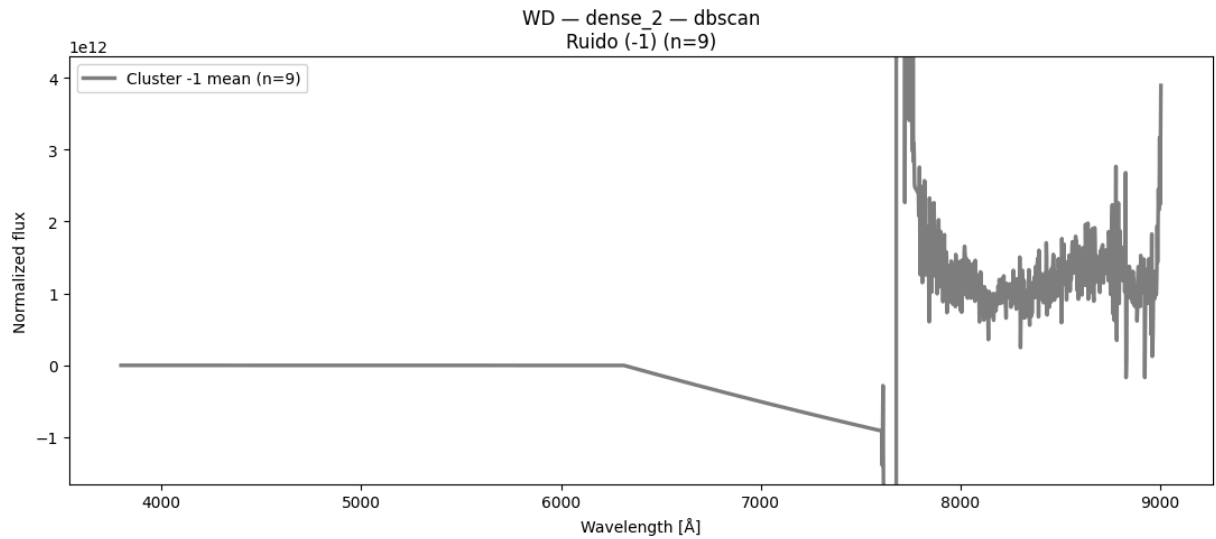
```
WD | dense_2 | dbscan | eps=1.5  
clusters=1 noise=9 sil=N/A DB=N/A CH=N/A  
DBCV = N/A (rango [-1,1], específico DBSCAN)
```

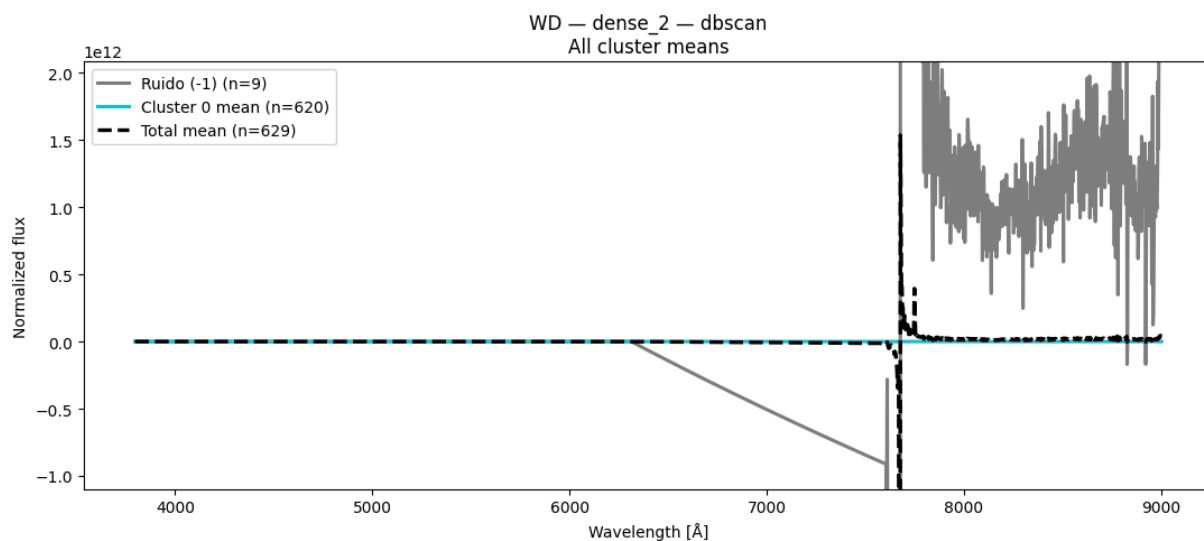
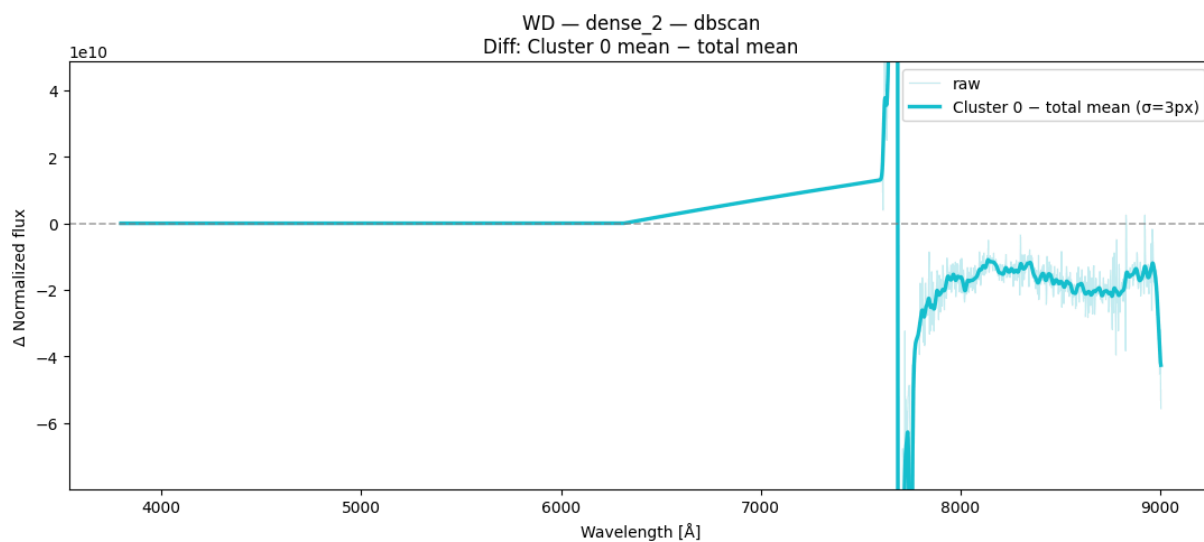
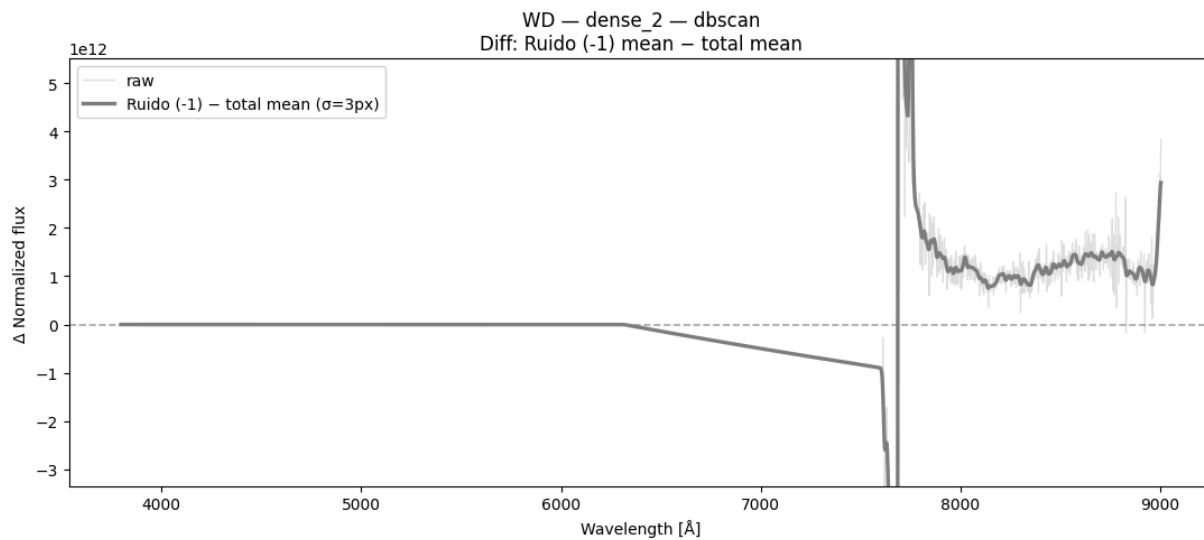
[illegible]

Espectros obtenidos para dense_2 con DBSCAN y $n = 700$

```
In [85]: plot_spectra_dbscan(
        X_wd_spec,
        labels_dbs_dense2_wd,
        W_wd[0],
        class_name="WD",
        layer_name="dense_2"
    )
```

```
C:\Users\javip\AppData\Local\Temp\ipykernel_52368\531566132.py:28: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get_cmap()`` or ``pyplot.get_cmap()`` instead.
cmap      = plt.cm.get_cmap("tab10", n_clusters)
```






```
Out[85]: {'total_mean': array([1.8663167e+00, 1.4631674e+00, 2.3618588e+00, ..., 3.2000662e
+10,
      4.3846500e+10, 5.5692263e+10], dtype=float32),
'group_means': {-1: array([-1.9543549e+01, -4.9689087e+01,  6.3476996e+00, ...,
      2.2364907e+12,  3.0643831e+12,  3.8922706e+12], dtype=float32),
      0: array([2.177106 , 2.2057014 , 2.304      , ..., 0.51832837, 0.5622155 ,
      0.5662164 ], dtype=float32)},
'unique_clusters': array([-1,  0], dtype=int64)}
```

Tabla de evaluación con métricas

```
In [86]: # construir tabla para WD con n=700
metrics_wd = build_metrics_table(
    repr_wd_32,
    class_name="WD",
    spectral_k_values=[2, 3, 4, 5, 6],
    dbscan_eps_values=[0.5, 1.0, 1.5, 2.0, 2.5, 3.0],
    dbscan_min_samples=10
)
metrics_wd
```

Out[86]:

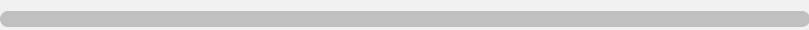
	class	layer	method	param	param_value	n_clusters	n_noise	silhouette	davies_b
0	WD	flatten	spectral	k	2.0	2	0	0.197746	2.4
1	WD	flatten	spectral	k	3.0	3	0	0.327003	1.9
2	WD	flatten	spectral	k	4.0	4	0	0.443064	1.7
3	WD	flatten	spectral	k	5.0	5	0	0.478238	1.4
4	WD	flatten	spectral	k	6.0	6	0	0.503960	2.0
5	WD	flatten	dbscan	eps	0.5	4	64	0.998674	0.0
6	WD	flatten	dbscan	eps	1.0	4	63	0.996886	0.0
7	WD	flatten	dbscan	eps	1.5	4	63	0.996886	0.0
8	WD	flatten	dbscan	eps	2.0	3	63	0.747332	0.3
9	WD	flatten	dbscan	eps	2.5	2	63	0.598638	0.4
10	WD	flatten	dbscan	eps	3.0	1	63	NaN	
11	WD	dense	spectral	k	2.0	2	0	0.496635	2.1
12	WD	dense	spectral	k	3.0	3	0	0.170522	2.4
13	WD	dense	spectral	k	4.0	4	0	0.299950	2.8
14	WD	dense	spectral	k	5.0	5	0	0.375520	2.5
15	WD	dense	spectral	k	6.0	6	0	0.446932	2.0
16	WD	dense	dbscan	eps	0.5	6	109	0.999998	0.0
17	WD	dense	dbscan	eps	1.0	6	109	0.999998	0.0
18	WD	dense	dbscan	eps	1.5	6	109	0.999998	0.0
19	WD	dense	dbscan	eps	2.0	6	109	0.999998	0.0
20	WD	dense	dbscan	eps	2.5	5	86	0.651567	0.5
21	WD	dense	dbscan	eps	3.0	2	78	0.498785	0.8
22	WD	dense_1	spectral	k	2.0	2	0	0.024737	1.5
23	WD	dense_1	spectral	k	3.0	3	0	0.128934	2.2
24	WD	dense_1	spectral	k	4.0	4	0	0.246754	1.9
25	WD	dense_1	spectral	k	5.0	5	0	0.250763	2.6
26	WD	dense_1	spectral	k	6.0	6	0	0.278213	1.7
27	WD	dense_1	dbscan	eps	0.5	5	381	0.659410	0.6
28	WD	dense_1	dbscan	eps	1.0	4	163	0.581505	0.6
29	WD	dense_1	dbscan	eps	1.5	4	103	0.524051	0.7

	class	layer	method	param	param_value	n_clusters	n_noise	silhouette	davies_b
30	WD	dense_1	dbscan	eps	2.0	4	82	0.502727	0.7
31	WD	dense_1	dbscan	eps	2.5	2	68	0.335052	1.0
32	WD	dense_1	dbscan	eps	3.0	2	58	0.321077	1.0
33	WD	dense_2	spectral	k	2.0	2	0	0.246113	2.6
34	WD	dense_2	spectral	k	3.0	3	0	0.101788	2.5
35	WD	dense_2	spectral	k	4.0	4	0	-0.055605	2.5
36	WD	dense_2	spectral	k	5.0	5	0	-0.117476	2.4
37	WD	dense_2	spectral	k	6.0	6	0	-0.076407	2.4
38	WD	dense_2	dbscan	eps	0.5	1	13	NaN	
39	WD	dense_2	dbscan	eps	1.0	1	10	NaN	
40	WD	dense_2	dbscan	eps	1.5	1	9	NaN	
41	WD	dense_2	dbscan	eps	2.0	1	9	NaN	
42	WD	dense_2	dbscan	eps	2.5	1	8	NaN	
43	WD	dense_2	dbscan	eps	3.0	1	8	NaN	

In [87]: `get_best_configs(metrics_wd, top_n=10)`

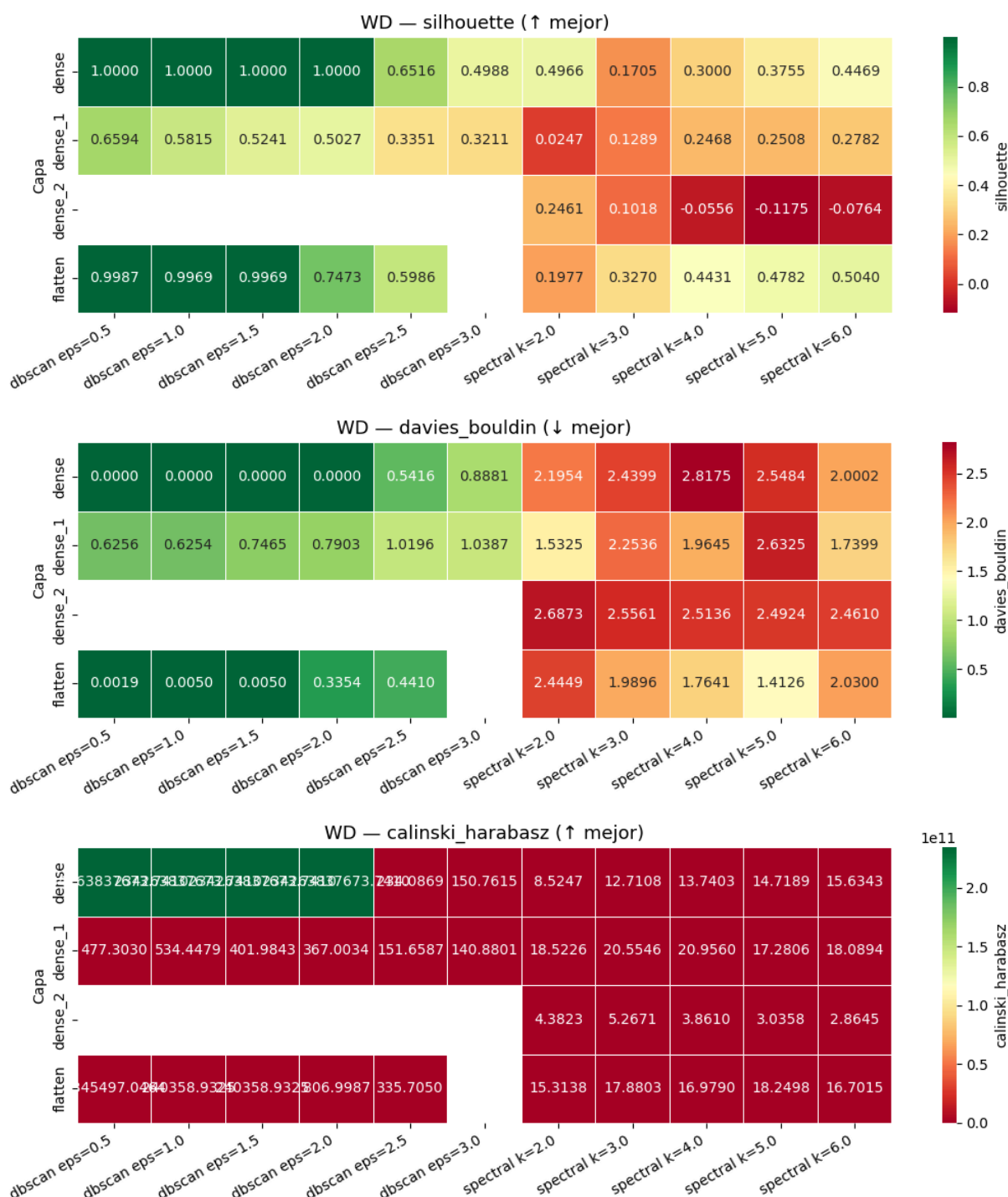
Out[87]:

	class	layer	method	param_value	n_clusters	n_noise	silhouette	davies_bouldin	ca
0	WD	dense	dbscan	2.0	6	109	0.999998	0.000013	
1	WD	dense	dbscan	1.5	6	109	0.999998	0.000013	
2	WD	dense	dbscan	1.0	6	109	0.999998	0.000013	
3	WD	dense	dbscan	0.5	6	109	0.999998	0.000013	
4	WD	flatten	dbscan	0.5	4	64	0.998674	0.001930	
5	WD	flatten	dbscan	1.0	4	63	0.996886	0.005012	
6	WD	flatten	dbscan	1.5	4	63	0.996886	0.005012	
7	WD	flatten	dbscan	2.0	3	63	0.747332	0.335428	
8	WD	dense_1	dbscan	0.5	5	381	0.659410	0.625551	
9	WD	dense_1	dbscan	1.0	4	163	0.581505	0.625378	

<  >

In [88]: `plot_metrics_heatmap(metrics_wd, metric="silhouette", class_name="WD")`
`plot_metrics_heatmap(metrics_wd, metric="davies_bouldin", class_name="WD")`

```
plot_metrics_heatmap(metrics_wd, metric="calinski_harabasz", class_name="WD")
```



interactive —> Spectral Clustering (tSNE + PCA)

```
In [94]: spectral_explorer(repr_wd_32, class_name="WD", suffix="wd")
```

```
interactive(children=(Dropdown(description='Capa:', options=('flatten', 'dense', 'dense_1', 'dense_2'), value=...
```

interactive —> DBSCAN (tSNE + PCA)

```
In [92]: dbscan_explorer(repr_wd_32, class_name="WD", suffix="wd")
```

```
interactive(children=(Dropdown(description='Capa:', options=('flatten', 'dense', 'dense_1', 'dense_2'), value=...
```

Explorador de espectros por cluster (Spectral + DBSCAN)

Visualiza los espectros promedio de cada cluster a partir de experimentos ya ejecutados anteriormente

Controles:

- **Método / Capa / k** (Spectral) o **eps + min_samples** (DBSCAN)
- **Media clase** — muestra/oculta el espectro promedio de la clase en el gráfico combinado
- **Balmer** — superpone las líneas de Balmer
- **$\pm 1\sigma$ individual** — banda de desviación estándar en los gráficos individuales

```
In [93]: spectra_viewer(X_wd_spec, W_wd[0], class_name="WD", suffix="wd")
```

```
VBox(children=(HBox(children=(Dropdown(description='Metodo:', layout=Layout(width='190px'), options=('spectral...
```